Virtual University Of Pakistan

Subject BIF604

Special Topics in Bioinformatics

Create By

VU Bio Mates Team

https://www.facebook.com/groups/vubiomates/

Lesson-1

Instructor Introduction

Mirza Jawad Ul Hasnain

MS-Bioinformatics

Next Gen. Solution

Research Area

Next Generation Sequencing

RNASeq Data Analysis

Statistical Data Analysis

- Gene NetworksMirza Jawad Ul Hasnain
- MS-Bioinformatics
- Next Gen. Solution
- Research Area
    - Next Generation Sequencing
    - RNASeq Data Analysis
    - Statistical Data Analysis
    - Gene Networks



Case Studies

Assignments will also include the performance and write up scheme.



Lesson-2

Course Introduction

Areas covering in this course

- Sequencing Techniques
- Next generation techniques
- R language for Bioinformatics
- Python for Bioinformatics
- Advance techniques in Bioinformatics

Outcomes of the course

- Sequencing Techniques
    - Alignment
    - Assembly
    - Gene Regulation
    - Gene Annotation
    - Tools for Next Generation Sequencing
- Linux an operating system
- Python for Bioinformatics
    - Biopython
- R for Bioinformatics
- Metagenomics
- Advance techniques in Bioinformatics

Recent Techniques In Sequencing

- High throughput sequencing also called next generation sequencing (NGS) have the capacity to sequence full genomes.
- These technologies includes Roche's 454 GS FLX, Illumina's Solexa Technology, ABI's SOLiD Technology and Ion Torrent Technology.

Lesson-3

Next Generation Techniques

Sequencing

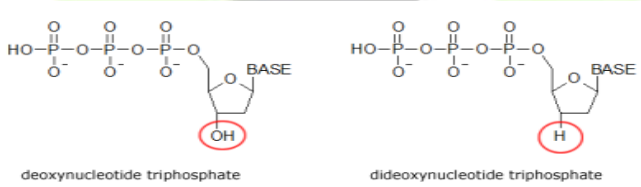
Sequencing is the process to determine the precise order of nucleotides or amino acids in DNA or RNA

molecule respectively.

- DNA Sequencing
- RNA Sequencing
- Single molecule detection/sequencing

History of Sequencing

- Early Sequencing was performed with transfer RNA (tRNA) via a technique developed by Richard Holley, a published in 1964.
- Technique involves in breaking down RNA molecule and then baffling the fragments back together.
- This technique is Time consuming due to its large molecular size
- In 1988 Fredrick Sanger prospered a method that allowed sequencing of up to 50 nucleotide in length.
- In 1975 Sanger developed "the plus and minus method" based on the principal of chain termination during polymerization and sequenced a complete genome of φX 174 bacteriophage



- . Locat addition of ddNTPs will terminate chain elongationion of ddNTP insertion within a nucleotide chain can be determined using gel separation.

Three innovations came about that greatly expedited the sequencing process:

- Shot-gun sequencing
- PCR (Polymerization Chain Reaction)
- Automation of sequencing

Lesson-4

Current and Emerging Sequencing Techniques

Sequencing By Hybridization (SBH)

- The array contains all possible oligonucleotide sequences of a given length.
- DNA of unknown sequence is incubated with the array.

What is Nanopore Sequencing?

Sequencing by hybridization (SBH)

- The target hybridizes to the array wherever there is complementation to a portion of the target.
- Hybridization of oligos are detected by fluorescence.

- The probes are organized by overlaps with one another to reconstruct the target sequence.

Limitations:

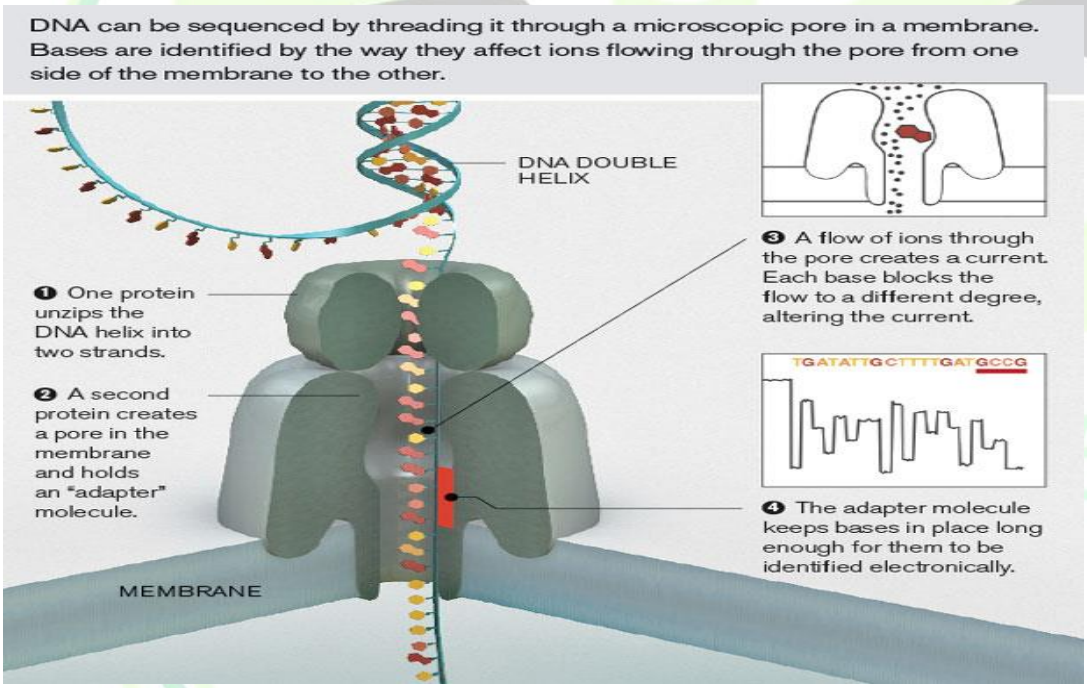
- Difficult to reconstruct long sequences.
- Very large libraries are required.
- The normal approach to SBH is also sensitive to errors.

Lesson-5

Nanopore Sequencing

Nanopore Sequencing

- Determine the sequence of DNA fragments by passing DNA through a protein (or other) pore in a membrane.
- Transmembrane porins

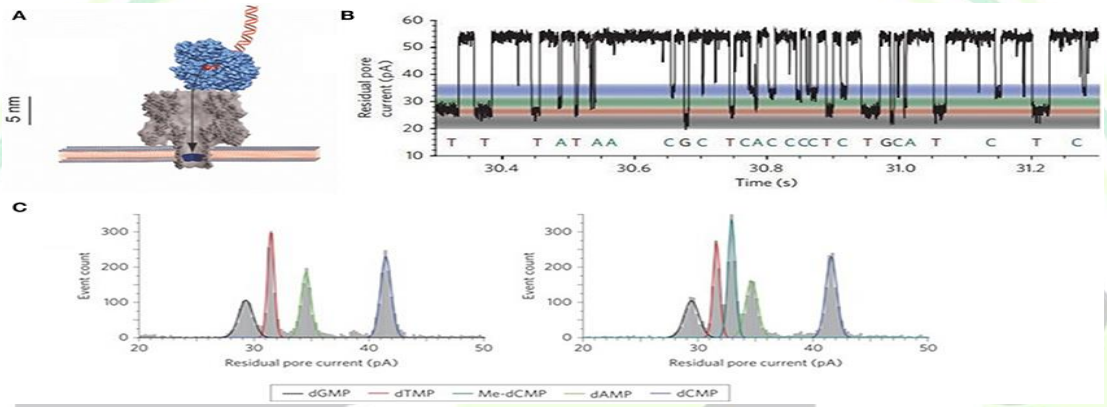


- Oxford Nanopore became the first company to provide a commercially available nanopore sequencer in 2015 (available to community in 2012)



- Nanopore is a disruptive technology:
- Sequencer Size

- Read Length 10,000 bp to 15,000 bp average
- Potential direct RNA sequencing
- Biology Problem with Data Velocity Issues
    - Currently ~400GB/24 hours needs to be processed
- ~50 (250-400 with new R9 pore) base pairs per second pass through a pore
- Need to segment signal into individual events representing base pairs and determine to which base each event corresponds



Lesson-6

Sequencing-By-Synthesis (SBS)

Sequencing-by-synthesis (SBS)

- In NGS, a huge number of short length reads are sequenced parallel to each other in a single run.
- The input sample is first cleaved into short length fragments. 100-150 bp.
- These fragments are ligated to the generic adaptors and annealed to slide via adaptors where sequencing takes place.
- Pcr amplifies each read, thus creating a spot with many copies of a same read. The sequences are then separated into single strands that are ultimately to be sequenced.
- The slide is occupied by a large number of nucleotides and DNA polymerases
- The nucleotides are fluorescently labeled with the corresponding color of the bases they are related to.
- A terminator also exists, to make sure that only one base is added at a time.
- Each slide is captured, and in each location, there will be a fluorescent signal that would be indicating the base that has been added.
- The slide is then prepared for the following cycle.
- The terminators are removed, this allows the next base to be added, and the fluorescent signal is removed so that it does not happen to contaminate the next image.

- The process is repeated, ensuring addition of one nucleotide at a time and imaging side by side.
- Computational forces are then used to detect the base at each location in each image and then these are used to construct a complete sequence

Advantages

- Allows parallel sequencing.
- Use of photons requires no additional chemical reagents.
- Clean products with no need of subsequent purification.

Lesson-7a

Other Sequencing Techniques

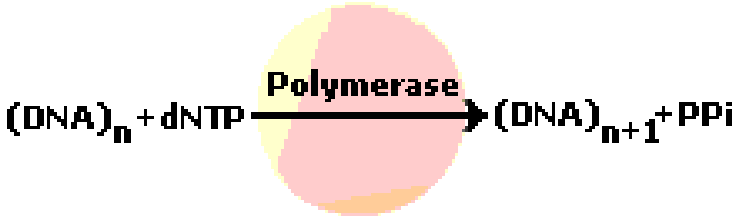
Single-nucleotide addition (SNA)

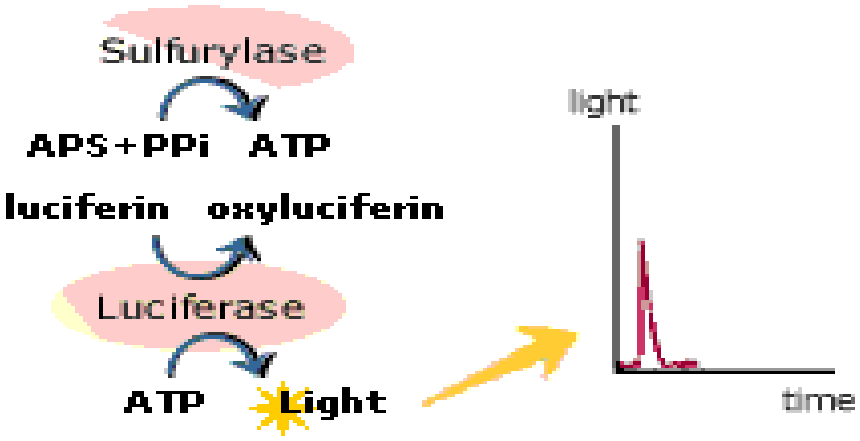
Pyrosequencing

- Non-sanger nonfluorescence technique that quantitatively measures released PPi
- Pyrogram corresponds to complementary base

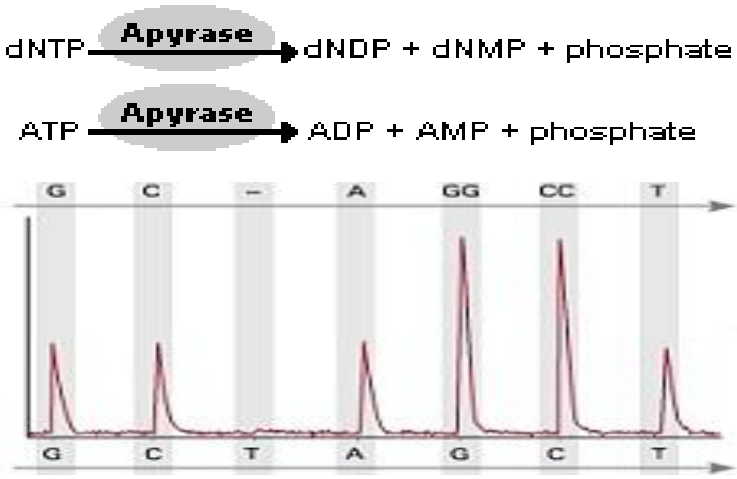
Applications and advantages

- SNP analysis
- Ideal for rapidly mutating organisms
- Quantifications provide additional data

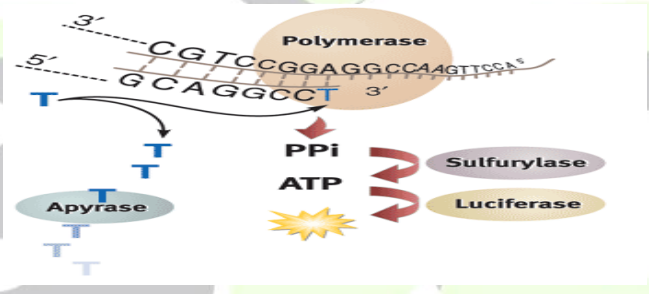




nucleotide incorporation generates light seen as a peak in the pyrogram



- deoxyadenosine alfa-thio triphosphate (dATPαS) is used as a substitute for the natural deoxyadenosine triphosphate (dATP) since it is efficiently used by the DNA polymerase, but not recognized by the luciferase



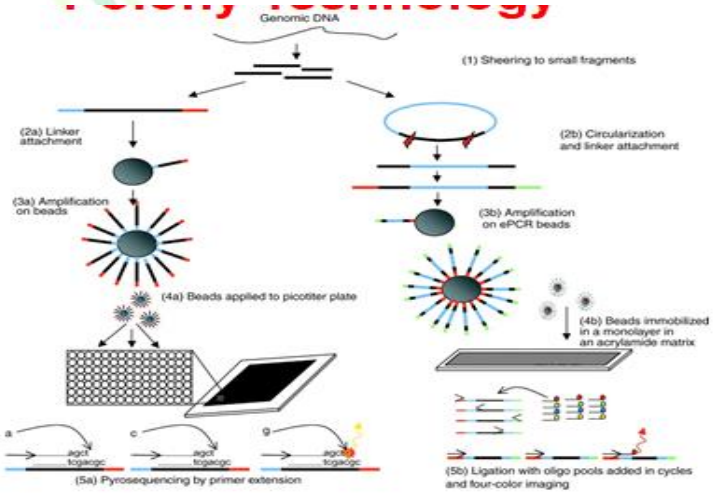
Limitations

- Short sequence reads
- Homopolymer repeat problems

Lesson-7b

Other Sequencing Techniques

Polony Technology



- Polony (polymerase colony) is amplified product from single DNA molecule in acrylamide gel.
- Sequencing done by the incorporation of cleavable fluorescent labeled nucleotide.

Advantage

- Scalability is easy by using 1µm magnetic beads.

Disadvantage

- Failure in cleaving dye moiety.

Lesson-7c

Other Sequencing Techniques

Comparative genome sequencing

- Test DNA is hybridized with reference DNA to identify regions of genomic differences.
- Genomic different regions are sequenced to identify SNPs.

Advantages

- Fast, accurate sequencing of the regions of interest

Table 1. Comparative genome sizes of humans and other model organisms

Organism	Estimated size (base pairs)	Chromosome number	Estimated gene number
Human (*Homo sapiens*)	3 billion	46	~25,000
Mouse (*Mus musculus*)	2.9 billion	40	~25,000
Fruit fly (*Drosophila melanogaster*)	165 million	8	13,000
Plant (*Arabidopsis thaliana*)	157 million	10	25,000
Roundworm (*Caenorhabditis elegans*)	97 million	12	19,000
Yeast (*Saccharomyces cerevisiae*)	12 million	32	6,000
Bacteria (*Escherichia coli*)	4.6 million	1	3,200

Cyclic reversible terminator (CRT)

- Sequencing by CRT consists of three steps; incorporation, imaging and deprotection. The reversible terminator must be cleaved efficiently with photocleaving groups like 2-nitrobenzyl group.

Advantages

- Avoids gel electrophoresis, functions in highly parallel fashion, high throughput, speed and accuracy.

Lesson-8a

NGS data formats

File Types

- Many software packages have been developed for the analysis of DNA and protein sequences.

- A variety of different file formats have been developed to store or analyse DNA and protein sequence information
- The various software packages will usually only accept a specific file format.
- The situation is made worse by the fact that different databases hold the information in different file formats
- An essential skill is be able to recognize the different formats and to be able to interconvert files between formats

Why different formats?

- Type of information
- Software requirement
- Database requirement

Main file formats used in Bioinformatics

- **ASN.1** **Abstract Syntax notation One**
- EMBL, SwissProt
- FASTA, FASTq
- GCG
- GeneBank
- Phylip/PIR

An example sequence in EMBL format is:

```
ID   AB000263 standard; RNA; PRI; 368 BP.
XX
AC   AB000263;
XX
DE   Homo sapiens mRNA for prepro cortistatin like peptide, complete cds.
XX
SQ   Sequence 368 BP;
     acaagatgcc attgtccccc ggcctcctgc tgctgctgct ctccggggcc acggccaccg        60
     ctgccctgcc cctggagggt ggccccaccg gccgagacag cgagcatatg caggaagcgg       120
     caggaataag gaaaagcagc ctcctgactt tcctcgcttg gtggtttgag tggacctccc       180
     aggccagtgc cgggcccctc ataggagagg aagctcggga ggtggccagg cggcaggaag       240
     gcgcacccca ccagcaatcc gcgcgccggg acagaatgcc ctgcaggaac ttcttctgga       300
     agaccttctc ctcctgcaaa taaaacctca cccatgaatg ctcacgcaag tttaattaca       360
     gacctgaa                                                               368
//
```

```
>AB000263 |acc=AB000263|descr=Homo sapiens mRNA for prepro cortistatin like peptide, complete cds.|len=368
ACAAGATGCCATTGTCCCCCGGCCTCCTGCTGCTGCTGCTCTCCGGGGCCACGGCCACCGCTGCCCTGCC
CCTGGAGGGTGGCCCCACCGGCCGAGACAGCGAGCATATGCAGGAAGCGGCAGGAATAAGGAAAAGCAGC
CTCCTGACTTTCCTCGCTTGGTGGTTTGAGTGGACCTCCCAGGCCAGTGCCGGGCCCCTCATAGGAGAGG
AAGCTCGGGAGGTGGCCAGGCGGCAGGAAGGCGCACCCCCCCAGCAATCCGCGCGCCGGGACAGAATGCC
CTGCAGGAACTTCTTCTGGAAGACCTTCTCCTCCTGCAAATAAAACCTCACCCATGAATGCTCACGCAAG
TTTAATTACAGACCTGAA
```

```
@SEQUENCE_ID
GTGGAAGTTCTTAGGGCATGGCAAAGAGTCAGAATTTGAC
+
FAFFADEDGDBGEGGBCGGHE>EEBA@@=
```

```
ID   AB000263 standard; RNA; PRI; 368 BP.
XX
AC   AB000263;
XX
DE   Homo sapiens mRNA for prepro cortistatin like peptide, complete cds.
XX
SQ   Sequence 368 BP;
AB000263  Length: 368  Check: 4514  ..
       1  acaagatgcc attgtccccc ggcctcctgc tgctgctgct ctccggggcc acggccaccg
      61  ctgccctgcc cctggagggt ggccccaccg gccgagacag cgagcatatg caggaagcgg
     121  caggaataag gaaaagcagc ctcctgactt tcctcgcttg gtggtttgag tggacctccc
     181  aggccagtgc cgggcccctc ataggagagg aagctcggga ggtggccagg cggcaggaag
     241  gcgcacccca ccagcaatcc gcgcgccggg acagaatgcc ctgcaggaac ttcttctgga
     301  agaccttctc ctcctgcaaa taaaacctca cccatgaatg ctcacgcaag tttaattaca
     361  gacctgaa
```

```
LOCUS       AB000263                368 bp    mRNA    linear   PRI 05-FEB-1999
DEFINITION  Homo sapiens mRNA for prepro cortistatin like peptide, complete
            cds.
ACCESSION   AB000263
ORIGIN
        1 acaagatgcc attgtccccc ggcctcctgc tgctgctgct ctccggggcc acggccaccg
       61 ctgccctgcc cctggagggt ggccccaccg gccgagacag cgagcatatg caggaagcgg
      121 caggaataag gaaaagcagc ctcctgactt tcctcgcttg gtggtttgag tggacctccc
      181 aggccagtgc cgggcccctc ataggagagg aagctcggga ggtggccagg cggcaggaag
      241 gcgcacccca ccagcaatcc gcgcgccggg acagaatgcc ctgcaggaac ttcttctgga
      301 agaccttctc ctcctgcaaa taaaacctca cccatgaatg ctcacgcaag tttaattaca
      361 gacctgaa
//
```

SRA format

- The SRA is a "raw data" archive, and requires per-base quality scores for all submitted data. Therefore, FASTA and other sequence-only formats are not sufficient for submission! FASTA can, however, be submitted as a reference sequence(s) for BAM files or as part of a FASTA/QUAL pair.
- Text formats, such as FASTQ, are supported, but are not the preferred file format for SRA submission.
- SRA prefers files such as BAM, SFF, and HDF5 formats.

sign of bright future

Lesson-8b

NGS data formats

BAM files

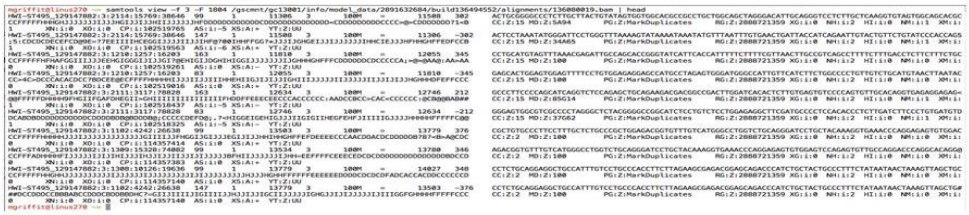
- Binary alignment/map files (BAM) are the preferred SRA submission format. BAM is a compressed version of the sequence alignment/map (SAM) format BAM files can be decompressed to a human-readable text format (SAM) using SAM/bam-specific utilities and can contain unaligned sequences as well.

Example of SAM/BAM file format

Example SAM/BAM header section (abbreviated)



Example SAM/BAM alignment section (only 10 alignments shown)



SAM Files

- Sequence Alignment Map format. A TAB-delimited text format consisting of a header section and alignment body/section. The Each line of header sections start with @ sign and alignment section don't. 11 compulsory fields having alignment information will be present in each alignment line such as aligner specific information and mapping position etc.

11 columns of SAM File

1. Read ID
2. The SAM flag
3. Chromosome/contig read aligned to
4. PosiCon which read aligned to
5. Mapping quality score
6. Cigar string
7. Chromosome/contig which read pair aligned to
8. PosiCon which read pair aligned to
9. Insert Size
10. Sequence in bases
11. Quality score for each base

11 columns of SAM File

1.	ERR001268.25	Read ID
2.	147	The SAM flag
3.	chr22	Chromosome/conCg read aligned to
4.	44549174	PosiCon which read aligned to
5.	60	Mapping quality score

6. 36M Cigar string
7. = Chromosome/contig which read pair aligned to
8. 44548985 PosiCon which read pair aligned to
9. -225 Insert Size
10. GGTTGGATGTGTATTTTT Sequence in bases
11.)(1)+.5+<.@9A%<;=0IIIHCII?III;IIIII Quality score for each base

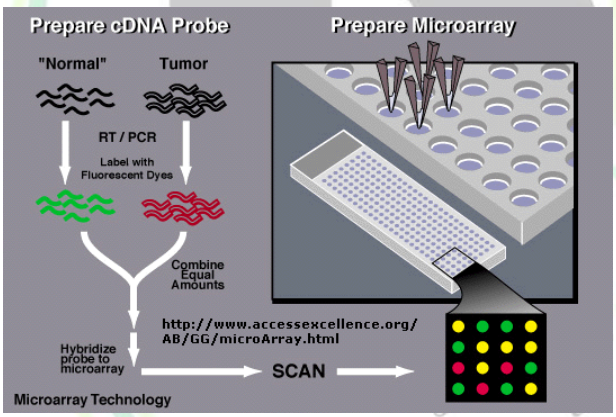
SAM/BAM Format

- SAM (Sequence Alignment Map) format is a text based format that stores alignment data.
- BAM (Binary Alignment Map) format is the binary version of SAM.
- This is the generally accepted file format for aligned sequence data. SAM tools is a utility that can be used to convert between SAM and BAM format. http://samtools.sourceforge.net/

Lesson-9

Gene Expression Omnibus (GEO)

Microarray in general



Gene Expression and Molecular abundance Data Repository

- A public repository for the archiving and distribution of gene expression data submitted by the scientific community.
- Miame compliant data.
  - Minimum information about a microarray experiment http://www.Mged.Org/workgroups/MIAME/miame.Html
- Convenient for deposition of gene expression data, as required by funding agencies and journals.
- Curated, online resource for gene expression data browsing, query, analysis and retrieval.

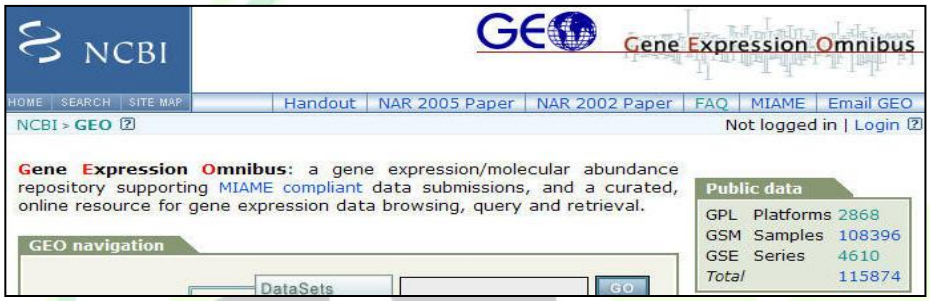
GEO Architecture

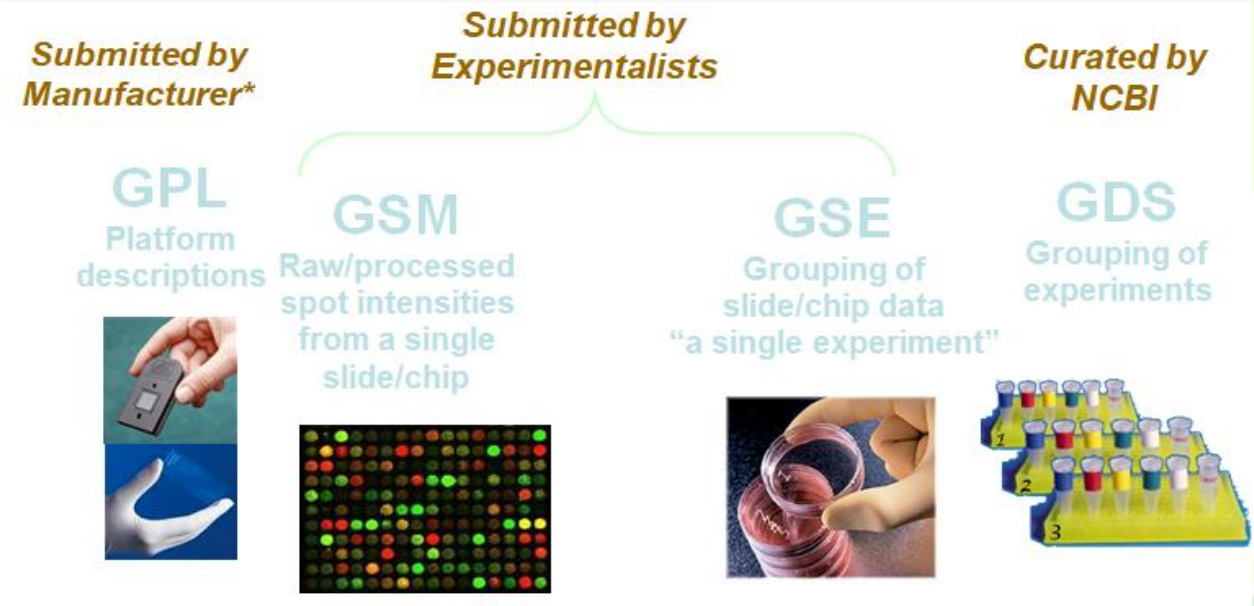
GEO has four kinds of data records

- Platform (GPL) = the technology used and the features detected.
- Sample (GSM) = preparation and description of the sample.
- Series (GSE) defines a set of samples and how they are related.
- Datasets (GDS) sample data collections assembled by geo staff.

Submitters may provide raw data

- Original microarray scans
- Raw quantification data

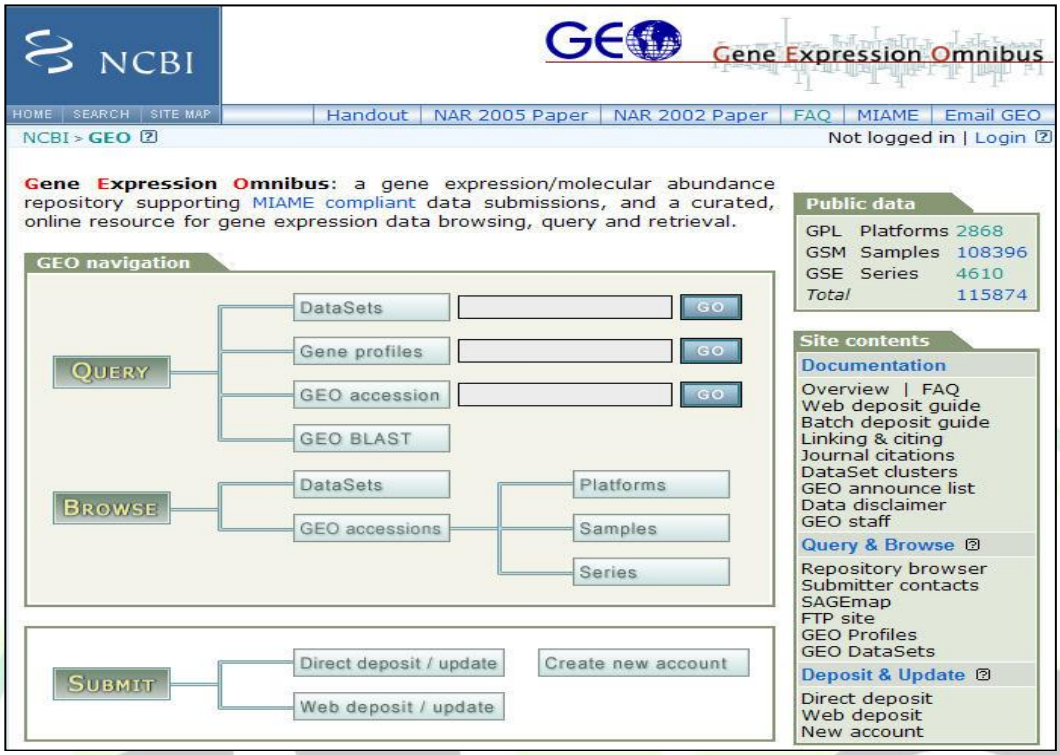


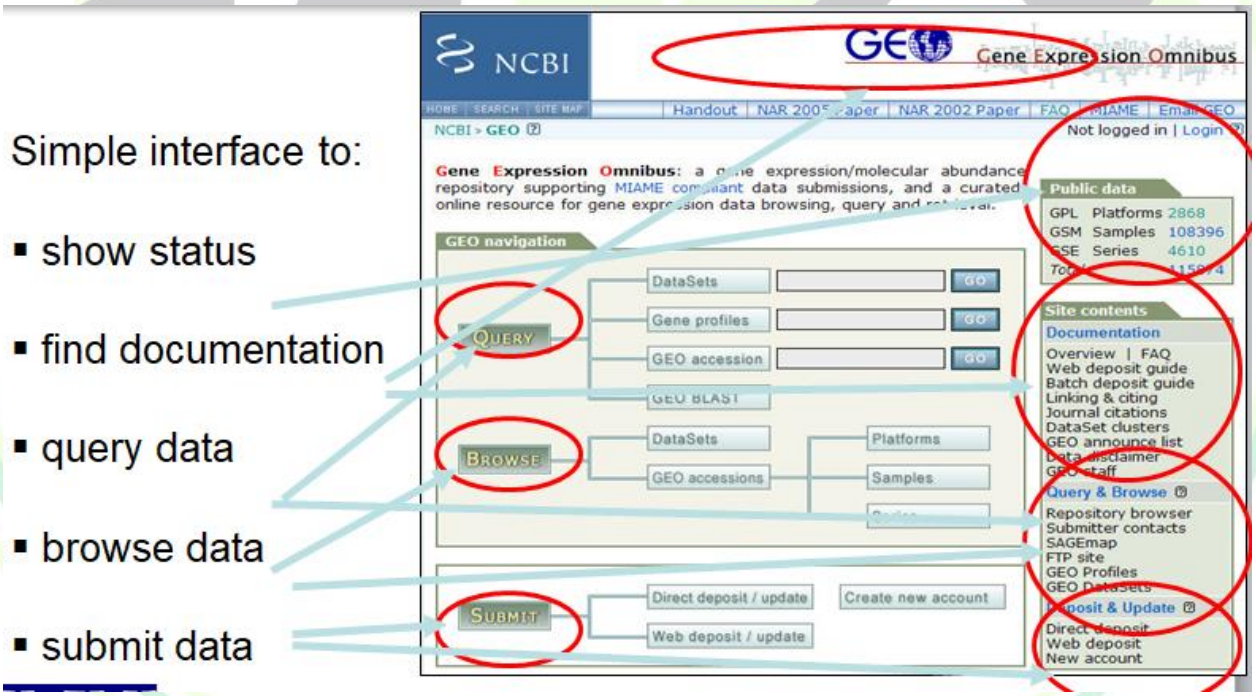


GEO Home Page

Simple interface to:

- show status
- find documentation
- query data
- browse data
- submit data





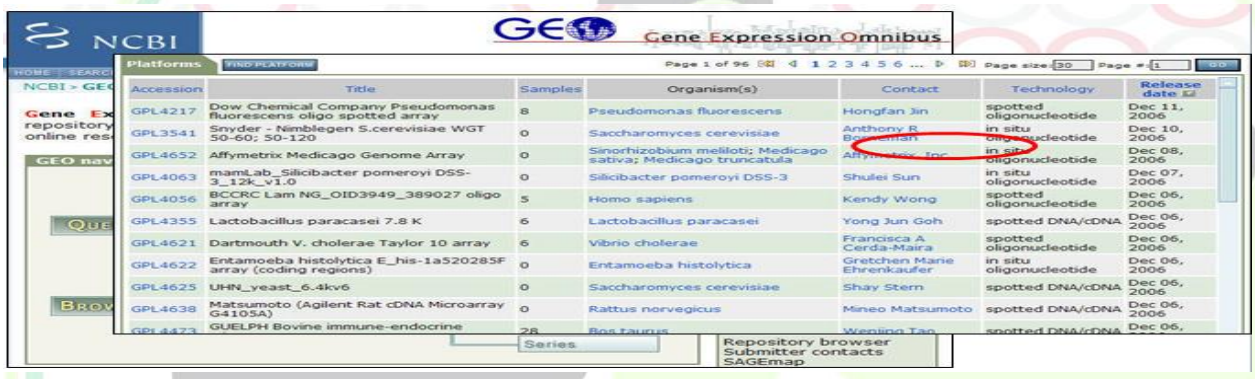
Basic Search: Repository Browser

Selecting the total public data or repository browser links on the GEO home page, takes you to the repository browser, listing:

- Number of each type of submitted file, both public and unreleased
- The total number of each technology type under platforms
- The total number of each sample type



Basic Search: Browser Platforms

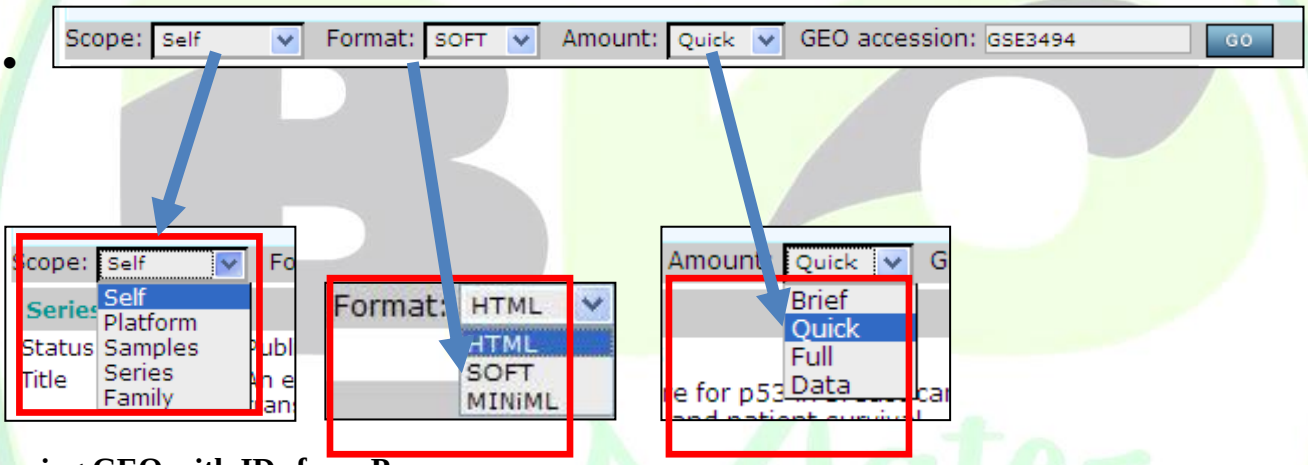


- All GEO submissions need to be associated with a platform file. These describe the features on a given platform, required to understand the data.
- A platform file must be submitted if one is not already present in geo.
- Commercial array platform files are submitted to geo by the manufacturer

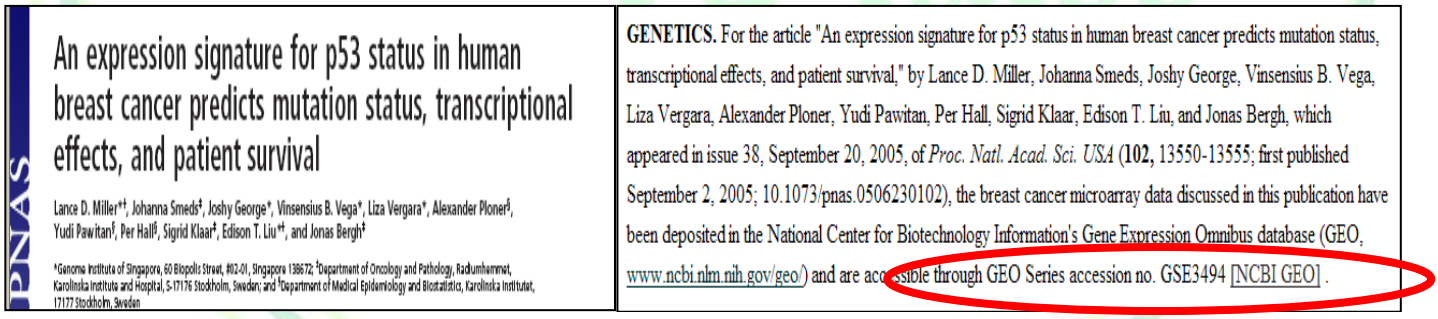
Data Retrieval: Series Accession Page

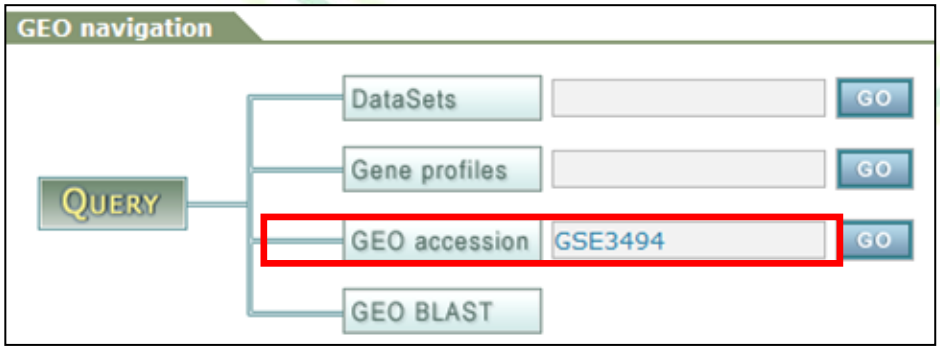


GEO Accession Results Display Options

- All GEO accession results pages have the same header that allows different views and formats for the data to be displayed
- Scope controls what information is displayed:
  - Self
  - Platform, Samples or Series
  - Family
- Format controls how information is displayed:
  - Html
  - SOFT (simple omnibus format in text)
  - Miniml (MIAME notation in markup language)
- 

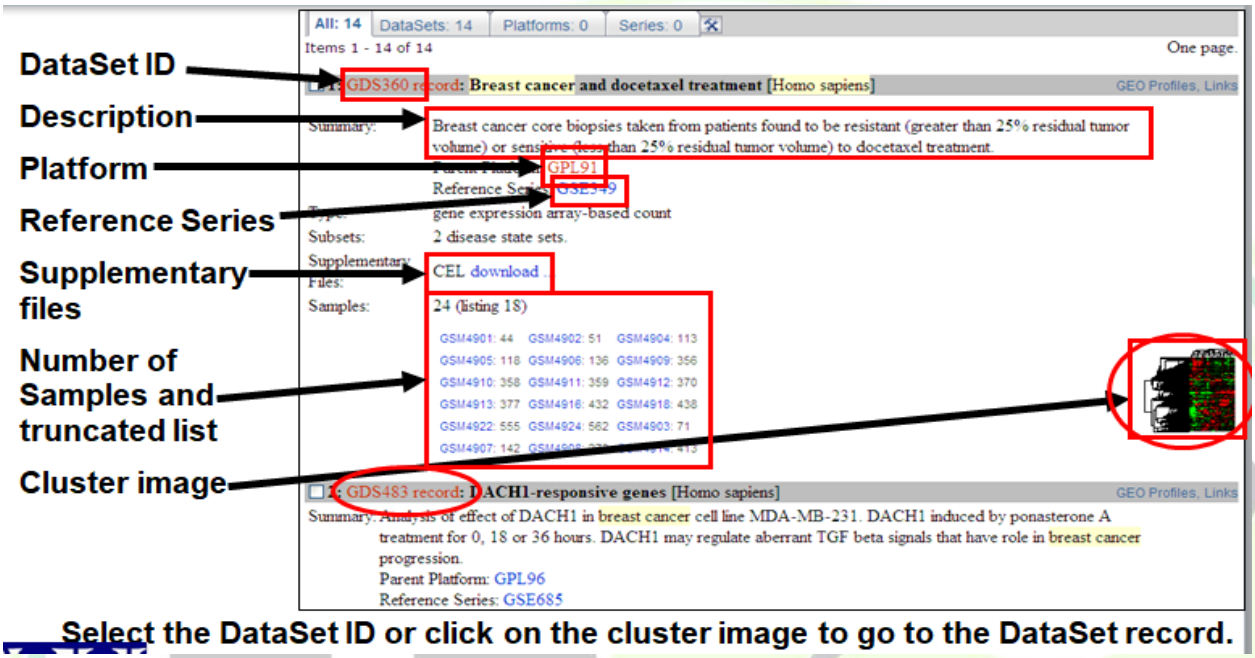
Querying GEO with IDs from Papers





- A common way to access GEO data is through accessions from papers.
- Online journals include hyperlinks to the geo accession page.
- Or, at the geo home page enter the accession into the query>geo accession text box

DataSet Search Result

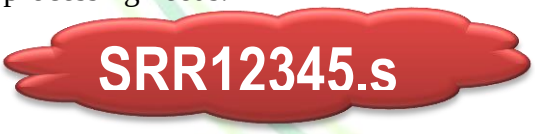


Lesson-10

SRA Files and SRA File Handling & SAM File

SRA Files and SRA File Handling & SAM File

Sequence Read Archive (SRA) files are a common format used by the NCBI, EBI, and others for storing reads and read alignments. While the format provides good compression and data accessibility, some work is often needed to transform SRA-formatted files into a form suitable for visualization and other analytical processing needs.



The SRA is one of the International Nucleotide Sequence Databases and this Collaboration (INSDC) sets policies and goals for the partner databases.

Goals

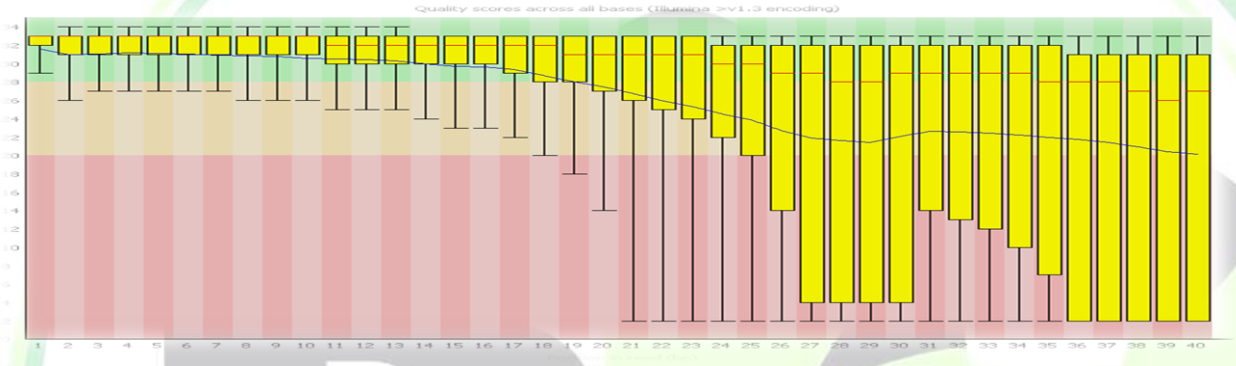
Guide for submitters of sequencing data in order to:

- Specify which data formats are currently supported by SRA.

- Enable submitters to validate and convert data prior submission to avoid unnecessary data transfers.
- Improve the speed of submission processing.
- Reduce the probability of failed submissions.
- Improve other services provided by SRA by freeing up time previously spend to correct and transform data.

The SRA is a "raw data" archive, and requires per-base quality scores for all submitted data. Thus, unlike GenBank and some other NCBI repositories, FASTA and other sequence-only formats are not sufficient for submission. FASTA can, however, be submitted as a reference sequence(s) for BAM file.

The SRA data model has transitioned from "dumps" of whole flowcell lanes or production runs into a semi-curated database of sample-specific sequencing libraries.



- The SRA generally prefers to obtain "container files". Container in this context means an unambiguous binary file. These are objects that contain both the data and a description or specification of the data. Examples include BAM, SFF, and PacBio HDF5 formats. Containers have the following advantages:
- All data for a given library is contained in one file.
- Data are indexed for random access.
- Data are compressed so *gzip* and other compression utilities are discouraged.
- Data are streamable (can be read from one input handle).
- Data are self-identifying (file type can be interrogated with *file*).
- Data come with run-time configuration and execution parameters, including run date, instrument name, flow cell name, processing program and version, etc.

Installation *SRA toolkit*

For Window users

https://trace.ncbi.nlm.nih.gov/Traces/sra/sra.cgi?view=software

For Linux users

wget*http://ftp-trace.ncbi.nlm.nih.gov/sra/sdk/current/sratoolkit.current-centos_linux64.tar.gz*

Unpacking the Toolkit

For Window users

use an archiving and compression utility (e.g., WinZip, 7-Zip, etc.), or simply double-click on the .zip file and drag the ‘ sratoolkit ...' folder to the preferred install location.

For Linux users

tar -xzf sratoolkit.current-centos_linux64.tar.gz

~/[user_name]/sra-toolkit/fastq-dump

Root->directory->sra-toolkit folder->fastq-dump folder

For linux

fastq-dump

Open a terminal or command prompt and "cd" into the directory containing the toolkit executables (e.g., [download_location]/sratoolkit[version]/bin/).

Linux/Mac OSX:

./fastq-dump -X 5 -Z SRR390728

Windows:

fastq-dump.exe -X 5 -Z SRR390728

SRA Toolkit to convert data into different format

- fastq-dump: Converts data to FASTq and FASTA format.
- sam-dump: Converts data to SAM (human-readable bam). Data submitted as aligned bam are output as aligned SAM, while other formats are output as unaligned SAM.
- sff-dump: Converts data to SFF format. Note that only data submitted as SFF can be converted back to this format.
- abi-dump: Converts data to csFASTA / csqual format. Note that data submitted in base-space can be represented in color-space.
- illumina-dump: Converts data to Illumina native and qseq formats.
- vdb-dump: Exports the vdb-formatted data of the .sra file.

Lesson-11

Important Terminologies in NGS Analysis

Terminologies and Concepts in Next Generation Sequencing

Template

A DNA/RNA sequence part of which is sequenced on a sequencing machine or assembled from raw sequences.

Segment

A contiguous sequence or subsequence.

Read

- A raw sequence that comes off a sequencing machine. A read may consist of multiple segments. For sequencing data, reads are indexed by the order in which they are sequenced.
- Range of a read varies from 90 bp to 180 bp even till 1000 bp depending upon the sequencing machine used.

Chimeric alignment

- An alignment of a read that cannot be represented as a linear alignment. A chimeric alignment is represented as a set of linear alignments that do not have large overlaps.
- Typically, one of the linear alignments in a chimeric alignment is considered the "representative" alignment, and the others are called "supplementary" and are distinguished by the supplementary alignment flag.
- All the SAM records in a chimeric alignment have the same QNAME and the same values for 0x40 and 0x80 flags the decision regarding which linear alignment is representative is arbitrary.

Read alignment

A linear alignment or a chimeric alignment that is the complete representation of the alignment of the read.

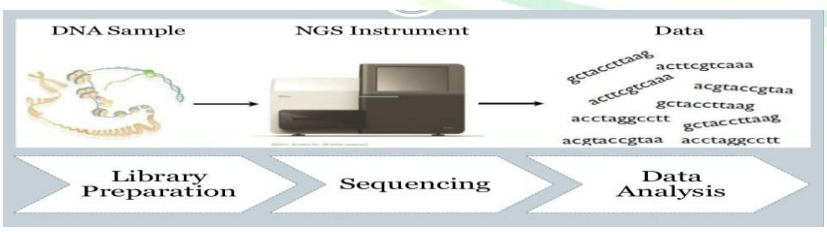
Multiple mapping

- The correct placement of a read may be ambiguous, e.g., Due to repeats.
- In this case, there may be multiple read alignments for the same read. One of these alignments is considered primary.
- All the other alignments have the secondary alignment flag set in the SAM records that represent them.
- All the SAM records have the same QNAME and the same values for 0x40 and 0x80 flags. Typically the alignment designated primary is the best alignment, but the decision may be arbitrary.

Lesson-12

RNA Seq Reads
Alignment vs Assembly

NGS sequencing pipeline



What does one do with RNA-seq reads

- Mapping (also called alignment)
- (de novo) Assembly

Alignment vs Assembly

- Imagine a book being ripped to pieces with word or sentence fragments ending up on each piece of paper.
- If you have a copy of the book that you can compare the pieces to, you have a mapping problem.
- If you have no copy of the book, you have a de novo assembly problem

Mapping to reference genome

CAATCAGA**T**G**A**TCCCACTGTGG

AGACGATCCCACTGTGGGGTG

GTGAAGTGTCCGTAGATGTGTG

GCAAATGCAATCAGACGATCCC

…GCAAATGCAATCAGACGGTCCCACTGTGGGGTGTGAAGTGTCCGTAGAGCTGTGAGAGAA

Gene(ot transcript) sequence

AGACGATCCCACTGTGGGGTG

GTGAAGTGTCCGTAGATGTGTG

GCAAATGCAATCAGACGATCCC

CAATCAGATGATCCCACTGTGG

…GCAAATGCAATCAGACGGTCCCACTGTGGGGTGTGAAGTGTCCGTAGAGCTGTGAGAGAA…

Gene(ot transcript) sequence

GTGAAGTGTCCGTAGATGTGTG

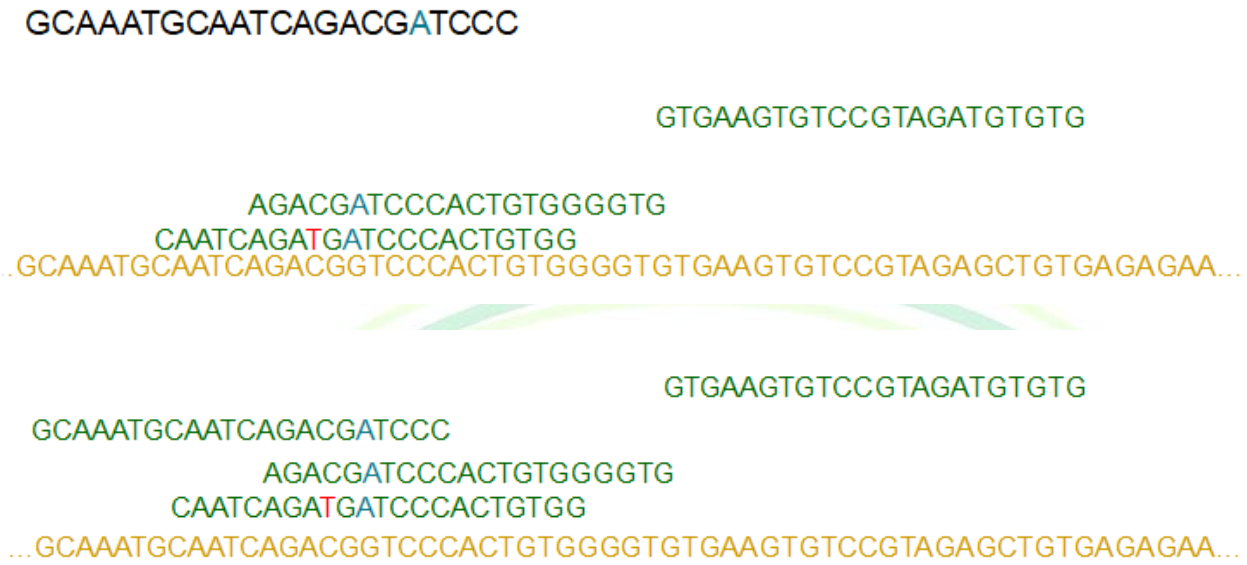
GCAAATGCAATCAGACGATCCC

AGACGATCCCACTGTGGGGTG

CAATCAGATGATCCCACTGTGG

…GCAAATGCAATCAGACGGTCCCACTGTGGGGTGTGAAGTGTCCGTAGAGCTGTGAGAGAA…

Gene(ot transcript) sequence



vs. the genome

- Can (in principal) detect new transcripts, splice variants
- Less sensitive, need a lot of coverage to discover new things
- Need a "splice-aware" aligned such as TopHat, MapSplice, RUM

Vs. the transcriptome

- Not unbiased anymore, tied to existing annotation
- Fatser, more sensitive, need less coverage
- The best of both worlds?

  Tools like TopHat (v1.4 and up) now do both

De-novo assembly



Most popular de novo assemblers build de Bruijn graphs where overlapping kmers are connected to each other. The programs then try to find paths through the graph

Typically needs a Lot of RAM. Can try to pre-process using "digital normalization"

Tools

- Trinity
- Velvet
- CLC Bio (commercial)

Work Flow

- Clean the reads properly(remove Adapters, low-quality reads) (FastQC, PRINSEQ, FASTX)
  - Run assembly tool of choice,resulting in a set of contigs
  - BLAST the contigs against nucleotide database, check for % overlap by transcript in related organisms
  - Map your original reads back to the contigs and count the reads overlapping each.

Lesson-13 [not available]

Lesson-14

CONTIG & SCAFFOLD FORMATION

Contig

- The unbroken series of clones assembled using overlapping sequences.
- More precisely, a set of overlapping DNA segments that together represent a consensus region of DNA

Bottom-up Vs Top-Down Sequencing:

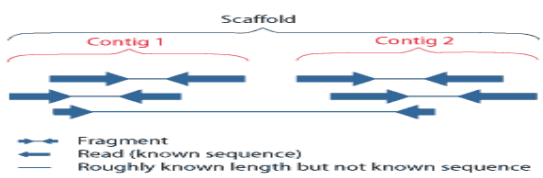
- Bottom-up sequencing:

  contig refers to overlapping sequence data (reads)

- Top-down sequencing:

  contig refers to the overlapping clones that form a physical map of the genome

Contig to Scaffolds

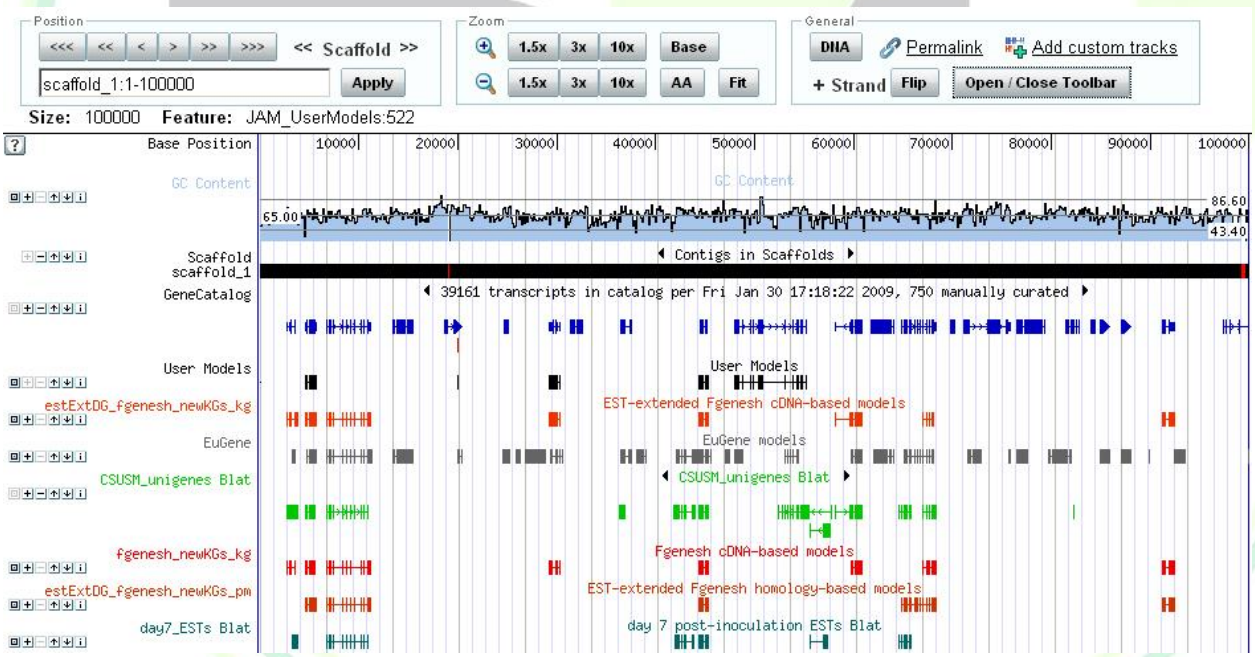


Overlapping reads from paired-end sequencing form contigs; contigs and gaps of known length form scaffolds.

Scaffold

- A scaffold is a portion of the genome sequence reconstructed from end to end-sequenced whole-genome shotgun clones
- Scaffolds are composed of contigs and gaps.
- A **contig** is a contiguous length of genomic sequence in which the order of bases is known to a high confidence level.
- **Gaps** occur where reads from the two sequenced ends of at least one fragment overlap with other reads in two different contigs
- The goal of whole-genome shotgun assembly is to represent each genomic sequence in one scaffold
- Scaffolds are normally numbered approximately from largest to smallest.
- Some scaffolds may ultimately be filtered out of the assembly, resulting in skipped scaffold numbers.
- In some cases, scaffolds can overlap.
- Thus, a sequence that exists in only one location in the genome may appear on more than one scaffold.

Scaffolds in Genome Viewer



Gaps are shown in the Genome Viewer as red lines or rectangles in the scaffold track.

Lesson-15

GTF Files

GTF Format

- The Gene transfer format (GTF) is a file format used to hold information about gene structure.

- It is a tab-delimited text format based on the general feature format (GFF)
- But has additional structure that warrants a separate definition and format name.

GTF FILES

GTF Field Definitions

- **<seqname>**
  The name of the sequence. Commonly, this is the chromosome ID or contig ID**.**
- **<source>**
  The source column should be a unique label indicating where the annotations came from
- **feature>**
  The following feature types are required: "CDS", "start_codon", "stop_codon"
- **<start> <end>**

Integer start and end coordinates of the feature relative to the beginning of the sequence named in <seqname>

- **<score>**
  The score field indicates a degree of confidence in the feature's existence and coordinates.
- **<frame>**

  One of '0', '1' or '2'.

[attributes]
All nine features have the same two mandatory attributes at the end of the record:

gene_id value; A globally unique identifier for the genomic locus of the transcript.

transcript_id value; A globally unique identifier for the predicted transcript. If empty, no transcript is associated with this feature.

- Structure is as GFF, so the fields are:
- <seqname> <source> <feature> <start> <end> <score> <strand> <frame> [attributes] [comments]

Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8	Col 9
chr21	HAVANA	transcript	10862622	10863067	.	+	.	gene_id "ENSG00000169..
chr21	HAVANA	exon	10862622	10862667	.	+	.	gene_id "ENSG00000169..
chr21	HAVANA	CDS	10862622	10862667	.	+	0	gene_id "ENSG00000169..
chr21	HAVANA	start_codon	10862622	10862624	.	+	0	gene_id "ENSG00000169..
chr21	HAVANA	exon	10862751	10863067	.	+	.	gene_id "ENSG00000169..
chr21	HAVANA	CDS	10862751	10863064	.	+	2	gene_id "ENSG00000169..
chr21	HAVANA	stop_codon	10863065	10863067	.	+	0	gene_id "ENSG00000169..
chr21	HAVANA	UTR	10863065	10863067	.	+	.	gene_id "ENSG00000169..

Lesson-16

RNA Seq Read Alignment-GALAXY a tool

GALAXY for Bioinformatics

Doing and redoing experiments

- If you do something once, you usually don’t need a script. Do it hundreds or thousands of times, you will want something to help you.

- Want to share what you did, providing a script is usually a good way.
- Sometimes though, scripts are too complicated, and don't capture all that is needed to do an experiment. For example: the version of a tool you used!

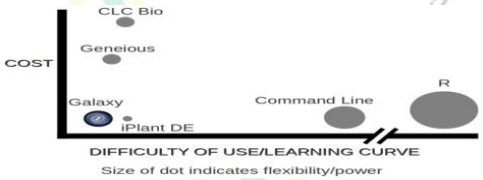
What does "Big Data" really means

- As the cost of sequencing decreases, it's much easier to generate huge amounts of data.
- Rapid technology change means software has to keep up with changing data characteristics.
- Both features encourage centralized approach to data storage and analysis.
- So, where can the data be stored and accessed, with relative ease, from my laptop?

Bioinformatics Platforms

- **Linux command line**
  - cutting edge, flexible, highest learning curve (~months to years)
- **Galaxy**
  - deployed on many public servers, lots of history
  - open source and relatively easy to develop for
- **iPlant Collaborative (link)**
  - younger than Galaxy, but more funding, personnel
  - not open source yet
- **Commercial products (Geneious, CLC, etc.)**
  - black boxes (no public review for bugs)
  - they develop new tools for you … or, they don't

Comparison of Bioinformatics Interface



Some of these are easier to transition between than others.

Lesson-17

Galaxy A Tool Introduction

GALAXY for Bioinformatics

Some requirements for easy computation:

- Open Source

- Solution should be useful to large community
- Well supported (by community and funding agency)
- Flexible
- Expandable
- Scalable
- Cloud-aware
- User friendly?



Open Source

Introduction

Advantages

○ User-friendly interface with drop-down menus

○ Workflows facilitate processing data

○ All Galaxy tools are available for command line ○ Not a "Black Box"

Disadvantages

○ All parameters may not be available for some tools ○ Installation of new software and writing new tools usually requires use of the command line.

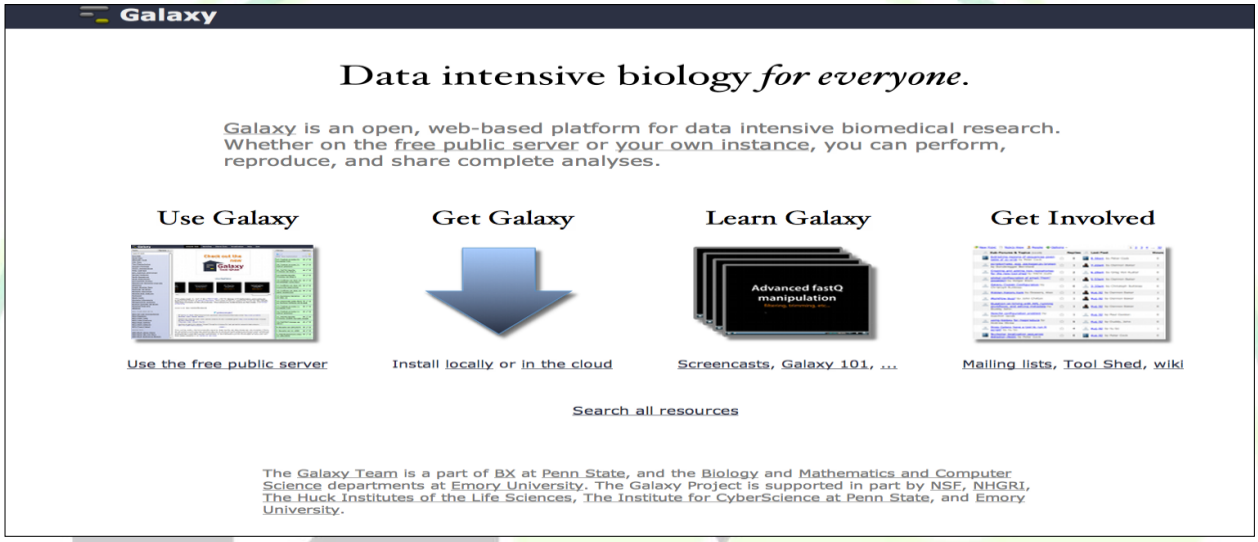
Why Galaxy?

- Galaxy integrates input data sources
- Galaxy allows you to use many tools that you don't need to install and maintain.
- Galaxy allows you to maintain workflows, reuse them, and share them.
- Galaxy has fully entered the "next-gen" space.
- Galaxy works in the cloud.

Why is it useful to a biologist?

- Easy to use!
- Allows data import from popular resources
- Provides access to best practice bioinformatics tools
- Allows you to build analysis pipelines and share them

- Provides multiple ways to visualize your data
- Easy to use!
- Allows data import from popular resources
- Provides access to best practice bioinformatics tools
- Allows you to build analysis pipelines and share them
- Provides multiple ways to visualize your data



http://galaxyproject.org/



Lesson-18

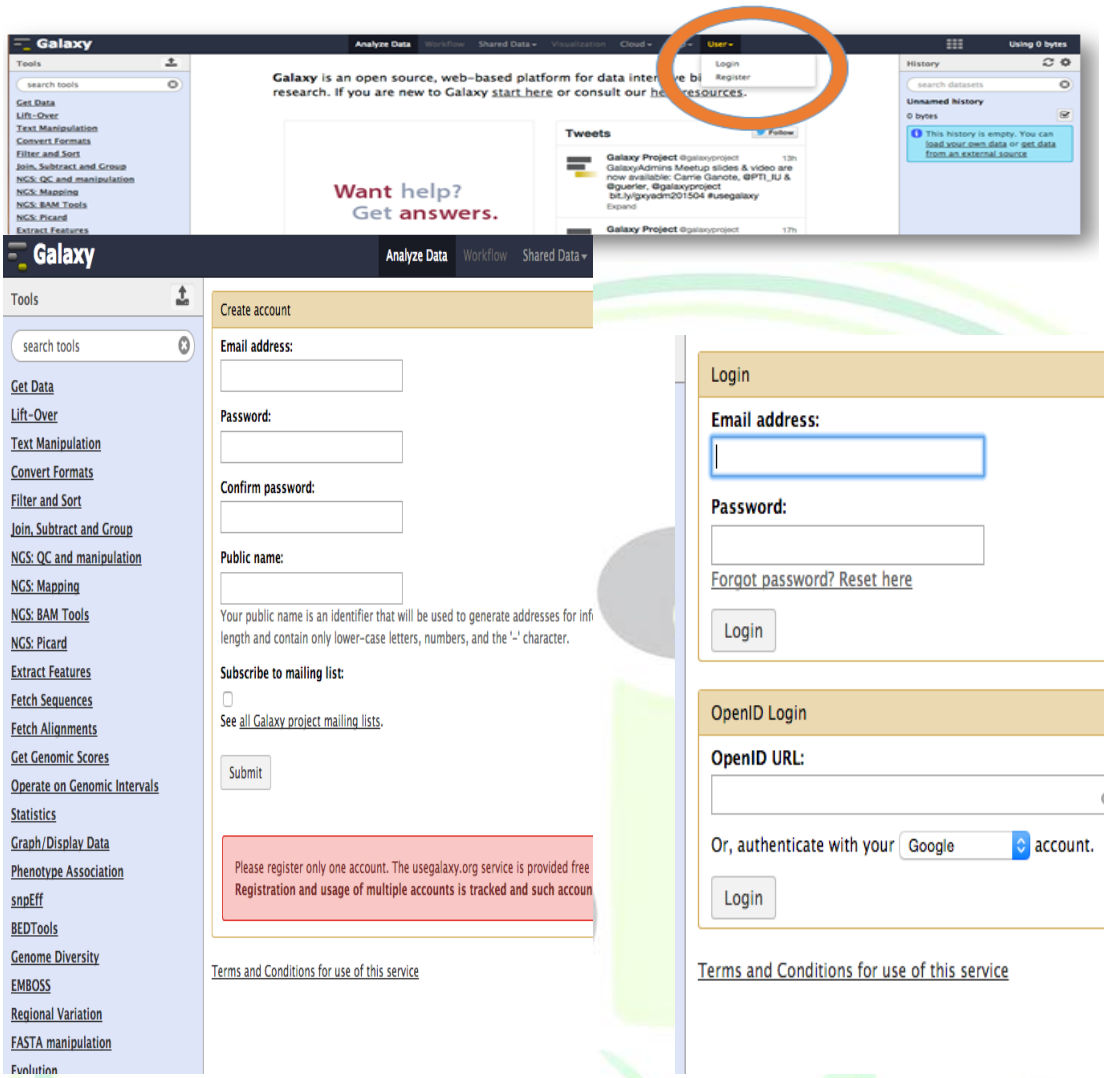
GALAXY a tool Module-I

GALAXY for Bioinformatics

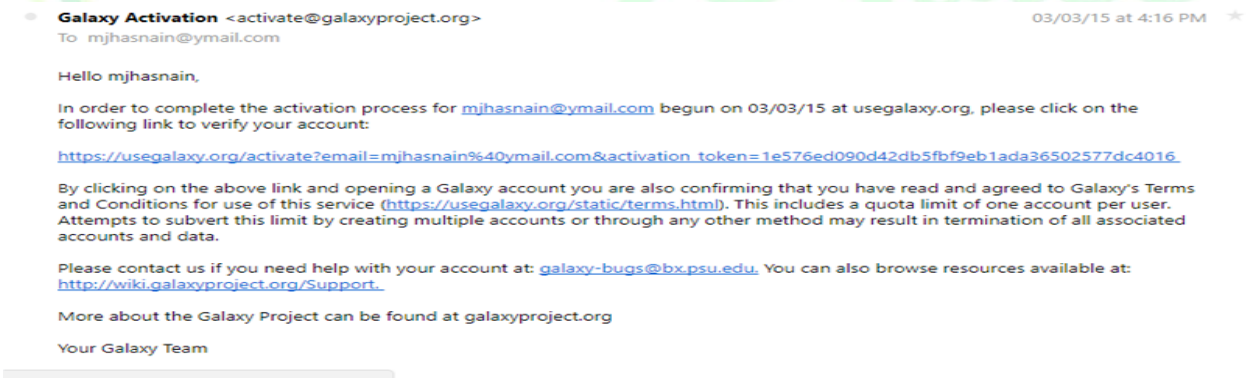
http://galaxyproject.org/

First, you should log in

This will allow you to store your data, share it, and keep your history.



New User **Returning User**



Galaxy Activation <activate@galaxyproject.org> 03/03/15 at 4:16 PM
To mjhasnain@ymail.com

Hello mjhasnain,

In order to complete the activation process for mjhasnain@ymail.com begun on 03/03/15 at usegalaxy.org, please click on the following link to verify your account:

https://usegalaxy.org/activate?email=mjhasnain%40ymail.com&activation_token=1e576ed090d42db5fbf9eb1ada36502577dc4016

By clicking on the above link and opening a Galaxy account you are also confirming that you have read and agreed to Galaxy's Terms and Conditions for use of this service (https://usegalaxy.org/static/terms.html). This includes a quota limit of one account per user. Attempts to subvert this limit by creating multiple accounts or through any other method may result in termination of all associated accounts and data.

Please contact us if you need help with your account at: galaxy-bugs@bx.psu.edu. You can also browse resources available at: http://wiki.galaxyproject.org/Support.

More about the Galaxy Project can be found at galaxyproject.org

Your Galaxy Team

New User



Your account is already active. Nothing has changed.
Go to login page.

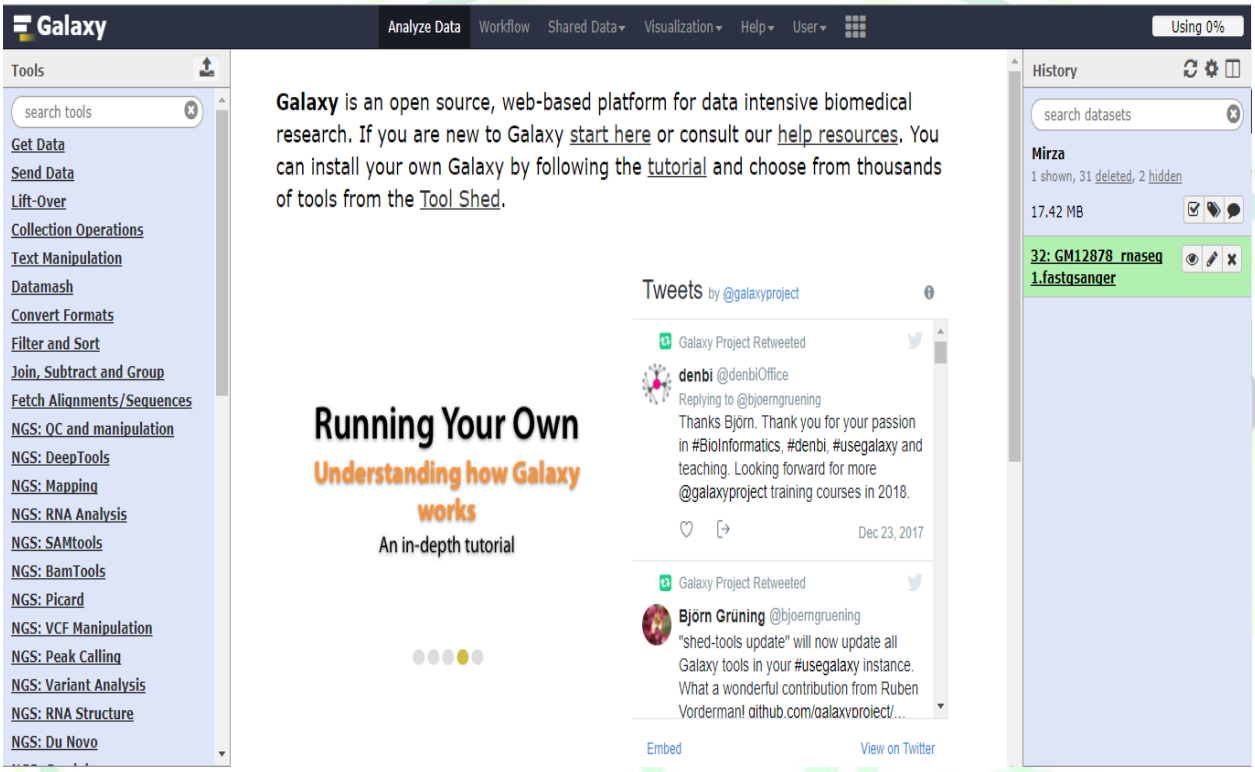
New User

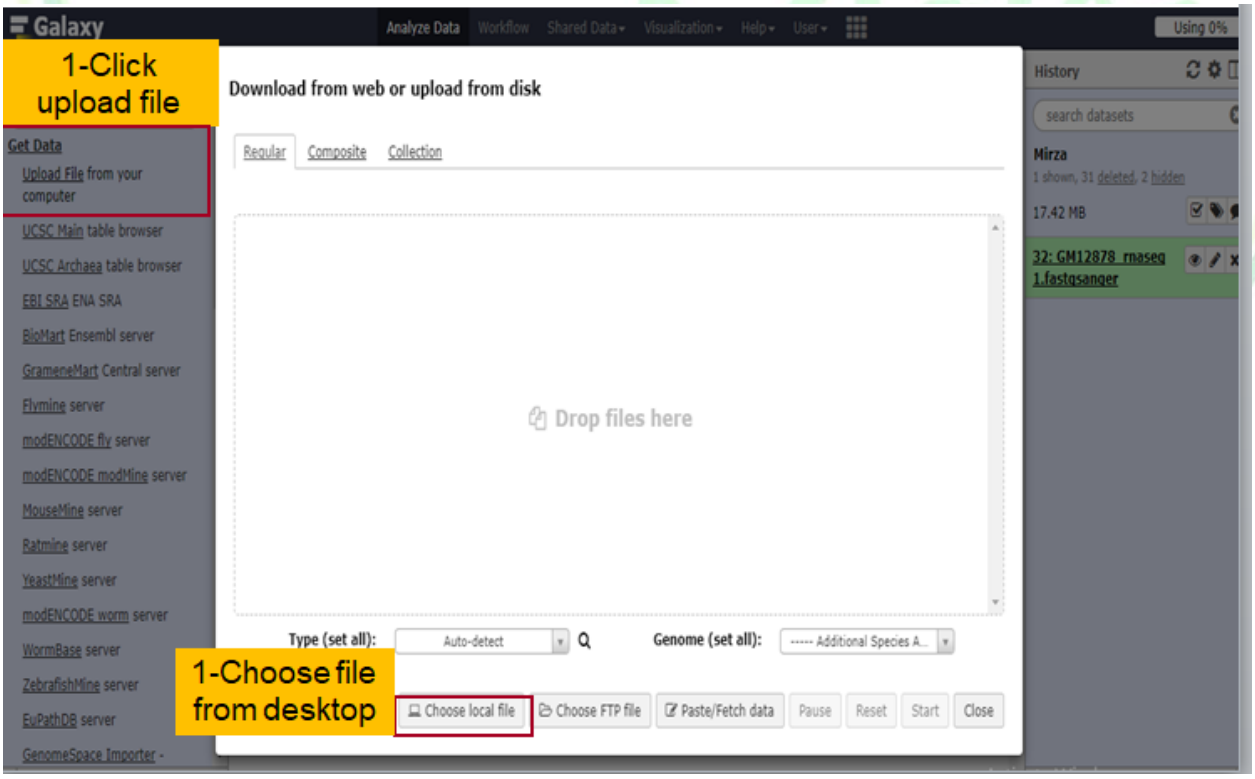
Lesson-19

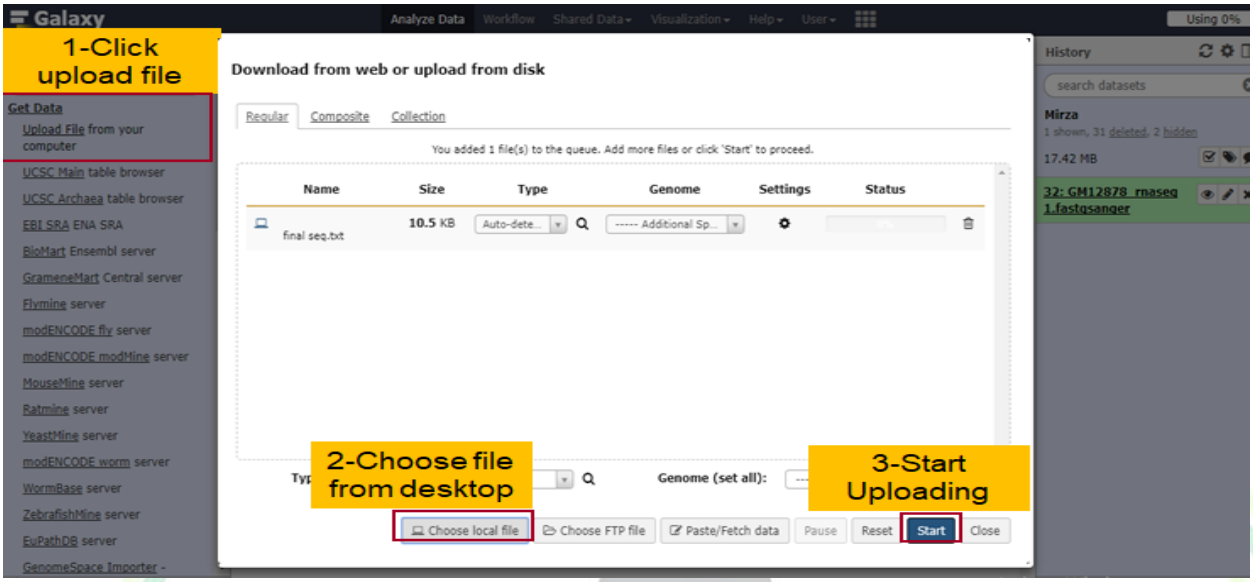
GALAXY a tool Module-II
Input Data

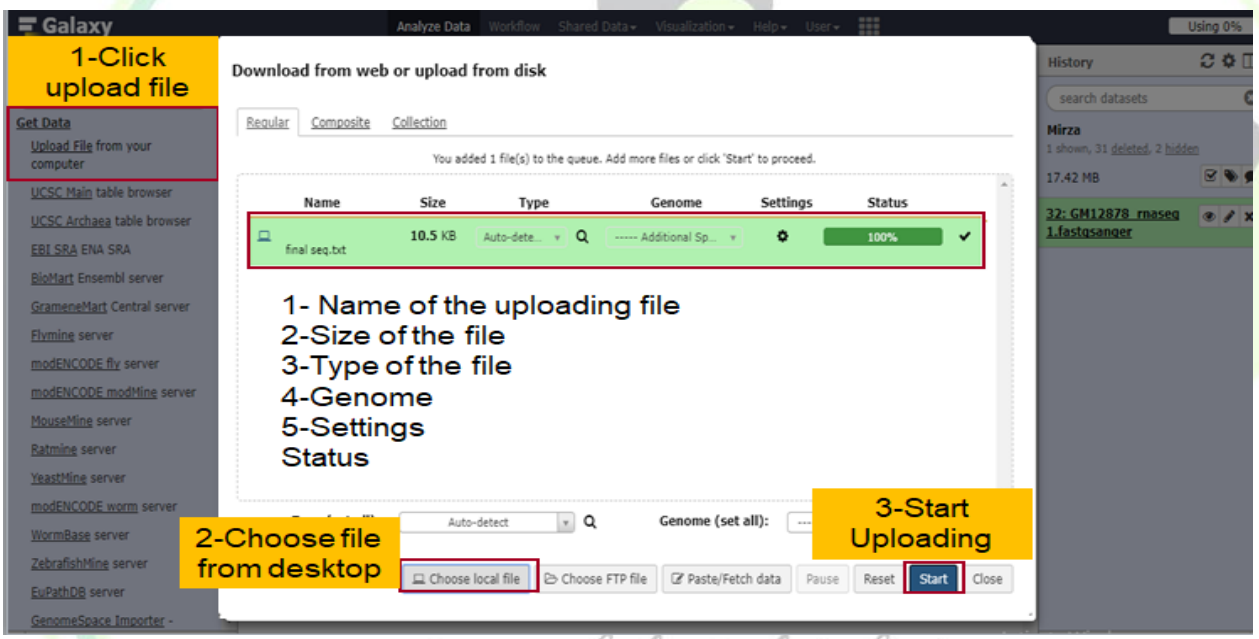
GALAXY for Bioinformatics

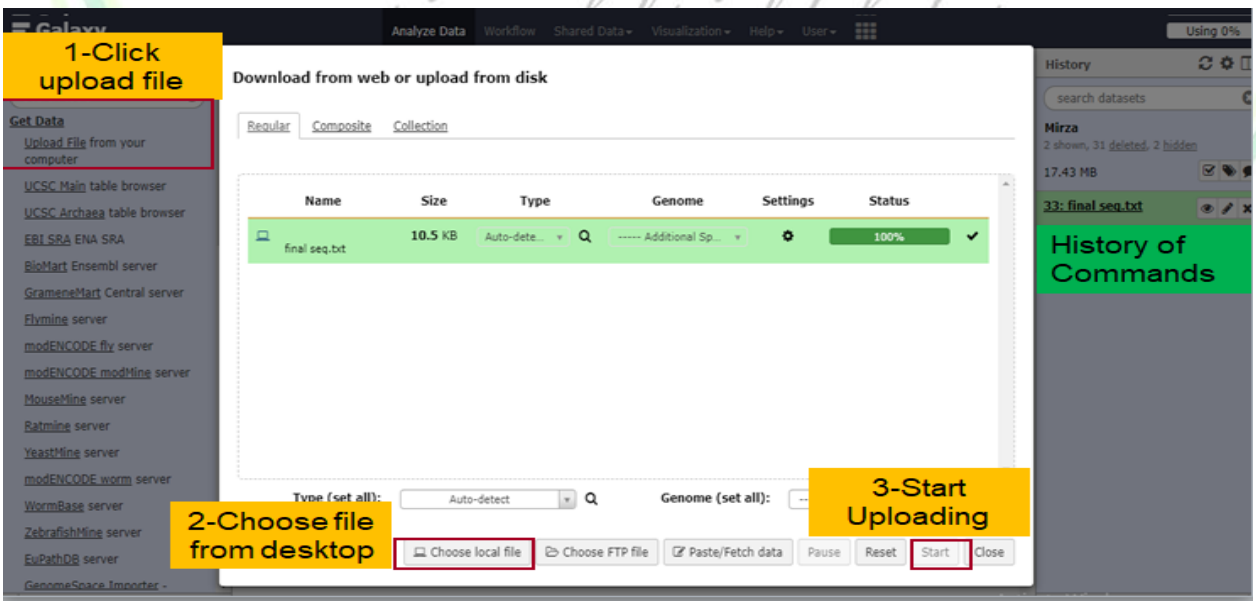
Getting Data

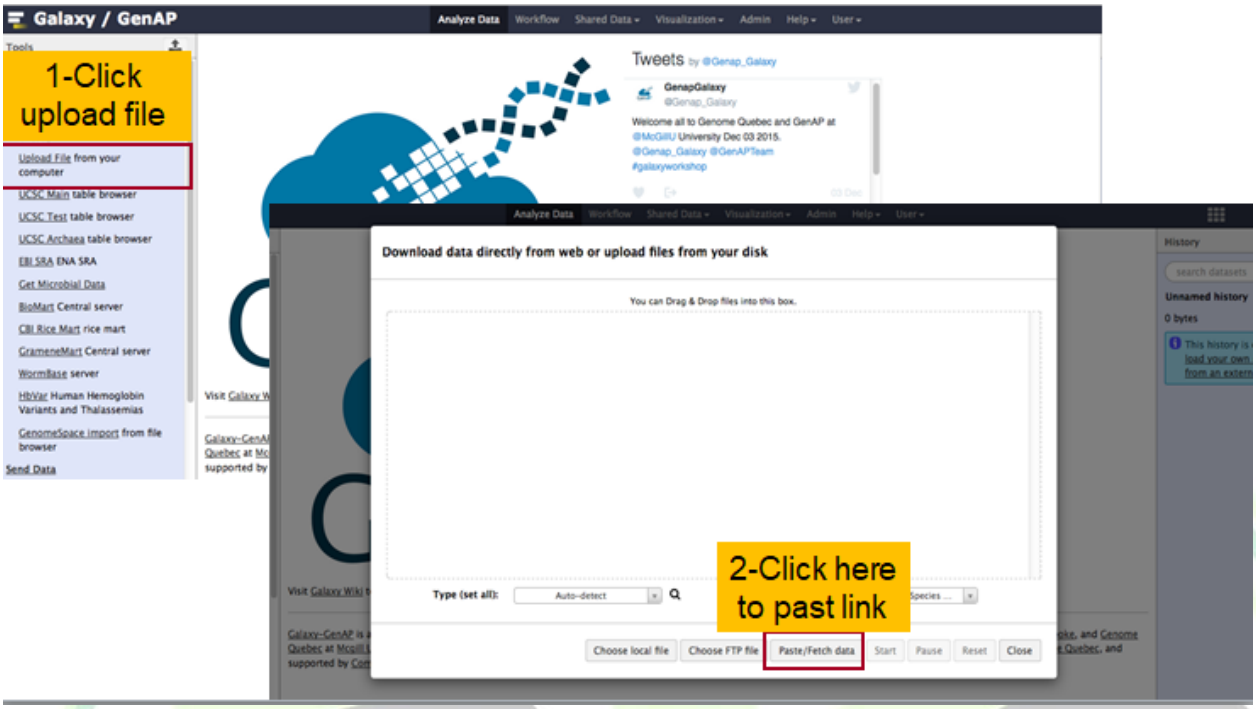




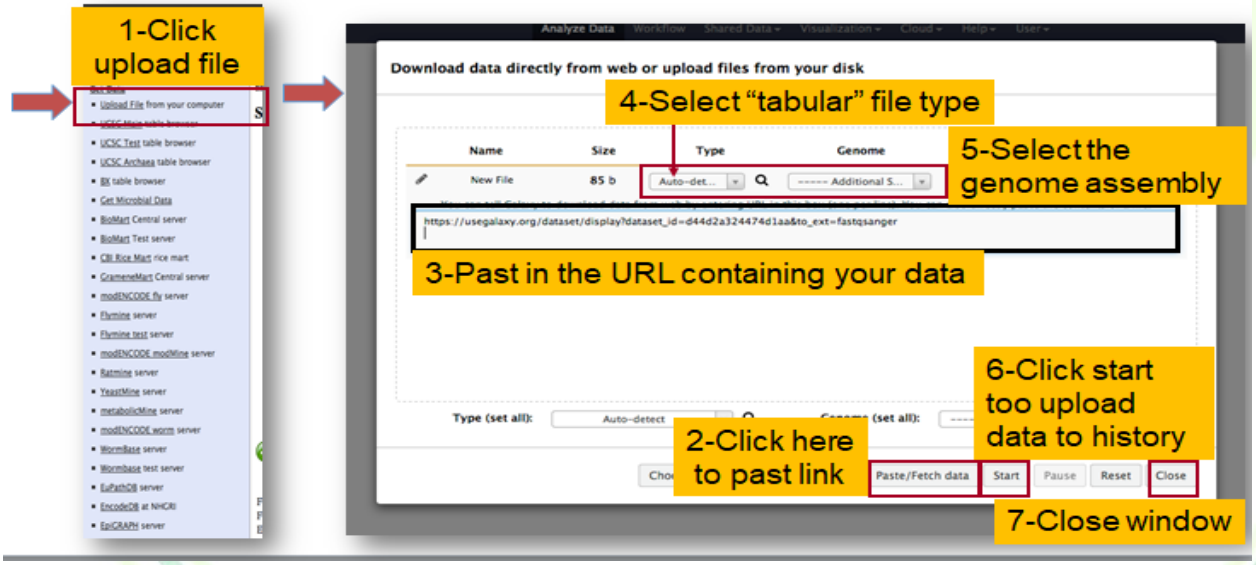

1-Click upload file
Download from web or upload from disk
2-Choose file from desktop
3-Start Uploading
Choose local file
Choose FTP file
Paste/Fetch data
Start


1-Click upload file
Download from web or upload from disk
1- Name of the uploading file
2-Size of the file
3-Type of the file
4-Genome
5-Settings
Status
2-Choose file from desktop
3-Start Uploading
Choose local file
Choose FTP file
Paste/Fetch data
Start


1-Click upload file
Download from web or upload from disk
History of Commands
2-Choose file from desktop
3-Start Uploading
Choose local file
Choose FTP file
Paste/Fetch data
Start



- Most of time, you will get from a file on your computer, or from a URL.



Data uploaded to your history!

The history keeps track of all steps in your analysis



UCSC Genome Browser: source of data for Galaxy

- Browse many Eukaryotic genomes (yeast to human)
- Most annotations are there
- Important evolutionary and variation data representation.
- Very flexible and configurable views
- Graphical and table views (Galaxy uses this)
- Upload your data into custom tracks and share with colleagues

Lesson-20 [not available]

Lesson-21

GALAXY a tool Module-II
Input Data
Student Practice

GALAXY for Bioinformatics

1.Open the Get Data → Upload File tool.

2. Enter the following URL into the text-entry box
http://chagall.med.cornell.edu/galaxy/rnaseq/GM12878_rnaseq1.fastqsanger

3. Change the File-Format pull down menu from 'Auto-detect' to 'Fastqsanger'.

Although Galaxy is usually quite good at recognizing formats, it is always safer, if you know the format of your file, not to leave its recognition to chance. This is especially important with FastQ formats, as they all look similar but will give different results if Galaxy assumes the wrong format.

4. Click Execute.

sign of bright future

Upload data from a File

1.Open the Get Data → Upload File tool.

2. Enter the following URL into the text-entry box (either by typing it or right-click the link on the course webpage, select 'Copy Link' and paste the link into the text-entry box.)
http://chagall.med.cornell.edu/galaxy/rnaseq/GM12878_rnaseq1.fastqsanger

Upload data from a File

3. Change the File-Format pull down menu from 'Auto-detect' to 'Fastqsanger'.

Although Galaxy is usually quite good at recognizing formats, it is always safer, if you know the format of your file, not to leave its recognition to chance. This is especially important with FastQ formats, as they all look similar but will give different results if Galaxy assumes the wrong format.

4. Click Execute.

History

The right hand panel of the Galaxy window is called the History panel. Histories are the way that datasets are stored in an organized fashion within Galaxy.

To change the name of your history, click on the Unnamed history text. This will highlight the history name window and allow you to type the new name of the history.

Histories can also be associated with tags and annotations which can help to identify the purpose of a history.

As the number of histories in your Galaxy account grows, these tags/annotations become more and more important to help you keep track of what the function of each history is.

There are many icons associated with each dataset.

1. The eye icon will show the contents of that dataset in the main Galaxy panel.

2. The pencil icon will allow you to edit any attributes associated with the dataset.

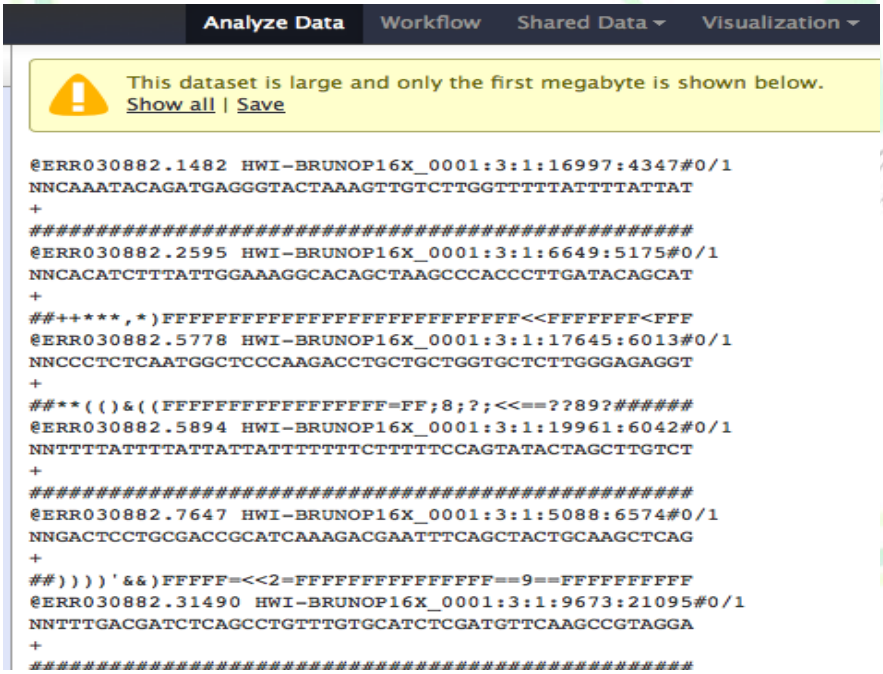
3. The X icon deletes the dataset from the history. Note that a dataset is not permanently deleted unless you choose to make it so.

4. The disk icon allows you to download the dataset to your computer.

5. The "i" icon gives you details about how you obtained that dataset (i.e., if you downloaded it from somewhere, or if it is the result of running a job within the Galaxy framework.)

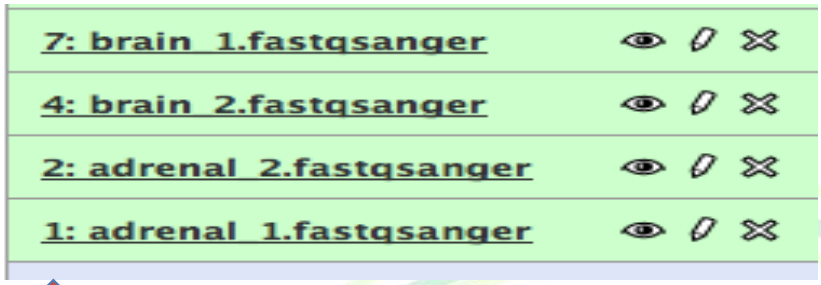
6. The scroll icon allows you to associate tags with the dataset. Similarly to the tags for the history itself, these tags can help to quickly identify particular datasets.

7. The post-it-note icon allows you associate annotations with a dataset. These annotations are also accessible via the Edit Attributes pencil icon.



"poke the eye"

You can inspect and edit the attributes of your data sets



"Numbers may vary with usage"

To make a history accessible to other users:

1. Click the gear icon at the top of the history panel that you want to share.

2. Choose the Share or Publish option.

a. To make the history accessible via a specific URL, click the Make History Accessible via Link button.

b. To publish the history on Galaxy's Published Histories section, click the Make History Accessible and Publish option.

To see the other histories that have been made accessible at this site, click the Shared Data tab at the top of the page and select Published Histories.

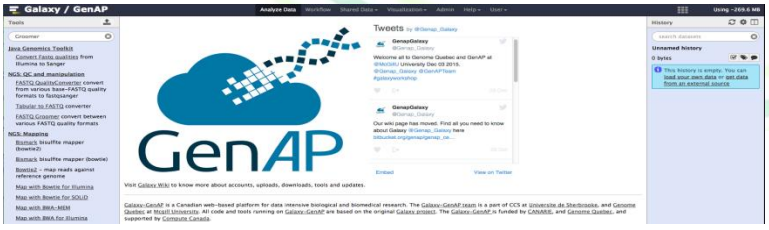
To go back to your current Galaxy history, click the Analyze Data tab at the top of the page. To see a list of all of your current histories, click the gear icon at the top of the history panel and select Saved Histories.

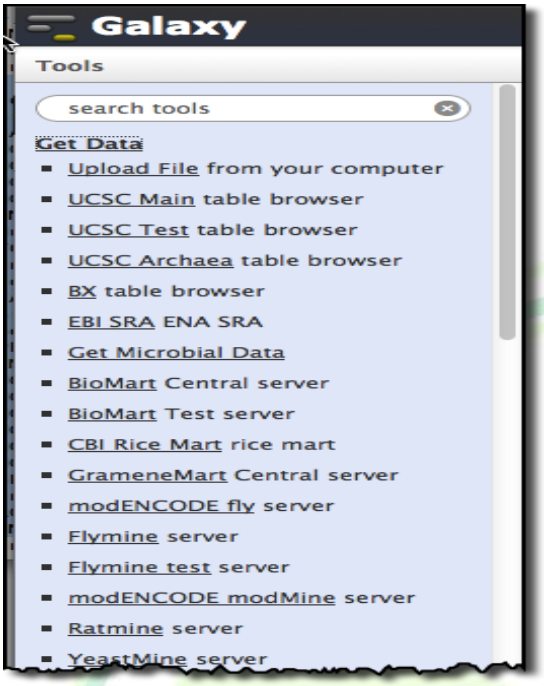
Lesson-22

GALAXY a tool Module-III
Format Conversion

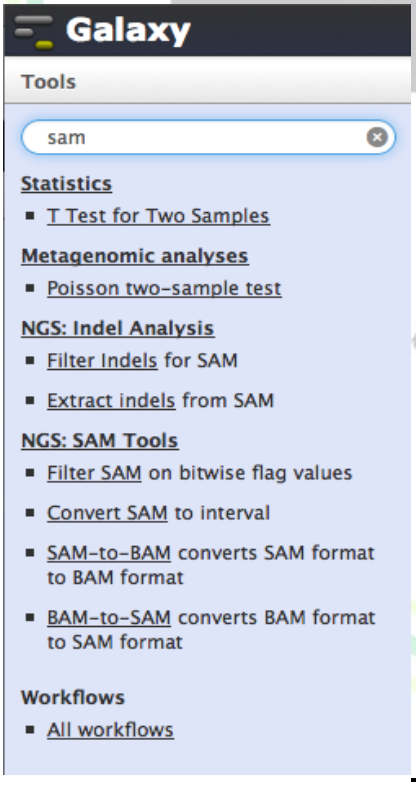
Tools

- Galaxy has hundreds of tools.
- You can search them by keyword or directly from the menu.

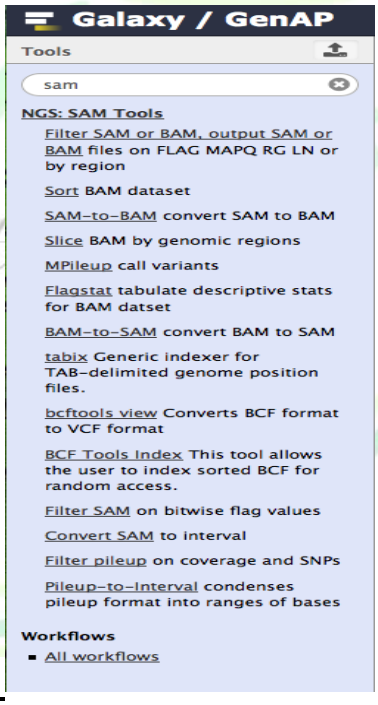




- … and each item, when you click on it expands to lots more choices!
- What I find most useful when I know the name of the tool I'm looking for is to simply using the search tool.
- E.g. look for "sam"

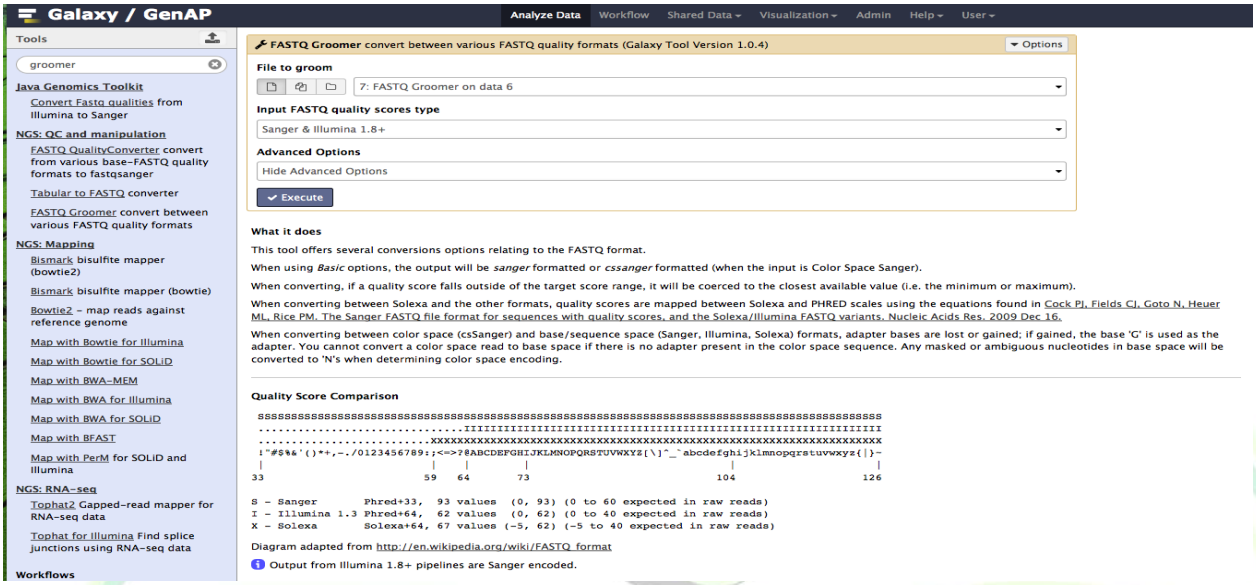




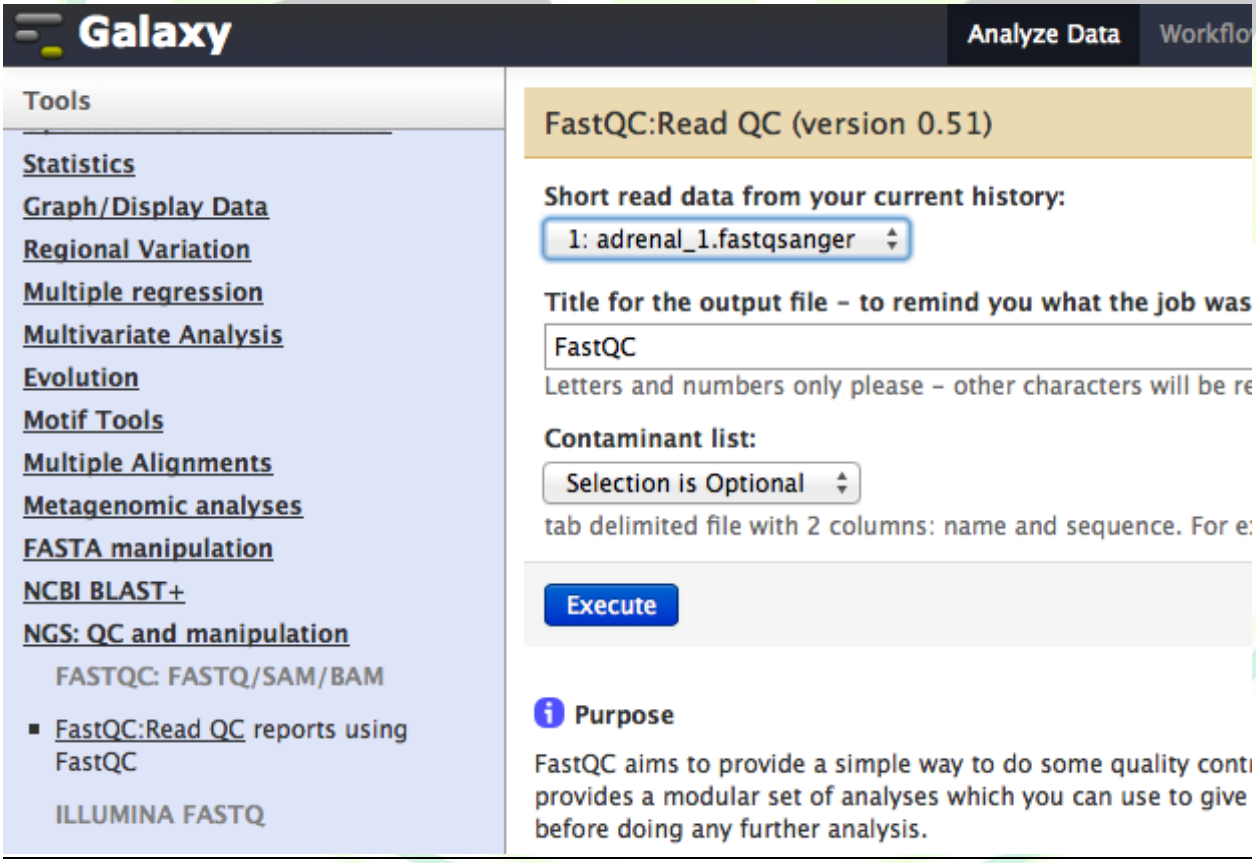


Galaxy Cloud **usegalaxy.org** **GenAP**

Select the tool and choose its parameters

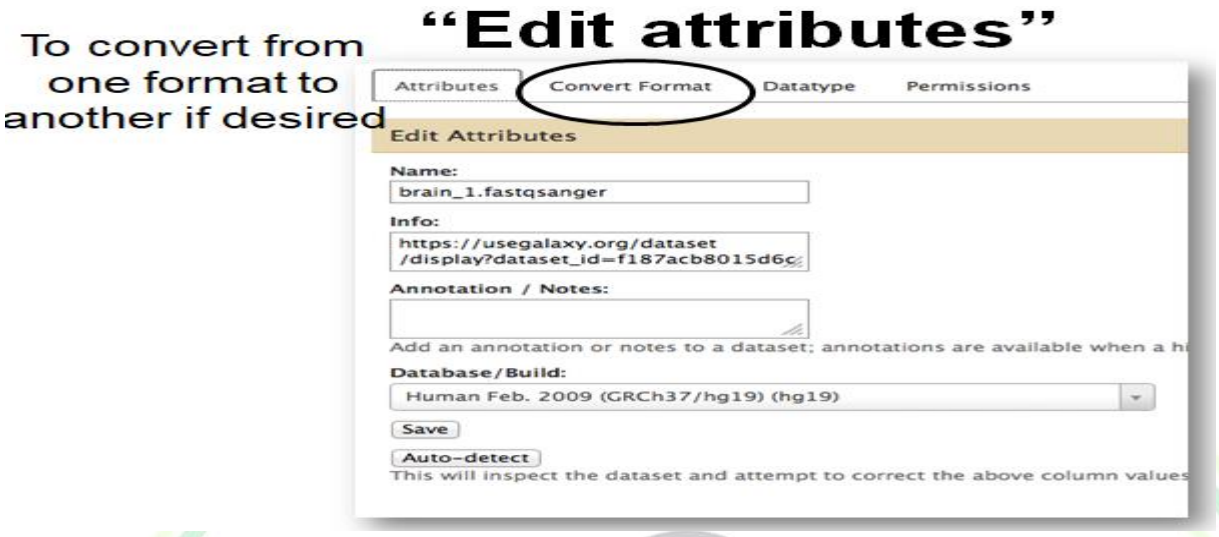


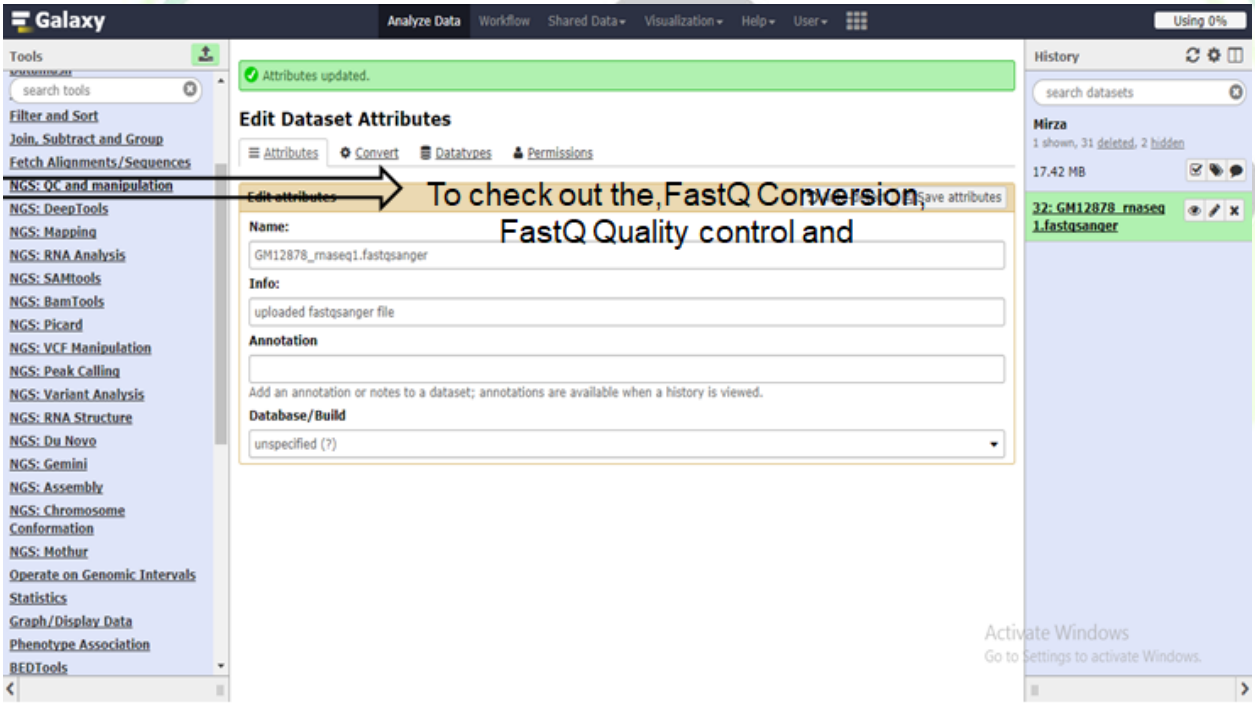
Example: quality control



Changing dataset formats, editing attributes

If Galaxy does not detect the format of your dataset correctly, you can always change the format of your dataset manually by editing the attributes of the dataset (the pen icon to the upper right of the dataset link in your history.) Specifically, a “Change data type” section, can be used to change format.





sign of bright future

Lesson-23 a

FastQ Quality Control

FastQ Quality Control

Summarizing and visualizing statistics about FastQ reads

NGS: QC and manipulation → FASTQ Summary Statistics

Graph/Display Data → Boxplot tool

Exercise

Run a quality control on the GM12878 dataset that we previously downloaded.

1. Open the NGS: QC and manipulation → FASTQ Summary Statistics tool. Make sure the GM12878 dataset is selected.

2. Click Execute.
3. Open the Graph/Display Data → Boxplot tool. Make sure the input dataset is the output from the Summary Statistics tool in the last step.
4. Change the X-axis label to “read position” and the Y-axis label to “quality score". Click Execute.

Lesson-23 b

FastQ Quality Control

FastQC quality control NGS: QC and manipulation → FastQC

- Basic Statistics
- Per Base Sequence Quality
- Per Sequence Quality Score
- Per Base Sequence Content
- Per Base GC Content
- Per Sequence GC Content
- Per Base N Content
- Duplicate Sequence
- Over presented Sequences
- Over Presented Kmers

Exercise Run a quality control on the GM12878 dataset using the FastQC tool.

1. Open the NGS: QC and manipulation → FastQC tool.
2. Select the GM12878 dataset.
3. Click Execute

Lesson-24

FastQ Manipulation

FastQ Quality Control

Trimming the ends off reads

NGS: QC and manipulation → FastQ Trimmer [under Generic FASTQ manipulation]

Removing individual sequences

NGS: QC and manipulation → Filter FastQ [under Generic FASTQ manipulation]

More complex manipulations

NGS: QC and manipulation → Manipulate FASTQ [under Generic FASTQ manipulation]

Exercises

The GM12878 dataset seems to be of poor quality.

1. Trim the reads in the GM12878 dataset using the Generic FASTQ manipulation → FastQ Trimmer tool. Determine from the boxplot and FastQC figures where the quality of the reads begins to drop off sharply. Calculate how many bases have to be trimmed from the end and use that number as the Offset from 3' end.
2. Using the Generic FASTQ manipulation → Filter FastQ tool, filter out all sequences with any bases that have a quality less than 20. How many sequences do you have left in your dataset?
3. Run another QC (summary statistics and boxplot) on both of the new datasets from steps 1 and 2.
4. Modify the trimmed dataset from step 1 so that all Thymines are replaced by Uracils.

Lesson-25 a

Mapping Illumina data with BWA

Mapping Reads to Transcriptome with TopHat

Next Generation Sequencing Mappers

- There are two mappers available in Galaxy: Bowtie and BWA.
- The crucial difference between these mappers is that BWA performs gapped alignments, whereas Bowtie does not.

Alignment reads with BWA[Burrows-Wheeler Alignment]

Two small datasets of under 100,000 single 75-bp reads from the ENCODE GM12878 cell line and ENCODE h1-hESC cell line, and compare the transcriptomes between the two cell lines.

1. Open the Get Data → Upload File tool.
2. Get the two RNA-Seq Analysis datasets by entering the following URLs in the text entry box.
   http://chagall.med.cornell.edu/galaxy/rnaseq/GM12878_rnaseq1.fastqsanger

   http://chagall.med.cornell.edu/galaxy/rnaseq/h1hESC_rnaseq2.fastqsanger
3. Change the File-Format pull-down menu from 'Auto-detect' to 'Fastqsanger
4. Do a quality control check on the data before beginning any analysis.

   a. Run the NGS: QC and manipulation → FASTQ Summary Statistics tool.

   b. Use the Graph/Display Data → Boxplot tool to visualize the quality score data summary.

In the case of the GM12878 dataset, there is a dramatic decrease in quality around base 60, so we want to trim off the last 16 bases from each read.

1. Open the NGS: QC and manipulation → FASTQ Trimmer tool.
2. Use 16 as offset from the 3' end.

The next step is to align the now filtered reads to the genome using TopHat.

1. Open the NGS: RNA Analysis → Tophat for Illumina tool.
2. Select the hg19 Full genome from the organism pull down menu.
3. Change the settings from defaults to display the full parameter list. In general, it is usually okay to keep most of the default parameters, but it is usually good practice to go down the list and make sure that they all look appropriate for the current analysis. Note that some of the options are only applicable to paired end reads (e.g., Library type and Use closure search).
4. Reduce the maximum intron length (both for initial (whole read) searches and split segment searches) down to 100000.
5. Turn off indel search and coverage search, to speed up the analysis.
6. Do this for both datasets.

TopHat Parameters

Each TopHat run will result in four files: a list of accepted mapped reads in BAM format , and three BED files: one for raw splice junctions (which can then be fed into a subsequent TopHat analysis), one each for detected insertions and deletions (although these will be empty if the indel search was disabled.)

The splice junctions file is formatted as a BED file, using all optional columns. This will enable the visualization of splice junctions as a pair of boxes joined by a thin line. The column headings are:

1. Chromosome name
2. Start position of junction representation
3. End position of junction representation
4. Name of the junction
5. Junction score (how many reads support the junction)
6. Strand
7. Position at which to start drawing the first box
8. Position at which to end drawing the last box
9. Color with which to paint the junction representation
10. Number of thick boxes (almost always 2)
11. Size of each of the thick boxes
12. Distance of the start position of each box from the start position of the junction representation (first number always 0)

The accepted hits file is a standard BAM file (with the standard SAM information). To verify the contents of this file, we can convert it to a human-readable SAM formatted file with the

NGS: SAM Tools → BAM-to-SAM tool.

Lesson-25 b

Mapping Illumina data with BWA [not available]

Lesson-26

TopHat Parameters

TopHat Parameters

Mean inner distance between mate pairs

If dealing with paired end reads, TopHat needs to be told the expected distance between those paired reads.

Library type

Determines to which strand TopHat will attempt to align reads.

Anchor length

This is the minimum number of bases from aligned reads that have to be present on either side of a junction for it to be recognized by TopHat as a potential junction Default: 8.

Maximum number of mismatches in anchor region

This defines the maximum number of mismatches allowed in the anchor region defined by the Anchor Length option. Default: 0.

Minimum intron length

This is the minimum amount of distance that can separate two exons

Maximum intron length

This is the maximum amount of distance that can separate two exons

Allow indel search

Checking this option will allow TopHat to include insertions and deletions in your reads relative to the genome sequence

Minimum isoform fraction

For each junction, the average depth of read coverage is computed for the left and right flanking regions of the junction separately

Maximum number alignments

Discards from further analysis reads that map to an excessive number of different regions on the genome

Minimum intron length in split-segment search

The minimum intron length that may be found during split-segment search. Default: 50.

Maximum intron length in split-segment search.

The maximum intron length that may be found during split-segment search. Default: 500000.

Number mismatches allowed in initial read mapping

The maximum number of mismatches that may appear in the initial (unsegmented) alignment. Default: 2.

Number of mismatches allowed in each segment alignment

The maximum number of mismatches that may appear in each segment of a segmented read. Default: 2.

Minimum length of read segments

The length of the segments that the reads are split into for the split-segment search.

Use own junctions

This allows you to give TopHat a set of junctions that it can add into its database of potential junctions.

Use gene annotation model

TopHat uses the supplied exon records to build a set of known splice junctions for each gene and will add those junctions to its potential junction database

Use raw junctions

This allows you to supply TopHat with a list of raw junctions, usually from a previous TopHat run. Records are defined as [chrom] [left][right][+/-]

Only look for supplied junctions

Forces TopHat to not add any more junctions from initial mapping results to its database and only use the junctions that you have supplied (either as gene annotations or as raw junctions.)

Use closure search

Enables the mate pair closure-based search for junctions.

Use coverage search

Enables the coverage based search for junctions.

Microexon search

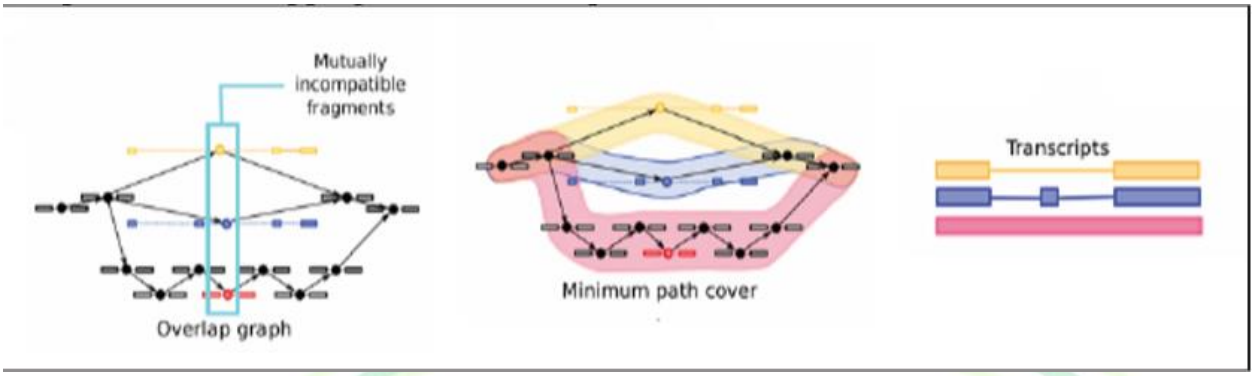
pipeline will attempt to map reads to microexons (very short exons, usually about 25 bases or less) by cutting up reads into smaller segments and attempting to align them to the genome. Works only for reads 50bp or longer.

Lesson-27

Assembly via CuffLinks in GALAXY

Assembly via CuffLinks in GALAXY

CuffLinks uses a directed acyclic graph algorithm to identify the minimum set of independent transcripts that can explain the reads observed in an RNA-Seq experiment



input to CuffLinks must be a SAM/BAM file sorted by reference position.

If you are starting with a BAM file, convert it to a SAM file first,

and then back to BAM format after sorting

1. Open the Filter and Sort → Select tool.
2. Use the pattern ^@ as the criterion for selecting lines. This will select all the SAM header lines.
3. Click Execute.
4. Open the Filter and Sort → Select tool again.
5. Use the same pattern as before, but this time change the pull-down menu to say NOT matching. This ignores all the header lines and selects all the alignment lines that now need to be sorted by reference position.
6. Open the Filter and Sort → Sort tool.
7. Sort the alignment lines file on column 3 (the reference chromosome number), alphabetically in ascending order.
8. Add a second column selection and sort on column 4 (the chromosome position), numerically, in ascending order.
9. Open the Text Manipulation → Concatenate datasets tool.
10. Ensure the first dataset selected is the SAM headers dataset from step 2.
11. Click the Add a new dataset button.
12. Add the sorted alignment lines dataset from step 8

Once the reads have been mapped, we want to assemble the reads into complete transcripts which can then be analyzed for differential splicing events, or differential expression.

This is done using CuffLinks.

1. Run NGS: RNA Analysis → Cufflinks on the two accepted_hits datasets produced by our earlier TopHat analysis (one for each of our two original FastQ input datasets).

2. Change the max intron length to 100000.
3. Click Execute.

Lesson-28

Variant Calling Gene/Genome variants

Variant Calling in GALAXY

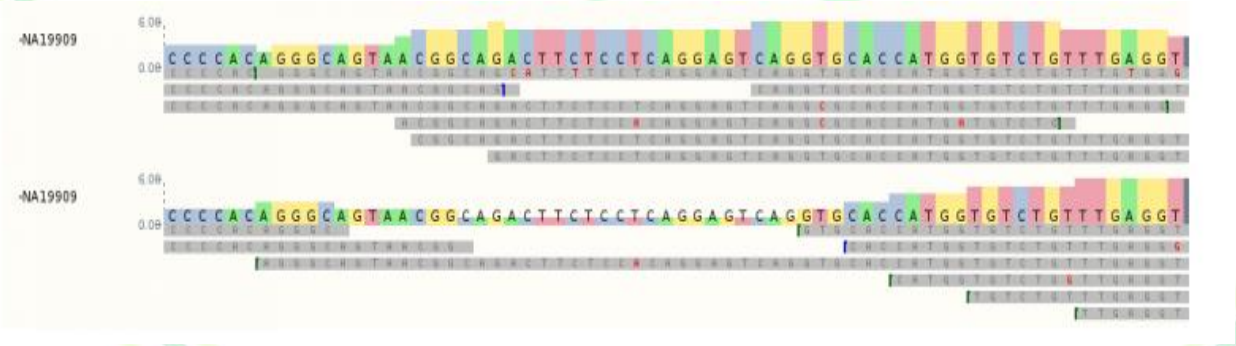
Gene variants:

Genetic variant may refer to:

- A single-nucleotide polymorphism (SNP), in case it is a common genetic variant
- A mutation, in a case where it is a rare genetic variant
- A copy-number variation

Variant calling:

- Variant calling is the process by which we identify variants from sequence data.
- For whole genome or exome variant calling we follow a three-step process:
- Carry out whole genome or whole exome sequencing to create FASTQ files.
- Align the sequences to a reference genome, creating BAM or CRAM files.
- Identify where the aligned reads differ from the reference genome and write to a VCF file.



A CRAM file aligned to a reference genomic region as visualized in Ensemble. Differences are highlighted in red in the reads, and will be called as variants.

Somatic Versus germline variant calling

A Variant calling is a complex field that was significantly propelled by advances in DNA sequencing and efforts of large scientific consortia such as the 1000 Genomes.

- **Variant calling** - identification of positions where the sequenced sample is different from the reference sequence (or reference genome graph);
- **Genotype calling** - identifying individual's genotype at variable sites.

Steps in discovery of genetic variants:

- A typical workflow for variation discovery involves the following steps (e.g., see Nielsen et al. 2011):
- Mapping reads against the reference genome;
- Thresholding BAM datasets by, for example, retaining paired, properly mapped reads;
- Performing quality score recalibration;
- Performing realignment
- Performing variant calling/genotype assignment;
- Performing filtering and genotype quality score recalibration;
- Annotating variants and performing downstream analyses.

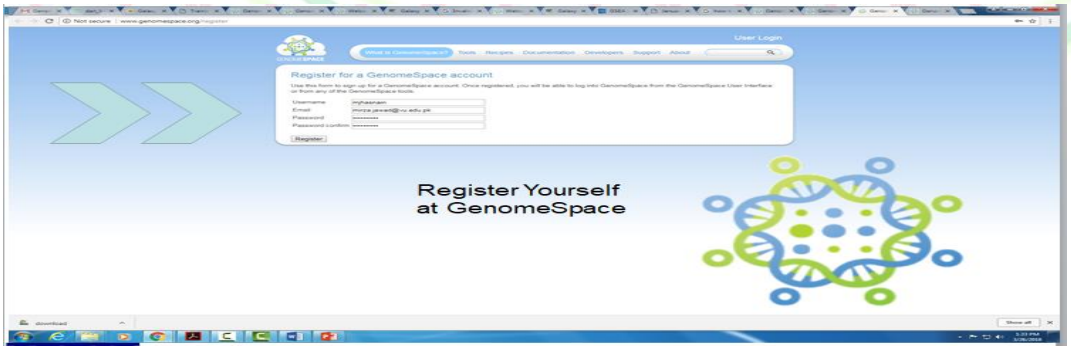
Example:

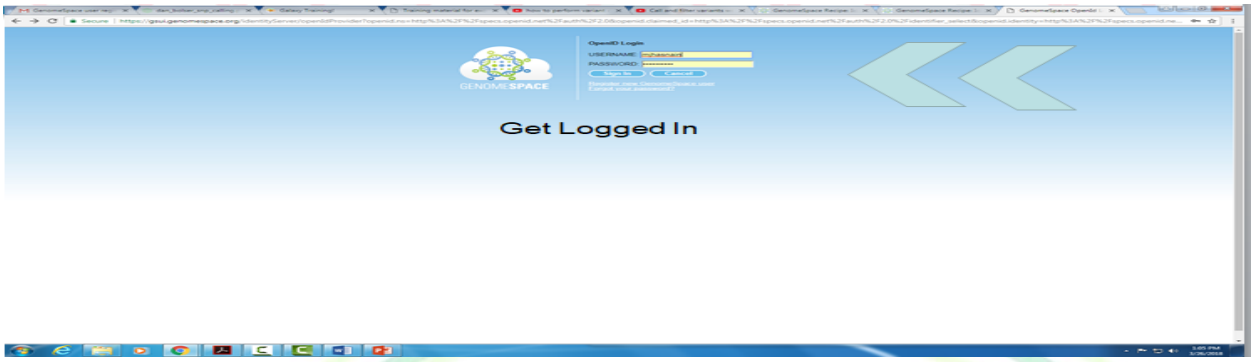
- Consider a set of sequencing reads derived from a diploid individual:
- REFERENCE: atcatgacggcaGtagcatat-
- READ1: atcatgacggcaGtagcatat
- READ2: tgacggcaGtagcatat
- READ3: atcatgacggcaAtagca
- READ4: cggcaGtagcatat
- READ5: atcatgacggcaGtagc
- A true variant;
- A library preparation artifact (e.g., a PCR error);
- A base call error;
- A misalignment (though unlikely in the above example).

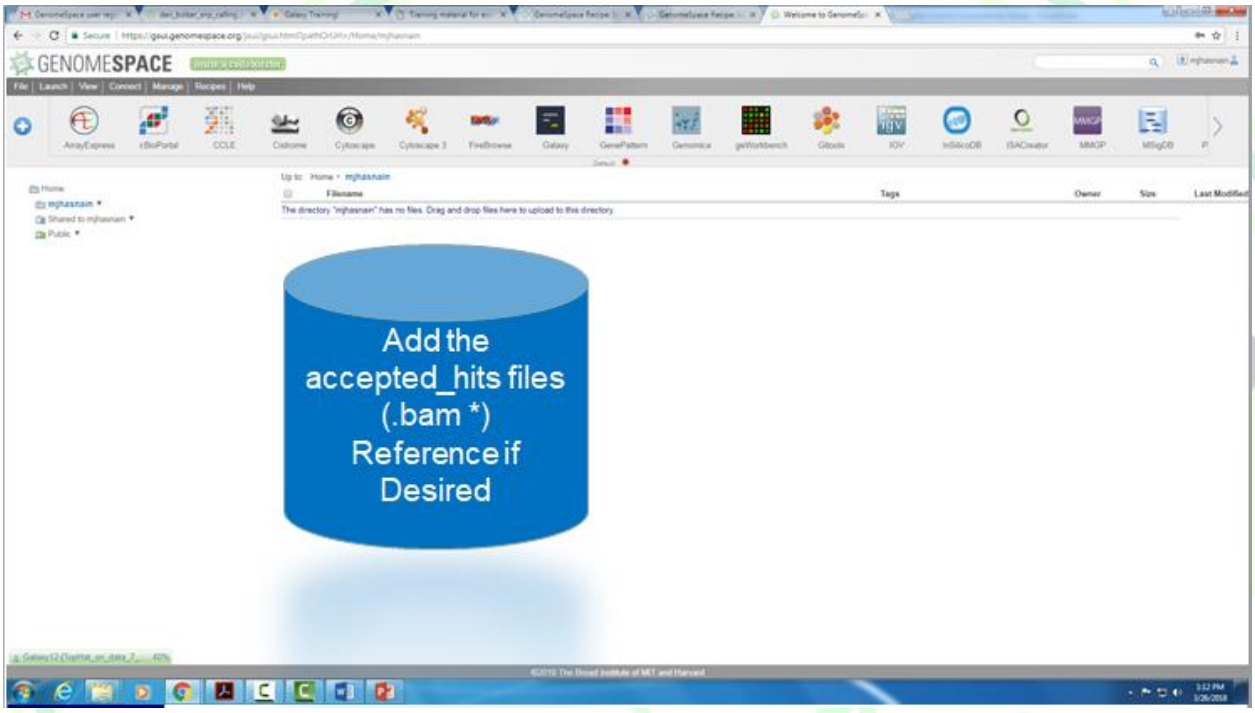
sign of bright future

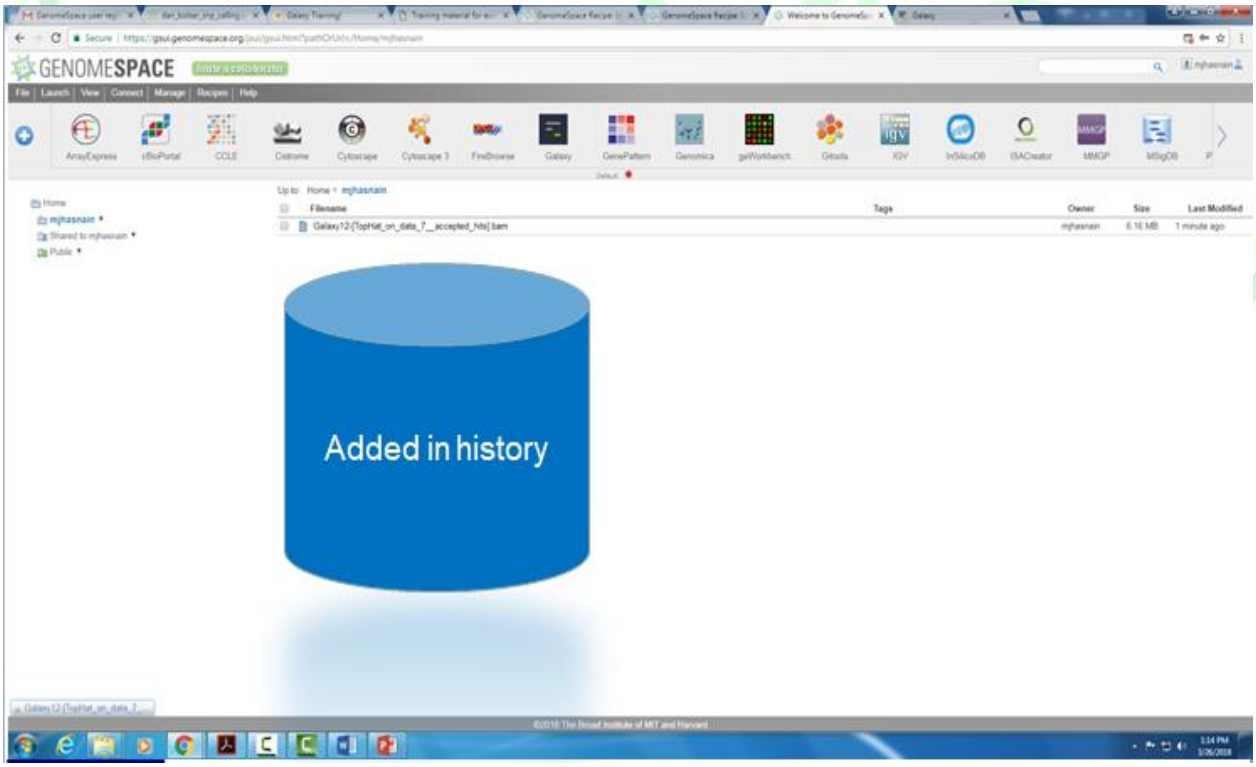
Lesson-29

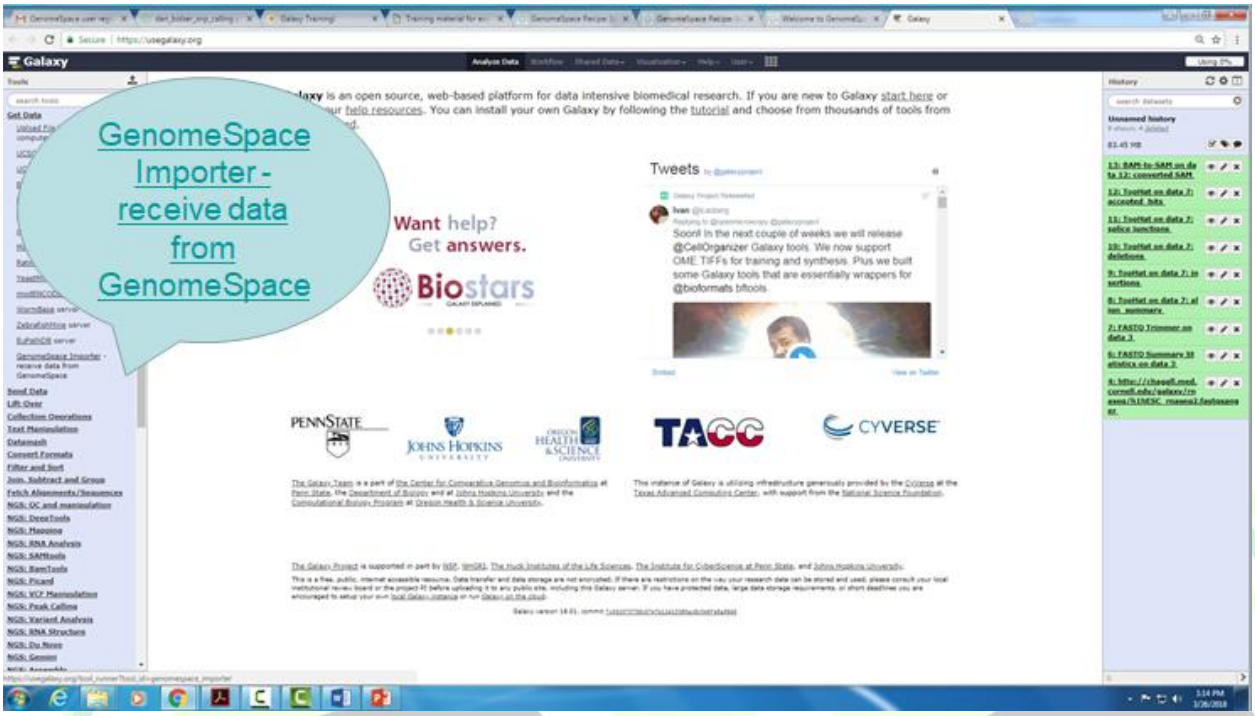
Variant Calling in GALAXY

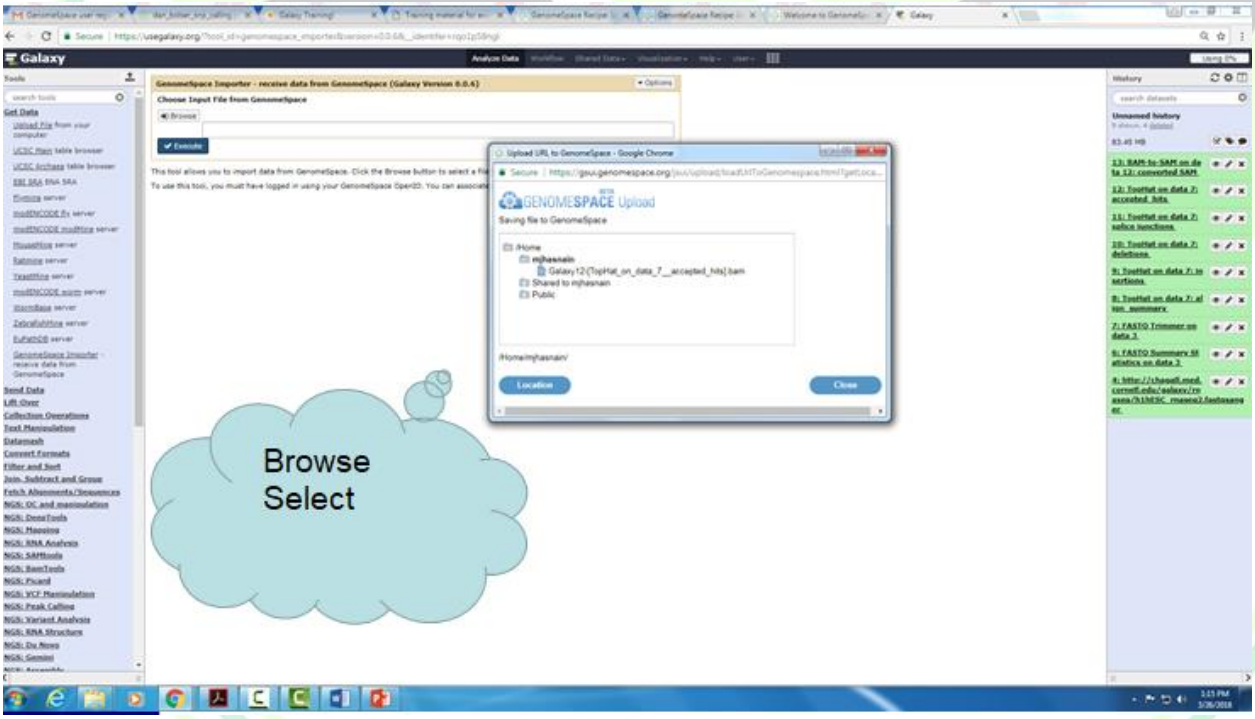


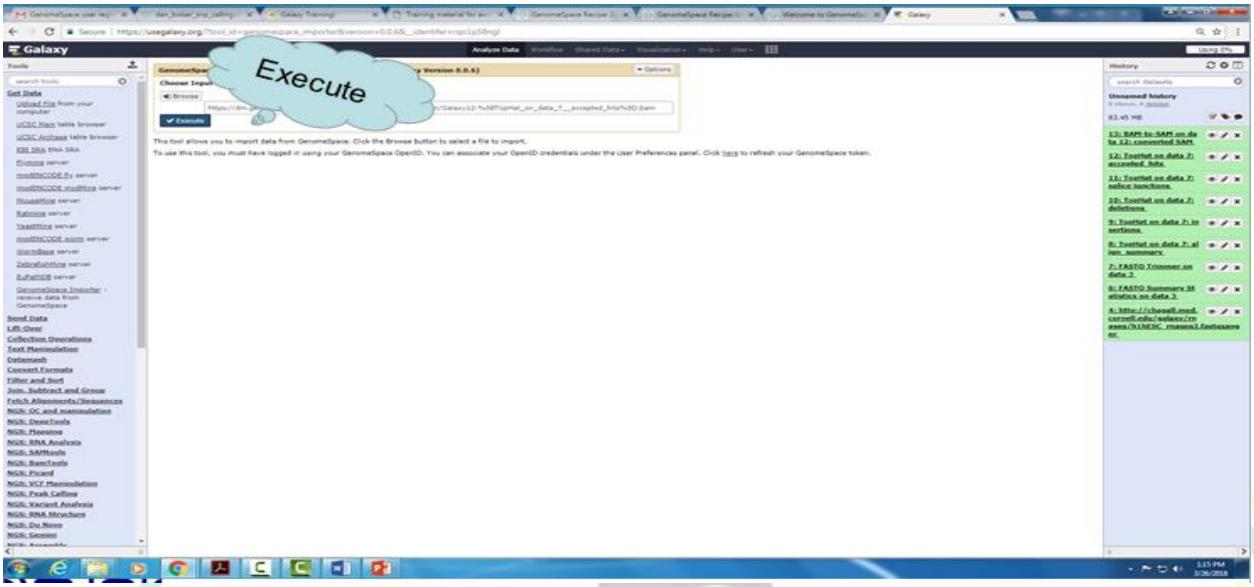

Get Logged In

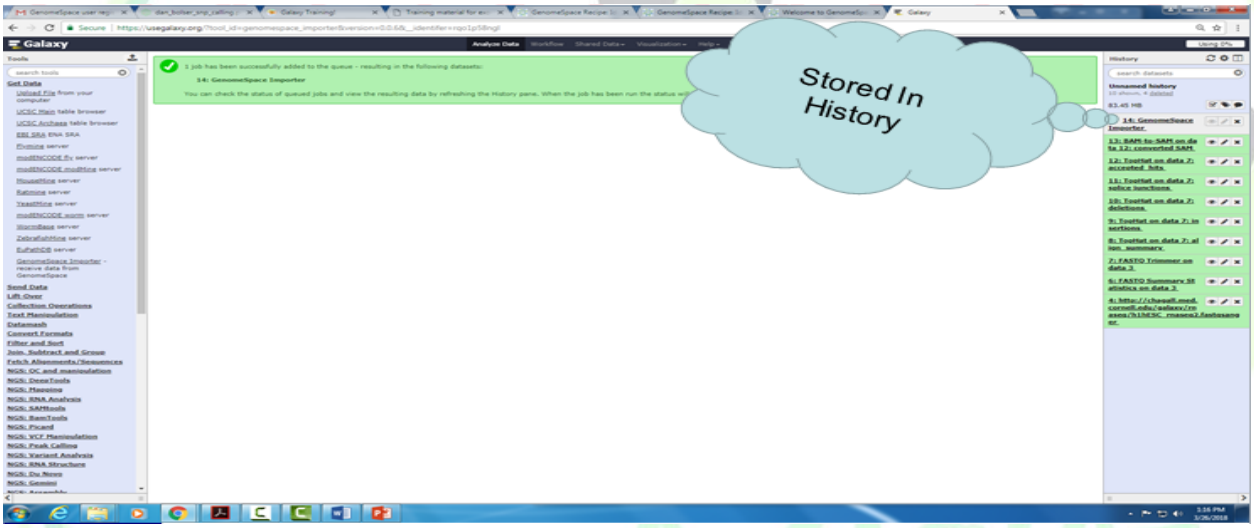

GENOMESPACE
Add the accepted_hits files (.bam *) Reference if Desired

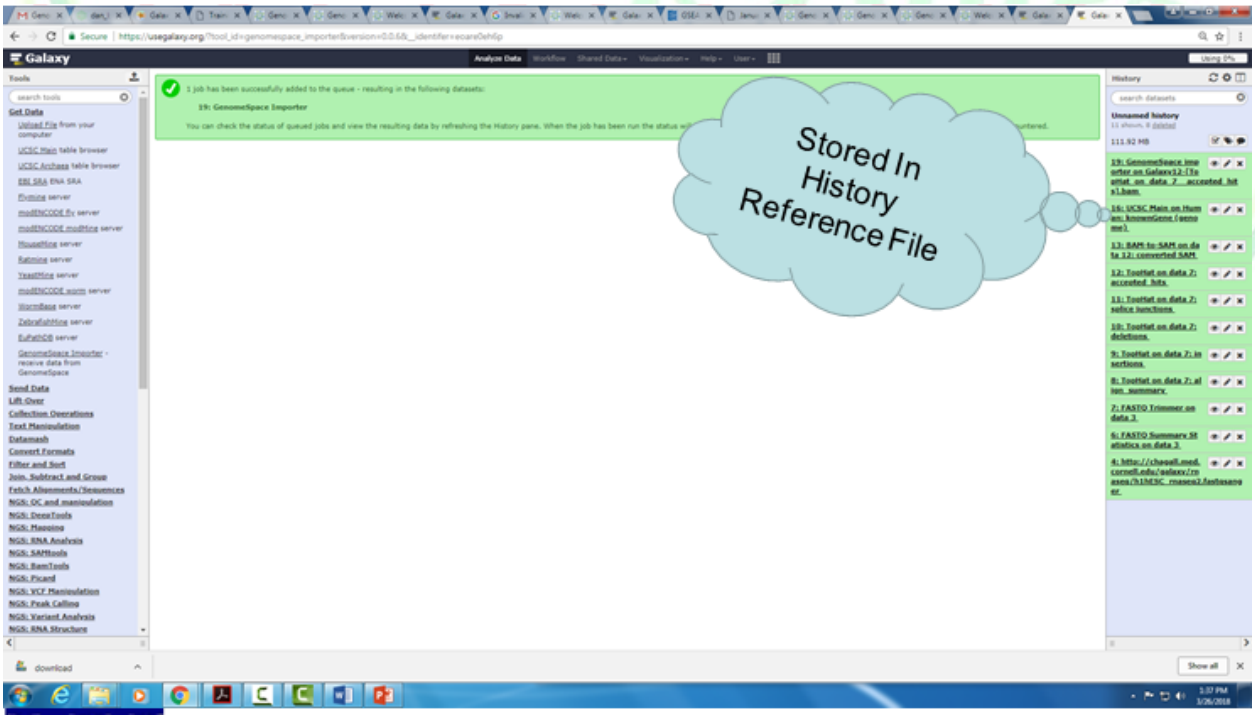

GENOMESPACE
Added in history

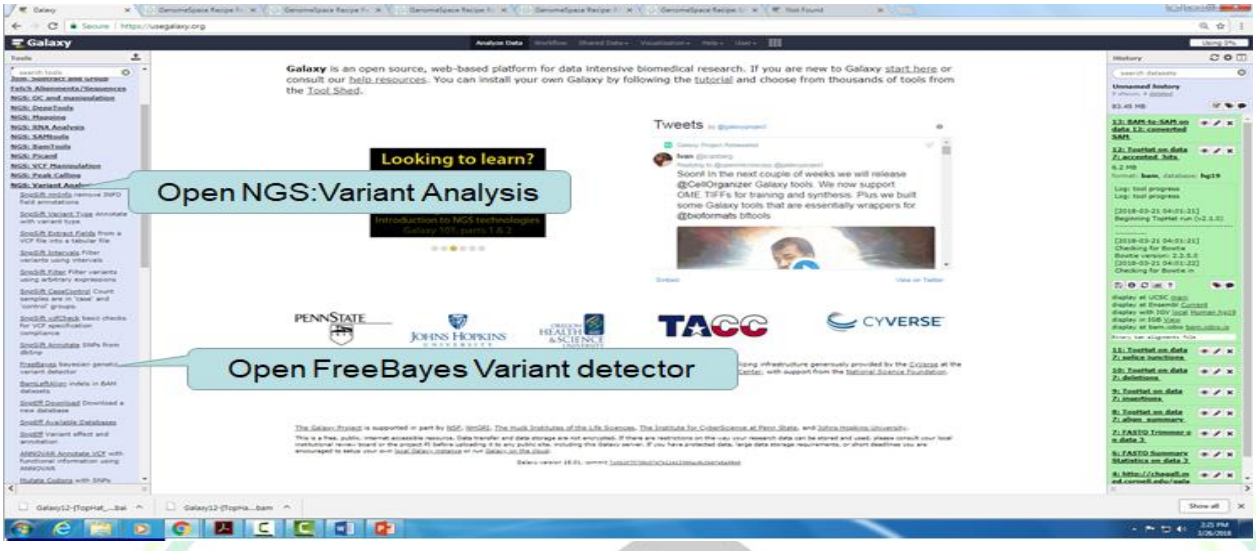

GenomeSpace Importer - receive data from GenomeSpace

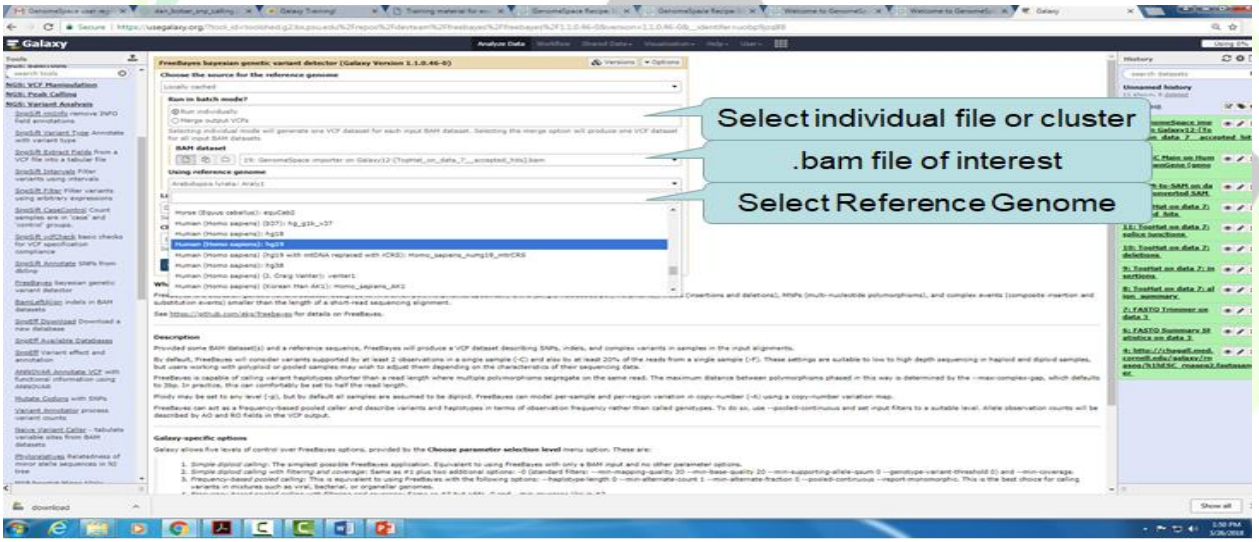

Browse
Select

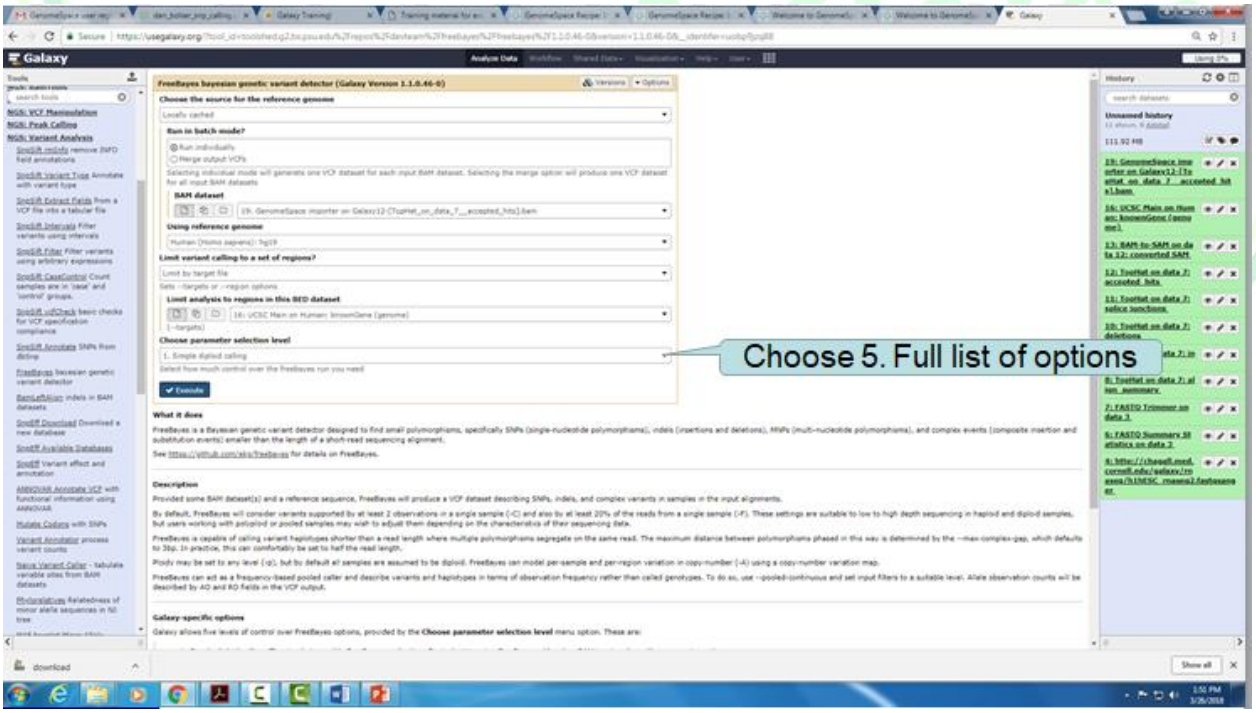

Execute

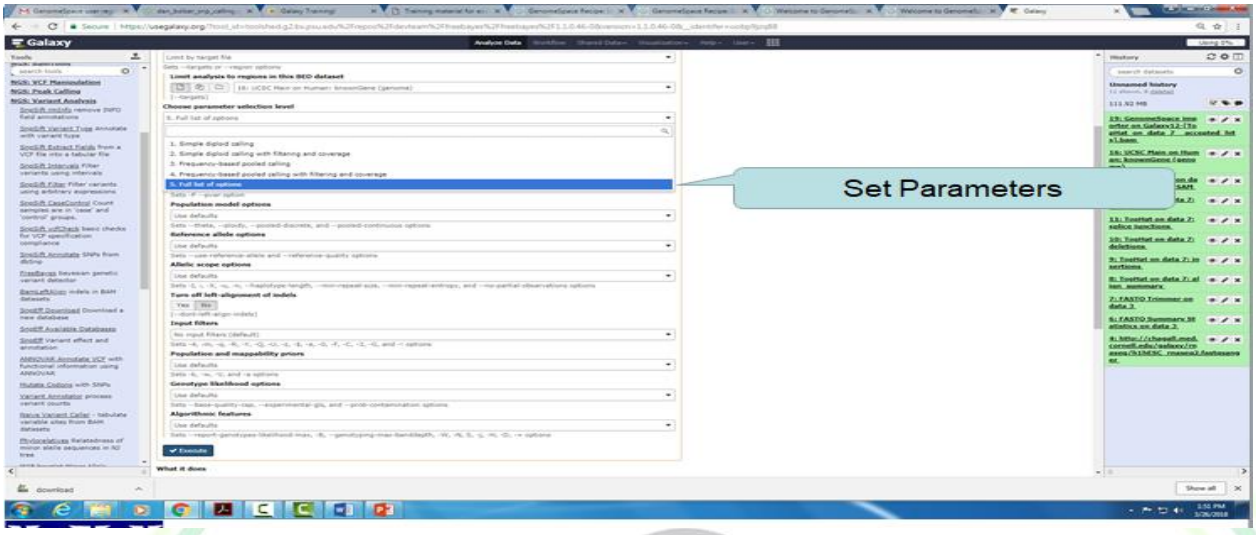

Stored In
History

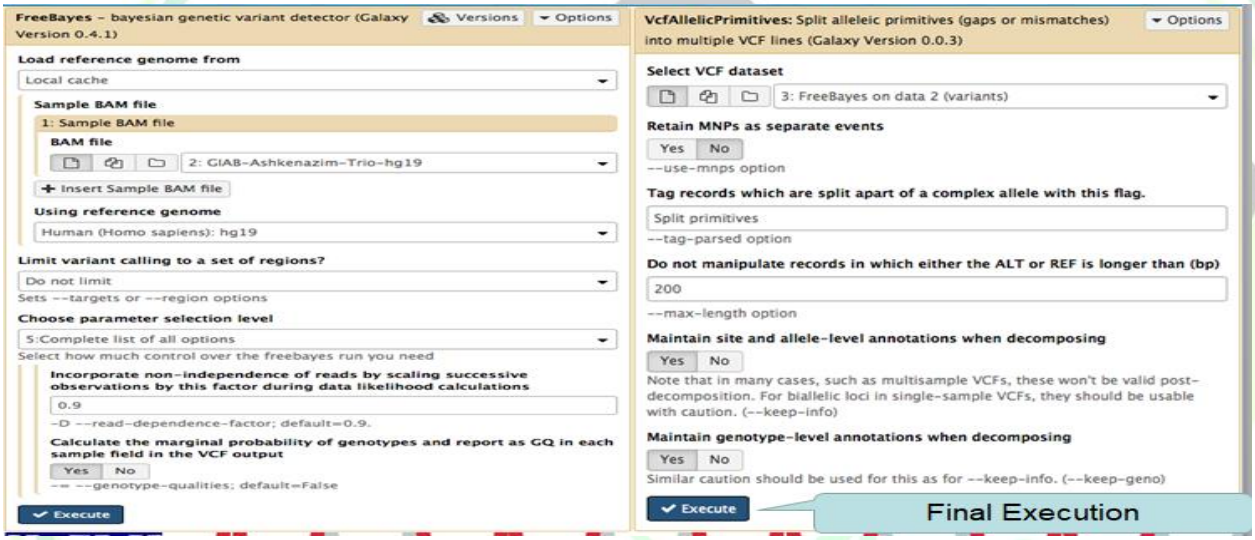

Stored In
History
Reference File

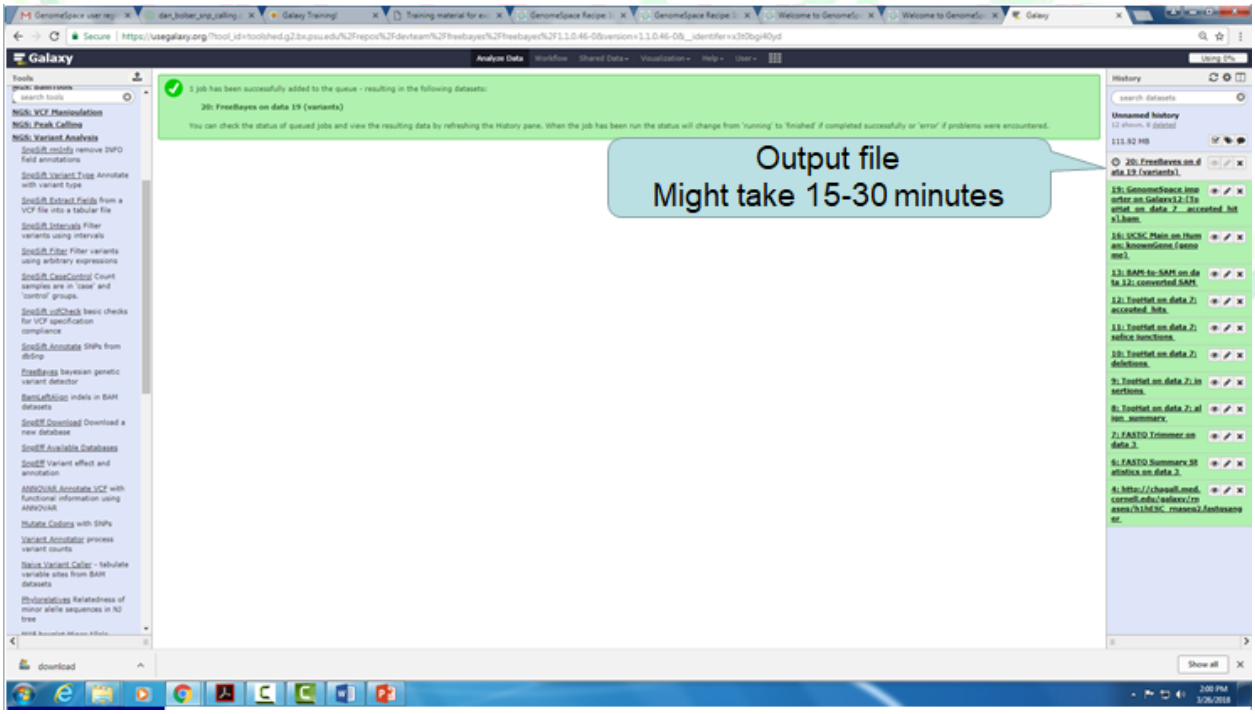

Looking to learn?
Open NGS:Variant Analysis
Open FreeBayes Variant detector

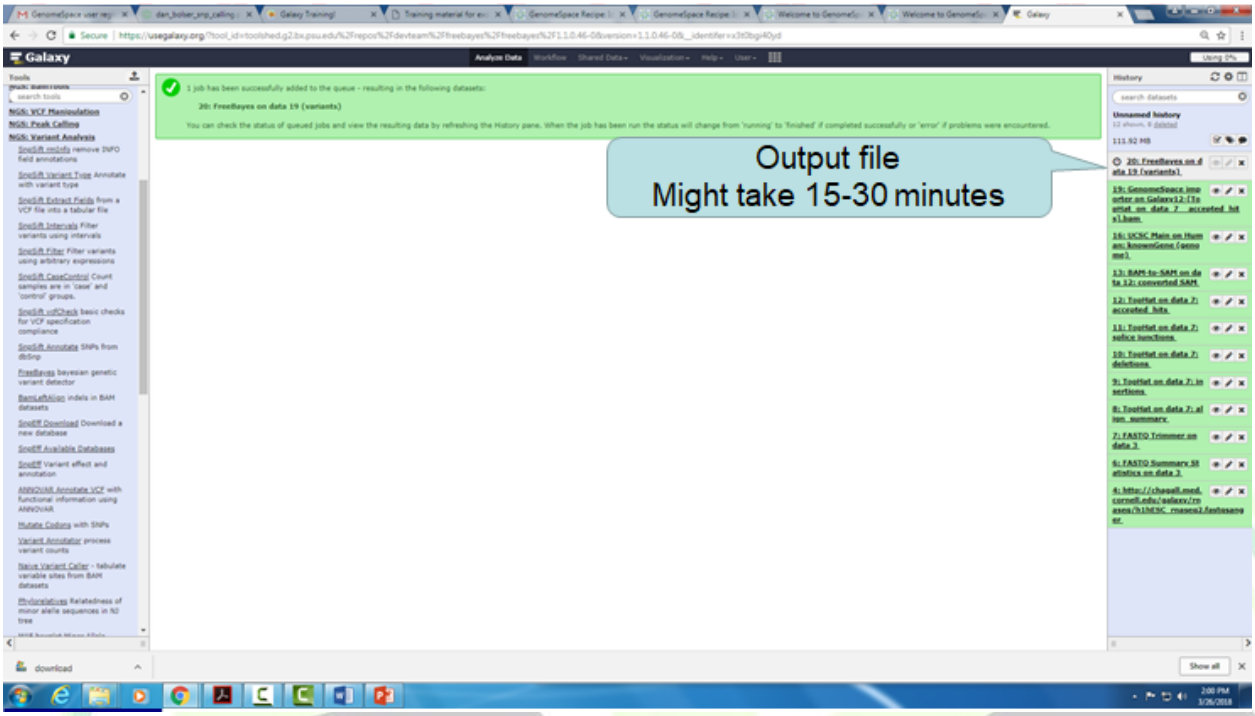

Select individual file or cluster
.bam file of interest
Select Reference Genome

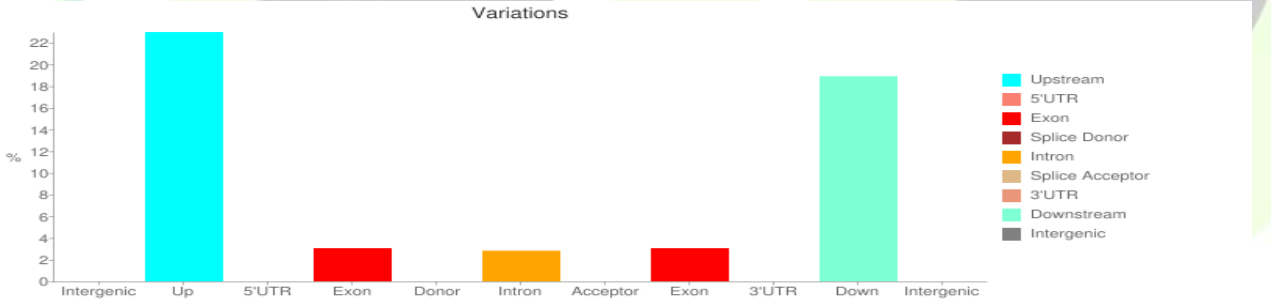

Choose 5. Full list of options


Set Parameters


FreeBayes - bayesian genetic variant detector (Galaxy Version 0.4.1)
Versions
Options
Load reference genome from
Local cache
Sample BAM file
1: Sample BAM file
BAM file
2: GIAB-Ashkenazim-Trio-hg19
Insert Sample BAM file
Using reference genome
Human (Homo sapiens): hg19
Limit variant calling to a set of regions?
Do not limit
Sets --targets or --region options
Choose parameter selection level
5:Complete list of all options
Select how much control over the freebayes run you need
Incorporate non-independence of reads by scaling successive observations by this factor during data likelihood calculations
0.9
-D --read-dependence-factor; default=0.9.
Calculate the marginal probability of genotypes and report as GQ in each sample field in the VCF output
Yes
No
--genotype-qualities; default=False
Execute
VcfAllelicPrimitives: Split alleleic primitives (gaps or mismatches) into multiple VCF lines (Galaxy Version 0.0.3)
Options
Select VCF dataset
3: FreeBayes on data 2 (variants)
Retain MNPs as separate events
Yes
No
--use-mnps option
Tag records which are split apart of a complex allele with this flag.
Split primitives
--tag-parsed option
Do not manipulate records in which either the ALT or REF is longer than (bp)
200
--max-length option
Maintain site and allele-level annotations when decomposing
Yes
No
Note that in many cases, such as multisample VCFs, these won't be valid post-decomposition. For biallelic loci in single-sample VCFs, they should be usable with caution. (--keep-info)
Maintain genotype-level annotations when decomposing
Yes
No
Similar caution should be used for this as for --keep-info. (--keep-geno)
Execute
Final Execution


Galaxy
Output file
Might take 15-30 minutes

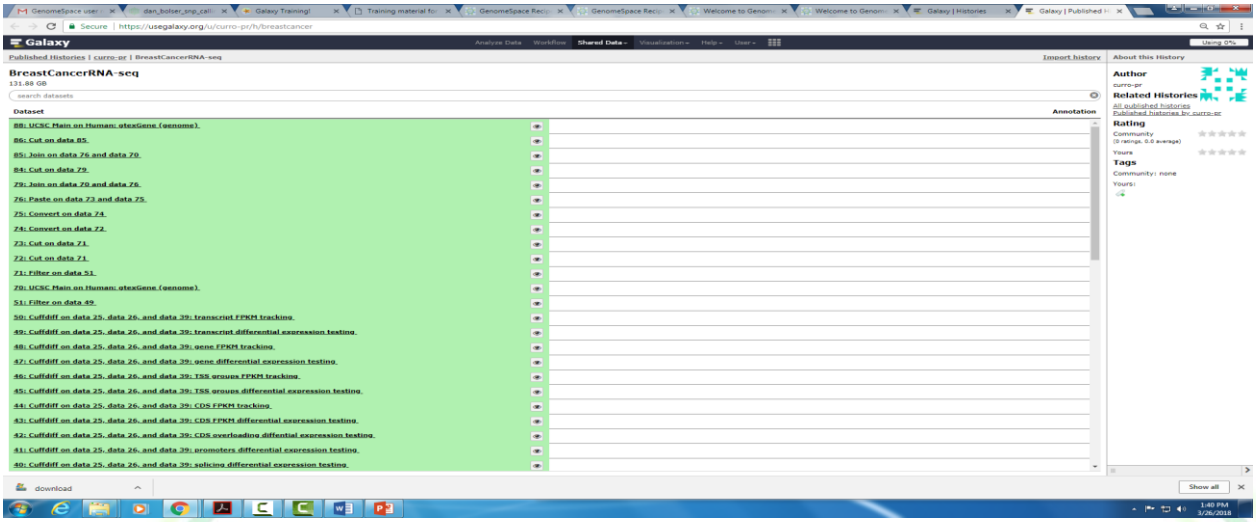


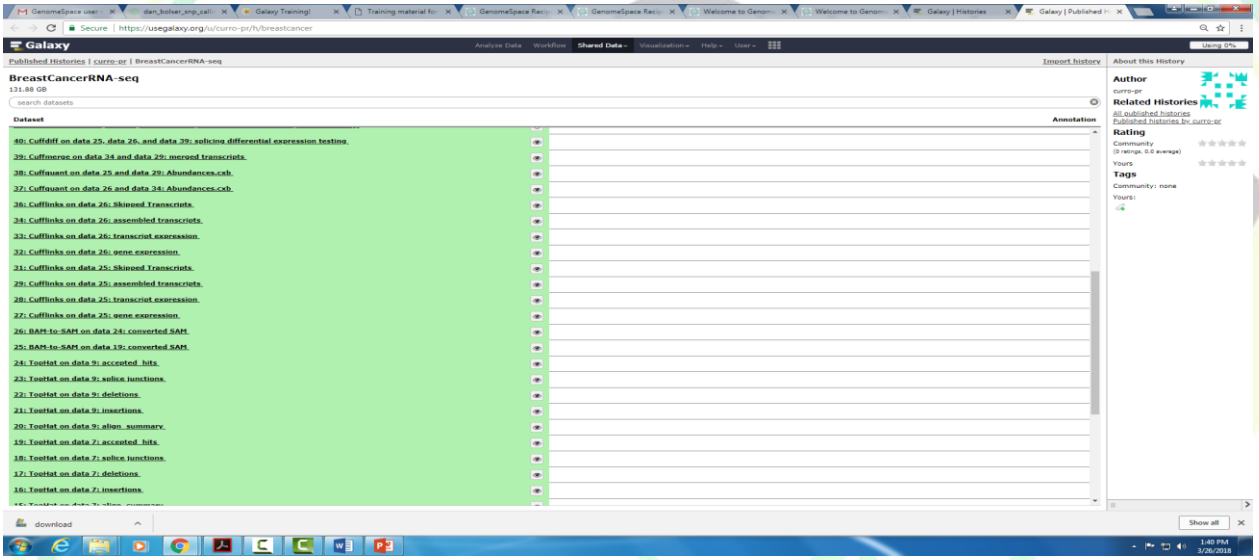


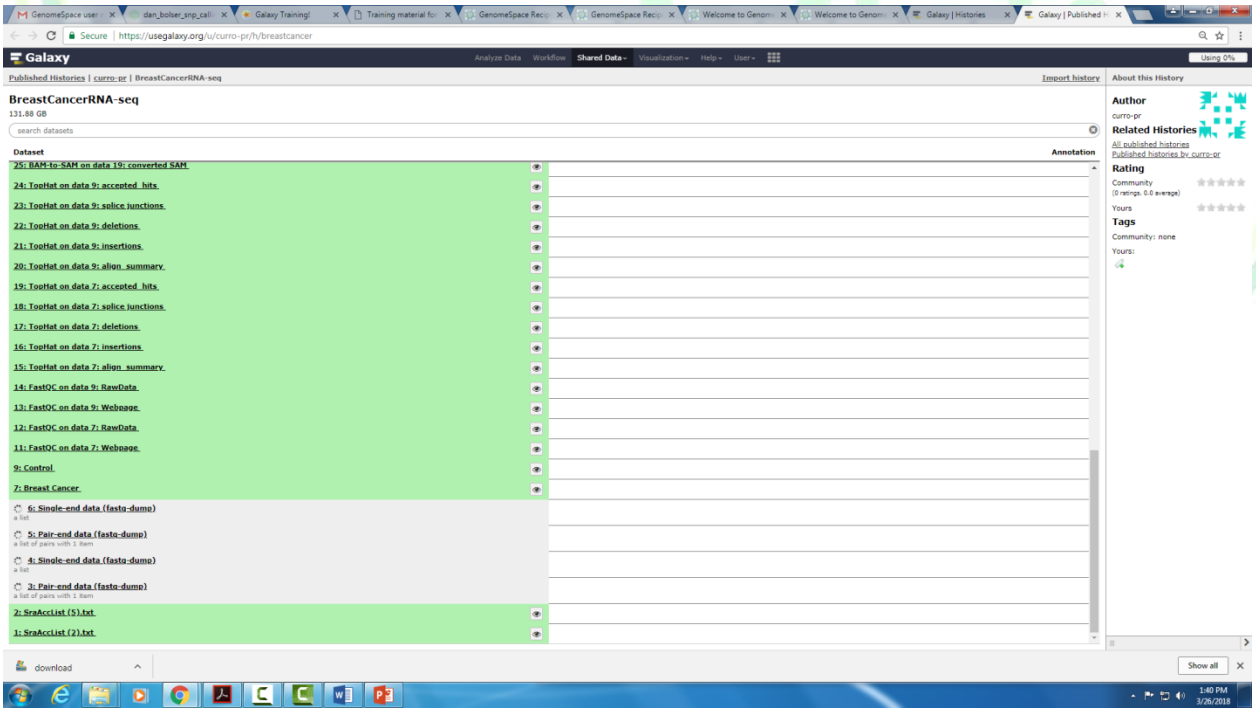
- Download hg19 database with SnpEff Download
- Launch annotation of your variants with SnpEff from Annotation, using the downloaded database (reference genome from your history)

SnpEff will generate two outputs: (1) an annotated VCF file and (2) an HTML report. The report contains a number of useful metrics such as distribution of variants across gene features

	ATC	ATT	CAA	CAC	CAT	CCC	CCG	CTG	GAA	GAC	GAG	GAT	GCA	GCC	GCT	GGA	GGG	GTG	TAC	TCC	TGG	TTG
ATC		2																				
ATT																						
CAA				1																		
CAC					2	1																
CAT																						
CCC							1															
CCG																						
CTG																						
GAA																1						
GAC												1										
GAG																	2					
GAT																						
GCA																						
GCC																						
GCT													1	1								
GGA																						
GGG																						
GTG																	1					
TAC																				1		
TCC																						
TGG																	1					
TTG								1														


Galaxy
BreastCancerRNA-seq
131.88 GB
Dataset
Annotation
88: UCSC Main on Human: qtexGene (genome)
86: Cut on data 85
85: Join on data 76 and data 70
84: Cut on data 79
79: Join on data 70 and data 76
76: Paste on data 73 and data 75
75: Convert on data 74
74: Convert on data 72
73: Cut on data 71
72: Cut on data 71
71: Filter on data 51
70: UCSC Main on Human: qtexGene (genome)
51: Filter on data 49
50: Cuffdiff on data 25, data 26, and data 39: transcript FPKM tracking
49: Cuffdiff on data 25, data 26, and data 39: transcript differential expression testing
48: Cuffdiff on data 25, data 26, and data 39: gene FPKM tracking
47: Cuffdiff on data 25, data 26, and data 39: gene differential expression testing
46: Cuffdiff on data 25, data 26, and data 39: TSS groups FPKM tracking
45: Cuffdiff on data 25, data 26, and data 39: TSS groups differential expression testing
44: Cuffdiff on data 25, data 26, and data 39: CDS FPKM tracking
43: Cuffdiff on data 25, data 26, and data 39: CDS FPKM differential expression testing
42: Cuffdiff on data 25, data 26, and data 39: CDS overloading diffential expression testing
41: Cuffdiff on data 25, data 26, and data 39: promoters differential expression testing
40: Cuffdiff on data 25, data 26, and data 39: splicing differential expression testing
Author
Related Histories
Rating
Tags
download
Show all


Galaxy
BreastCancerRNA-seq
131.88 GB
Dataset
Annotation
40: Cuffdiff on data 25, data 26, and data 39: splicing differential expression testing
39: Cuffmerge on data 34 and data 29: merged transcripts
38: Cuffquant on data 25 and data 29: Abundances.cxb
37: Cuffquant on data 26 and data 34: Abundances.cxb
36: Cufflinks on data 26: Skipped Transcripts
34: Cufflinks on data 26: assembled transcripts
33: Cufflinks on data 26: transcript expression
32: Cufflinks on data 26: gene expression
31: Cufflinks on data 25: Skipped Transcripts
29: Cufflinks on data 25: assembled transcripts
28: Cufflinks on data 25: transcript expression
27: Cufflinks on data 25: gene expression
26: BAM-to-SAM on data 24: converted SAM
25: BAM-to-SAM on data 19: converted SAM
24: TopHat on data 9: accepted_hits
23: TopHat on data 9: splice junctions
22: TopHat on data 9: deletions
21: TopHat on data 9: insertions
20: TopHat on data 9: align_summary
19: TopHat on data 7: accepted_hits
18: TopHat on data 7: splice junctions
17: TopHat on data 7: deletions
16: TopHat on data 7: insertions
Author
Related Histories
Rating
Tags
download
Show all


Galaxy
BreastCancerRNA-seq
131.88 GB
Dataset
Annotation
25: BAM-to-SAM on data 19: converted SAM
24: TopHat on data 9: accepted_hits
23: TopHat on data 9: splice junctions
22: TopHat on data 9: deletions
21: TopHat on data 9: insertions
20: TopHat on data 9: align_summary
19: TopHat on data 7: accepted_hits
18: TopHat on data 7: splice junctions
17: TopHat on data 7: deletions
16: TopHat on data 7: insertions
15: TopHat on data 7: align_summary
14: FastQC on data 9: RawData
13: FastQC on data 9: Webpage
12: FastQC on data 7: RawData
11: FastQC on data 7: Webpage
9: Control
7: Breast Cancer
6: Single-end data (fastq-dump)
5: Pair-end data (fastq-dump)
4: Single-end data (fastq-dump)
3: Pair-end data (fastq-dump)
2: SraAccList (5).txt
1: SraAccList (2).txt
Author
Related Histories
Rating
Tags
download
Show all

Lesson-30[not available]

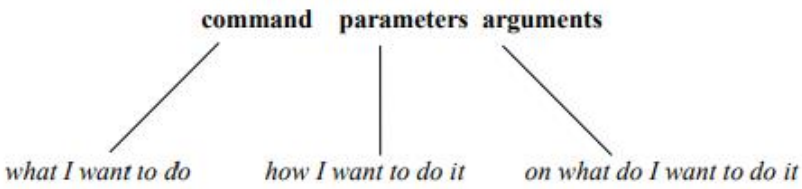
Lesson-31

LINUX Introduction-Basic Command

Introduction

Anatomy of a Command

Linux/Unix commands usually take the form:



ls -l bioinf_files

Basic Linux/Unix tips for filenames

a.) Certain characters should not be used in filenames in

Linux/Unix.

b.) Linux/Unix cannot deal with spaces in filenames!

If you end up with filenames with spaces in them, you will need to enclose the entire filename in quotation marks so that Linux/Unix understands that the space is part of the name.

Basic Linux/Unix tips for filenames

Alternatively, you can "escape" the space using a backslash. For example, if I have a file called

my document

"my document"

my\ document

A common practice is to replace the space with an underscore. For example:

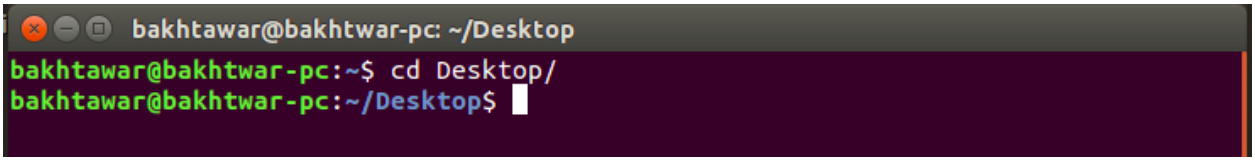
mv "my document"

my_document

Present Working Directory

```
bakhtawar@bakhtwar-pc:~$ pwd
/home/bakhtawar
bakhtawar@bakhtwar-pc:~$
```

Change Directory



To change to a directory one below you are in, just use the **cd** command followed by the subdirectory name:

cd subdir_name

To change directory to the one above the one your are in, use the shorthand for "the directory above" ..

cd ..

If you need to change directories to one far away on the system, you could explicitly state the full path:

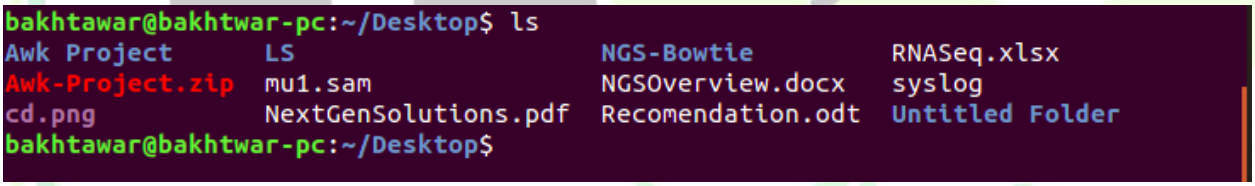
cd /usr/local/bin

If you wish to return to your home directory at any time, just type cd by itself.

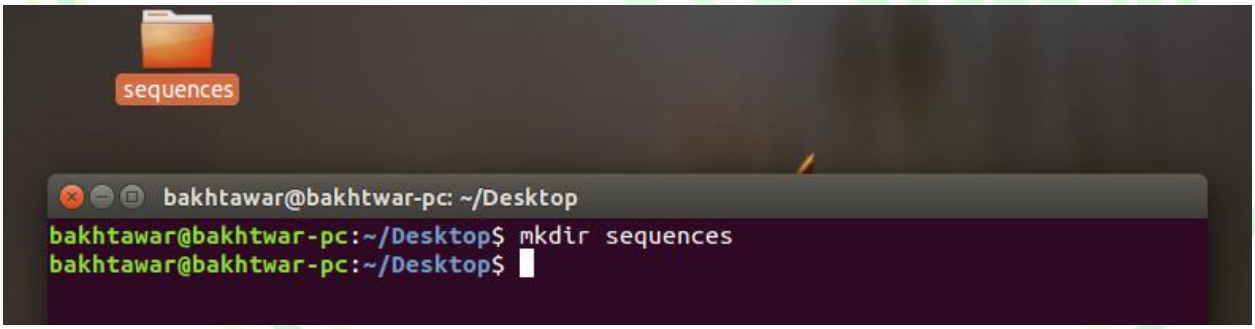
cd

To switch between directories, you can use the "**cd –**" command. This returns you to the last directory you were working in before this one.

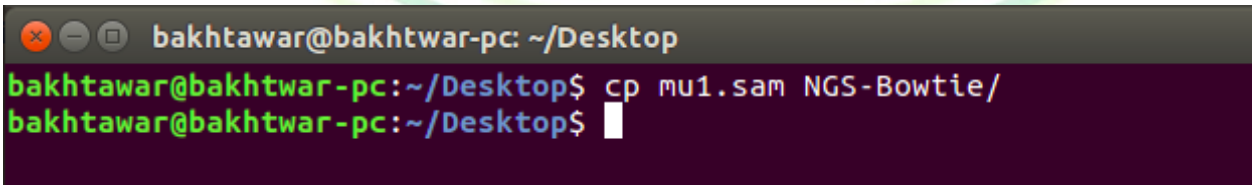
List (ls):



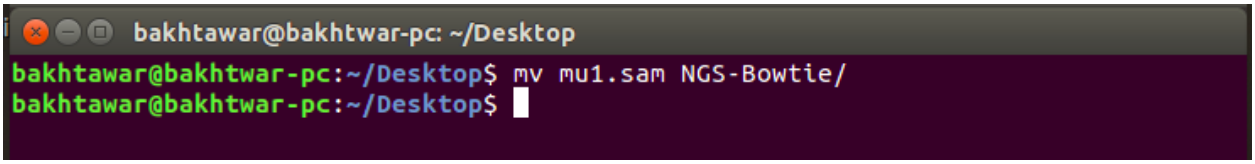
Make Directory (folder):



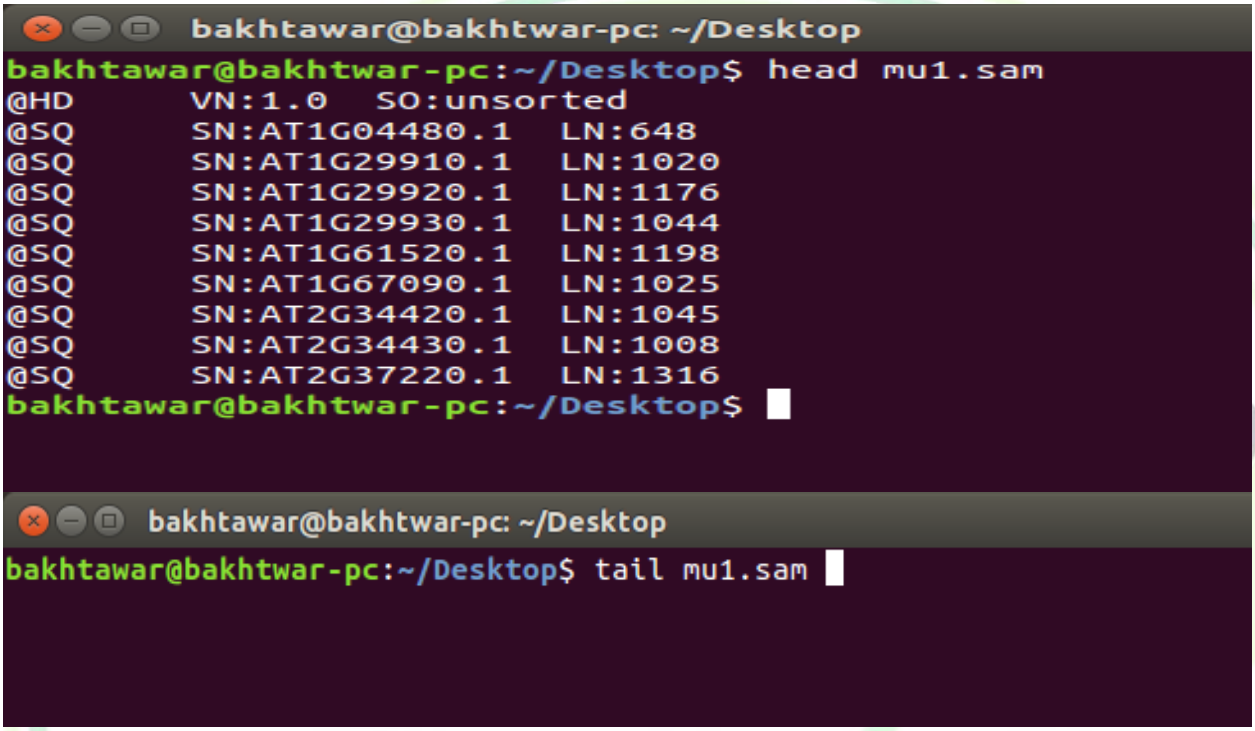
Copy files (cp):



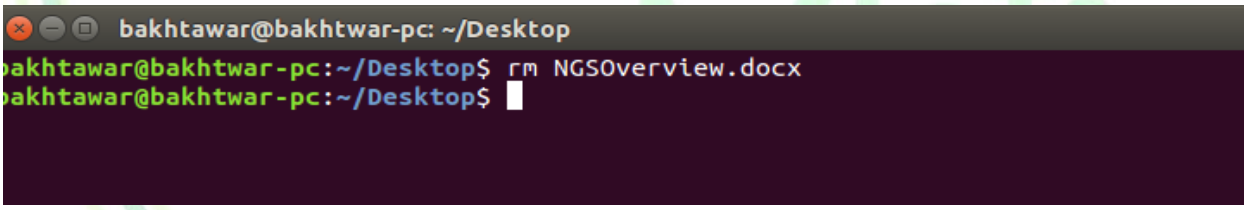
Move Files (mv):



Head and Tail



Remove a file or directory (rm):



Exercise

- Change directory from your home directory to the directory bioinf_files by typing **cd bioinf_files.**
- Type **pwd** to see the full path to where you are.
- Go back to your home directory by typing **cd.**
- Change directory into the /usr/local/bin directory by typing **cd /usr/local/bin**.
- List the files in this directory. Many of the programs in **/usr/local/bin** are bioinformatics programs. Go back to your home directory by typing **cd.**
- Make a new directory called **testdir**
- The graphical view of your account in Konqueror will be automatically updated to show this new directory.

- Move into the new directory **testdir** by typing **cd testdir**.
- Using information in the **Linux/Unix shorthand and shortcuts** section, try to move back into the bioinf_files directory using a single command.

Lesson-32

LINUX for Mapping and Assembly

Mapping to reference genome

- **vs. the genome**
  - Can (in principal) detect new transcripts, splice variants
  - Less sensitive, need a lot of coverage to discover new things
  - Need a "splice-aware" aligned such as TopHat, MapSplice, RUM
- **Vs. the transcriptome**
  - Not unbiased anymore, tied to existing annotation
  - Fatser, more sensitive, need less coverage
  - The best of both worlds?
    - Tools like TopHat (v1.4 and up) now do both

Download Data

Download a single file using wget

wget http://www.website-name.com/file

Download a whole website, recursively

wget -r http://www.website-name.com

Download specific type of file (say pdf and png) from a website

wget -r -A png,pdf http://www.website-name.com

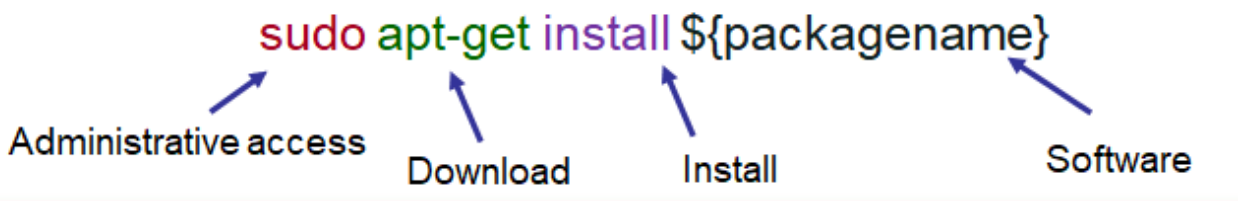
Download and Install Package

To install any software, just use remove

apt-get install ${packagename}

To remove/uninstall any software, just use remove

apt-get remove ${packagename}



RNASeq analysis (using Ubuntu linux)

Bowtie:

- CD to bowtie folder and issue commands as under
  - extract bowtie_0.12.7-1_amd64.deb with command

```
bakhtawar@bakhtwar-pc: ~/Desktop/NGS-Bowtie
bakhtawar@bakhtwar-pc:~/Desktop/NGS-Bowtie$ sudo dpkg -i extract bowtie_0.12.7-1
_amd64.deb
[sudo] password for bakhtawar:
dpkg: error processing archive extract (--install):
 cannot access archive: No such file or directory
(Reading database ... 175384 files and directories currently installed.)
Preparing to unpack bowtie_0.12.7-1_amd64.deb ...
Unpacking bowtie (0.12.7-1) over (0.12.7-1) ...
Setting up bowtie (0.12.7-1) ...
Processing triggers for doc-base (0.10.6) ...
Processing 1 changed doc-base file...
Processing triggers for man-db (2.7.4-1) ...
Errors were encountered while processing:
 extract
```

Lesson-33

LINUX for Mapping and Assembly Indexes Formation and Alignment

RNASeq analysis (using Ubuntu linux)

Bowtie:

- build indexes against which the alignments will be done

```
bakhtawar@bakhtwar-pc: ~/Desktop/NGS-Bowtie
bakhtawar@bakhtwar-pc:~/Desktop/NGS-Bowtie$ bowtie-build alignment.txt my-indexe
s
```

my-index1

my-index2 Forward

my-index3 Reverse

Align the reads with indexes and store them in file xyz.sam

```
bakhtawar@bakhtwar-pc: ~/Desktop/NGS-Bowtie
bakhtawar@bakhtwar-pc:~/Desktop/NGS-Bowtie$ bowtie my-indexs mu1.txt > mu1.sam
# reads processed: 1000
# reads with at least one reported alignment: 108 (10.80%)
# reads that failed to align: 892 (89.20%)
Reported 108 alignments to 1 output stream(s)
bakhtawar@bakhtwar-pc:~/Desktop/NGS-Bowtie$
```

- Get read counts while using counter script (python code) 'getCountMatrix.py' as;
- You must place the aligned bowtie outputs (mu1.sam and mu2.sam) in a folder called **bowtieOutputs**
- Make sure that the **getCountMatrix.py**, **GeneIDs.txt** and **outputs folder** are in the same folder. CD to that folder, while opening the terminal in Ubuntu
- Issue the following command;

```
bakhtawar@bakhtwar-pc: ~/Desktop/NGS-Bowtie
bakhtawar@bakhtwar-pc:~/Desktop/NGS-Bowtie$ python getCounts.py
```

Lesson-34

LINUX for Mapping and Assembly
Indexes Formation and Alignment By BWA

Burrow's Wheel Aligner (BWA):

Use the word "index" for indexing your reference now..

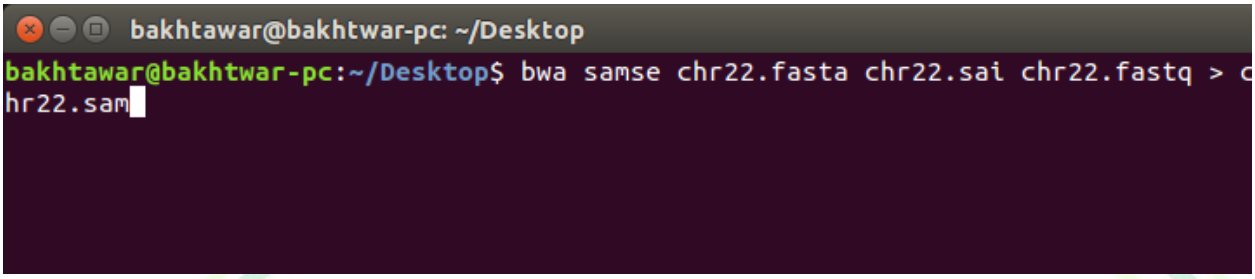
```
bakhtawar@bakhtwar-pc: ~/Desktop
bakhtawar@bakhtwar-pc:~/Desktop$ bwa index chr22.fasta
```

Align your query sequence on the Reference sequence and store it in a .sai file, (.sai being Suffix Array Indexing, an intermediate between .fasta and .SAM file)

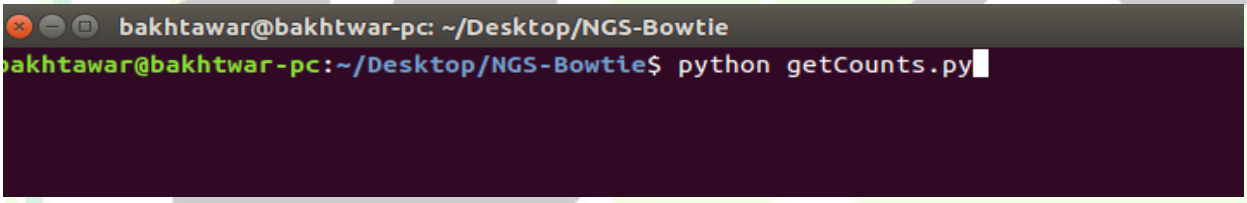
```
bakhtawar@bakhtwar-pc: ~/Desktop
bakhtawar@bakhtwar-pc:~/Desktop$ bwa aln chr22.fasta chr22.fastq > chr22.sai
```

Generate a SAM file

- Paired end data- use sampe
- Single end data – use samse

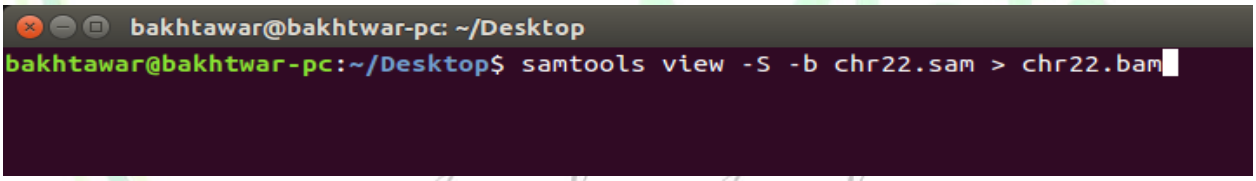


- Get read counts while using counter script (python code) 'getCountMatrix.py' as;
- You must place the aligned bowtie outputs (mu1.sam and mu2.sam) in a folder called **bowtieOutputs**
- Make sure that the **getCountMatrix.py**, **GeneIDs.txt** and **outputs folder** are in the same folder. CD to that folder, while opening the terminal in Ubuntu
- Issue the following command;

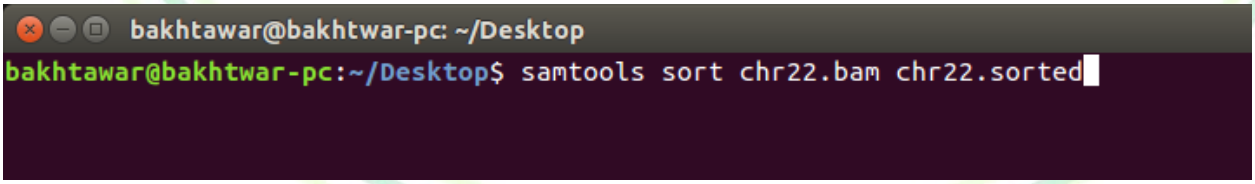


SAMtools

- Convert your Sam (bits format) file to a BAM (interpretable format) file, use "view".

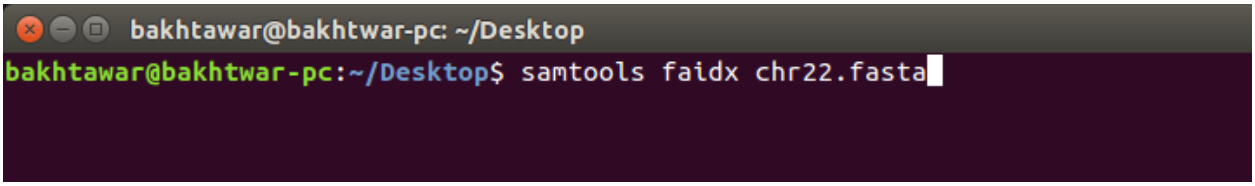


- Sort BAM for SNPcalling



Index the reference FASTA

Command "faidx" helps the SAMtools index the reference genome indexing as per its requirements.



"mpileup" → Generate VCF format

```
bakhtawar@bakhtwar-pc: ~/Desktop
bakhtawar@bakhtwar-pc:~/Desktop$ samtools mpileup -f chr22.fasta chr22.sorted.ba
m > chr22.mpileup
```

SNPcalling:

```
bakhtawar@bakhtwar-pc: ~/Desktop
bakhtawar@bakhtwar-pc:~/Desktop$ java -jar VarScan.v2.3.9.jar pileup2snp chr22.m
pileup > chr22.snp.txt
```

Indels Calling:

```
bakhtawar@bakhtwar-pc: ~/Desktop
bakhtawar@bakhtwar-pc:~/Desktop$ java -jar VarScan.v2.3.9.jar pileup2indel chr22
.mpileup > chr22.idels.txt
```

Lesson-35

R- a language for Data analysis(Overview)

R- a language for Data analysis(Overview)

R is a programming language and software environment for statistical analysis, graphics representation and reporting. R was created by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand, and is currently developed by the R Development Core Team.





- R, a programming language for statistical data analysis, is becoming increasingly important now a days because it is not only very flexible in reading, manipulating, and writing data, but all its outcomes are directly available as objects for further programming.
- R is a rapidly growing language making basic as well as advanced statistical programming easy
- From an educational point of view, R provides the possibility to combine the learning of statistical concepts by mathematics, programming, and visualization. The plots and tables produced by R can readily be used in typewriting systems such as Emacs, LATEX, or Word.
- The core of R is an interpreted computer language which allows branching and looping as well as modular programming using functions. R allows integration with the procedures written in the C, C++, .NET, Python or FORTRAN languages for efficiency.
- R is freely available under the GNU General Public License, and pre-compiled binary versions are provided for various operating systems like Linux, Windows and Mac.

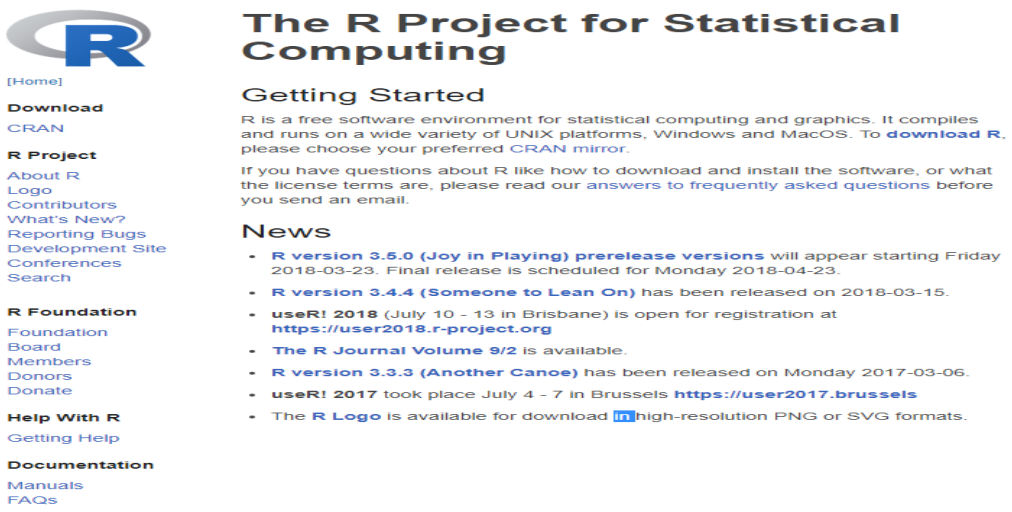


Lesson-36

R-Installation

R-Installation

- R can be downloaded from the following link.
- https://www.r-project.org/
- Follow the steps to download and install in your computer.
- Step 1- Open the link.
- You will get following screen.



Step 2-

Then click on CRAN and choose a mirror for you. R is avaliable at several links.

have to chose one.

The Comprehensive R Archive Network is available at the following URLs, please choose a location close to you. Some statistics on the status of the mirrors can be found here: main page, windows release, windows old release.

If you want to host a new mirror at your institution, please have a look at the CRAN Mirror HOWTO.

0-Cloud	
https://cloud.r-project.org/	Automatic redirection to servers worldwide, currently sponsored by Rstudio
http://cloud.r-project.org/	Automatic redirection to servers worldwide, currently sponsored by Rstudio
Algeria	
https://cran.usthb.dz/	University of Science and Technology Houari Boumediene
http://cran.usthb.dz/	University of Science and Technology Houari Boumediene
Argentina	
http://mirror.fcaglp.unlp.edu.ar/CRAN/	Universidad Nacional de La Plata
Australia	
https://cran.csiro.au/	CSIRO
http://cran.csiro.au/	CSIRO
https://mirror.aarnet.edu.au/pub/CRAN/	AARNET
https://cran.ms.unimelb.edu.au/	School of Mathematics and Statistics, University of Melbourne
https://cran.curtin.edu.au/	Curtin University of Technology
Austria	
https://cran.wu.ac.at/	Wirtschaftsuniversität Wien
http://cran.wu.ac.at/	Wirtschaftsuniversität Wien
Belgium	
http://www.freestatistics.org/cran/	K.U.Leuven Association
https://lib.ugent.be/CRAN/	Ghent University Library
http://lib.ugent.be/CRAN/	Ghent University Library
Brazil	
http://nbcgib.uesc.br/mirrors/cran/	Center for Comp. Biol. at Universidade Estadual de Santa Cruz
https://cran-r.c3sl.ufpr.br/	Universidade Federal do Parana
http://cran-r.c3sl.ufpr.br/	Universidade Federal do Parana
https://cran.fiocruz.br/	Oswaldo Cruz Foundation, Rio de Janeiro

Step 3-

After choosing mirror you will be headed to following page.

Click on download R for windows.

The Comprehensive R Archive Network

CRAN
Mirrors
What's new?
Task Views
Search

About R
R Homepage
The R Journal

Software
R Sources
R Binaries
Packages
Other

Documentation
Manuals
FAQs
Contributed

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- Download R for Linux
- Download R for (Mac) OS X
- Download R for Windows

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

Source Code for all Platforms

Windows and Mac users most likely want to download the precompiled binaries listed in the upper box, not the source code. The sources have to be compiled before you can use them. If you do not know what this means, you probably do not want to do it!

- The latest release (2018-03-15, Someone to Lean On) R-3.4.4.tar.gz, read what's new in the latest version.
- Sources of R alpha and beta releases (daily snapshots, created only in time periods before a planned release).
- Daily snapshots of current patched and development versions are available here. Please read about new features and bug fixes before filing corresponding feature requests or bug reports.
- Source code of older versions of R is available here.
- Contributed extension packages

Questions About R

- If you have questions about R like how to download and install the software, or what the license terms are, please read our answers to frequently asked questions before you send an email.

What are R and CRAN?

Step 4-

Click on base as you are installing R for the first time



CRAN
Mirrors
What's new?
Task Views
Search

About R
R Homepage
The R Journal

Software
R Sources
R Binaries
Packages
Other

Documentation
Manuals
FAQs
Contributed

R for Windows

Subdirectories:

base	Binaries for base distribution. This is what you want to **install R for the first time**.
contrib	Binaries of contributed CRAN packages (for R >= 2.13.x; managed by Uwe Ligges). There is also information on third party software available for CRAN Windows services and corresponding environment and make variables.
old contrib	Binaries of contributed CRAN packages for outdated versions of R (for R < 2.13.x; managed by Uwe Ligges).
Rtools	Tools to build R and R packages. This is what you want to build your own packages on Windows, or to build R itself.

Please do not submit binaries to CRAN. Package developers might want to contact Uwe Ligges directly in case of questions / suggestions related to Windows binaries.

You may also want to read the R FAQ and R for Windows FAQ.

Note: CRAN does some checks on these binaries for viruses, but cannot give guarantees. Use the normal precautions with downloaded executables.

Step 5-

Download latest version by clicking on download link



CRAN
Mirrors
What's new?
Task Views
Search

About R
R Homepage
The R Journal

Software
R Sources
R Binaries
Packages
Other

Documentation
Manuals
FAQs
Contributed

R-3.4.4 for Windows (32/64 bit)

Download R 3.4.4 for Windows (62 megabytes, 32/64 bit)
Installation and other instructions
New features in this version

If you want to double-check that the package you have downloaded matches the package distributed by CRAN, you can compare the md5sum of the .exe to the fingerprint on the master server. You will need a version of md5sum for windows: both graphical and command line versions are available.

Frequently asked questions

- Does R run under my version of Windows?
- How do I update packages in my previous version of R?
- Should I run 32-bit or 64-bit R?

Please see the R FAQ for general information about R and the R Windows FAQ for Windows-specific information.

Other builds

- Patches to this release are incorporated in the r-patched snapshot build.
- A build of the development version (which will eventually become the next major release of R) is available in the r-devel snapshot build.
- Previous releases

Note to webmasters: A stable link which will redirect to the current Windows binary release is <CRAN MIRROR>/bin/windows/base/release.htm.

Last change: 2018-03-15

Lesson-37

R- Strengths and Weakness

Strengths

The strengths of R are of course many, such as:

- Compact expression of ideas
- One-liners
- Seamless integration of new code

- Easy construction of simulation studies
- Operation on model objects (e.g. prediction on new data)
- The flexible graphics system
- The User Community.
- This is fundamentally linked with R's position as a Free/Open Source Software
- The price is of course important to some, but the high degree of community involvement is probably the most important aspect overall.

Weakness

Despite the strengths, there are some concrete challenges also which are:

- Inefficiency
- Semantic complications
- Rejuvenation of the community
- Some speed and space limitations are caused by the fact that R is an interpreted language executing in main memory
- However, specific aspects of the language lead to semantic complications
- This makes compiling hard and memory management difficult
- Unfortunately, they are also rather useful.
- R emerged out of a "historical coincidence" where a number of people turned out to have both similar and complementary abilities, in areas that were not actually being taught in any systematic fashion
- It is necessary to formalize and systematize these abilities in a way that **can** be taught at a general level
- This may not be easy in a world where the current trend in university education goes toward international standardization and away from innovation of the curriculum

Lesson-38

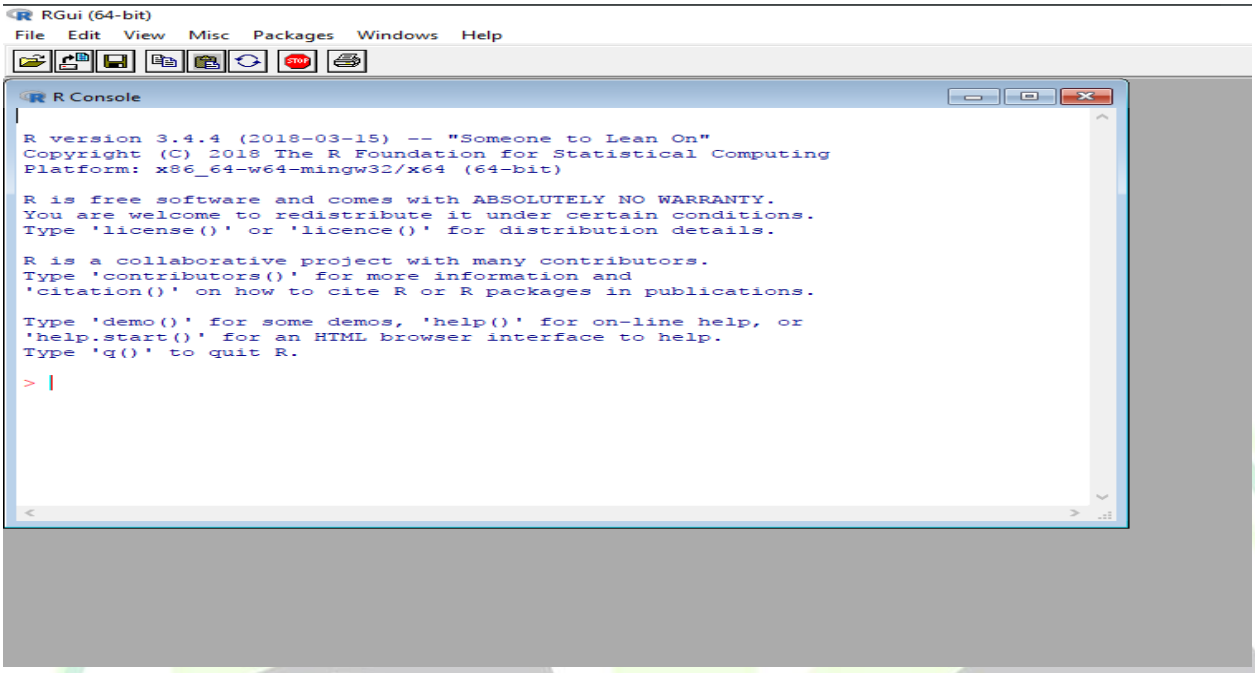
R-User Interface

R-User Interface

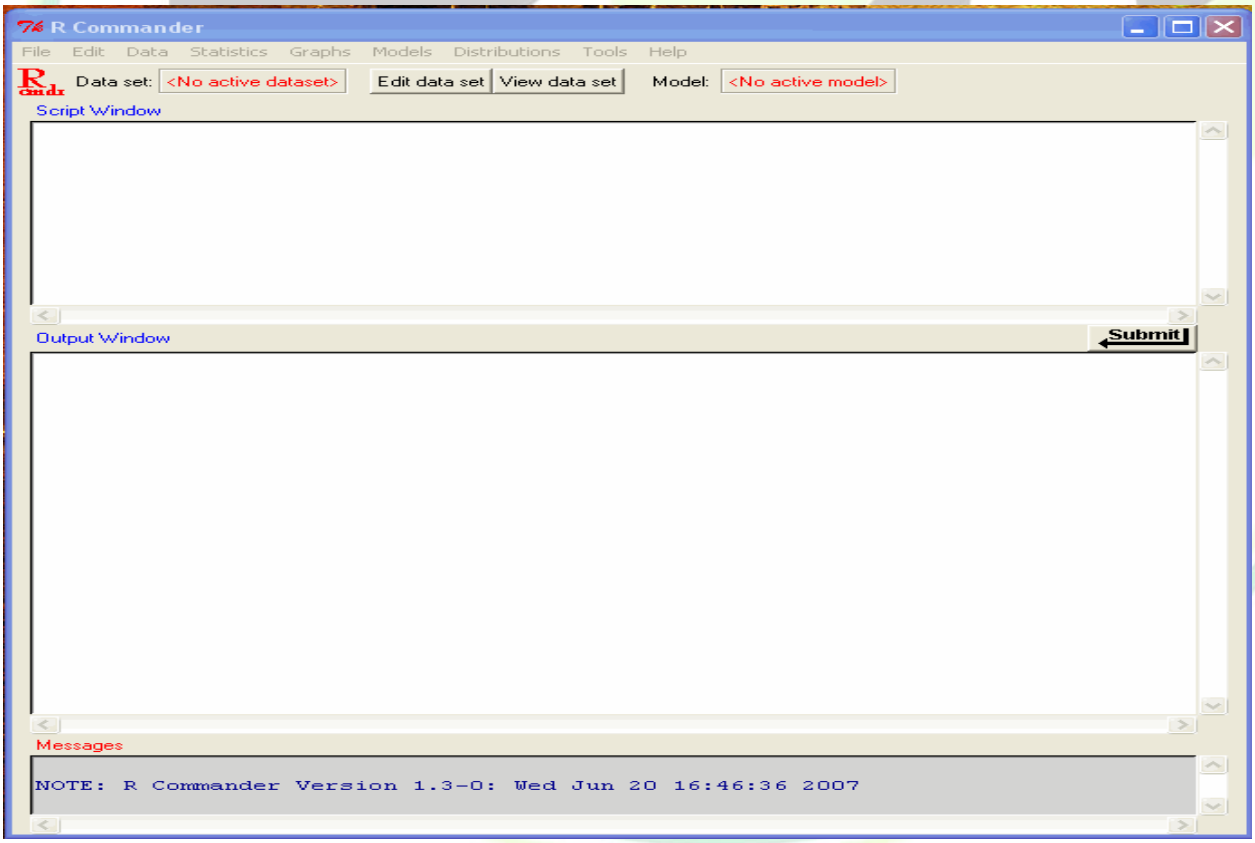
- You can downloaded R freely from http://cran.r-project.org.
- Click on your favorite operating system (Windows, Linux or MacOS) and simply follow the instructions. After a little patience you should be able to start R

Graphical User Interfaces for R	
RStudio	Integrated development environment (IDE) for R
Rattle	Gnome cross platform GUI for Data Mining using R
Red-R	Open source visual programming interface for R
Deducer	Intuitive, cross-platform graphical data analysis system
RKWard	Easy to use, transparent frontend
JGR	Universal and unified graphical user interface for R
R Commander	Basic-Statistics GUI for R

- When you open R, its user interface looks like this.



When you open RStudio, its user interface looks like this.



- To use R it is essential to install Bioconductor on your PC or network. Bioconductor is primarily based on R and can be installed, as follows.

>source("http://www.bioconductor.org/biocLite.R")

> biocLite()

Getting help

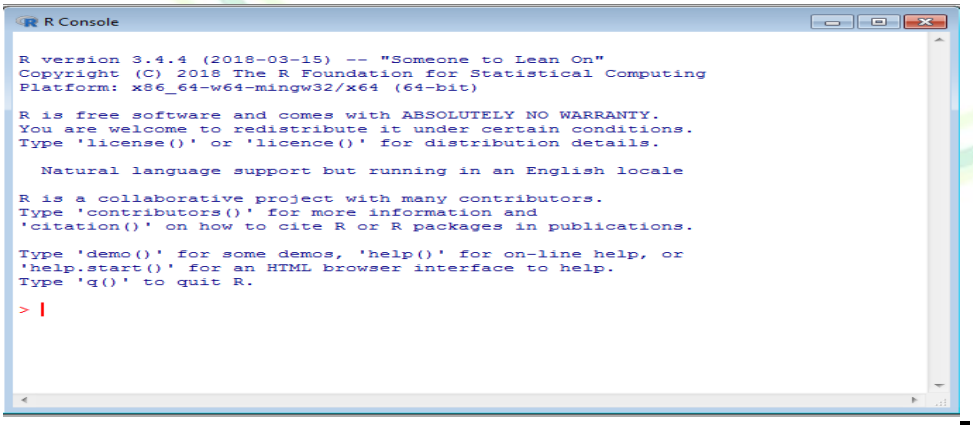
- All functionalities of R are well-organized in so-called packages.
- Use the function library() to see which packages are currently installed on your operating system.
- The packages stats and base are automatically installed, because these contain many basic functionalities.
- To obtain an overview of the content of a package use >ls(package:stats) or >library(help="stats").
- Help on the purpose of specific functions can be obtained from the (package) manual by typing a question mark in front of a function. For instance, ?sum gives details on summation.
- In case you are seeking help on a function which uses if, simply type apropos("if").
- When you are starting with a new concept such as "boxplot", it is convenient to have an example showing output (a plot) and programming code. Such is given by example(boxplot).
- The function history can be useful for collecting previously given commands.
- Type help.start() to launch an HTML page linking to several well written R manuals such as: "An Introduction to R", "The R Language Definition", "R Installation and Administration", and "R Data Import/Export".
- Further help can be obtained from http://cran.r-project.org.

Lesson-39

R-Basic Syntax

R-Basic Syntax

- When you open R, a screen is opened with the prompt >.
- All the code is written after prompt >
- All the information required to run R is in packages.
- Packages can be installed by following code: >install.packages(c("affycoretools"),repo="http://cran.r-project.org", dep=TRUE)





- After installing packages you load it by function library() for example:

  library(affycoretools)

- General help on loaded Bioconductor packages becomes available by

  openVignette().

- we shall often use packages from Bioconductor, a very useful open source software project for the analysis and comprehension of genomic data. It is essential to install Bioconductor on your PC or network.

- Bioconductor is primarily based on R and can be installed, as follows.

  >source("http://www.bioconductor.org/biocLite.R")

  > biocLite()

- Type
  - class() to obtain the class of the object.
  - str() to obtain its structure or content.

- Type objects() or ls() to view the currently loaded objects, a list probably growing soon to be large.

- To prevent conflicting definitions, it is wise to remove them all at the end of a session by rm(list=ls()).

- To quit a session, type q(), or simply click on the cross in the upper right corner of your screen.

sign of bright future

Lesson-40

R-Basic Syntax II

R-Basic Syntax

- First Hello world Program in R can be written as:

> mystring <- “Hello, World!”

>Print(mystring)

- Following result will be produced by this piece of code:

[1] “Hello, World!”

- Comments are like helping text in your R program and they are ignored by the interpreter while executing your actual program. Single comment is written using # in the beginning of the statement as

follows:

> #my first program in R programming

Lesson-41

R-As a Calculator and for Data Computing

R- As a Calculator

- R can be used as a simple calculator.
- For instance, to add 2 and 3 we simply
- insert the following.
- > 2+3
- [1] 5
- In many calculations the natural base e = 2:718282 of exponential functions is used.
- Such type of functions can be called as follows.
- > exp(1)
- [1] 2.718282
- To compute $e^2 = e.e$ we use $\exp(2)^6$. So, indeed, we have ex =exp(x), for any value of x. The sum 1 + 2 + 3 + 4 + 5 can be computed by
- > sum(1:5)
- [1] 15
- and the product 5! = 5 . 4 . 3 . 2 . 1 by
- > prod(1:5)
- [1] 120

Generating a sequence and a factor

- In order to compute so-called quantiles of distributions or plots of functions, we need to generate sequences of numbers. The easiest way to construct a sequence of numbers is by
- > 1:5
- [1] 1 2 3 4 5
- This sequence can also be produced by the function seq, which allows for various sizes of steps to be chosen. For instance, in order to compute per- centiles of a distribution we may want to generate numbers between zero and one with step size equal to 0.1.
- > seq(0,1,0.1)

- [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
- For plotting and testing of hypotheses we need to generate yet another type of sequence, called a \factor". It is designed to indicate an experimental condition of a measurement or the group to which a patient belongs.7 When, for instance, for each of three experimental conditions there are measurements from five patients, the corresponding factor can be generated as follows.
- > factor <- gl(3,5)
- > factor
- [1] 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3
- Levels: 1 2 3

Computing on a data vector

- A data vector is simply a collection of numbers obtained as outcomes from measurements. This can be illustrated by a simple example on expression values of a gene. Suppose that gene expression values 1; 1:5; and 1:25 from the persons "Eric", "Peter", and "Anna" are available. To store these in a vector we use the concatenate command c(), as follows.
- > gene1 <- c(1.00,1.50,1.25)
- > gene1
- [1] 1.00 1.50 1.25
- Now we have created the object gene1 containing three gene expression values. To compute the sum, mean, and standard deviation of the gene expression values we use the corresponding built-in-functions.
- > sum(gene1)
- [1] 3.75
- > mean(gene1)
- [1] 1.25
- > sum(gene1)/3
- [1] 1.25
- > sd(gene1)
- [1] 0.25
- > sqrt(sum((gene1-mean(gene1))^2)/2)
- [1] 0.25

Constructing a data matrix

In bioinformatics gene expression values (from several groups of patients) are stored as rows such that each row contains the expressions values of the patients corresponding to a particular gene and each column contains all gene expression values for a particular person.

- Expression values on four genes from Eric, Peter, and Anna.8
- > gene1 <- c(1.00,1.50,1.25)
- > gene2 <- c(1.35,1.55,1.00)
- > gene3 <- c(-1.10,-1.50,-1.25)
- > gene4 <- c(-1.20,-1.30,-1.00)
- Before constructing the matrix it is convenient to add the names of the rows and the columns. To do so we construct the following list.
- > rowcolnames <- list(c("gene1","gene2","gene3","gene4"),
- + c("Eric","Peter","Anna"))
- After the last comma in the first line we give a carriage return for R to come up with a new line starting with + in order to complete a command. Now we can construct a matrix containing the expression values from our four genes, as follows.
- > gendat <- matrix(c(gene1,gene2,gene3,gene4), nrow=4, +ncol=3,byrow=TRUE, dimnames = rowcolnames)
- A matrix such as gendat has two indices [i,j], the first of which refers to rows and the second to columns. Thus, if you want to print the second element of the first row to the screen, then type gendat[1,2]. If you want to print the first row, then use gendat[1,]. For the second column, use gendat[,2].
- > write.table(gendat,file="D:/data/gendat.Rdata")
- > gendatread <- read.table("D:/data/gendat.Rdata")
- > gendatread
- _____
- > write.csv(gendat,file="D:/data/gendat.Rdata")

sign of bright future

Lesson-42

R-Variables and Vectors

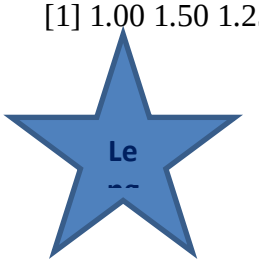
R-Variables and Vectors

- A data vector is simply a collection of numbers obtained as outcomes from measurements.
- This can be illustrated by a simple example on expression values of a gene. Suppose that gene expression values 1; 1:5; and 1:25 from the persons "Eric", "Peter", and "Anna" are available. To store these in a vector we use the concatenate command c(), as follows.

> gene1 <- c(1.00,1.50,1.25)

> gene1

[1] 1.00 1.50 1.25



- Now we have created the object gene1 containing three gene expression values. To compute the sum, mean, and standard deviation of the gene expression values we use the corresponding built-in-functions.

> sum(gene1)

[1] 3.75

> mean(gene1)

[1] 1.25

> sum(gene1)/3

[1] 1.25

> sd(gene1)

[1] 0.25

> sqrt(sum((gene1-mean(gene1))^2)/2)

[1] 0.25

- A variable provides us with named storage that our programs can manipulate.
- A variable in R can store an atomic vector, group of atomic vectors or a combination of many R objects.
- A valid variable name consists of letters, numbers and the dot or underline characters. The variable name starts with a letter or the dot not followed by a number.

Variable Name	Validity	Reason
Var_name2.	Valid	Has letters, numbers, dot and underscore
Var_name%	Invalid	Has the character '%'. Only dot(.) and underscore allowed.
2var_name	Invalid	Starts with a number
.var_name,var.name	Valid	Can start with a dot(.) but the dot(.)should not be followed by a number.
.2var_name	Invalid	The starting dot is following by a number
_var_name	Invalid	Starts with _ which is not valid

- The variables can be assigned values using leftward, rightward and equal to operator. The values of

the variables can be printed using print() or cat()function. The cat() function combines multiple items into a continuous print output.

- For example:
- Var.1=c(0,1,2,3)
- Var.2 <- c("Learn", "R")
- > x <- c(1, 5, 4, 9, 0)
- > typeof(x)
- ---------- ?
- > length(x)
- ---------- ?
- > x <- c(1, 5.4, TRUE, "hello")
- > x
- ---------- ?
- > typeof(x)
- ----------

Lesson-43

R-Data Frame

- A data frame is used for storing data tables. It is a list of vectors of equal length. For example, the following variable df is a data frame containing three vectors n, s, b.
- ```
  > n = c(2, 3, 5)
  > s = c("aa", "bb", "cc")
  > b = c(TRUE, FALSE, TRUE)
  > df = data.frame(n, s, b)
  ```
- # df is a data frame
- We can use built-in data frames in R. For example, here is a built-in data frame in R, called **mtcars**.
- > mtcars

```
                  mpg cyl disp  hp drat   wt ...
Mazda RX4         21.0   6  160 110 3.90 2.62 ...
Mazda RX4 Wag 21.0   6  160 110 3.90 2.88 ...
Datsun 710        22.8   4  108  93 3.85 2.32 ...
```

- Here is the cell value from the first row, second column of mtcars.
- > mtcars[1, 2]
  [1] 6
```

- Moreover, we can use the row and column names instead of the numeric coordinates.
- > mtcars["Mazda RX4", "cyl"]
  [1] 6
- Lastly, the number of data rows in the data frame is given by the nrow function.
- > nrow(mtcars) # number of data row
  [1] 32
- And the number of columns of a data frame is given by the ncol function.
- > ncol(mtcars) # number of columns
  [1] 11
- Further details of the mtcars data set is available in the R documentation.
- > help(mtcars)
- **Conversion of Arguments**:

as.data.frame(optional=TRUE)

---

stringsAsFactors

Is.vector

As.matrix

default.stringsAsFactors

getOption("stringsAsFactors")

Lesson-44

R-Data Types and Subsetting

Vectors:

- R has a wide variety of data types including scalars, vectors (numerical, character, logical), matrices, data frames, and lists.
- a <- c(1,2,5.3,6,-2,4) # numeric vector
- b <- c("one","two","three") # character vector
- c < c(TRUE,TRUE,TRUE,

FALSE,TRUE,FALSE) #logical vector

Matrices:

- All columns in a matrix must have the same mode(numeric, character, etc.) and the same length.

- The general format is mymatrix<-matrix(vector,nrow=r, ncol=c, byrow=FALSE, dimnames=list(char_vector_rownames, char_vector_colnames))
- byrow=TRUE indicates that the matrix should be filled by rows. byrow=FALSE indicates that the matrix should be filled by columns (the default). dimnames provides optional labels for the columns and rows.

Arrays:

- Arrays are similar to matrices but can have more than two dimensions. See help(array)for details.

Data Frames:

- A data frame is more general than a matrix, in that different columns can have different modes (numeric, character, factor, etc.). This is similar to SAS and SPSS datasets.

Lists:

- An ordered collection of objects (components). A list allows you to gather a variety of (possibly unrelated) objects under one name.

Factors:

- Tell R that a variable is **nominal** by making it a factor. The factor stores the nominal values as a vector of integers in the range [1... k] (where k is the number of unique values in the nominal variable), and an internal vector of character strings (the original values) mapped to these integers.

Subsetting data:

- R has powerful indexing features for accessing object elements. These features can be used to select and exclude variables and observations. The following code snippet demonstrate ways to keep or delete variables and observations and to take random samples from a dataset.
- # select variables v1, v2, v3
  myvars <- c("v1", "v2", "v3")
  newdata <- mydata[myvars]
- The **subset( )** function is the easiest way to select variables and observations. In the following example, we select all rows that have a value of age greater than or equal to 20 or age less then 10. We keep the ID and Weight columns.
- # using subset function
  newdata <- subset(mydata, age >= 20 | age < 10,
  select=c(ID, Weight))

hsb2.small <- **read.csv**("https://stats.idre.ucla.edu/stat/data/hsb2_small.csv")

using the names function to see names of the variables and which column of

data to which they correspond

names(hsb2.small)

[1] "id" "female" "race" "ses" "schtyp" "prog" "read"

[8] "write" "math" "science" "socst"

(hsb3 <- hsb2.small[, c(1, 7, 8)])

(hsb3 <- hsb2.small[, c(1, 7, 8)])

```
##      id  read  write
## 1    70    57     52
## 2   121    68     59
## 3    86    44     33
## 4   141    63     44
## 5   172    47     52
## 6   113    44     52
## 7    50    50     59
## 8    11    34     46
## 9    84    63     57
## 10   48    57     55
## 11   75    60     46
## 12   60    57     65
## 13   95    73     60
## 14  104    54     63
## 15   38    45     57
## 16  115    42     49
## 17   76    47     52
## 18  195    57     57
## 19  114    68     65
## 20   85    55     39
## 21  167    63     49
## 22  143    63     63
## 23   41    50     40
## 24   20    60     52
## 25   12    37     44
```

(hsb4 <- hsb2.small[, 1:4])

```
(hsb4 <- hsb2.small[, 1:4])
##      id      female race ses
## 1    70        0    4   1
## 2   121        1    4   2
## 3    86        0    4   3
## 4   141        0    4   3
## 5   172        0    4   2
## 6   113        0    4   2
## 7    50        0    3   2
## 8    11        0    1   2
## 9    84        0    4   2
## 10   48        0    3   2
## 11   75        0    4   2
## 12   60        0    4   2
## 13   95        0    4   3
## 14  104        0    4   3
## 15  38        0    3   1
## 16  115        0    4   1
## 17   76        0    4   3
## 18  195        0    4   2
## 19  114        0    4   3
## 20   85        0    4   2
## 21  167        0    4   2
## 22  143        0    4   2
## 23  41        0    3   2
## 24  20        0    1   3
## 25  12        0    1   2
```

Lesson-45

R-Functions and Operators

Functions:

- Functions are a fundamental building block of R, to master many of the more advanced techniques you need a solid foundation in how functions work.
- The most important thing to understand about R is that functions are objects in their own right. You can work with them exactly the same way you work with any other type of object.

```
function (arglist) {body}
```

All R functions have three parts:

- the body(), the code inside the function.
- the formals(), the list of arguments which controls how you can call the function.
- the environment(), the "map" of the location of the function's variables.

Some useful functions:

- length(object)# number of elements or components
- str(object) # structure of an object
- class(object) # class or type of an object
- names(object) # names
- c(object,object,...) # combine objects into a vector
- cbind(object, object, ...) # combine objects as columns
- rbind(object, object, ...) # combine objects as rows
- object # prints the object
- ls() # list current objects
- rm(object) # delete an object
- newobject <- edit(object) # edit copy and save as newobject
  fix(object) # edit in place

User Defined Functions

```
function.name <- function(arguments)
{
        computations on the arguments
        some other code
```

{r eval=FALSE} ? OurFunctionName

Operators:

- R's binary and logical operators will look very familiar to programmers. Note that binary operators work on vectors and matrices as well as scalars.

Arithmetic Operators

Operator	Description
+	addition
-	subtraction
*	multiplication
/	division
^ or **	exponentiation
x %% y	modulus (x mod y) 5%%2 is 1
x %/% y	integer division 5%/%2 is 2

Logical Operators

Operator	Description
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	exactly equal to
!=	not equal to
!x	Not x
x | y	x OR y
x & y	x AND y
isTRUE(x)	test if X is TRUE

- # An example
  x <- c(1:10)
  x[(x>8) | (x<5)]
  # yields 1 2 3 4 9 10

Lesson-46

R-Branching and Loops

R-Branching and Loops

- "Looping", "cycling", "iterating" or just replicating instructions is an old practice that originated well before the invention of computers. It is nothing more than automating a multi-step process by organizing sequences of actions or 'batch' processes and by grouping the parts that need to be repeated.
- According to the R base manual, among the control flow commands, the loop constructs are for, while and repeat, with the additional clauses break and next.

For loop:

- A **For loop** is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

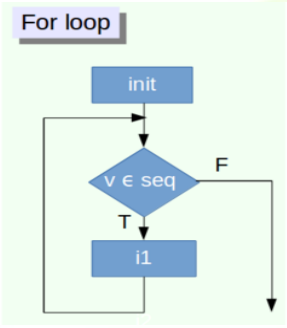
- Like a while statement, except that it tests the condition at the end of the loop body.

Syntax:

- for (value in vector) { statements }

Example:

- v <- LETTERS[1:4] for (i in v) { print(i) }



While loop:

Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.

Syntax:

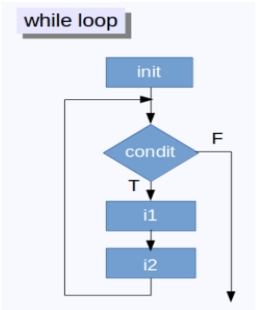
while (test_expression) { statement }

Example:

v <- c("Hello","while loop")

cnt <- 2

while (cnt < 7) { print(v) cnt = cnt + 1}



Repeat loop:

Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.

Syntax:

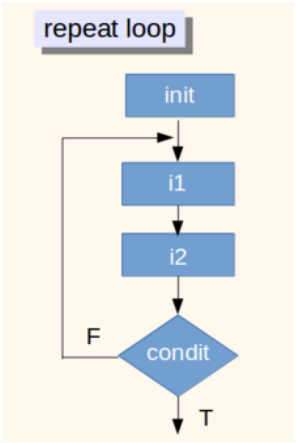
repeat { commands if(condition) { break }}

Example:

```
v <- c("Hello","loop")

cnt <- 2

repeat {   print(v)   cnt <- cnt+1      if(cnt > 5) {      break   }}
```



Break Statement:

Terminates the **loop** statement and transfers execution to the statement immediately following the loop.

Next Statement:

The **next** statement simulates the behavior of R switch.

Lesson-47

R-Storing Data

R-Storing Data

- We will look at some of the ways that R can store and organize data. This lecture demonstrate the different kinds of information R can handle.

Variable Types:

Numbers:

- The most basic way to store a number is to make an assignment of a single number:
- a <- 3

Strings:

You are not limited to just storing numbers. You can also store strings. A string is specified by using quotes. Both single and double quotes will work.

Example:

- a <- "hello"

- > a [1] "hello"
- > b <- **c**("hello","there")
- > b [1] "hello" "there"

Factors:

- Another important way R can store data is as a factor. Often times an experiment includes trials for different levels of some explanatory variable. **For example**, when looking at the impact of carbon dioxide on the growth rate of a tree you might try to observe how different trees grow when exposed to different preset concentrations of carbon dioxide. The different levels are also called factors.

Data Frames:

- Another way that information is stored is in data frames. This is a way to take many vectors of different types and store them in the same variable. The vectors can be of all different types.
- **For example,** a data frame may contain many lists, and each list might be a list of factors, strings, or numbers.

Logical:

Another important data type is the logical type.

There are two predefined variables, *TRUE* and *FALSE*:

- >a = TRUE
- > typeof(a) [1] "logical"
- > b = FALSE
- > typeof(b) [1] "logical

Table:

Another common way to store information is in a table.

Tables can be both one way and two way.

Example:

- >a < factor(c("A","A","B","A","B","B","C","A","C"))
- > results <- table(a)
- > results

Lesson-48

R-Importing and Exporting Data

Importing Data:

Importing data into R is fairly simple. Example of importing data are provided below.

From Comma delimited File:

```
# first row contains variable names, comma is separator
# assign the variable id to row names
# note the / instead of \ on mswindows systems
mydata < read.table("c:/mydata.csv", header=TRUE, sep=",",row.names="id")
```



From Excel:

One of the best ways to read an Excel file is

to export it to a comma delimited file and import

it using the method above.

Alternatively you can use the xlsx package to

access Excel files. The first row should contain variable/column names.

```
# read in the first worksheet from the workbook myexcel.xlsx
# first row contains variable names
library(xlsx)
mydata <- read.xlsx("c:/myexcel.xlsx", 1)

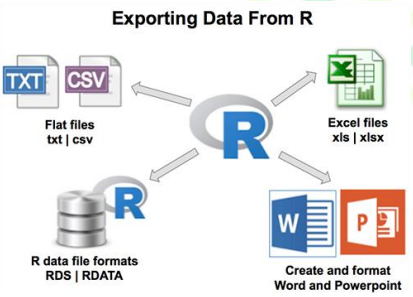
# read in the worksheet named mysheet
mydata <- read.xlsx("c:/myexcel.xlsx", sheetName = "mysheet")
```

Exporting Data:

There are numerous methods for exporting R objects into other formats . For SPSS, SAS and Stata, you will need to load the foreign packages. For Excel, you will need the xlsReadWrite package.

To a tab delimited Text file:

```
write.table(mydata,"c:/mydata.txt",sep="\t")
```

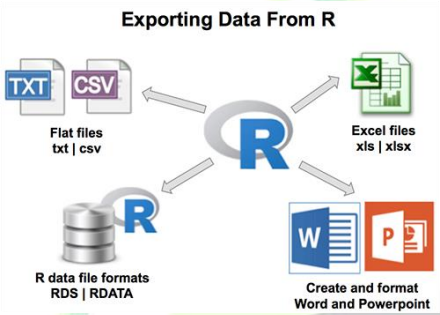


To an excel spread sheet:

```
library(xlsx)
write.xlsx(mydata, "c:/mydata.xlsx")
```

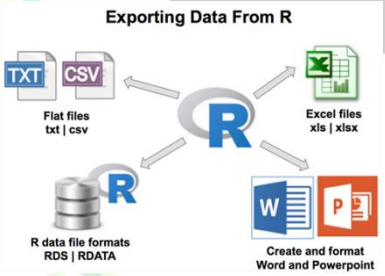
To SPSS:

```
# write out text datafile and
# an SPSS program to read it
library(foreign)
write.foreign(mydata, "c:/mydata.txt", "c:/mydata.sps",   package="SPSS")
```



To SAS:

```
# write out text datafile and
# an SAS program to read it
library(foreign)
write.foreign(mydata, "c:/mydata.txt", "c:/mydata.sas",   package="SAS")
```



sign of bright future

Lesson-49

R-DEG Analysis

Differential gene expression analysis

Differential expression analysis means taking the normalised read count data and performing statistical analysis to discover quantitative changes in expression levels between experimental groups.

For example, we use statistical testing to decide whether, for a given gene, an observed difference in read counts is significant, that is, whether it is greater than what would be expected just due to natural random variation.

After DEG analysis we get up and down regulatory genes files

GeneID	Length	pre1	pre2	pre3	post1	post2	post3
ENSG00000282961	12726	2066	2262	1294	1643	1661	1002
ENSG00000280720	125691	7	1	1	12	0	0
ENSG00000280719	22618	468	1242	428	619	618	169
ENSG00000278848	3204	198	308	202	190	304	106
ENSG00000278067	154321	644	464	489	189	263	148
ENSG00000276600	26721	124	167	161	186	423	9
ENSG00000276034	12364	146	143	139	81	89	91
ENSG00000274212	233	66	64	49	49	217	18
ENSG00000269784	15678	166	162	170	121	69	61
ENSG00000269762	154321	172	192	188	172	166	161
ENSG00000269717	15678	192	213	189	126	142	121
ENSG00000269671	154321	233	231	120	179	14	136
ENSG00000269668	12364	81	72	39	81	24	32
ENSG00000269649	316893	1244	1387	1289	734	666	798
ENSG00000269642	15678	146	141	161	122	101	107
ENSG00000269641	4049	0	0	0	0	0	0
ENSG00000269618	125691	1	0	2	1	0	0

Getting started:

After installing R from, get the full latest version of bioconductor bioconductor (www.bioconductor.org) or Load the specific packages for RNASeq data. Bioconductor is an open source suit with two releases each year, 1024 software packages so far.

source("http://bioconductor.org/biocLite.R")

will connect to bioconductor website

#biocLite()

will get all the updated packages but it takes longer.

Loading packages:

The next step is to load packages.

biocLite("affycoretools")

will upload affycoretools directly from server

library(affycoretools)

You can load other packages the same way.

Input file

- The input file for this differential gene expression analysis can either be .text or .csv file containing raw read counts.
- Read counts may be of wild type and mutant samples. Or pre and post treatment samples.
- One excellent way to get input file is from GEO (gene expression omnibus) where thousands of datasets are present.

Packages for differential gene expression analysis:

There are several packages for DEG Analysis purpose but in this course we will study five:

- edgeR
- Limma

- DESeq
- NOISeq
- BAYSeq

Lesson-50

R-DESeq package

Opening file

raw.data <- read.csv("1st comp.csv",header=T) # will take first line as column headers

dim(raw.data)

Look at the first few lines of the data:

raw.data[1:5,]

head(raw.data)

tail(raw.data)

raw.data$GeneIDs

will print whatever we have in this column

names(raw.data)[names(raw.data)=="GeneIDs"] <- "ID"

we can rename any column

Creating target file

A target file keeps track of the experimental design, i.e what treatments are there Iin #### each column, how many total treatments, replicates etc.

let's create one in R:

targets <- data.frame(FileName=colnames(raw.data)[3:8],Group=rep(c("WT","TFL"),each=3))

targets

#see what the targets object looks like

Find library sizes

library.sizes <- colSums(raw.data[,3:8])

x11()

#This will open a new graphing window; without it, previous graph will be replaced.

barplot(library.sizes)

#visualize library sizes as barplot

Counting rowSums

Let's sum the counts per gene for each group

- raw.data$WTTotal <- rowSums(raw.data[,3:5])
- raw.data$TFLTotal <- rowSums(raw.data[,6:8])
- raw.data$allTotal <- rowSums(raw.data[,3:8])
- raw.data[1:5,]

Counting Means

Same way we can take mean counts within each group

- raw.data$WTRawMean <- rowMeans(raw.data[,3:5])
- raw.data$TFLRawMean <- rowMeans (raw.data[,6:8])
- head(raw.data)

Gene filteration:

- What is the range of the allTotal counts?

  range(raw.data$allTotal)

- Somehow, there are genes in the data set with zero counts for any sample. How many?

  sum(raw.data$allTotal == 0)

- These need to be filtered out, as there is no useful information:

  raw.data <- raw.data[raw.data$allTotal != 0 ,]

  dim(raw.data)

- Before going on to DE testing, let's see how the DESeq package approaches count data. First, open the vignette:

  openVignette()

- Put in the number corresponding to DESeq; should be 18. DESeq uses it's own type of object, called a CountDataSet, that contains the counts and group information:

  cds <- newCountDataSet(raw.data[,3:8],targets$Group)

  What kind of object is it?

  class(cds)

Unlike the DGEList of edgeR which used $_name_ to access the items in the list,

- A CountDataSet object uses special functions to access items in the object.

  counts(cds)[1:5,]

- The group info and effective library sizes are in

```
pData(cds)
```

Normalization

- We need to tell DESeq how to normalize the data (sizeFactors). We could simply use the library.sizes, but DESeq has their own suggested normalization factor based on estimations from the counts themselves. See

```
?estimateSizeFactorsForMatrix for details.

cds <- estimateSizeFactors(cds)

pData(cds)
```

- DESeq has a recommended method for clustering samples, based on applying a Variance Stabilizing Transformation that changes the count data to non-integer data that can be used with regular hierarchical clustering and PCA:

```
First, estimate the regular variance:

cds <- estimateDispersions( cds )

Second, do the VST transformation:

vsd <- getVarianceStabilizedData( cds )

vsd[1:5,]
```

Differential Gene Expression

- Now do DE testing using DESeq. We already estimated the variances above using estimateVarianceFunctions(). The DESeq vignette goes through a series of QC plots to see whether the function adequately estimated the variances for this particular data set. We do not have time to go through all of that here, but you can do it on your own. We can go straight to DE testing:

```
DESeq.detailed <- nbinomTest( cds, "WT", "TFL" )

DESeq.detailed[1:5,]

#The explanation of the columns can be found by ?nbinomTest
```

Create output

- This table can be output by

```
write.csv(DESeq.detailed,file="DESeq_results.csv")
```

- By default, the p-values are also FDR-adjusted, so we can see how many genes are significant by:

```
sum(DESeq.detailed$padj < 0.01)
```

Lesson-51

R-edgeR Package

Opening file for analysis

After loading packaging we will start analysis by opening read count file.

```
raw.data <- read.csv("1st comp.csv",header=T) # will take first line as column headers
dim(raw.data)
# Look at the first few lines of the data:
raw.data[1:5,]
head(raw.data)
tail(raw.data)
raw.data$GeneIDs
# will print whatever we have in this column
names(raw.data)[names(raw.data)=="GeneIDs"] <- "ID" # we can rename any column
```

Creating target file

```
#### A target file keeps track of the experimental design, i.e what treatments are there Iin #### each column, how many total treatments, replicates etc.
#### let's create one in R:
targets <- data.frame(FileName=colnames(raw.data)[3:8],Group=rep(c("WT","TFL"),each=3))
targets
#see what the targets object looks like
```

Finding library sizes

```
library.sizes <- colSums(raw.data[,3:8])
Now create barplot of library sizes to visualize them
x11()
 #This will open a new graphing window; without it, previous graph will be replaced.
barplot(library.sizes)
```

Finding rowsums

Sums the count per gene for each group

```
raw.data$WTTotal <- rowSums(raw.data[,3:5])
raw.data$TFLTotal <- rowSums(raw.data[,6:8])
```

```
raw.data$allTotal <- rowSums(raw.data[,3:8])
raw.data[1:5,]
```

Taking meancounts

```
raw.data$WTRawMean <- rowMeans(raw.data[,3:5])
raw.data$TFLRawMean <- rowMeans (raw.data[,6:8])
head(raw.data)
```

Data Filteration:

Find range

```
range(raw.data$allTotal)
```

Find zero counts

```
sum(raw.data$allTotal == 0)
# These need to be filtered out, as there is no useful information:
raw.data <- raw.data[raw.data$allTotal != 0 ,]
dim(raw.data)
```

DEG Analysis

From the vignette, edgeR requires 3 pieces of data for DEG analysis:

1. counts: a matrix of counts where each row represents a gene/exon (or

```
# whatever genomic feature is being tracked) and each column is a different
# sample. The row names are transcript IDs."
# We have these in raw.data, but are the row names the gene IDs? Check the 1st few:
rownames(raw.data)[1:4]
# Now set the rownames to be the Gene ids
rownames(raw.data) <- raw.data$ID
head(raw.data)
```

2. group: a factor (with length equal to the number of columns of counts)

```
# denoting the experimental group."
# We have this in the targets object:
Targets
```

3. lib.size: vector of the same length as group giving the total number

```
# of reads sequenced for each sample."
```

We have this in library.sizes:

library.sizes

We need to put this information into an object of a specific class,

DGEList. There is a special function to create this object; access the help file to see how to use it:

?DGEList

put all 3 pieces into a DGEList object:

d <- DGEList(counts=as.matrix(raw.data[,3:8]), lib.size=library.sizes, group=targets$Group)

What kind of object is it?

class(d)

Since it's a list, we can find out the names of the items in the list:

names(d)

The counts are stored in the $counts:

d$counts[1:5,]

The group info and library sizes are in

d$samples

Notice that the norm.factors column are all 1s. In addition to library size, read counts can be affected by a few genes with very high expression. For example, here are the proportions of total counts for the top 10 genes in the mu1 sample:

sort(d$counts[,1]/library.sizes[1],decreasing=T)[1:10]

Normalization:

The top 2 genes each make up ~15% of the total reads! You can imagine that any

changes in these high abundance genes could affect the total pool of RNAs,

and hence the counts of other genes. edgeR suggest an additional normalization

factor using a TMM method; see ?calcNormFactors for details:

d <- calcNormFactors(d)

d$samples

Now do DE testing using edgeR. Accounting for the variance among replicates is termed "estimating dispersions" in edgeR. You can read about the different options in the edgeR vignette, but as this experiment has a single factor, we will use the qCML method, common dispersion estimate.

d <- estimateCommonDisp(d)

Expression differences can be tested using an exact test with a negative binomial distribution:

nbt<- exactTest(d,pair=c("WT","MU"))

See what is in the de.com object:

names(nbt)

The DEG we want are in the $table

nbt$table[1:6,]

The p.values have not been adjusted for multiple hypothesis testing; This can be done with the topTags function. It will perform FDR (False Discovery Rate correction)

nbt.corrected<- topTags(nbt,n=Inf)

nbt.corrected[1:5,] #see deg for top 5 genes

separate up-regulated and down-regulated genes

deg_up<- nbt.corrected[(nbt.corrected$FDR<0.05&nbt.corrected$logFC>0),]

deg_down<- nbt.corrected[(nbt.corrected$FDR<0.05&nbt.corrected$logFC<0),]

Output your deg

write.csv(deg_up,file="UPRegulated.csv")

write.csv(deg_down,file="DownRegulated.csv")

Lesson-52

R-Limma package

Opening file

raw.data <- read.csv("1st comp.csv",header=T) # will take first line as column headers

dim(raw.data)

Look at the first few lines of the data:

raw.data[1:5,]

head(raw.data)

tail(raw.data)

raw.data$GeneIDs

will print whatever we have in this column

names(raw.data)[names(raw.data)=="GeneIDs"] <- "ID"

we can rename any column

Creating target file

A target file keeps track of the experimental design, i.e what treatments are there Iin #### each column, how many total treatments, replicates etc.

let's create one in R:

```
targets <- data.frame(FileName=colnames(raw.data)[3:8],Group=rep(c("WT","TFL"),each=3))
targets
 #see what the targets object looks like
```

Find library sizes

```
library.sizes <- colSums(raw.data[,3:8])
x11()
#This will open a new graphing window; without it, previous graph will be replaced.
barplot(library.sizes)
#visualize library sizes as barplot
```

Counting rowSums

Let's sum the counts per gene for each group

```
raw.data$WTTotal <- rowSums(raw.data[,3:5])
raw.data$TFLTotal <- rowSums(raw.data[,6:8])
raw.data$allTotal <- rowSums(raw.data[,3:8])
raw.data[1:5,]
```

Counting Means

Same way we can take mean counts within each group

```
raw.data$WTRawMean <- rowMeans(raw.data[,3:5])
raw.data$TFLRawMean <- rowMeans (raw.data[,6:8])
head(raw.data)
```

Gene filteration:

What is the range of the allTotal counts?

```
range(raw.data$allTotal)
```

Somehow, there are genes in the data set with zero counts for any sample. How many?

```
sum(raw.data$allTotal == 0)
```

These need to be filtered out, as there is no useful information:

```
raw.data <- raw.data[raw.data$allTotal != 0 ,]
dim(raw.data)
```

RPKM Normalization

```
rpkm.data <- matrix(NA,nrow=nrow(raw.data),ncol=6,dimnames=list(raw.data$ID,targets$FileName))
```

look at what you created:

```
rpkm.data[,1:5]
```

Now, compute the RPKM values and put in the rpkm.data object:

```
for (i in 1:6) {
   rpkm.data[,i] <- raw.data[,2+i] / (raw.data$LENGTH/1000) / (library.sizes[i]/1000000)
   }
rpkm.data[1:5,]
```

Output RPKM File

We can have mean RPKM for both mutant and wild type samples

```
WTRPKMMean <- rowMeans(rpkm.data[,1:3])
TFLRPKMMean <- rowMeans(rpkm.data[,4:6])
```

In order to add a column to matrix, function cbind is used c stands for column

```
rpkm.data<-cbind(rpkm.data, WTRPKMMean, TFLRPKMMean)
head(rpkm.data)
rpkm.data<-as.data.frame(rpkm.data)
rpkm.data$ID<-rownames(rpkm.data)
datafile<-merge(raw.data, rpkm.data, by='ID', all.ID=T)
```

To output RPKM data to use in other programs:

```
write.csv(rpkm.data,file="rpkm_values.csv")
```

DEG Analysis by limma

Limma vignette is available in menu bar: Vignettes->limma. It's very comprehensive and demonstrates how to do analyses for many different kinds of experiment designs.

Create the design matrix; this case is simple to do by hand:

```
design <- cbind(WT=1,WTvsTFL=rep(1:0,each=3))
design<-model.matrix(~0+targets$Group)
rownames(design) <- targets$FileName  #It isn't necessary to change rownames, but it reminds you which are which rows
colnames(design)<-c("WT", "TFL")
design
```

Step 1: Estimate model coefficients from preprocessed data and design matrix

```
fit.rpkm <- lmFit(rpkm.data[,1:6],design)
names(fit.rpkm)   #gives the names of the items in the MArrayLM list
fit.rpkm$coefficients[1:5,]  #check the coefs for the first 5 genes (diff between MU and WT)
```

Step 2: Specify contrasts of interest

```
contrast.matrix<-makeContrasts(WT-TFL, levels=design)
fit.rpkm2 <- contrasts.fit(fit.rpkm, contrast.matrix)
```

Step 3: Do empirical Bayes "shrinkage" of variances and calculate test statistics

```
fit.rpkm2 <- eBayes(fit.rpkm2)
names(fit.rpkm2)
 #note additional items in list
```

Step 4: Correct for multiple tests and extract relvant data; 2 ways to do this:

A) get coded output and summary: 1 means sig up, -1 means sig down, and 0 means NS

```
?decideTests    #see what default correction is and default p-value cutoff
rpkm.coded <- decideTests(fit.rpkm2,p.value=0.1)
rpkm.coded[1:5,]  #see results for first 5 genes
summary(rpkm.coded)   #for counts of sig up and down genes
# TEST - how many genes are significant at fdr/BH p-value of 0.1? 0.2?
```

B) Get detailed output for a single contrast at a time:

```
?topTable
#check defaults; we are interested in the second coefficient; we can pull out all the genes instead of the top 10;
rpkm.detailed <- topTable(fit.rpkm2, number=Inf)
rpkm.detailed[1:5,]    #see results for top 5 genes
```

Output result

we want to combine the mean RPKM and Raw counts in order to see the real change in expression.

we can combine rpkm.detailed, rpkm.data and raw.data on a common column but that column must have the same name, we might rename the columns

```
limma_results<-merge(datafile, rpkm.detailed, all.ID=T)
head(limma_results)
limma_results<- limma_results[,c('ID', 'MuRawMean', 'WtRawMean', 'MuRPKMMean', 'WtRPKMMean',
```

'logFC', 'AveExpr', 't', 'P.Value', 'adj.P.Val', 'B')]

head(limma_results)

separate out up regulated and down regulated genes

limma_results_up<-rpkm.detailed[(rpkm.detailed$adj.P.Val<0.1 & rpkm.detailed$logFC>0),]

limma_results_up<- limma_results_up[order(limma_results_up$logFC, decreasing=T),]

dim(limma_results_up)

head(limma_results_up)

limma_results_down<- rpkm.detailed[(rpkm.detailed$adj.P.Val<0.1 & rpkm.detailed$logFC<0),]

limma_results_down<- limma_results_down[order(limma_results_down$logFC),]

dim(limma_results_down)

head(limma_results_down)

can be output by

write.csv(limma_results_up,file="RPKMlimma_results_up.csv")

write.csv(limma_results_down,file="RPKMlimma_results_down.csv")

Lesson-53

R-BAYSeq Package

R-BAYSeq Package

Opening Fille and creating design

library(baySeq)

raw.data <- read.csv("1st Comp.csv",header=T)

replicates <- c(1,1,1,2,2,2)

groups <- list(NDE = c(1,1,1,1,1,1),DE = c(1,1,1,2,2,2))

cname <- raw.data[,1]

data <- raw.data[,3:8]

data = as.matrix(data)

CD <- new("countData", data = data, replicates = replicates, groups = groups)

libsizes(CD) <- getLibsizes(CD)

Get library sizes and perform DEG Analysis

#CD@libsizes <- getLibsizes(CD)

```
#plotMA.CD(CD, samplesA = "1", samplesB = "2", col = c(rep("red", 100), rep("black", 900)))

library(snow)

cl <- makeCluster(6, "SOCK")

CD@annotation <- as.data.frame(cname)

CD <- getPriors.NB(CD, samplesize = 10000, seglens=raw.data[,2], estimation = "QL", cl = NULL)

CD <- getLikelihoods.NB(CD, pET = 'BIC', cl = NULL)

CD@estProps

topCounts(CD, group = "DE")
```

Output results:

```
blah <- topCounts(CD,group="DE",FDR=0.1)

baySeq.coded <- as.numeric(blah$FDR.DE < 0.1)

write.csv(blah, file="bayseq_de_1.csv")
```

Lesson-54

R-NOISeq package

Opening file

```
library(NOISeq)

raw.data <- read.csv("1st Comp.csv",header=T,row.names=1)

raw.data$WTTotal <- rowSums(raw.data[,2:4])

raw.data$TFLTotal <- rowSums(raw.data[,5:7])

raw.data$allTotal <- rowSums(raw.data[,2:7])

sum(raw.data$allTotal == 0)

raw.data <- raw.data[raw.data$allTotal != 0 ,]
```

Create Design

```
lengthdata <-

counts=raw.data[,2:7]

#names(newdata) <- row.names(raw.data)

myfactors = data.frame(Sample = c("WT", "WT", "WT", "TFL" , "TFL" , "TFL"), SampleRun=
c("WT1","WT2","WT3","TFL1","TFL2","TFL3"),Levels=c(rep("WT",3),rep("TFL",3)))

myfactors
```

Normalization

```
mylengths <- as.numeric(raw.data[,1])

names(mylengths) <- row.names(raw.data)

mydata <- readData(data = counts, length = mylengths ,factors = myfactors)

head(featureData(mydata)@data)

mycd = dat(mydata, type = "cd", norm = TRUE, refColumn = 1)

myRPKM = rpkm(assayData(mydata)$exprs, long = 1000, k = NULL)

myUQUA = uqua(assayData(mydata)$exprs, long = mylengths, lc = 0.5, k = 0)

myTMM = tmm(assayData(mydata)$exprs, long = mylengths, lc=0)
```

Filtration

```
myfilt = filtered.data(counts, factor = myfactors$Sample, norm = TRUE, depth = NULL, method = 1, cv.cutoff = 100, cpm = 1, p.adj = "fdr")

mynoiseqbio = noiseqbio(mydata, k = 0.5, norm = "tmm", factor = "Sample", lc = 1, r = 20, adj = 1.5, plot = FALSE, a0per = 0.9, random.seed = 12345, filter = 1)
```

Output result

```
mynoiseqbio.deg = degenes(mynoiseqbio, q = 0.90, M = NULL)

mynoiseqbio.deg1 = degenes(mynoiseqbio, q = 0.90, M = "up")

mynoiseqbio.deg2 = degenes(mynoiseqbio, q = 0.90, M = "down")

write.csv(mynoiseqbio.deg1,file="NOISeq_DowninTFL01.csv")

write.csv(mynoiseqbio.deg2,file="NOISeq_UpinTFL01.csv")
```

Lesson-55

Up regulating and Down regulating genes

Regulation of Gene Expression

Regulation of gene expression includes a wide range of mechanisms that enhance or suppress the working of a particular gene or production of the gene product.

- The regulation is of two types:
    - Up-regulation
    - Down-regulation
- Up-regulation of the gene leads to increase in the amount of its protein-product and in increase of the protein-product activity in the cell, as consequence.
- Downregulation of the gene leads to reduction in the amount of its protein-product and in reduction of the protein-product activity in the cell, consequently.

Regulators

- The regulators of these networks depends on the nature of process,
- A regulator can be;
  - A gene
  - DNA
  - RNA
  - Protein
- OR complexes on any of these correspondently

Upregulating and Down regulating genes

- When the regulators tend to be genes and they enhance or suppress the working of a correspondent particular gene they are said to upregulating or down regulating respectively.

Lesson-56

Database for Annotation, Visualization and Integrated Discovery (DAVID)

DAVID

- Free Bioinformatics Resource Software.
- Aims to provide functional interpretation of large datasets of genes derived from different genomic studies such as microarray.
- David has embedded five web-based functional annotation tool suits:
  - DAVID gene functional classification Tool
  - DAVID functional annotation Tool
  - DAVID gene ID conversion Tool
  - DAVID gene name viewer
  - DAVID NIAID Pathogen Genome Browser.



What DAVID does?

- Classifies large gene lists into functionally related groups
- Ranks the importance of the functional gene groups.
- Summarizes the major biology of gene groups discovered.
- Searches other functionally related genes that are not in the gene list but exist in the genome.
- Visualizes the annotations.

Significance

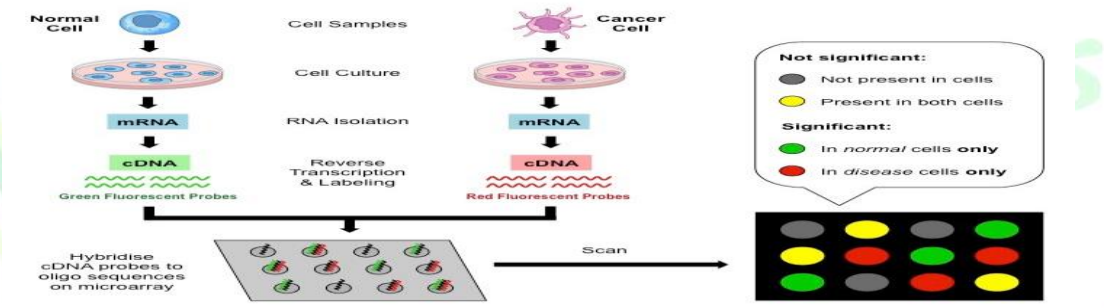
- Genes are highly organized so that they are readable and understandable.
- Ranking the genes help a quick focus on only the significant ones.
- Gene's annotations are displayed in a single view, easy for comparison
- The studies are extended by the digging up of functionally related genes.

Lesson-57

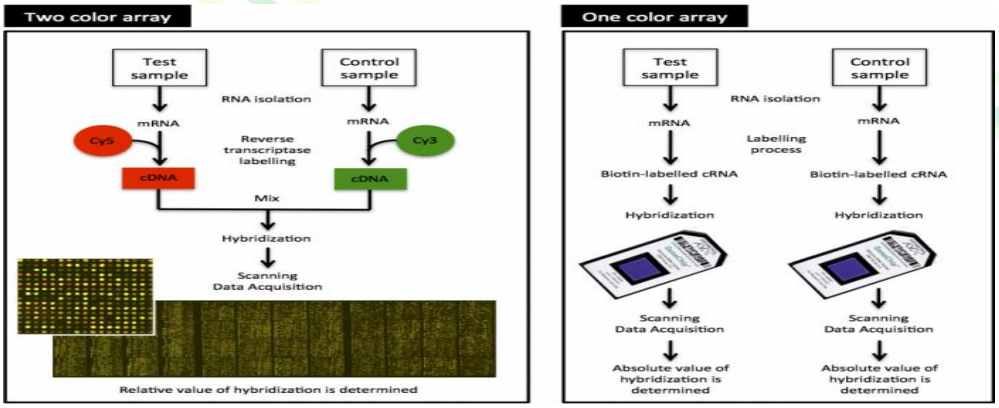
Microarray Data

Microarray

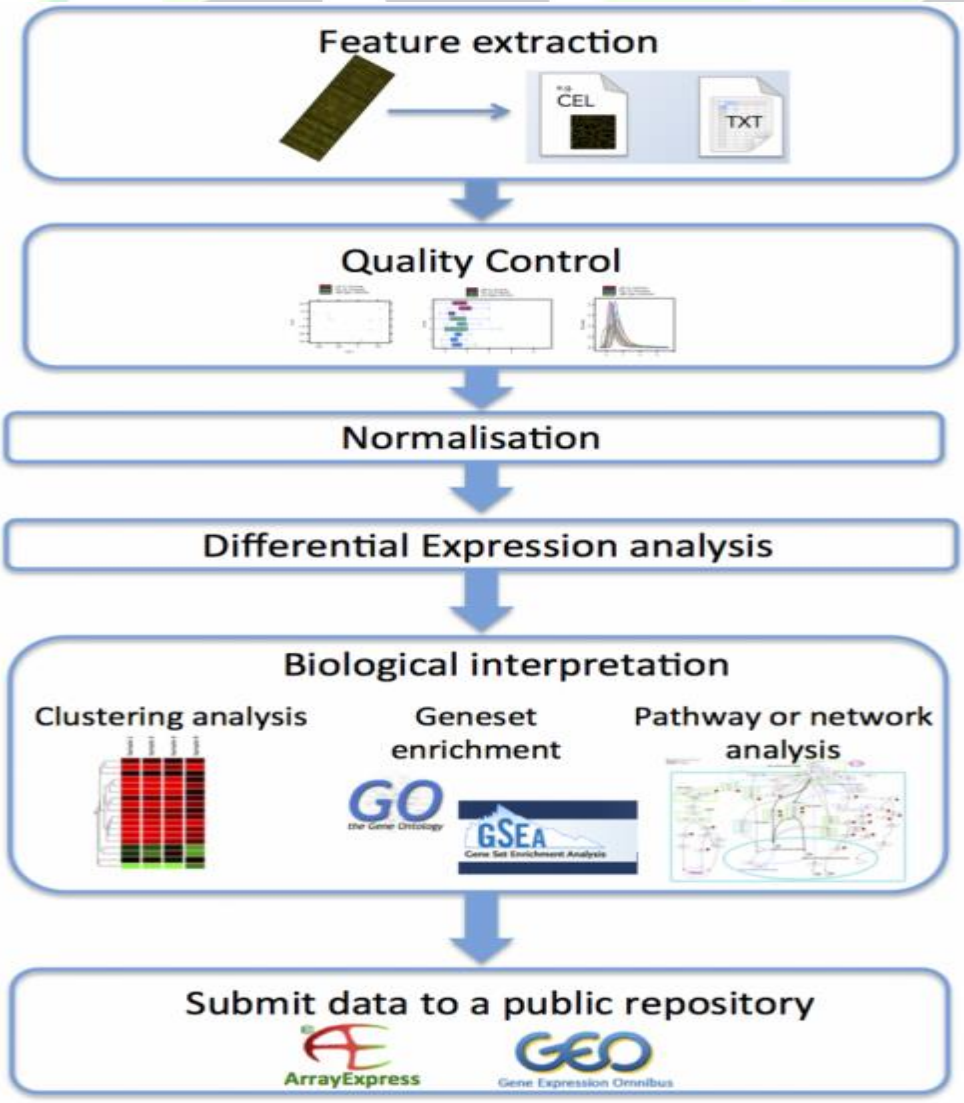
- A collection of microscopic DNA spots attached to a solid surface.
- Used to measure the expression levels of large numbers of genes simultaneously or to genotype multiple regions of a genome



One colour or Two colour arrays



- Microarrays can be used in many types of experiments including
  - genotyping,
  - epigenetics,
  - translation profiling and
  - gene expression profiling
- The process of analysing gene expression data in microarrays involves
  - feature extraction;
  - quality control;
  - normalisation;
  - differential expression analysis;
  - biological interpretation of the results;
  - submission of data to a public database.



Lesson-58

Challenges in Interpreting Gene Microarray Data

Microarray Data

- This technique makes it possible to examine the expression of thousands of genes simultaneously.
- This technology promises to lead to improvements in developing rational approaches to therapy as well as to improvements in cancer diagnosis and prognosis.
- Yet comes along with a lot of challenges to be dealt with its data.

Limitations

- Data variability, especially for genes with low expression levels
- Small sample amounts which limits replication
- Unequal labelling efficiency of flouresecnt dyes.
- Provides no information regarding protein expression levels and function.

Solutions

- Replication to reduce false positives.
- Use of amplified RNA (aRNA)
- Reciprocal labelling to confirm observations.
- Confirm with other biochemical analysis for protein expression (e.g. immunohistochemistry protein arrays)

Lesson-59

Biological Interpretation of Microarray data

sign of bright future

Interpreting the Microarray Data

- Many of the methods for visualizing and interpreting gene expression data can be used for microarray some of them are discussed here

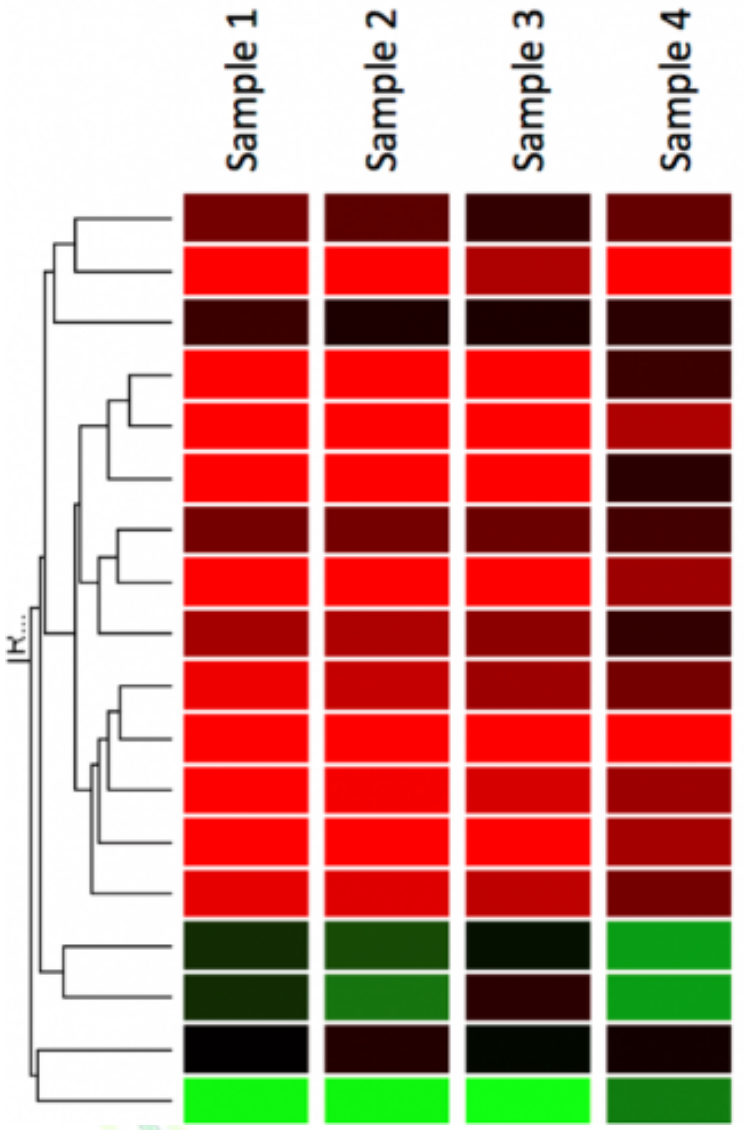
HeatMaps

- A common method of visualising gene expression data is to display it as a heatmap.
- The heatmap may also be combined with clustering methods which group genes and/or samples together based on the similarity of their gene expression pattern.
- This can be useful for identifying genes that are commonly regulated, or biological signatures associated with a particular condition (e.g a disease or an environmental condition)

HeatMaps-Example

- The data is displayed in a grid where each row represents a gene and each column represents a sample.

- The colour and intensity of the boxes is used to represent changes (not absolute values) of gene expression.
- In the example, red represents up-regulated genes and green represents down-regulated genes. Black represents unchanged expression.



Gene set enrichment analysis

- A common approach to interpreting gene expression data is gene set enrichment analysis based on the functional annotation of the differentially expressed genes.
- The Gene Ontology, containing standardised annotation of gene products, is commonly used for this purpose. It works by comparing the frequency of individual annotations in the gene list (e.g differentially expressed genes) with a reference list

Popular tools for gene set enrichment

- DAVID (Free online Tool)
- GSEA (Free)
- Ingenuity (Licensed)

- Reactome (Free)

Network analysis

- Network analysis is complementary to pathway analysis and can be used to show how key components of different pathways interact.
- This can be useful for identifying regulatory events that influence multiple biological processes and pathways

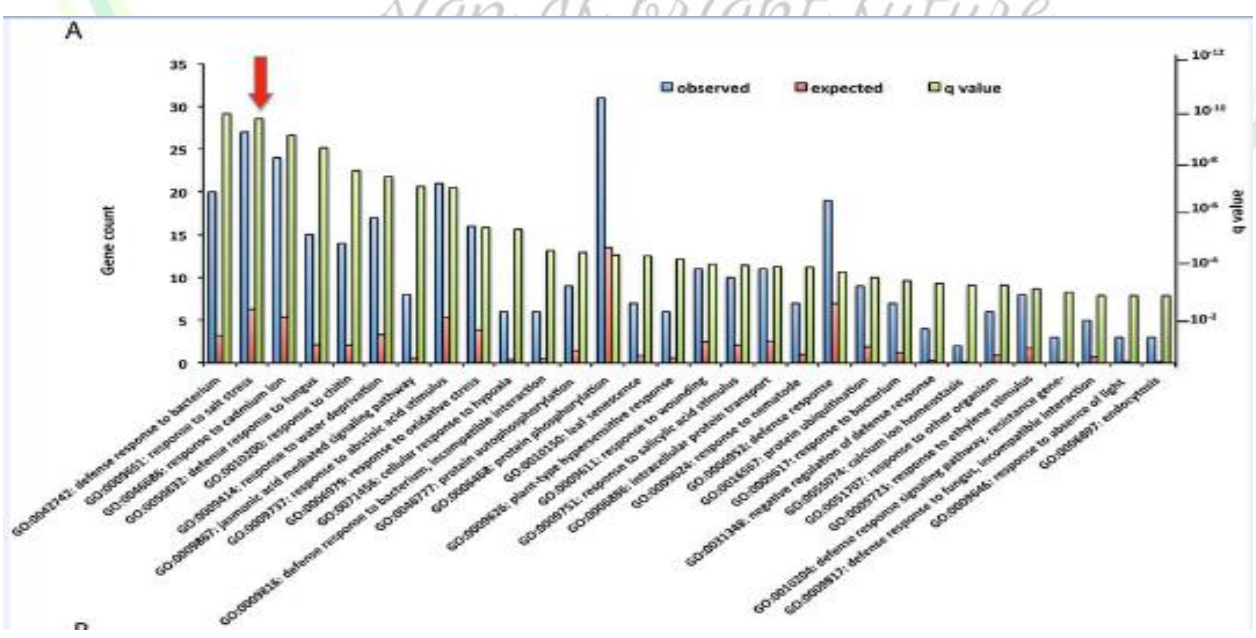
Lesson-60

GO Term and Functional Analysis

GO Term and Functional Analysis

- Gene Ontology (GO) term enrichment is a technique for interpreting sets of genes making use of the Gene Ontology system of classification, in which genes are assigned to a set of predefined bins depending on their functional characteristics
- The Gene Ontology (GO) provides a system for hierarchically classifying genes or gene products to terms organized in a graph structure called an ontology.
- The GO terms are grouped into three categories:
    - molecular function (describing the molecular activity of a gene),
    - biological process (describing the larger cellular or physiological role carried out by the gene, coordinated with other genes)
    - and cellular component (describing the location in the cell where the gene product executes its function). Each gene can be described (annotated with) multiple terms.
- Using the GO it is possible to retrieve the set of terms used to describe any gene, or conversely, given a term, return the set of genes annotated to that term.

Visualization



Lesson-61

Functional Profiling Tools in DAVID

DAVID

- David has embedded five web-based functional annotation tool suits:
  - DAVID gene functional classification Tool
  - DAVID functional annotation Tool
  - DAVID gene ID conversion Tool
  - DAVID gene name viewer
  - DAVID NIAID Pathogen Genome Browser.

DAVID functional annotation Tool

- DAVID functional annotation Tool, mainly provides typical batch annotation and gene-GO term enrichment analysis to highlight the most relevant GO terms associated with a given gene list.
- This function uses a novel algorithm to measure relationships among the annotation terms based on the degrees of their co-association genes to group the similar, redundant, and heterogeneous annotation contents from the same or different resources into annotation groups



Lesson-62

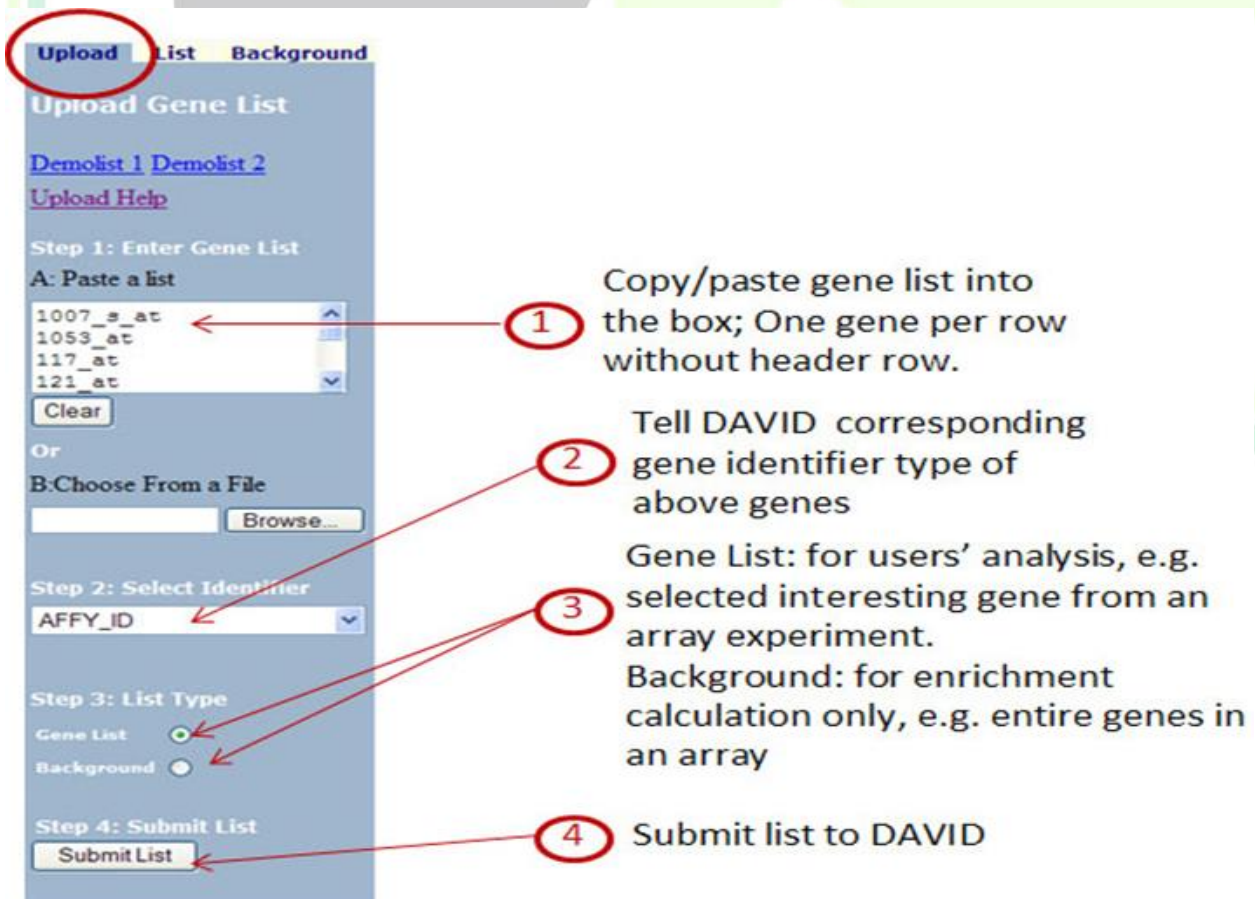
Functional Annotation Analysis Manual

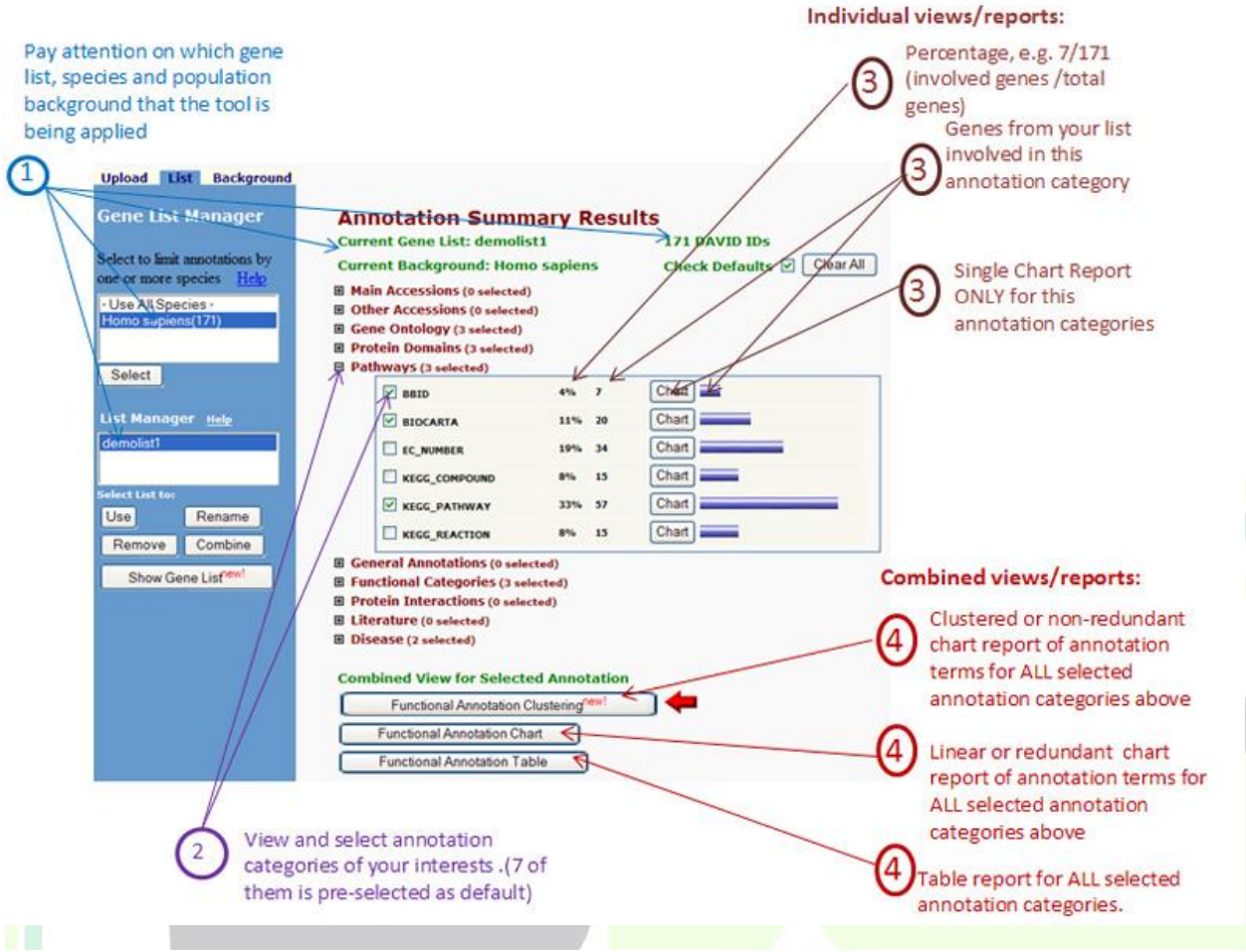
Functional Annotation Analysis

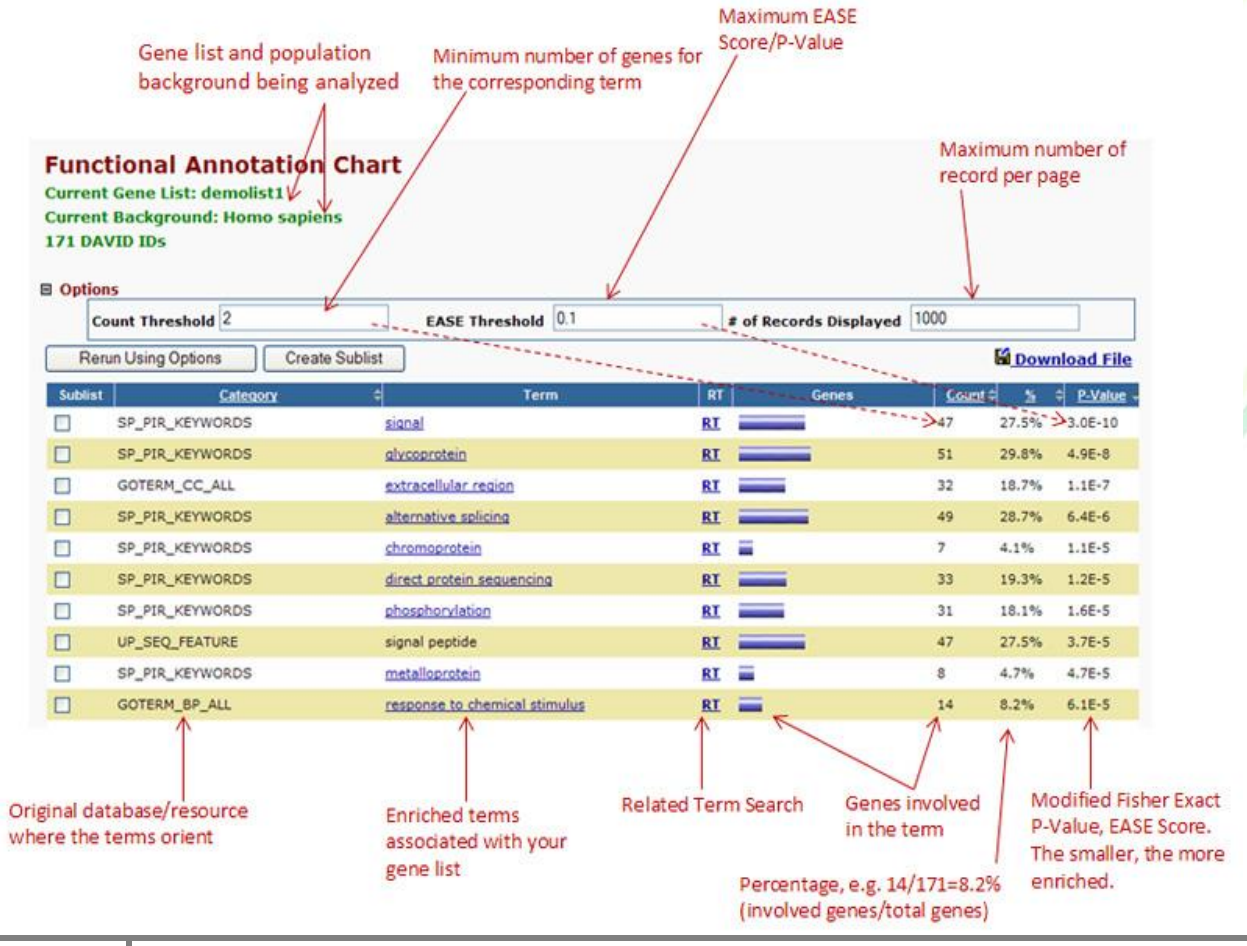


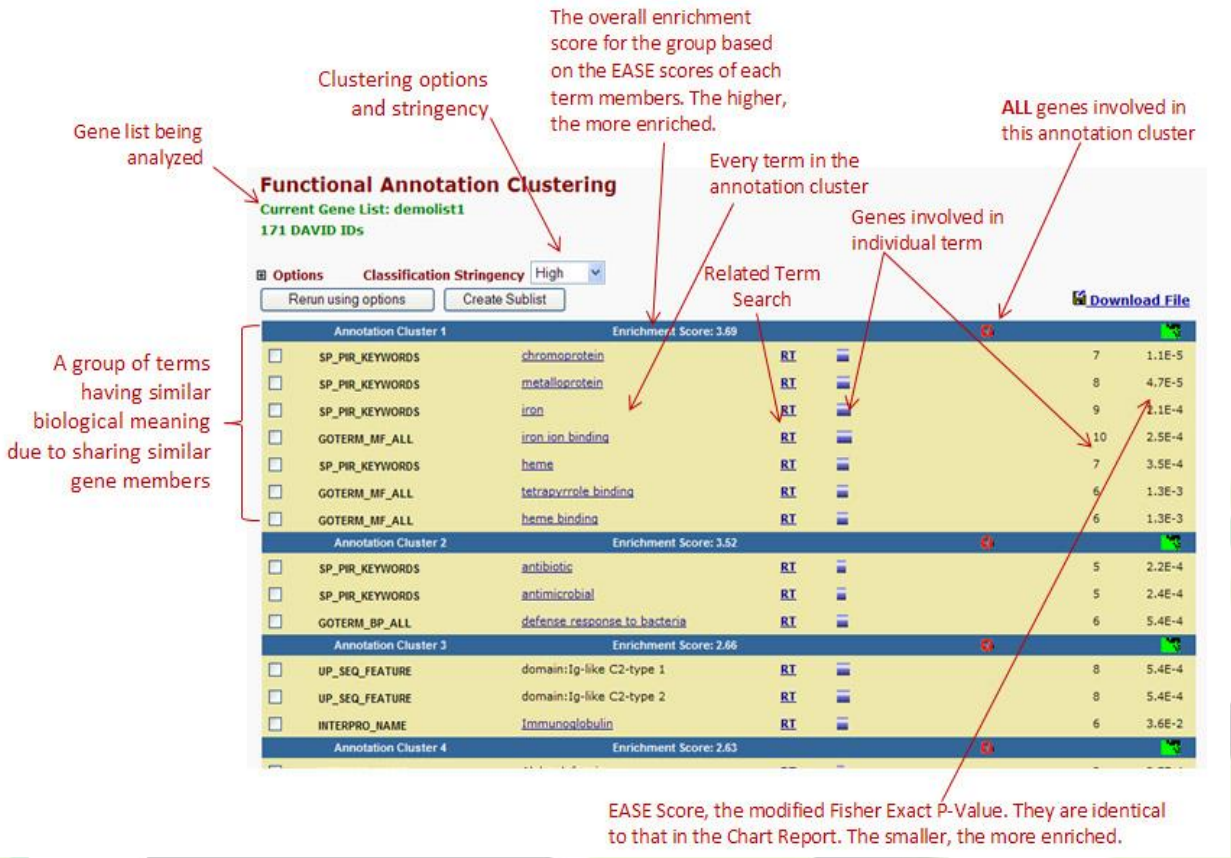
- **Workflow:**

Load Gene List -> View Summary Page -> Explore details through Chart Report, Table Report, Clustering Report, etc. -> Export and Save Results




Individual views/reports:
Pay attention on which gene list, species and population background that the tool is being applied
Percentage, e.g. 7/171 (involved genes /total genes)
Genes from your list involved in this annotation category
Single Chart Report ONLY for this annotation categories
Upload
List
Background
Gene List Manager
Select to limit annotations by one or more species Help
- Use All Species -
Homo sapiens(171)
Select
List Manager Help
demolist1
Select List to:
Use
Rename
Remove
Combine
Show Gene List new!
Annotation Summary Results
Current Gene List: demolist1
171 DAVID IDs
Current Background: Homo sapiens
Check Defaults
Clear All
Main Accessions (0 selected)
Other Accessions (0 selected)
Gene Ontology (3 selected)
Protein Domains (3 selected)
Pathways (3 selected)
BBID 4% 7 Chart
BIOCARTA 11% 20 Chart
EC_NUMBER 19% 34 Chart
KEGG_COMPOUND 8% 15 Chart
KEGG_PATHWAY 33% 57 Chart
KEGG_REACTION 8% 15 Chart
General Annotations (0 selected)
Functional Categories (3 selected)
Protein Interactions (0 selected)
Literature (0 selected)
Disease (2 selected)
Combined View for Selected Annotation
Functional Annotation Clustering new!
Functional Annotation Chart
Functional Annotation Table
Combined views/reports:
Clustered or non-redundant chart report of annotation terms for ALL selected annotation categories above
Linear or redundant chart report of annotation terms for ALL selected annotation categories above
Table report for ALL selected annotation categories.
View and select annotation categories of your interests .(7 of them is pre-selected as default)

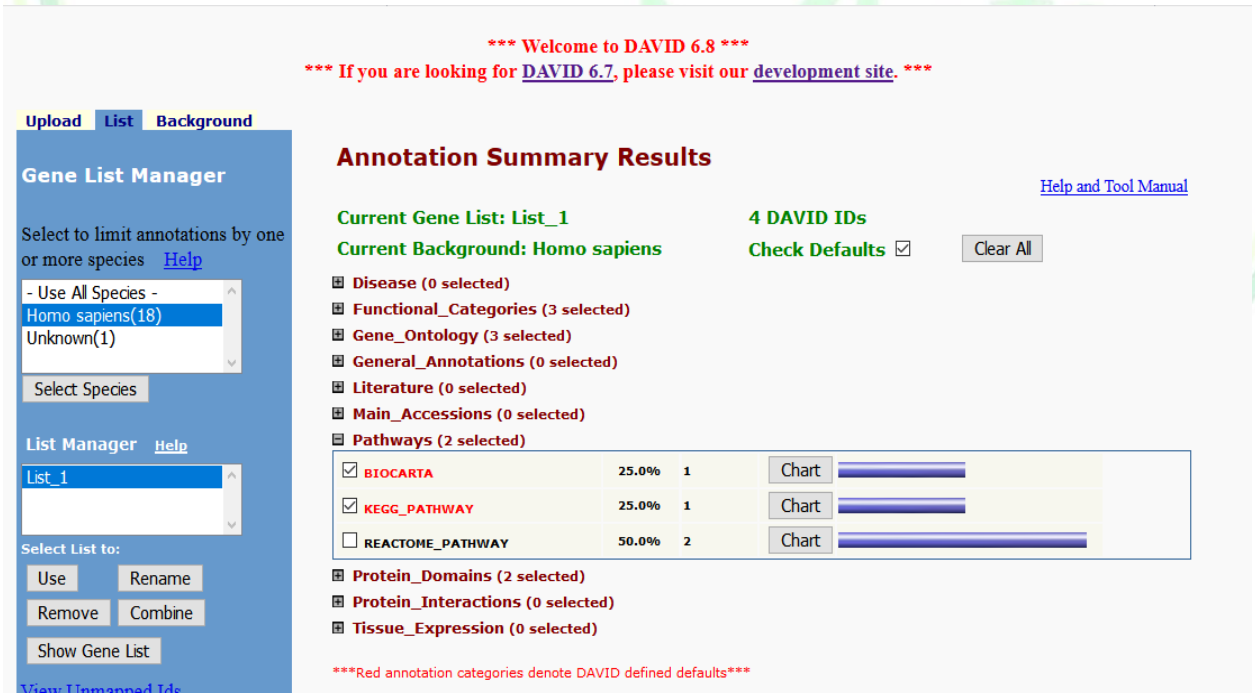

Gene list and population background being analyzed
Minimum number of genes for the corresponding term
Maximum EASE Score/P-Value
Maximum number of record per page
Functional Annotation Chart
Current Gene List: demolist1
Current Background: Homo sapiens
171 DAVID IDs
Options
Count Threshold 2
EASE Threshold 0.1
of Records Displayed 1000
Rerun Using Options
Create Sublist
Download File
Sublist | Category | Term | RT | Genes | Count | % | P-Value
SP_PIR_KEYWORDS | signal | RT | 47 | 27.5% | 3.0E-10
SP_PIR_KEYWORDS | glycoprotein | RT | 51 | 29.8% | 4.9E-8
GOTERM_CC_ALL | extracellular region | RT | 32 | 18.7% | 1.1E-7
SP_PIR_KEYWORDS | alternative splicing | RT | 49 | 28.7% | 6.4E-6
SP_PIR_KEYWORDS | chromoprotein | RT | 7 | 4.1% | 1.1E-5
SP_PIR_KEYWORDS | direct protein sequencing | RT | 33 | 19.3% | 1.2E-5
SP_PIR_KEYWORDS | phosphorylation | RT | 31 | 18.1% | 1.6E-5
UP_SEQ_FEATURE | signal peptide | RT | 47 | 27.5% | 3.7E-5
SP_PIR_KEYWORDS | metalloprotein | RT | 8 | 4.7% | 4.7E-5
GOTERM_BP_ALL | response to chemical stimulus | RT | 14 | 8.2% | 6.1E-5
Original database/resource where the terms orient
Enriched terms associated with your gene list
Related Term Search
Genes involved in the term
Percentage, e.g. 14/171=8.2% (involved genes/total genes)
Modified Fisher Exact P-Value, EASE Score. The smaller, the more enriched.



Lesson-63

KEGG Pathway

KEGG Pathway



*** Welcome to DAVID 6.8 ***
*** If you are looking for DAVID 6.7, please visit our development site. ***

Functional Annotation Table

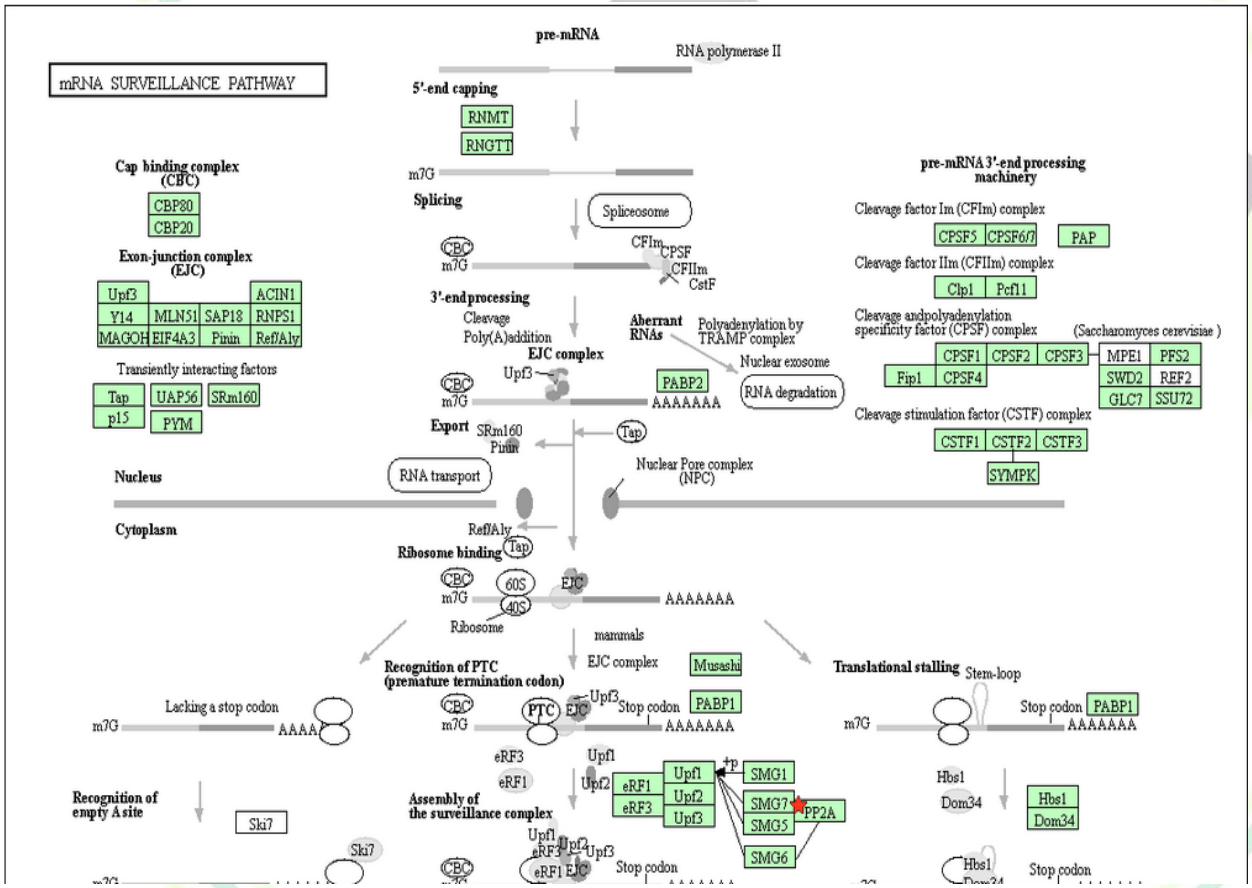
Help and Manual

Current Gene List: List_1
Current Background: Homo sapiens
4 DAVID IDs

1 record(s)

Download File

ENST00000477110, ENST00000381625, ENST00000468169, ENST00000496630, ENST00000390665, ENST00000479438, ENST00000445792	protein phosphatase 2 regulatory subunit B"beta(PPP2R3B)	Related Genes	Homo sapiens
KEGG_PATHWAY	mRNA surveillance pathway, Sphingolipid signaling pathway, PI3K-Akt signaling pathway, AMPK signaling pathway, Adrenergic signaling in cardiomyocytes, Dopaminergic synapse,		



Lesson-64

Gene Network-I

Gene Network

A gene interaction network is a set of genes (nodes) connected by edges representing functional relationships among these genes.

- These edges are named interactions, since the two given genes are thought to have either a physical interaction through their gene products, e.g., proteins, or one of the genes alters or affects the activity of other gene of interest.

- The Gene Regulatory Network runs in the cell to govern the gene expression levels of mRNA and proteins.
- These play a central role in various essential processes of cell and ultimately the body including, morphogenesis, the creation of body structures.

Regulators

- The regulators of these networks depends on the nature of process,
- A regulator can be;
  - DNA
  - RNA
  - Protein
- OR complexes on any of these correspondently

Interactions

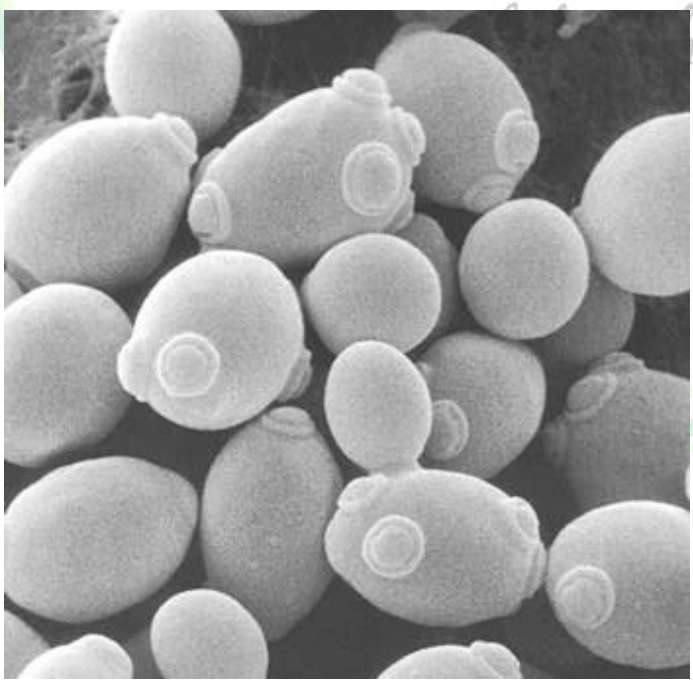
- The interactions between the entities can be both direct or indirect,

For Example,

each mRNA molecule goes on to make a specific protein (or set of proteins). In some cases this protein will be structural, and will accumulate at the cell membrane or within the cell to give it particular structural properties.

Uni-cellular organisms

- The regulatory networks respond to the external environment, optimising the cell at a given time for survival in this environment.
- Yeast cell, finding itself in a sugar solution, will turn on genes to make enzymes that process the sugar to alcohol



Multi-cellular Organisms

- In multicellular animals the same principle has been put in the service of gene cascades that control body-shape.
- Each time a cell divides, two cells result which, although they contain the same genome in full, can differ in which genes are turned on and making proteins.

Studying Gene Networks

- The functional products of genes, e.g., proteins, work together to achieve a particular task, and they often physically associate with each other to function or to form a more complex structure.
- Since these interactions are important to carry out most biological processes, knowledge about interacting proteins is crucial for understanding these biological functions

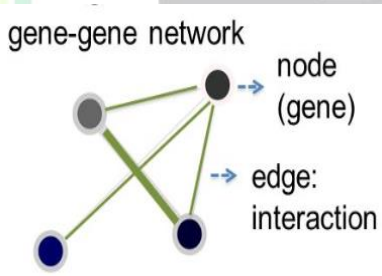
Lesson-65

Gene Network Representation

Nodes & Edges

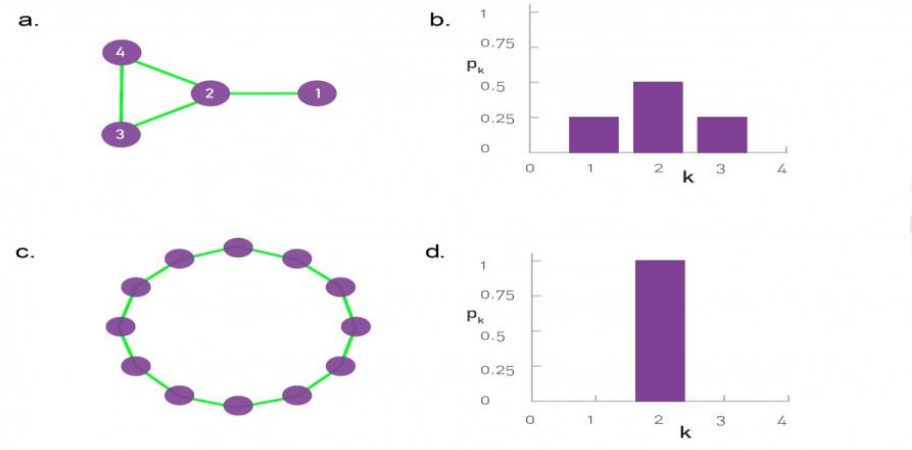
The basic components of any network are

- **Nodes;** Nodes represent units in the network,
- **Edges;** edges represent the interactions between the units



Degree of a Network

- Degree is the number of edges that connect a node, represented by k.

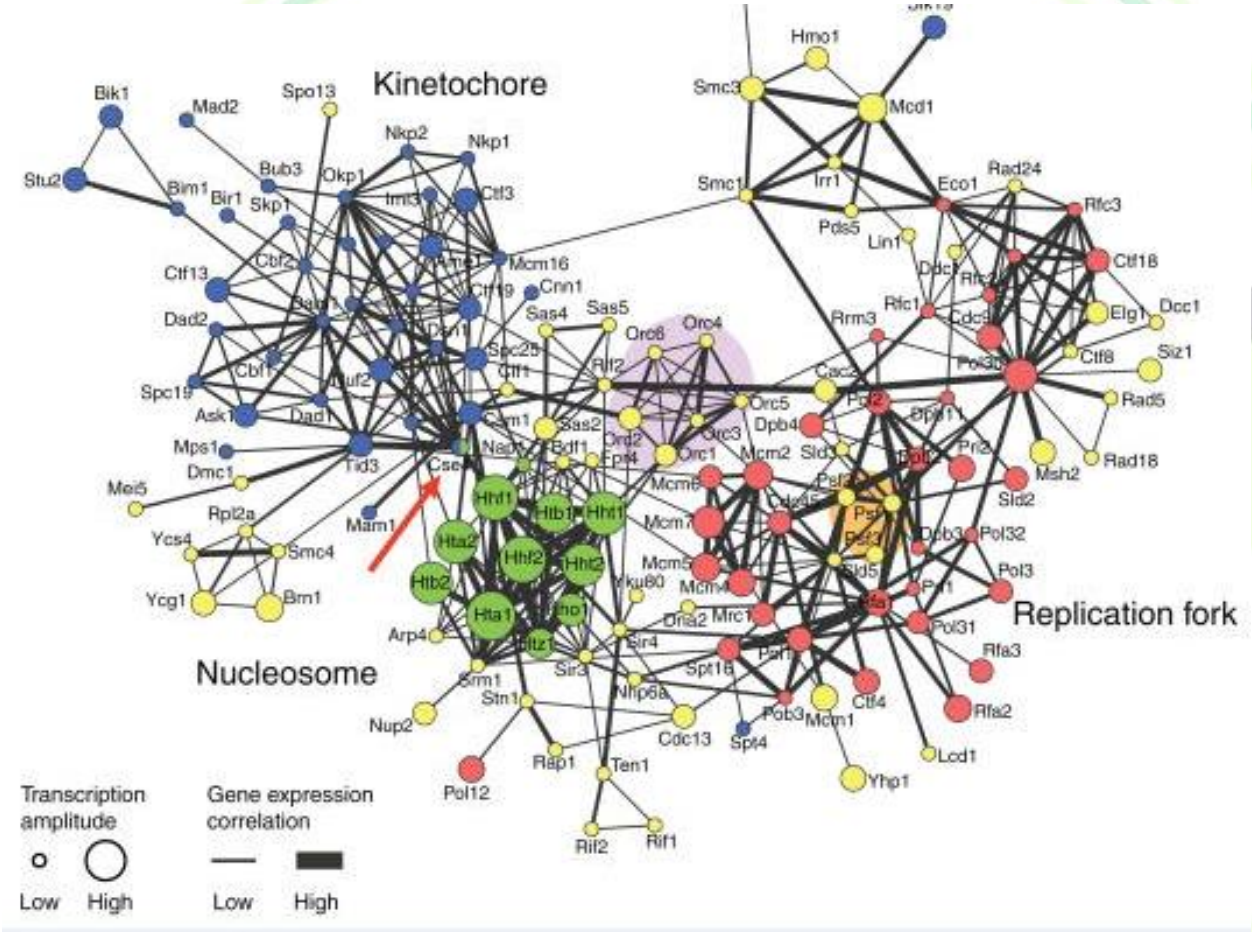


Visual Features

- The different 'visual features'—such as shapes, sizes, colors and line thicknesses depict different properties of a network.

Example

- Node color represents the subcellular localization of a gene. A gene is colored according to whether it localizes to the replication fork (red), nucleosome (green), kinetochore (blue) or other chromosome components (yellow).
- The size of a node and the thickness of an edge convey gene expression data



Example

- Simultaneously visualizing all of these attributes—localization (color), expression level (size) and expression correlation (edge thickness), helps generate consensus results about the gene networks under observation.

Lesson-66

STRING Structural and Functional Protein Networks-I

Introduction

- STRING is a database of known and predicted protein-protein interactions.
- The interactions include direct (physical) and indirect (functional) associations

Data Sources

The sources stem from computational prediction, from knowledge transfer between organisms, and from interactions aggregated from other (primary) databases.

- Genomic context predictions
- High-throughput lab experiments
- Co-expression
- Automated text-mining
- Previous knowledge in databases

How to access?

- The main entry point into the STRING website is the protein search box on its start page
- Users can also arrive via a number of external websites that maintain cross-links with STRING
- A third way to enter STRING is via logging on to the My Data section; this allows users to upload gene-lists

Main entry point



Interaction Scores

- In STRING, each protein-protein interaction is annotated with one or more 'scores'.
- These scores do not indicate the strength or the specificity of the interaction. Instead, they are indicators of confidence, i.e. how likely STRING judges an interaction to be true, given the available evidence
- The scores lie between 0-1; 1 being the highest confidence.

Example

As an example, the model organism 'Escherichia coli K12 MG1655' is shown below — indicating the number of interactions per score type, at a confidence of 'medium' or better (score >= 0.400);

gene neighborhood, normal:	7851 interactions
gene neighborhood, transferred:	11177 interactions
gene fusion:	514 interactions
gene cooccurrence:	35497 interactions
gene coexpression, normal:	12376 interactions
gene coexpression, transferred:	3154 interactions
experiments/biochemistry, normal:	5301 interactions
experiments/biochemistry, transferred:	4113 interactions
annotated pathways, normal:	6726 interactions
annotated pathways, transferred:	1727 interactions
textmining, normal:	27445 interactions
textmining, transferred:	7119 interactions
combined-score, total:	*210914 interactions*

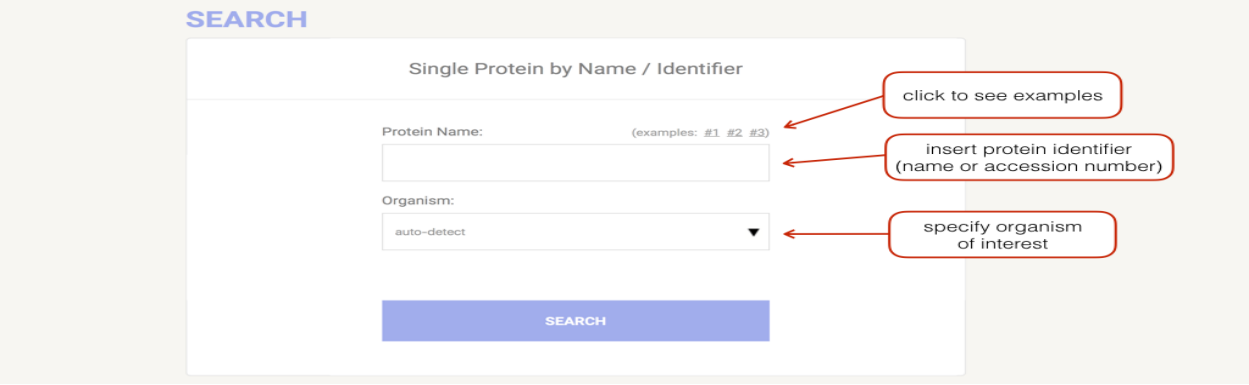
Lesson-67

STRING Structural and Functional Protein Networks-II

Starting Point

To specify your desired starting point of the analysis you have to use the input form at the STRING start page.

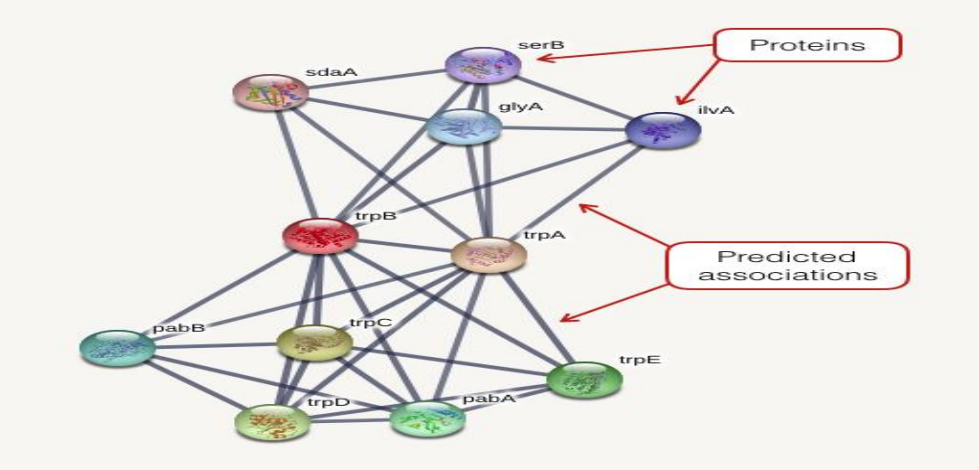
- Protein by name
- Protein by sequence
- Multiple proteins
- Multiple sequences
- Organisms
- Protein families (COGs)
- Examples
- Random entry



You can search STRING by single protein name, multiple names or by amino acid sequence (in any format)

Network

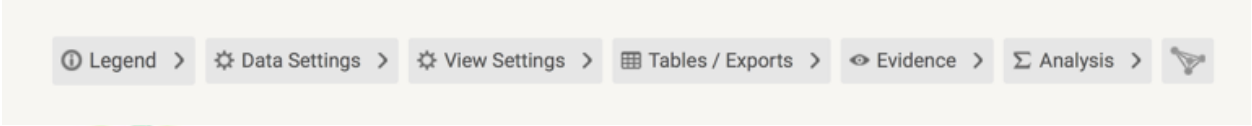
- The network view summarizes the network of predicted associations for a particular group of proteins.
- The network nodes are proteins whereas the edges represent the predicted functional associations.
- Clicking on a node gives several details about the protein. Clicking on an edge displays a detailed evidence breakdown.



Modes

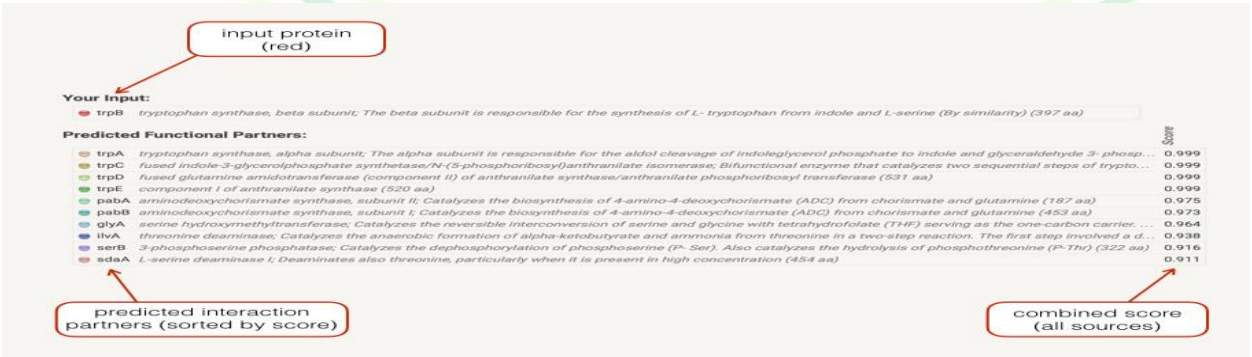
- In *evidence mode*, an edge may be drawn with up to 7 differently colored lines - these lines represent the existence of the seven types of evidence used in predicting the associations.
- In *confidence* mode the thickness of the line indicate the degree of confidence prediction of the interaction.
- *Action* mode show additional information about the prediction, such as, binding, activation, etc.

Navigation Buttons



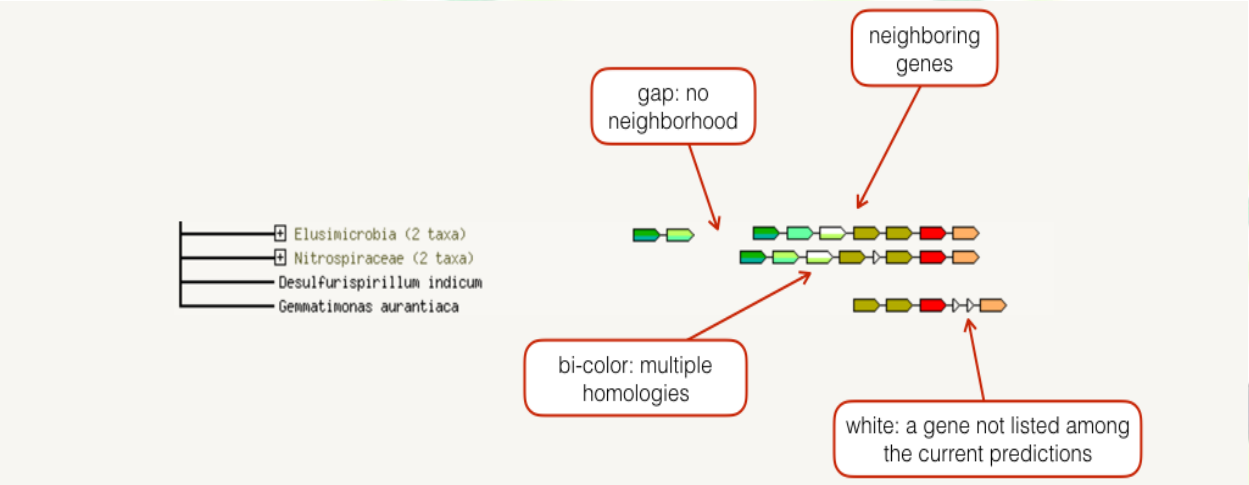
The navigation takes you to different aspects of the data, allowing you change parameters and to see the different types of evidence that supports the predicted associations.

Legend



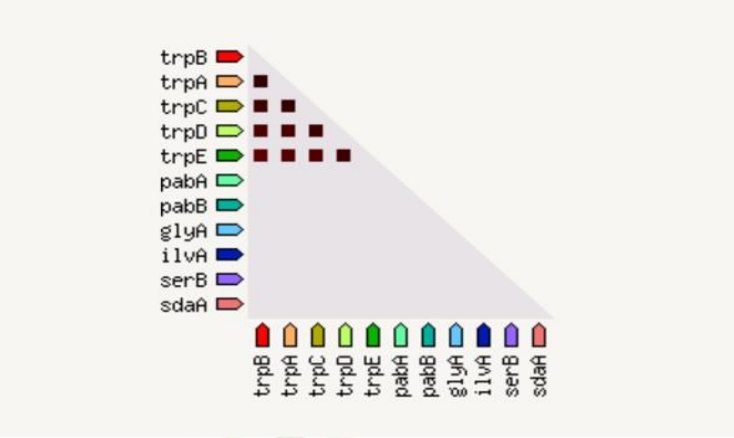
- In the Legend section a list your input(s) is shown. Predicted associations are shown immediately in a list below your input, sorted by score.
- Clicking on the score bullets gives you a breakdown of the individual prediction method scores.
- Clicking on a gene name gives you the protein sequence as well as a list of similar proteins in STRING.

Conserved Neighborhood



Co-expression

- The coexpression view shows the genes that are co-expressed in the same or in other species.
- Co-expression is shown by a red square: more intense color of the square represent a higher association score of the expression data.

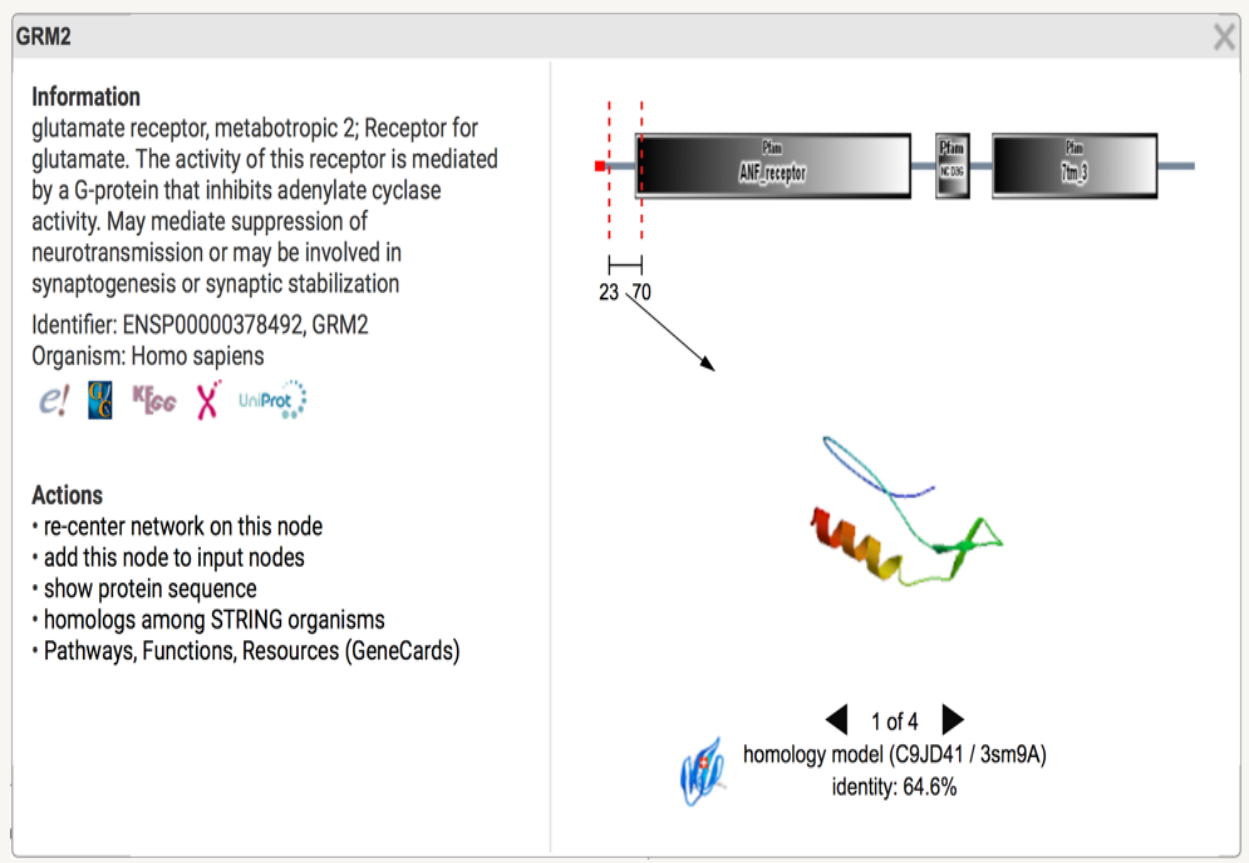


Lesson-68

STRING Structural and Functional Protein Networks-III

Protein Window

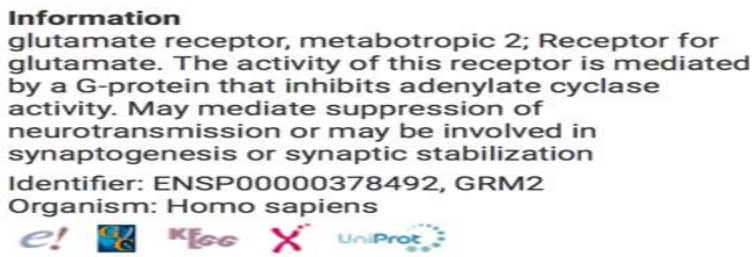
- The protein window is a popup window that appears when you click on a particular protein.
- The protein window shows information about the protein and also provide links to other sections of STRING or to other external websites.



- The protein window is divided in two parts: left and right.
- The right part is displayed only when we have the protein structure (or a model of the protein structure) stored in our database

Information

- A description of the protein is shown.
- The identifier of the protein generally correspond to the identifiers of the import source of the genomic information
- The organism is indicated for clarity.
- Also links to other resources are provided (e.g., UniProt, Ensembl, etc.).



Actions

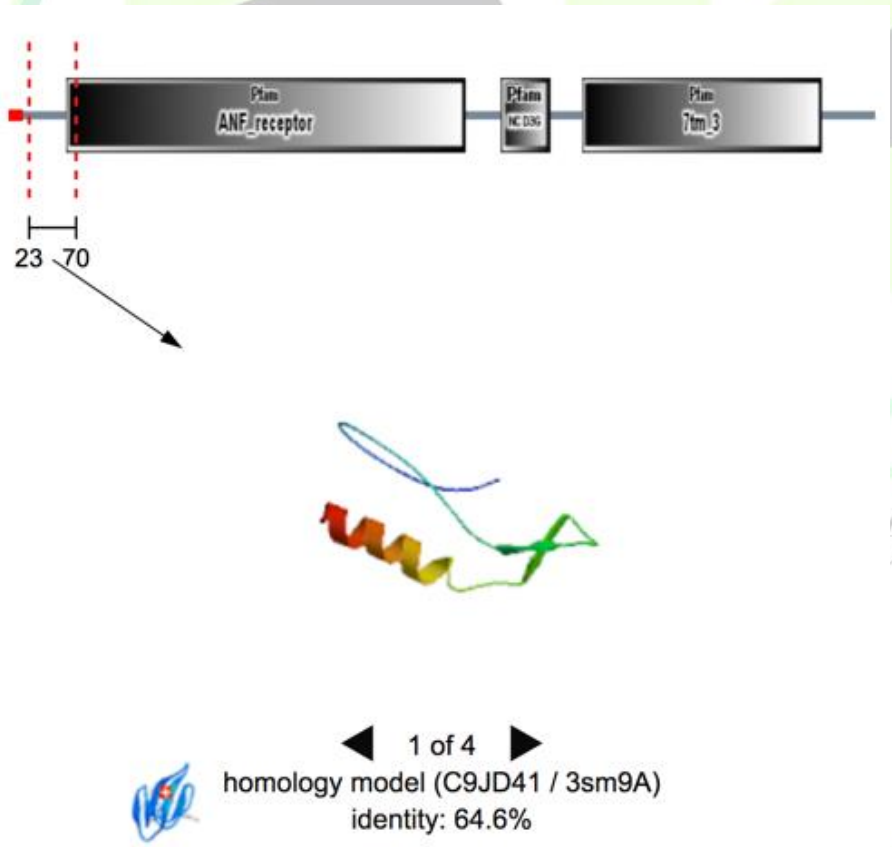
- The action are available are:
- *recenter the network on this node* - adjusts the network to this protein (i.e. look for all the proteins in

STRING that interact with the protein of the window). This is a shortcut with respect to move back to the input page and relaunch STRING with the corresponding protein identifier

- *add this node to the input node* - adds to the network also the proteins that interact with the protein of the window. This is a shortcut with respect to moving back to the input page and relaunch STRING with both the identifiers (i.e. the input protein and the selected protein)
- *show protein sequence* - opens popup window containing the amino acid sequence of the protein
- *homologs in STRING* - brings to a String page where you can see all the homologs of your protein in the same and also in other organisms.
- *Pathway, Functions, Resources (GeneCards)* - redirects to the relative entry in the GeneCards database (only for human proteins).
- *show protein domain (SMART)* - redirects to the relative entry in the SMART database.

Domain and Structure

If no crystal-structure or structure model can be found in our database for the protein (or for similar proteins), this part is not shown.



- Usually the image of the protein structure represents only a part of the protein (domain).
- For the PDB structures whole protein structure is displayed, when this is available.
- In this case, hovering the mouse cursor over the image will change the image of the whole protein structure with the image of the chain

Lesson-69

Visual Features in Cytoscape

Cytoscape- Menus

File

The File menu contains basic file functionality:

File → Recent Session for opening a recently opened Cytoscape session file

File → New for creating a new network.

File → Open... for opening a Cytoscape session file.

File → Save for saving a session file.

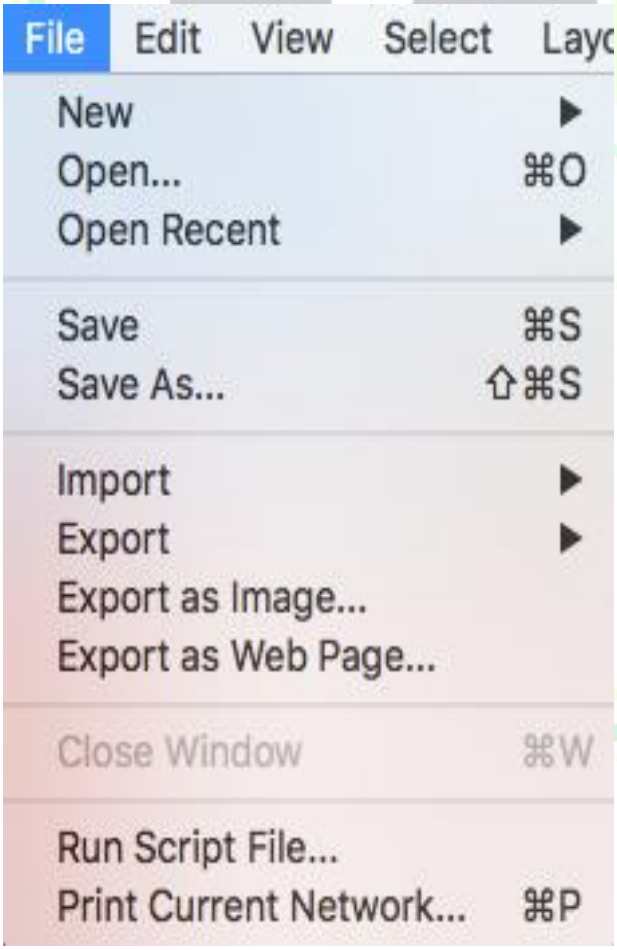
File → Save As... for saving a session file.

File → Import for importing data such as networks and table data.

File → Export for exporting data and images.

File → Run... for running a Cytoscape script file.

File → Print Current Network allows printing.



Edit

The Edit menu contains options for manipulating networks and network views:

Copy, Cut and Paste editing functions.

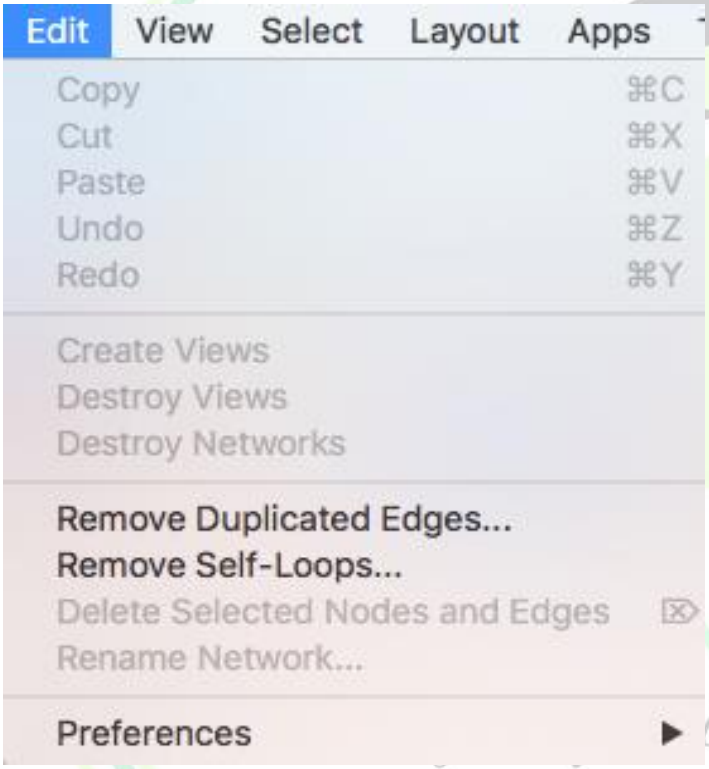
Undo and Redo functions which undo and redo edits made in the Table Panel and the Layout.

Options for creating and destroying views (graphical representations of a network) and networks, and for renaming the network

Options for deleting selected nodes and edges from the current network

Options for removing duplicated edges and self-loops.

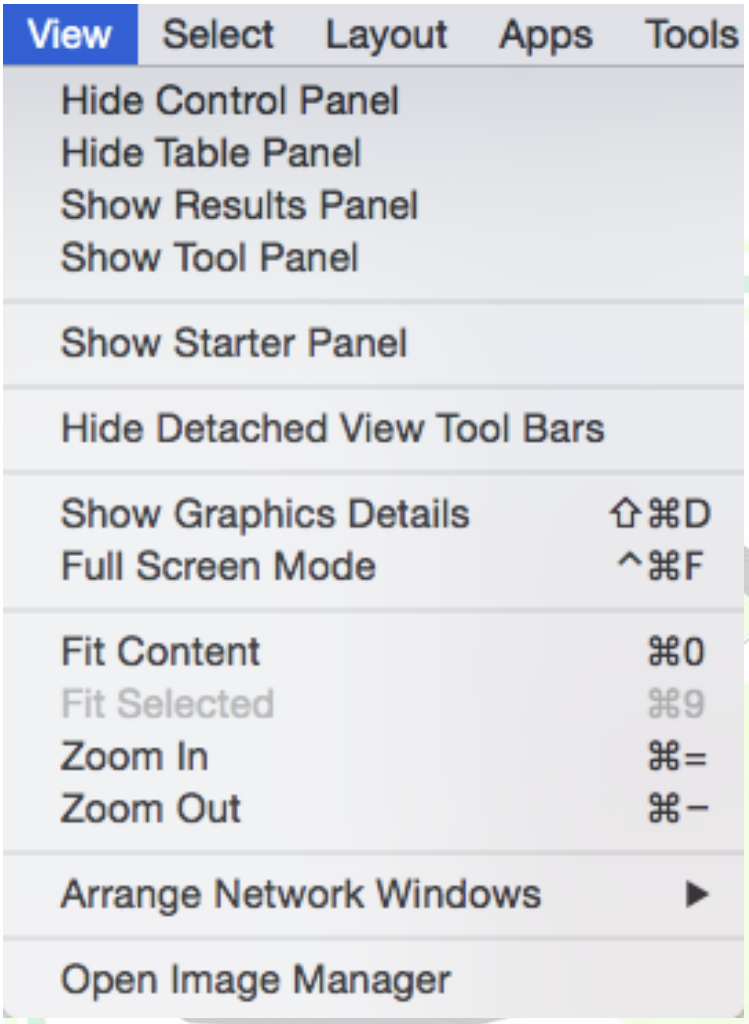
Edit → Preferences → Properties to edit preferences for properties and apps.



View

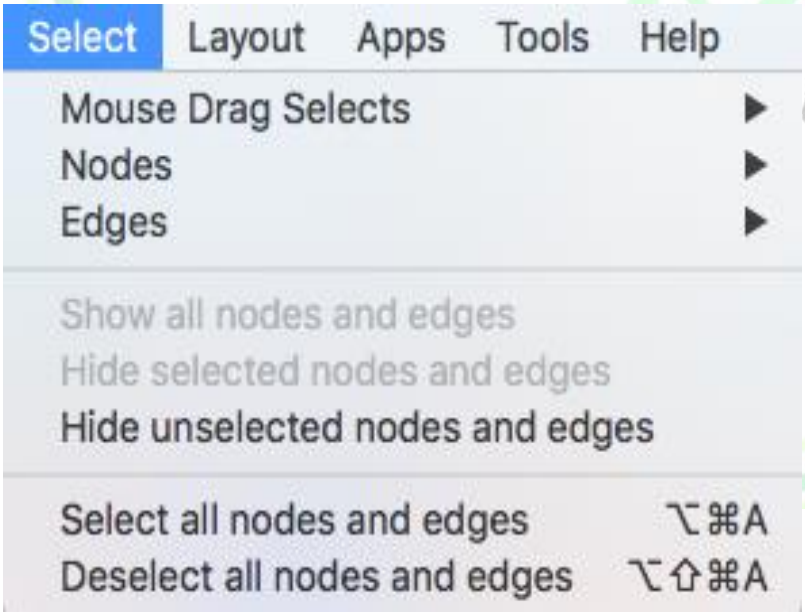
The View menu allows you to display or hide:

- The Control Panel.
- The Table Panel.
- Results Panel. When displayed, the Results Panel appears in the upper right corner of the Cytoscape window.
- Show Graphics Detail. Displays all graphics details, regardless of the current Level of Details set.
- Full Screen Mode utilizes the full display for Cytoscape.
- If multiple network windows are present, you can control how to display them under Arrange Network Windows.



Select

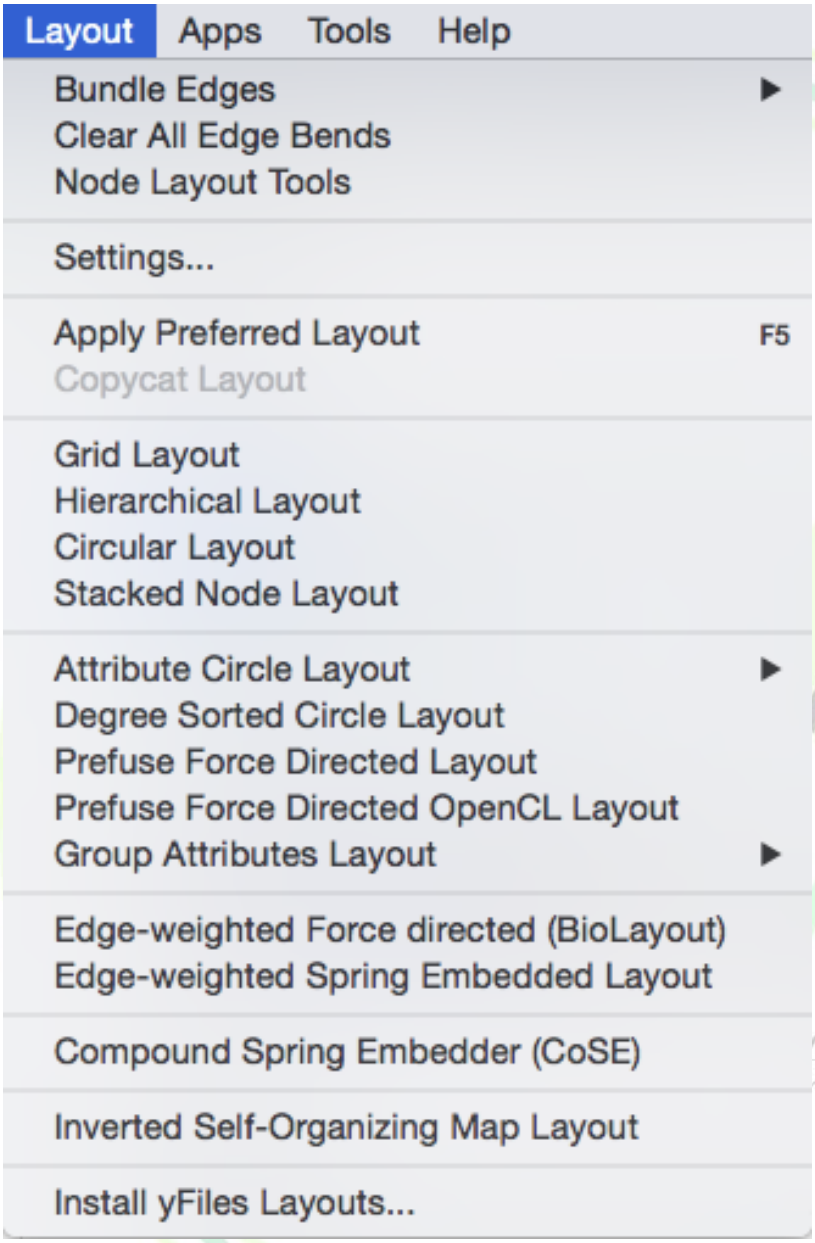
The Select menu contains different options for selecting nodes and edges.



Layout

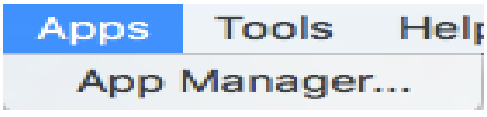
- The **Layout** menu has an array of features for visually organizing the network.

- The features in the top portion of the network (**Bundle Edges**, **Clear Edge Bends**, **Rotate**, **Scale**, **Align and Distribute**) are tools for manipulating the network visualization.
- The bottom section of the menu lists a variety of layout algorithms which automatically lay a network out.



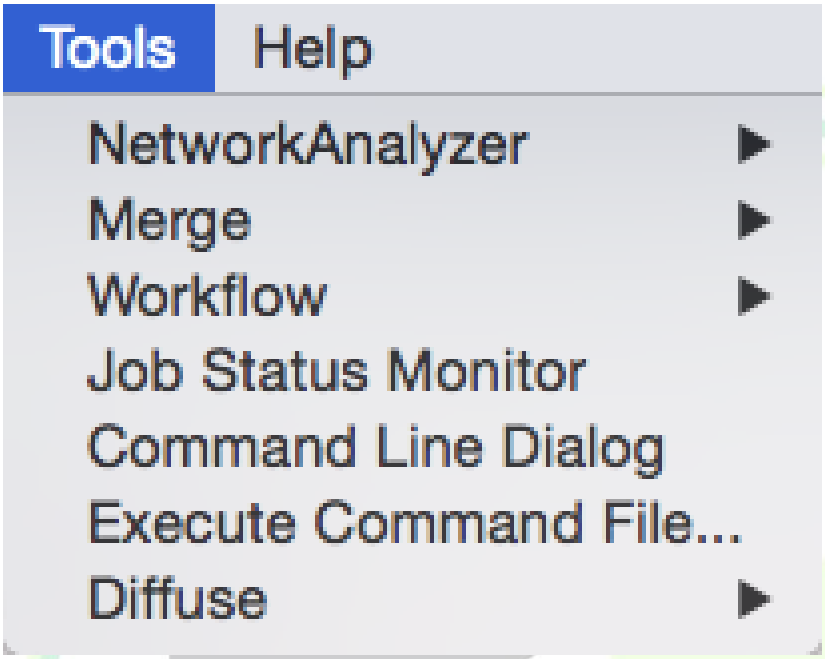
Apps

- The **Apps** menu gives you access to the **App Manager** (**Apps → App Manager**) for managing (install/update/delete) your apps and may have options added by apps that have been installed.
- Depending on which apps are loaded, the apps that you see may be different than what appear here.
- The picture shows a Cytoscape installation without installed apps.



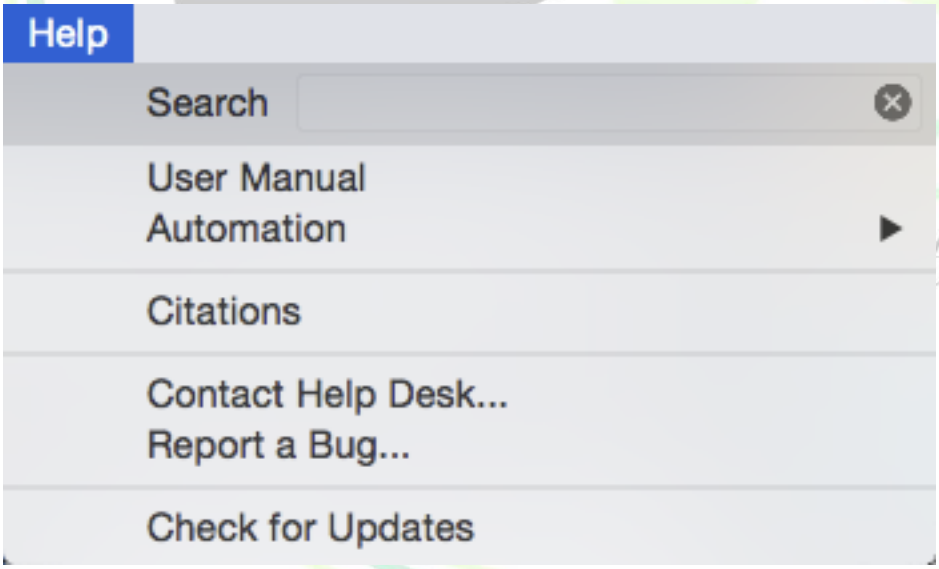
Tools

The **Tools** menu contains advanced features like the **Command Line Dialog**, **Network Analyzer**, **Network Merge** and **Workflow**.



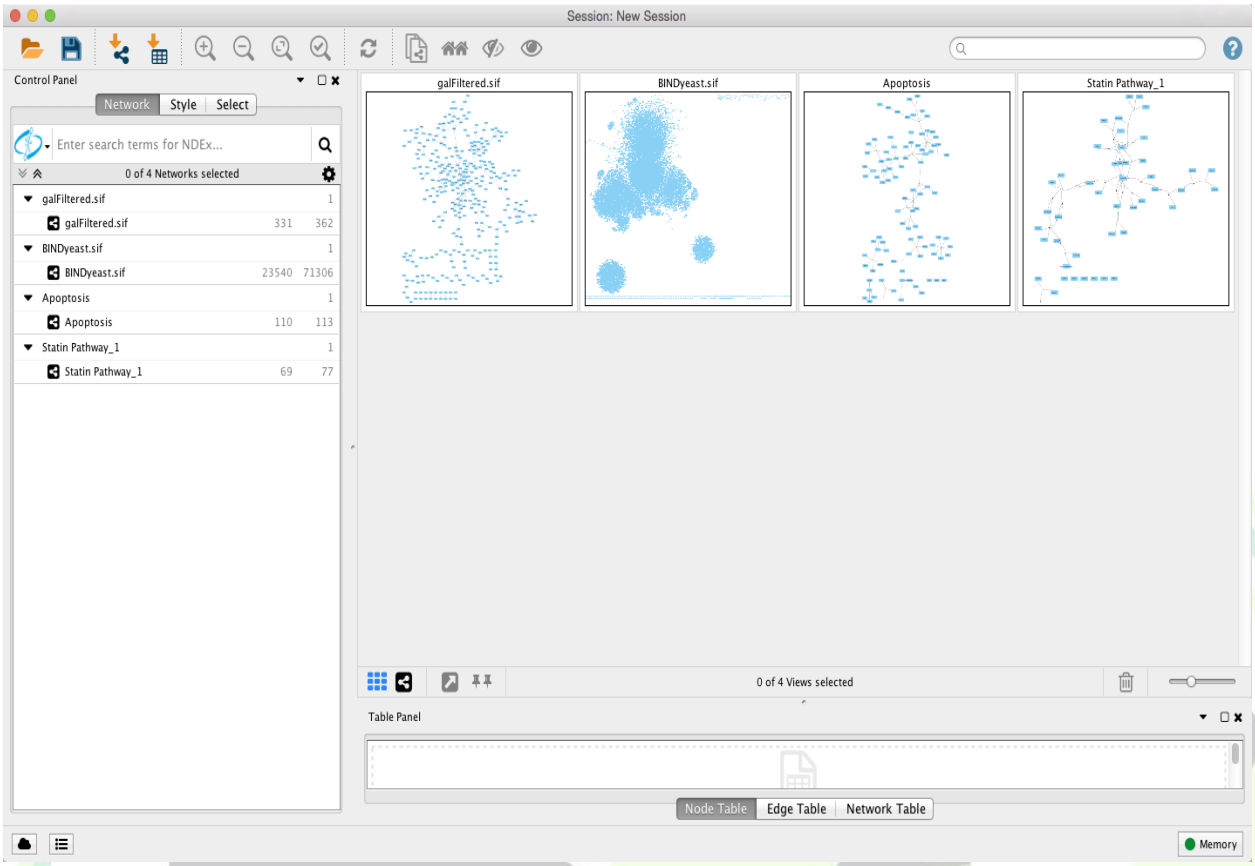
Help

The **Help** menu allows you to launch the online help viewer and browse the table of contents for this manual (**Contents**).



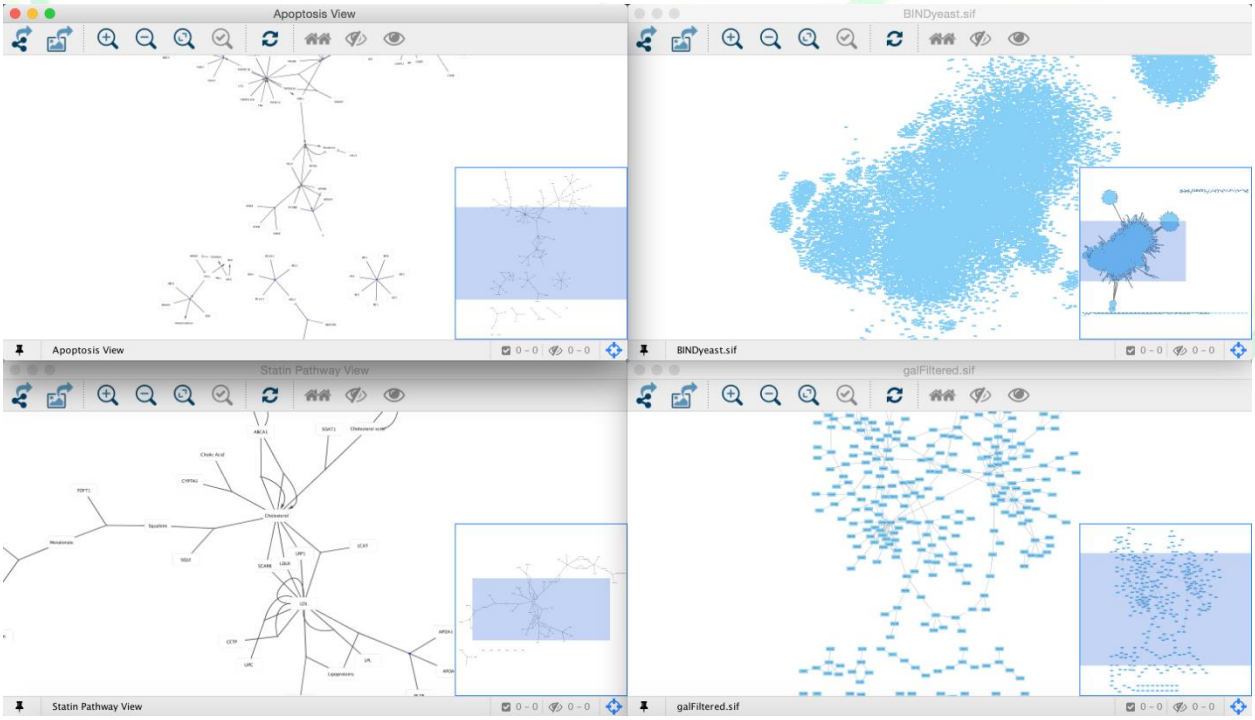
Network Management

- Cytoscape allows multiple networks to be loaded at a time, either with or without a view.
- A network stores all the nodes and edges that are loaded by the user and a view displays them.



Arrange Network Windows

- When you have detached network view windows, you can arrange them by selecting one of these options under **View → Arrange Network Windows**



Lesson-70

Visual Features in Cytoscape-II

Creating Networks

There are 4 different ways of creating networks in Cytoscape:

- Importing pre-existing, fixed-format network files.
- Importing pre-existing, unformatted text or Excel files.
- Importing data from from public databases.
- Creating an empty network and manually adding nodes and edges.

1.Import Fixed-Format Network Files

Network files can be specified in any of the formats described in the **Supported Network Formats** section.

Networks are imported into Cytoscape through the

File → Import → Network menu.

The network file can either be located directly on the local computer, or found on a remote computer

2. Import Networks from Unformatted Table Files

- Cytoscape supports the import of networks from delimited text files and Excel workbooks using **File → Import → Network → File...**.
- An interactive GUI allows users to specify parsing options for specified files.
- The screen provides a preview that shows how the file will be parsed given the current configuration.
- As the configuration changes, the preview updates automatically.

3. Import Networks from Public Databases

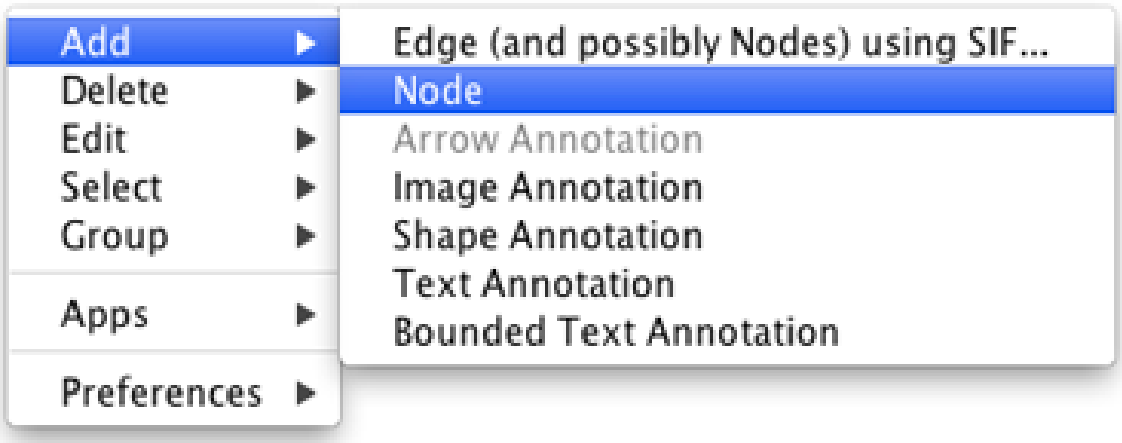
- Cytoscape has a feature called **Import Network from Public Databases**.
- Users can access various kinds of databases through this function, under **File → Import → Network → Public Databases...**.
- As of Cytoscape 3.6, a new **Search bar** is also available at the top of the **Network** panel in the **Control Panel**.
- From the search bar, you can directly access several public databases.

4. Create a New Network or Edit One Manually

- A new, empty network can also be created and nodes and edges manually added.
- To create an empty network, go to **File → New → Network → Empty Network**, and then manually add network components by right clicking on the network canvas or on a node.
- You can edit an existing network using the same process.

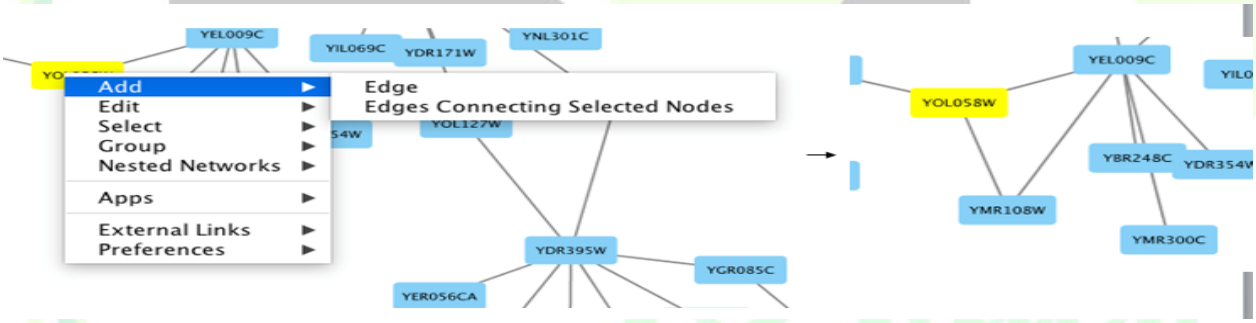
Adding a Node

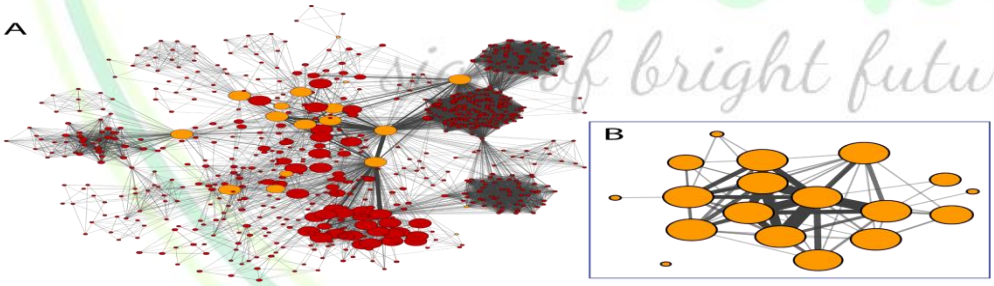
To add a new node, right-click on an empty space of the network view panel. Select Add → Node item from the pop-up menu.



Adding an Edge

- To add an edge to connect nodes, right-click on the source node. Select **Edit → Add Edge** from the pop-up menu. Next, click on the target node.





Lesson-71

Network Layout and Subnetwork

Network Layout

Manual Layout:

The simplest method to manually organize a network is to click on a node and drag it. All of the selected nodes are moved together.

In addition to the ability to click on a node and drag it to a new position, Cytoscape now has the ability to move nodes using the arrow keys on the keyboard.

By selecting one or more nodes using the mouse and clicking one of the arrow keys (←, ↑, →, ↓) the selected nodes will move one pixel in the chosen direction.

If an arrow key is pressed while holding the Shift key down, the selected nodes will move 15 pixels in the chosen direction.

Node Layout Tools

The Tool Panel is available via the menu command: View → Show Tool Panel.

It contains several Node Layout Tools that can help to automate or fine tune a layout.

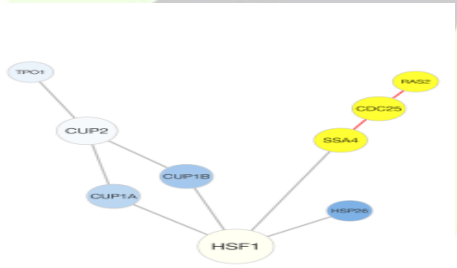
Scale:

Adjust the Scale slider to change the length of edges.

The position of the nodes will be scaled, not the node sizes.

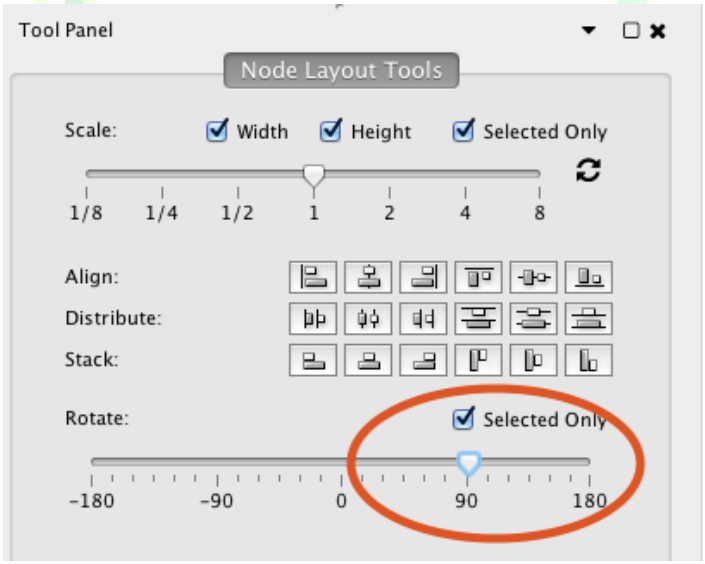
Node size can be adjusted using Styles.

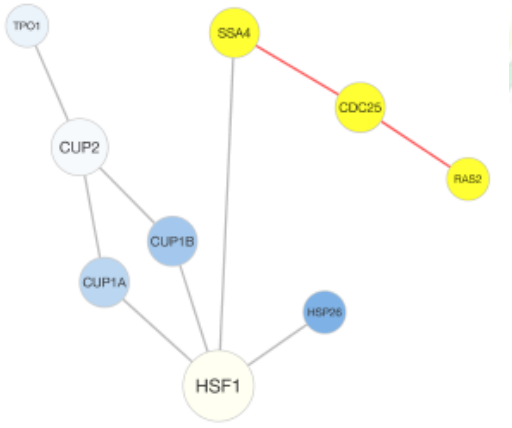




Rotate:

The Rotate function will change the orientation of the entire network or a selected portion of the network. The images below show a network with selected nodes rotated 90 degrees.





Align, Distribute and Stack

Align provides different options for either vertically or horizontally aligning selected nodes against a line.

The differences are in what part of the node gets aligned, e.g. the center of the node, the top of the node, the left side of the node.

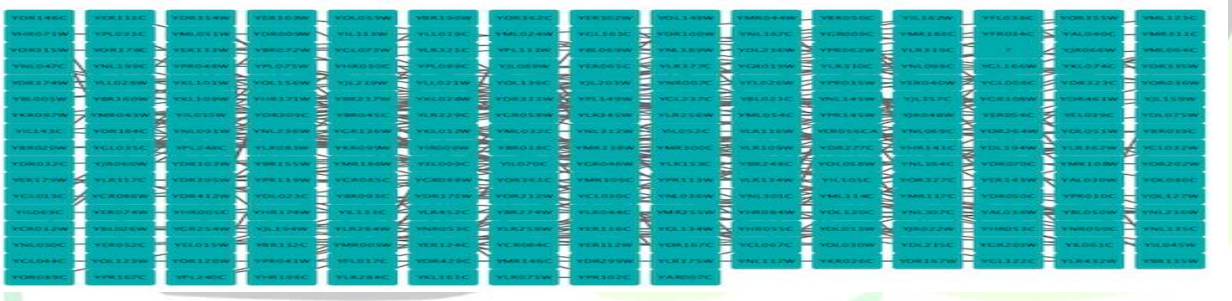
Distribute evenly distributes selected nodes between the two most distant nodes along either the vertical or horizontal axis. The differences are again a function what part of the node is used as a reference point for the distribution.

Stack vertically or horizontally stacks selected nodes with the full complement of alignment options. The table below provides a description of what each button does.

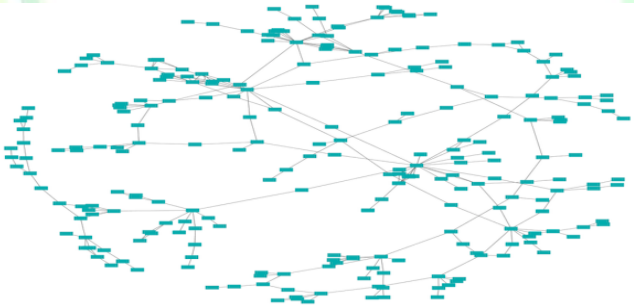
Automatic Layout Algorithms

- Cytoscape Layouts have the option to operate on only the selected nodes, and all provide a Settings... panel to change the parameters of the algorithm.
- Most of the Cytoscape layouts also partition the graph before performing the layout.
- In addition, many of these layouts include the option to take either node or edge columns into account.

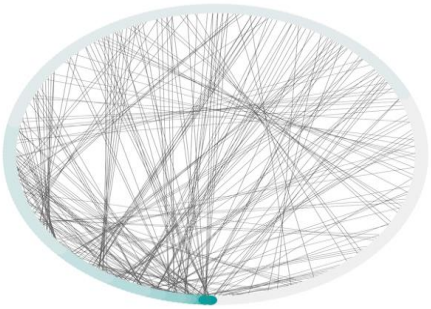
Grid Layout



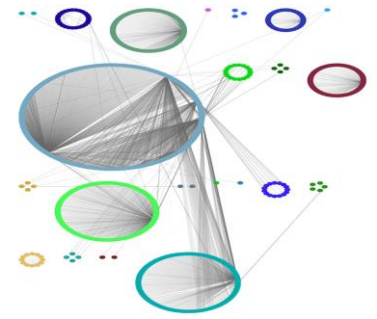
Edge-weighted Spring-Embedded Layout



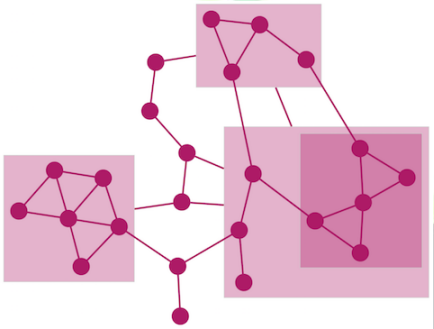
Attribute Circle Layout



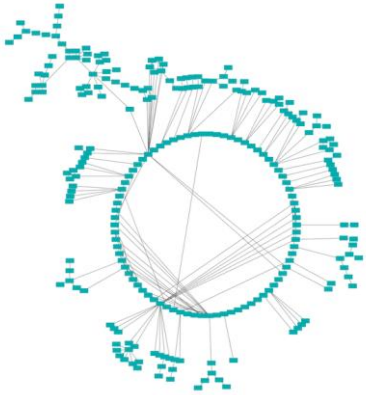
Group Attributes Layout



Compound Spring Embedder Layout



Circular Layout

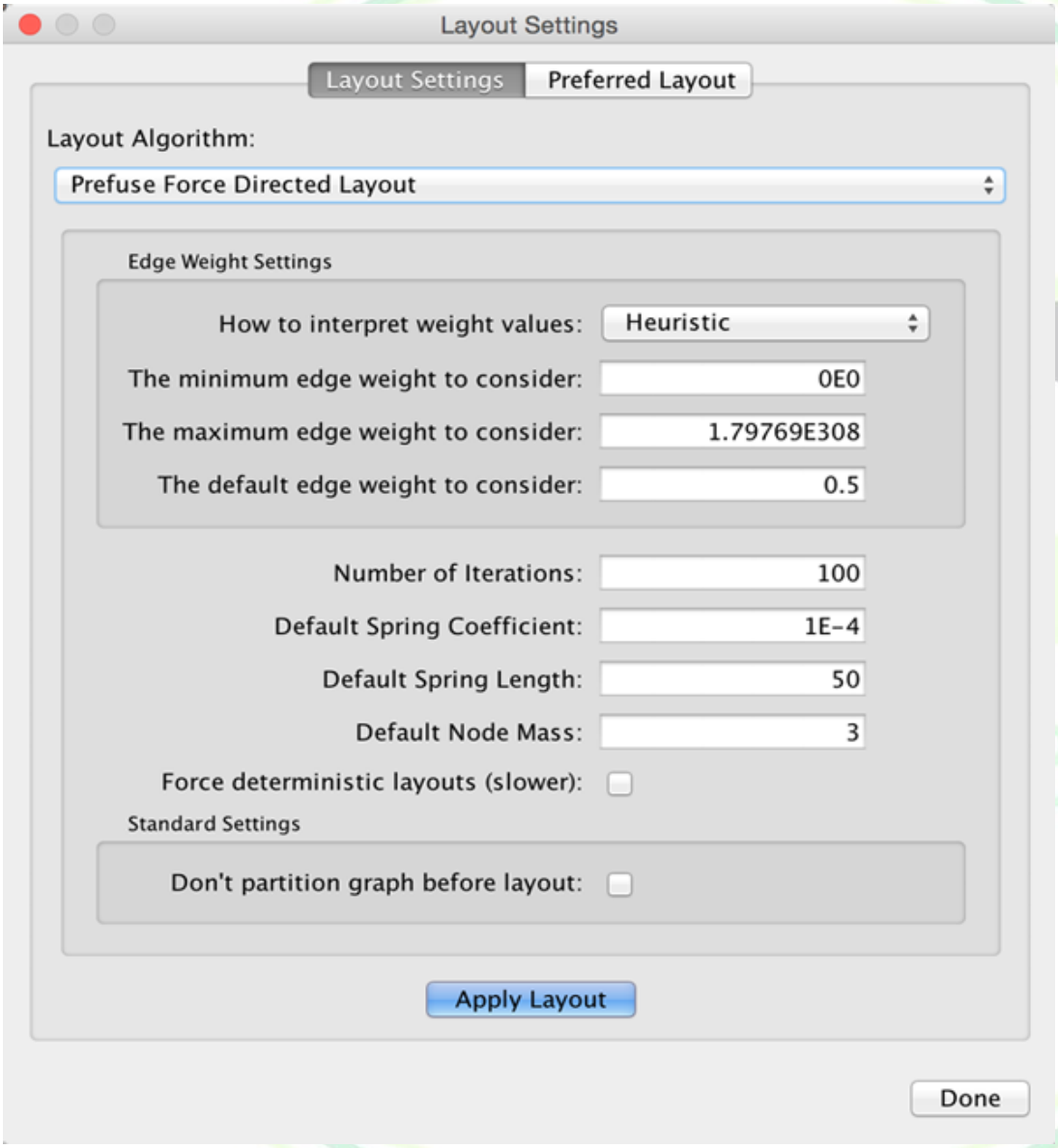


Hierarchical Layout



Layout Parameters

- Many layouts have adjustable parameters that are exposed through the Layouts → Settings... menu option.
- The Layout Settings dialog, which allows you to choose which layout algorithm settings to adjust, is shown below.
- The settings presented vary by algorithm and only those algorithms that allow access to their parameters will appear in the drop-down menu at the top of the dialog



Subnetwork Creation

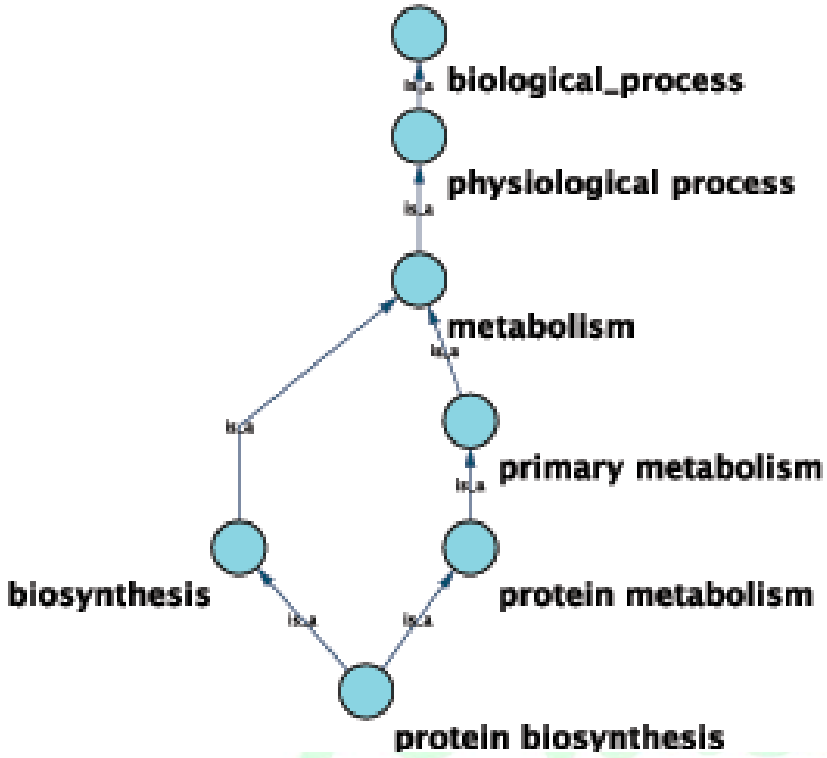
- NetworkAnalyzer allows for the creation of sub-networks of connected components.
- The user selects a number of connected components from a list and each selected component is visualized as a sub-network.
- To create sub-networks from connected components, select Tools → NetworkAnalyzer → Subnetwork Creation → Extract Connected Components.

Lesson-72

Gene Lists- Ontology and Annotation

Ontology and Annotation

- Annotations in Cytoscape are stored as a set of ontologies (e.g. the Gene Ontology, or GO).
- An ontology consists of a set of controlled vocabulary terms that annotate the objects.
- Cytoscape can use this ontology DAG (Directed Acyclic Graph) to annotate objects in networks.



Gene Association File

sign of bright future

- The **Gene Association** files provide annotation only for the Gene Ontology.
- It is a species-specific annotation file for GO terms. Gene Association files will only work with Gene Ontology annotation.

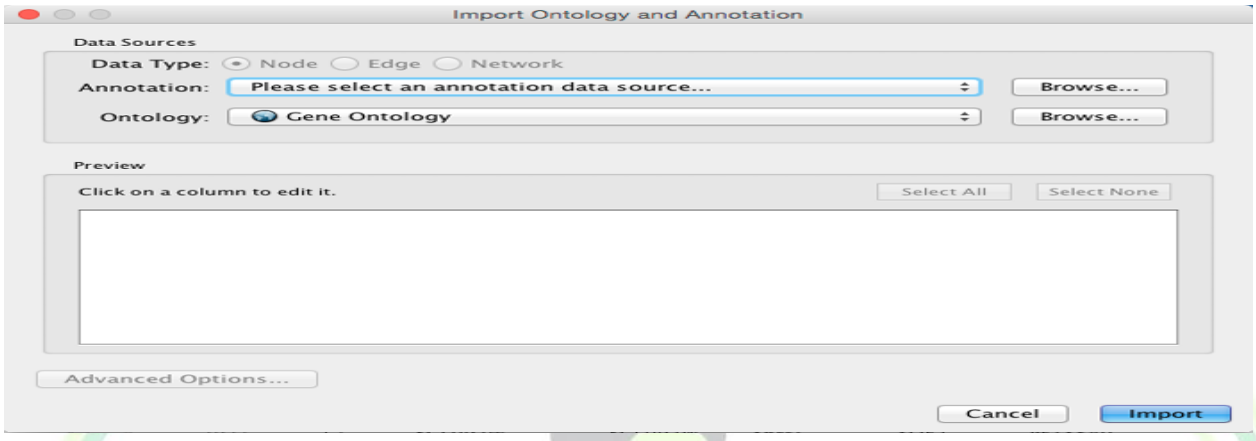
Sample Gene Association File

SGD	S000003916	AAD10	GO:0006081	SGD_REF:S000042151|PMID:10572264	ISS	P	aryl-alcohol dehydrogenase (putative)	YJR155W gene	taxon:4932
SGD	S000005275	AAD14	GO:0008372	SGD_REF:S000069584	ND	C	aryl-alcohol dehydrogenase (putative)	YNL331C gene	taxon:4932

Import Ontology and Annotation

- Select **File → Import → Ontology and Annotation...** to open the "Import Ontology and Annotation" interface.

- From the **Annotation** drop-down list, select a gene association file for your network.
- For example, if you want to annotate the yeast network, select "Gene Association file for Saccharomyces cerevisiae".



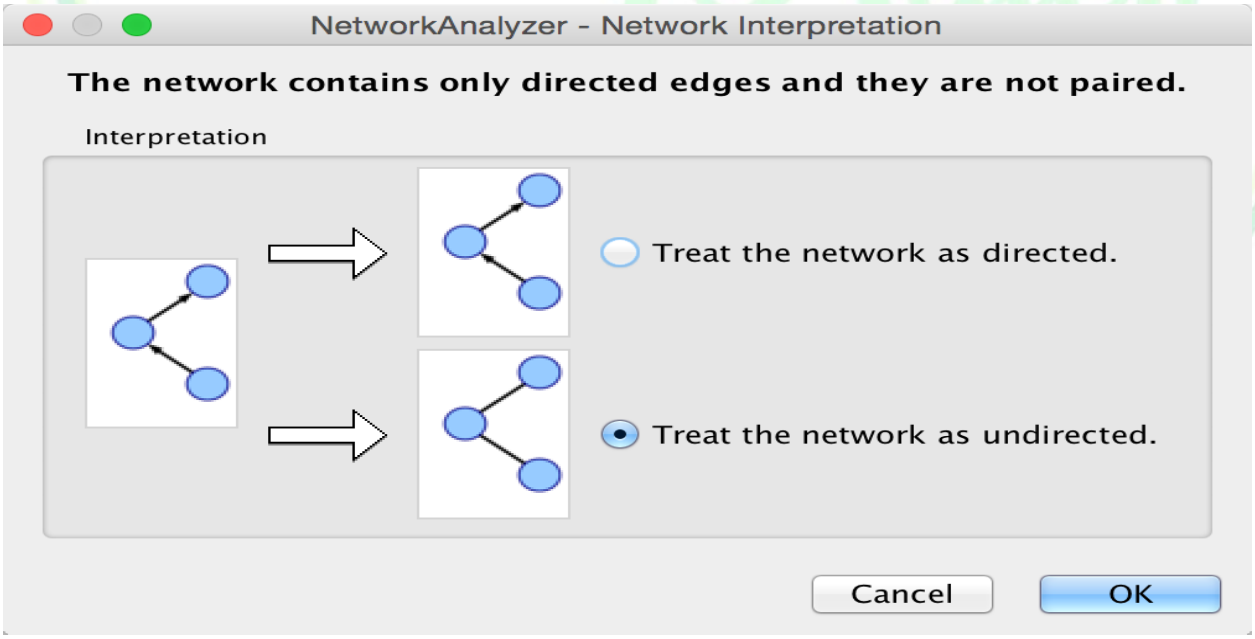
- Once you click the **Import** button, Cytoscape will start loading OBO and Gene Association files from the remote sources. If you choose GO Full it may take a while since it is a large data file.
- When Cytoscape finishes importing files, the import window will be automatically closed. All columns mapped by this function have the prefix "annotation" and look like this: annotation.[column_name].

Lesson-73

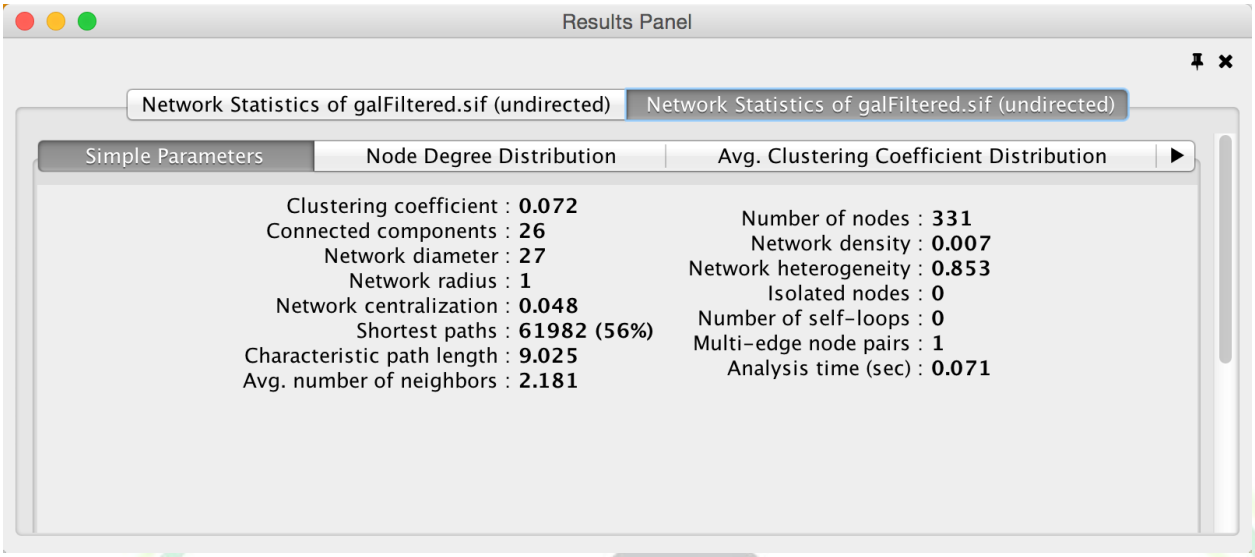
Network Analysis Overview

Network Analysis

- To run NetworkAnalyzer, select Tools → NetworkAnalyzer → Network Analysis → Analyze Network..



When results are calculated, they will appear in the Results Panel.



- The results have multiple tabs.
  - Simple Parameters
  - Node Degree Distribution
  - Avg. Clustering Coefficient Distribution
  - Topological Coefficients
  - Shortest Path Distribution
  - Shared Neighbors Distribution
  - Neighborhood Connectivity Distribution
  - Betweenness Centrality
  - Closeness Centrality
  - Stress Centrality Distribution

Analyze Subset of Nodes

- NetworkAnalyzer provides the Analyze Subset of Nodes option for computing local parameters for a subset of nodes.
- If one or more nodes in the network are selected before starting an analysis, only the sub-network induced by the selected nodes is analyzed.
- Moreover, only local parameters are computed. Shared neighbors distribution and shortest path lengths distribution, among others, are not displayed in the results.

Batch Analysis

- The Batch Analysis option is used to perform topological analysis on all networks stored in specific directory, using all possible interpretations for every network.
- Batch analysis consists of three simple steps:

- Selecting directories
- Analysis
- Inspection of results

Plot Parameters

- The Plot Parameters dialog offers a possibility to plot two parameters against each other.
- The parameters to be plotted can be chosen from two drop-down menus.
- The Table Column 1 menu provides the values for the domain/category axis, and the Table Column 2 menu specifies the values for the range/value axis.
- The plot is updated each time a different parameter is selected in one of the menus.

Lesson-74

BiNGO Plugin

BiNGO

- A Biological Network Gene Ontology tool
- BiNGO is a Java-based tool to determine which Gene Ontology (GO) categories are statistically overrepresented in a set of genes or a subgraph of a biological network.
- BiNGO is implemented as a plugin for Cytoscape, which is a an open source bioinformatics software platform for visualizing and integrating molecular interaction networks.
- BiNGO maps the predominant functional themes of a given gene set on the GO hierarchy, and outputs this mapping as a Cytoscape graph.
- Gene sets can either be selected or computed from a Cytoscape network (as subgraphs) or compiled from sources other than Cytoscape

Advantages:

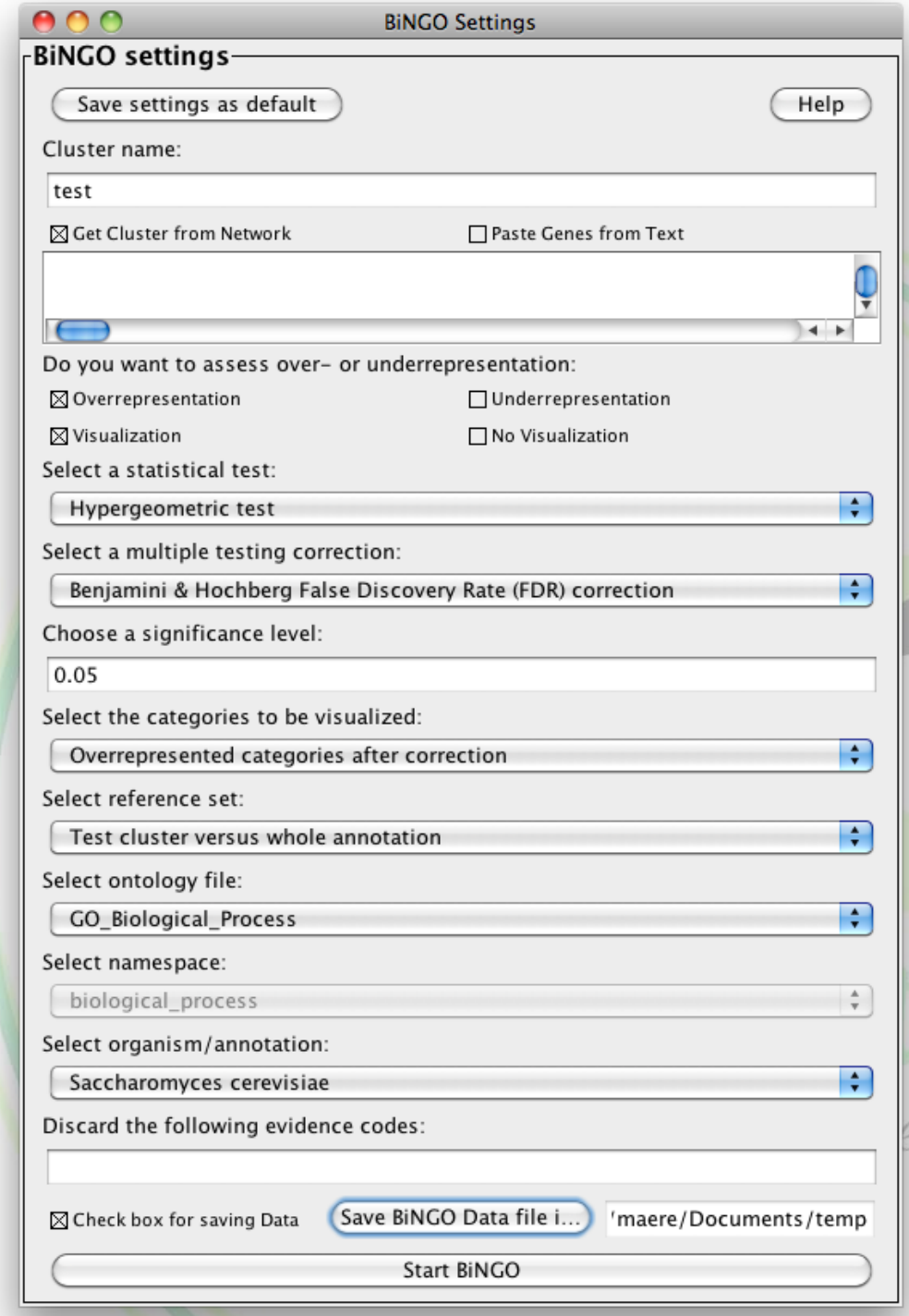
- The main advantage of BiNGO over other GO tools is the fact that it can be used directly and interactively on molecular interaction graphs.
- Another plus is that BiNGO takes full advantage of Cytoscape's versatile visualization environment.
- This allows you to produce customized high-quality figures.

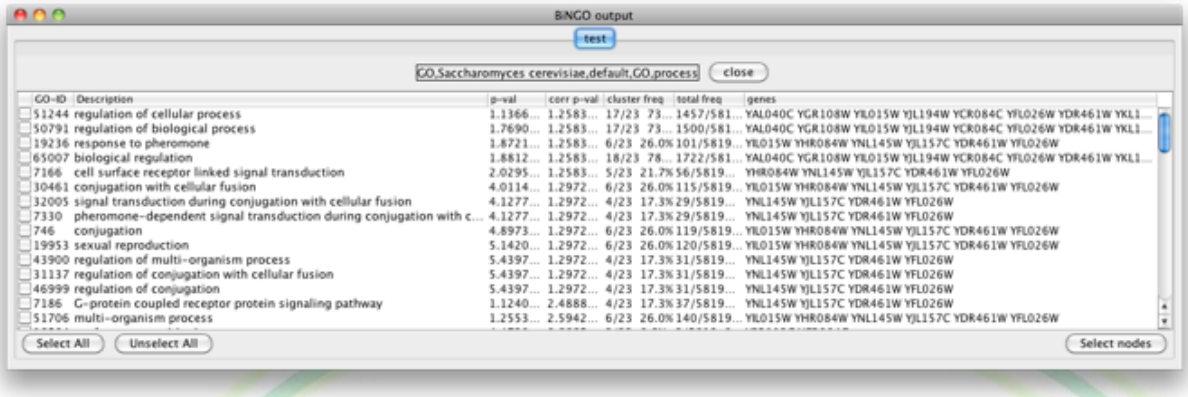
Features include :

- assessing overrepresentation or underrepresentation of GO categories
- Graph or gene list input
- batch mode : analyze several clusters simultaneously using same settings
- Different GO and GOSlim ontologies
- Wide range of organisms

- Evidence code filtering
- Hypergeometric or binomial test for overrepresentation
- Multiple testing correction using Bonferroni (FWER) or Benjamini&Hochberg (FDR) correction
- Interactive visualization of results mapped on the GO hierarchy.
- extensive results in tab-delimited text file format
- make and use custom annotations, ontologies and reference sets
- open source

Bio Mates
sign of bright future


BiNGO Settings
BiNGO settings
Save settings as default
Help
Cluster name:
test
Get Cluster from Network
Paste Genes from Text
Do you want to assess over- or underrepresentation:
Overrepresentation
Underrepresentation
Visualization
No Visualization
Select a statistical test:
Hypergeometric test
Select a multiple testing correction:
Benjamini & Hochberg False Discovery Rate (FDR) correction
Choose a significance level:
0.05
Select the categories to be visualized:
Overrepresented categories after correction
Select reference set:
Test cluster versus whole annotation
Select ontology file:
GO_Biological_Process
Select namespace:
biological_process
Select organism/annotation:
Saccharomyces cerevisiae
Discard the following evidence codes:
Check box for saving Data
Save BiNGO Data file i...
'maere/Documents/temp
Start BiNGO

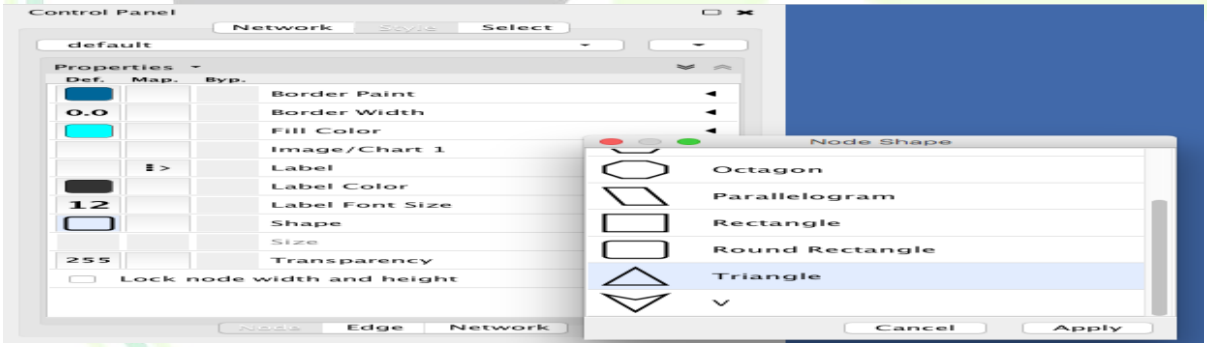


Lesson-75

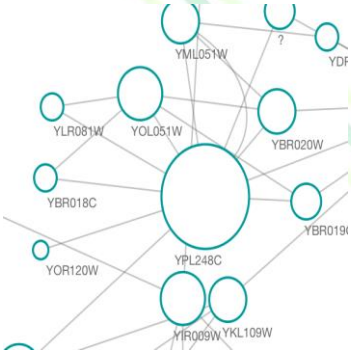
Styles

Styles

- One of Cytoscape's strengths in network visualization is the ability to allow users to encode any table data (name, type, degree, weight, expression data, etc.) as a property (such as color, size of node, transparency, or font type) of the network.
- A set of these encoded or mapped table data sets is called a Style and can be created or edited in the Style panel of the Control Panel.
- Specify a default color and shape for all nodes



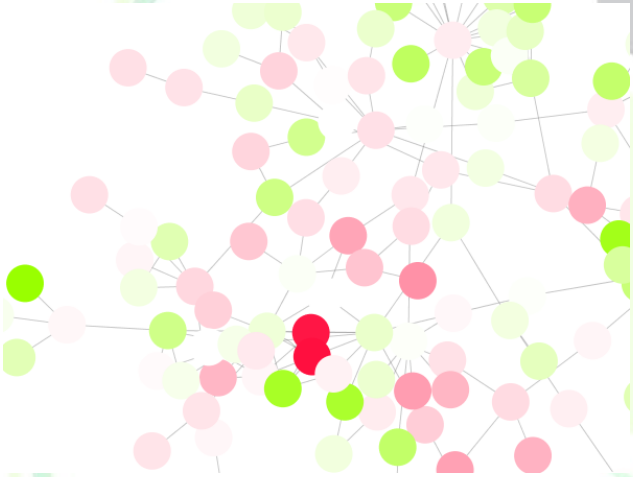
Set node sizes based on the degree of connectivity of the nodes. You can visually see the hub of a network...



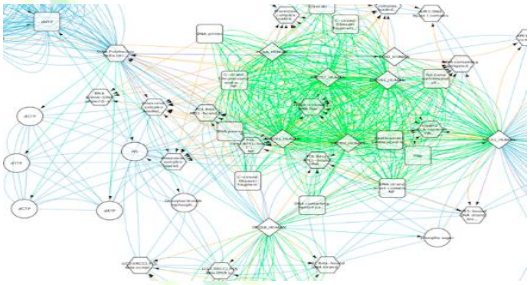
...or, set the font size of the node labels instead.



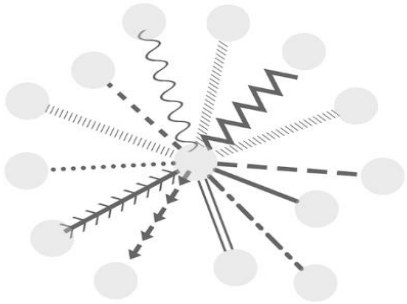
Visualize gene expression data along a color gradient.



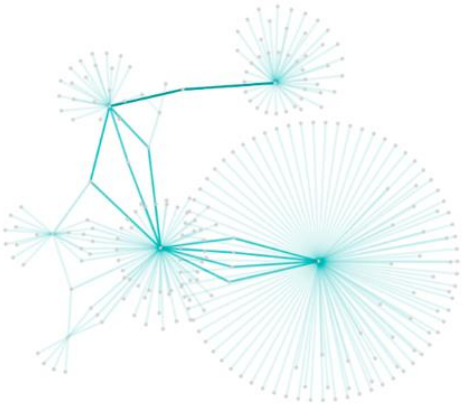
Encode specific physical entities as different node shapes.



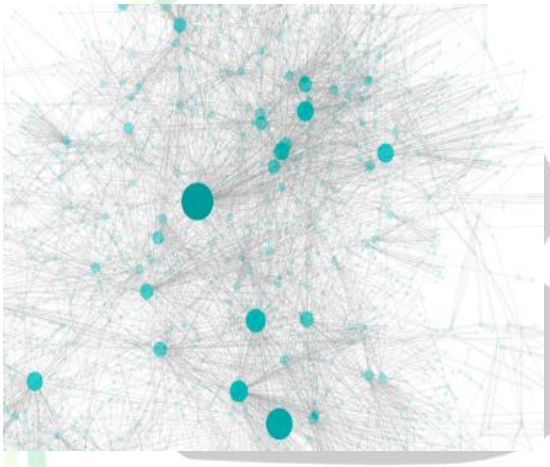
Use specific line types to indicate different types of interactions.



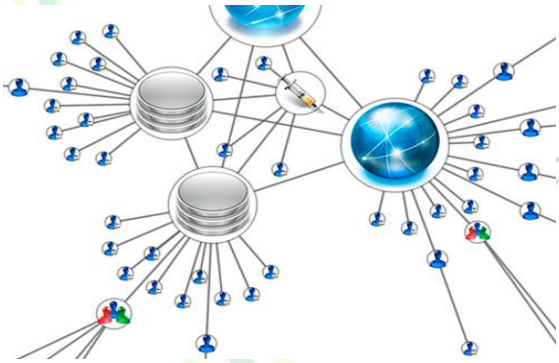
Control edge transparency (opacity) using edge weights.



Browse extremely-dense networks by controlling the opacity of nodes.



Add photo/image/graphics on top of nodes.



Lesson-76

Network Clustering

Network Clustering

- Cytoscape has the ability to associate a Nested Network/ Cluster with any node.
- A nested network is any other network currently defined in Cytoscape.
- This capability allows for creation of network hierarchies as well as circular relationships.

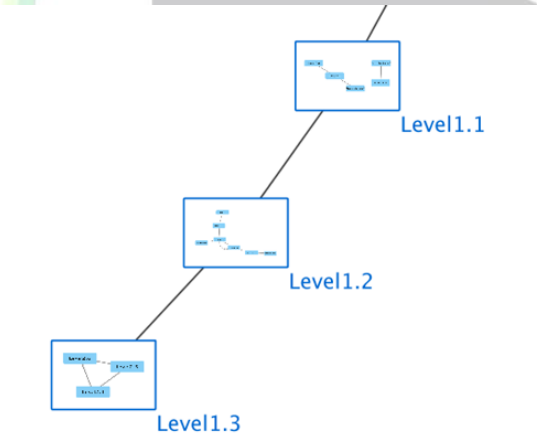
- Note that **Nested Networks** are similar in concept to **Groups**, but with very different consequences.
- A Nested Network incorporates an independent network that does not share nodes or attributes with the current network – it can be edited only as a separate network collection or in a separate Cytoscape session.
- Its nodes cannot be connected to nodes in the current network and cannot be found or filtered using Cytoscape functions or apps.

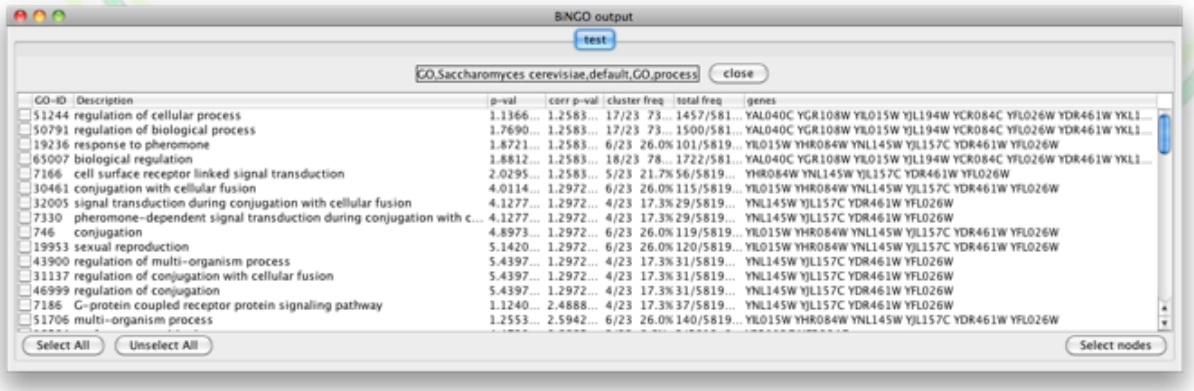
Creating Nested Networks

- There are currently two ways in which a user can create nested networks.
  - By importing a Nested Network Format (NNF) file.
  - By manually constructing networks and assigning nested networks to individual nodes through the right-click node context menu.

Visualization of Nested Networks

- Nodes containing nested networks that are zoomed in sufficiently display an image for the nested network.
- If no current network view exists for the nested network the image will be a default icon, otherwise it will be a low-resolution rendering of the nested network's current layout



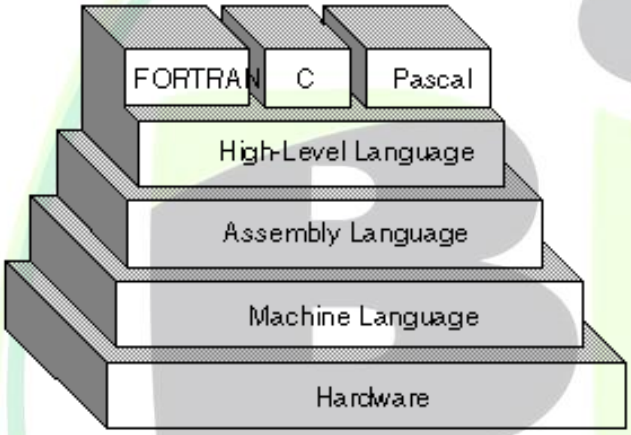


Lesson-77

Python Introduction

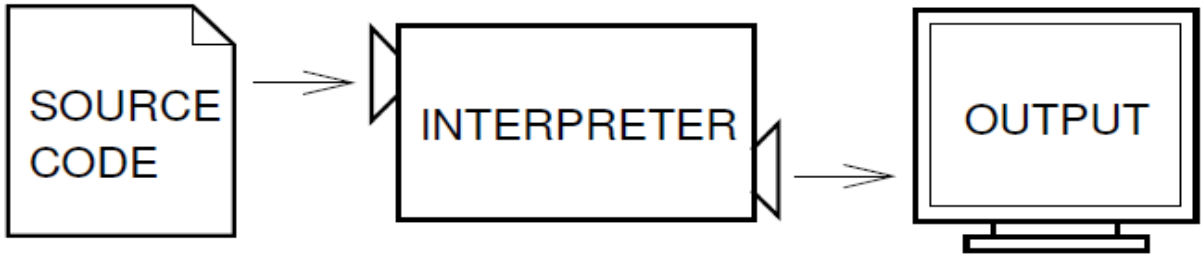
Programming Languages

- Low-level languages
  - Language a computer understands to execute programs.
- High-level languages:
  - Disadvantages?
  - Advantages?

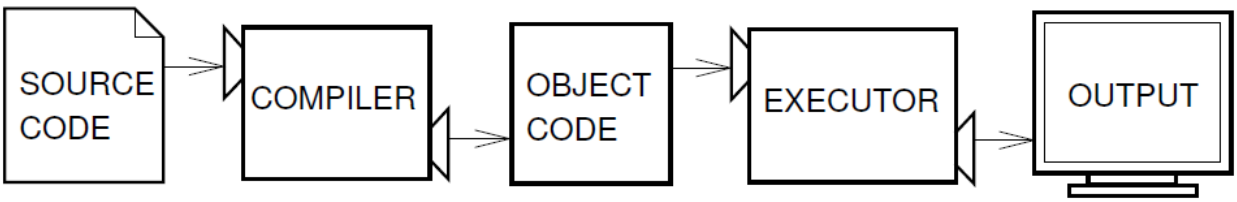


Processing of High-level language into a Low-level language

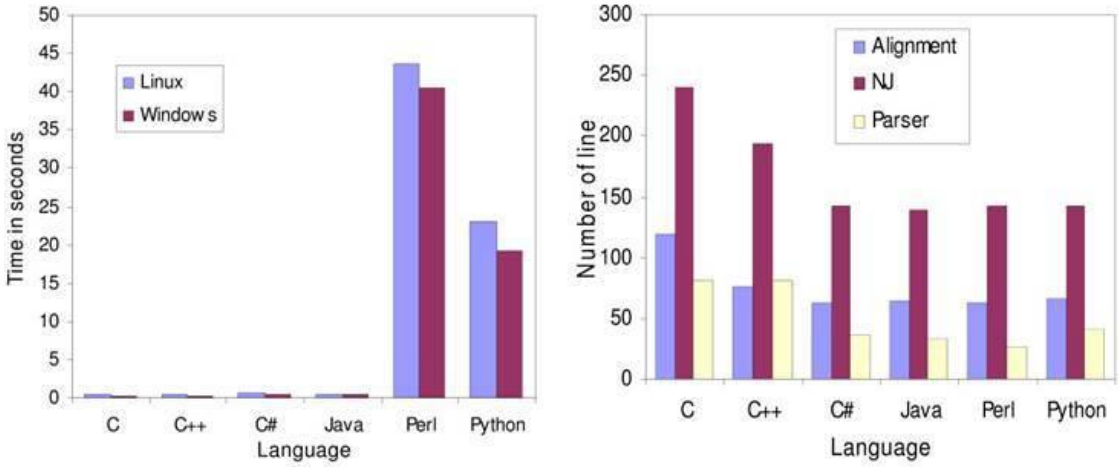
- **Interpreters:**



- **Compilers:**



A Comparison of Bioinformatics Programming Languages (2008)



Why Python?

- Easy to use
- Great for text files.
- Built-in data types for lists, dictionaries, strings. No need to import special classes like in java or C++
- Clean syntax, Portable

Installing Python

- Python for Win/Mac/Linux is available from www.python.org.
  - Installation is generally easy, follow the steps.
- Slight differences between versions 2.x and 3.x.
- In lab, version 3.4.0 is installed. It can be downloaded from https://www.python.org/downloads/release/python-340/

Basic Datatypes

- Integers

For example 2, 3, 5, 0, -1

- Floats (numbers with decimal point)

For example 1.50

- Strings

Either double quotes(“”) or single quotes(‘’) can be used. For example “python” and ‘python’ are same strings.

Unmatched ones can occur within the string. “datatype’s”

Comments

- Add notes in natural language inside code.
- Start comments with # – anything infront of this will be ignored.
- Can use # in a separate line or in front of any code line.

#add two numbers

2+3 #add two numbers

Variables

- Assignment statement will create a new variable.

Syntax: variable_name = value (string, int, float)

```
>>>  message = "Hello"
>>>  n = 17
>>> pi = 3.1415
```

- Variable types don't need to be declared.
- Python figures out the variable types on its own.

Multiple Assignment

- You can also assign to multiple names at the same time.

```
>>> a, b, c = 17, 3.14, "test"
>>> a
 17
>>> b
 3.14
>>> c
 'test'
```

Lesson-78

Python Basics

Printing Output

- You can print a value to the screen using "print."

```
>>> print("hello world")
hello world
```

"Print" automatically adds a newline to the end of the string.

- If you include a coma separated list, it will concatenate them with a space between them.

```
>>> print("hello world", 17)
hello world 17
```

Operators for Numbers

- + , - , *, /, % (for integers), ** (exponential)
- All operators produce results as expected except division operator. In 3.4.0, it's performing as expected

```
>>> 9/10  #integer division rounds down
0
>>> 9/10.0  # Alternative
0.9
```

- Order of operations: PEMDAS
- 2*(3-1) = 4 (1+1)**(5-2) = 8

Comparison Operators

x == *y* # *x* is equal to *y*

x != *y* # *x* is not equal to *y*

x > *y* # *x* is greater than *y*

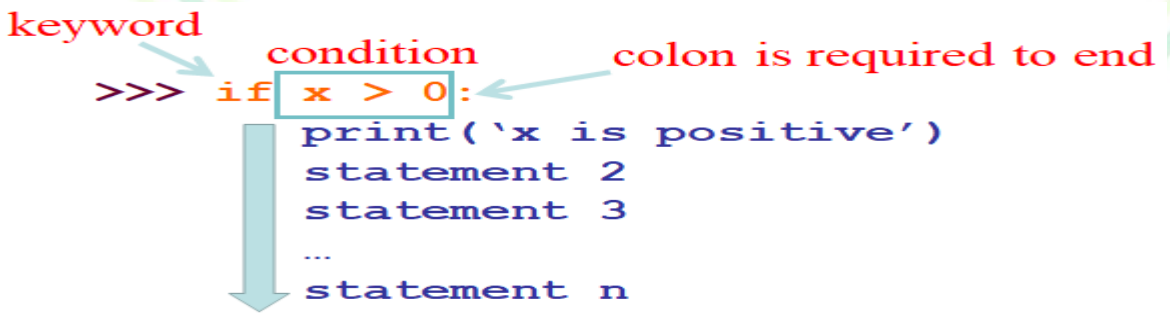
x < *y* # *x* is less than *y*

x >= *y* # *x* is greater than or equal to *y*

x <= *y* # *x* is less than or equal to *y*

Conditional Execution: if, elif, and else

- Allows us to change the behavior of program based on certain conditions.



Conditional: Two Possibilities

- Condition determines which one will be executed.

```
>>> if x >= 0:
      print('x is positive')
          else:
        print('x is negative')
```

- Indentation should match. Both if and else should have at least one statement.

Review of Functions: Calling Built-In Functions

```
>>> type("Hello world")
  <class, 'str'>
```

Function name: type

Input: takes a single value or variable called an *argument*

Output: displays the type of a value or variable. The result is called the return value

Type Conversion: String to Int

- Collection of built-in functions that convert values from one type to another.
- Convert strings to integers.

```
>>> int("17")  # string to integer
  17
>>> int("Python")
Traceback (most recent call last):
  File "<pyshell#24>", line 1, in <module>
    int('Python')
ValueError: invalid literal for int() with base 10: 'Python'
```

Type Coercion

- Remember Integer division problem

```
>>> 9/10
0
```

- Type coercion: automatic type conversion in Python. If one operand is float other is automatically converted to float.

```
>>> 9/10.0
0.9
```

Keyboard input

- Python provides built-in functions to get input from the user.

```
>>> input_value = input()
>>> prompt = “Enter your value=”
# use prompt as an argument to function
>>> input_value = input(prompt)
```

- When these functions are used, the program stops and waits for the user to provide a value and press “enter”.

Lesson-79

Python Loops

Strings

- Strings are enclosed in either single (') or double ("") quotation marks.
- Strings are a composite or compound data type
- Built from another data type, characters
- Within strings characters are stored in sequence, therefore, strings are called sequences.
- Strings can be of any length, for example empty_string = ''

Finding Length of a String

- Use len() function to find the number of characters in a string

```
>>> DNA = "ATGCAGCT"
>>> len(DNA)
8
>>> DNA = "" #empty string
>>> len(DNA)
0
```

Empty strings have zero characters

String Slices: Access Single Characters

```
>>> DNA = "ATGCAGCT"
>>> DNA[0]     #first character
A
>>> DNA[3]     #fourth character
C
>>> DNA[8]     #last character
```

Error

>>> DNA[7] #last character

T

>>> length = len(DNA) // 8

>>> DNA[length-1] #last character

T

String Slices: Segments

- Use a range to specify a slice (sub-string)
  - **Format:** *sample[start:end]*
  - Includes the start index but excludes the last index.

>>> DNA = "ATGCAGCT"

>>> DNA[0:4] #0-3 indices

ATGC

>>> DNA[5:8] #5-7 indices

GCT

>>> DNA[2:50] #2:7 indices

GCAGCT

- Omit the start index to select the start of a sting

>>> DNA = "ATGCAGCT"

>>> DNA[:3] #0-2 indices

ATG

- Omit the last index to select the end of a sting

>>> DNA[5:] #5-7 indices

GCT

String Slices: Segments with Negative Indices

- Omit the start index and use negative index to select the start of a sting

>>> DNA = "ATGCAGCT"

>>> DNA[:-1] #0-6 indices

ATGCAGC

- Omit the last index and use negative index to select the end of a sting

>>> DNA[-3:] #5-7 indices

GCT

Strings are Immutable

- We can't change individual characters in a string through assignment operator.

>>> DNA = "ATGCAGCT"

>>> DNA[0] = "G" # error

- A new string can be created with new value at zeroth index.

>>> new_str = "G" + DNA[1:]

>>> new_str

GTGCAGCT

Strings Membership

- Use "in" operator to check whether a character or substring exists in another string.

>>> DNA = "ATGCAGCT"

>>> base = "W"

>>> base in DNA

False

>>> base not in DNA

True

>>> not base in DNA

True

Some Useful String Functions

- Find substring:

>>> DNA = "ATGCAGCTTAC"

>>> DNA.find('GCT')

5

>>> DNA.find('TTT')

-1

>>> DNA.index('GCT')

5

```
>>> DNA.index('TTT')
ValueError: substring not found
```

- ***count()*** returns number of non-overlapping occurrences of the substring:

```
>>> DNA = "ATCGCGG"
>>> DNA.count('G')
3
>>> DNA.count('G', 0, 5)
1
>>> seq = "ATCCCCCGG"
>>> seq.count("CC")
2
```

- ***split()*** returns a list of the words after splitting string on a specified delimiter.

```
>>> DNA = 'ATGCTT'
>>> DNA.split('GC')
['AT', 'TT']
>>> DNA.split('GC')[0]
'AT'
```

Or

```
>>> a = DNA.split('GC')
>>> a[0]
'AT'
```

- ***replace()*** returns a copy of string with all occurrences of old substring replaced by a new substring:

```
>>> DNA = "ATCGCGG"
>>> DNA.replace('G', 'U')
ATCUCUU
>>> DNA.replace('G', 'U', 2)
ATCUCUG
```

Lesson-80

Python Loops-II

while Loop

- Loops are used to perform repetitive tasks.
- A *while* loop needs a condition that will eventually becomes False.

```
>>> DNA = "ATGCAGCT"
>>> i = 0
>>> while i < len(DNA):
      print(DNA[i])
      i += 1
   print('outside loop')
```

***for* Loop**

- for loop provides an easy way to access elements of the string.

```
>>> DNA = "ATGCAGCT"
>>> for base in DNA:
      print(base, end="")
   print("\noutside loop")
```

Output:

ATGCAGCT

Outside loop

break, continue, pass,* and loop *else

break: jumps out of the closest enclosing loop.

continue: jumps to the header line of the closest enclosing loop

pass: does nothing at all

Loop else block: this gets executed if and only if the loop is exited normally (i. e., without any break statement)

General Format

```
while <test1>:
        <statements1>
        if <test2>:
```

```
        break      #Exit loop now, skip else
    if <test3>:
        continue   #Go to test1 of top loop
else:          #Run if didn't hit a 'break'
    <statements2>
```

- Usually *break* and *continue* are nested further inside *if* to specify an action in response to a certain condition.
- *break* and *continue* can appear anywhere inside the body of the while or for loop.

```
while True:
    <statements>
    break      #Exit loop now
while True:
    continue   #Go to header of while loop
    <statements>
```

- The <statements> after the continue never gets executed.

Example: *continue*

- continue jumps to the header line of the closest enclosing loop.
- For example, continue will skip all odd and only even gets printed.

```
x = 10
while x:      # x = 0 will terminate loop
   x = x−1        # Or, x -= 1
   if x % 2 != 0:
      continue    # Odd -- skip print
   print(x)
```

Example: break

- *break* jumps out of the closest enclosing loop. In the example, *break* will terminate the loop when user enters "stop".

```
while True:
   name = input('Enter name:')
   if name == 'stop':
      break
```

```
age = input('Enter age: ')
print('Age is', int(age) ** 2)
```

pass

- *pass*: does nothing at all. It can be used when the syntax requires a statement and you don't have any useful statement.

```
while True:  # infinite loop
    pass
```

OR

```
x = 1
if x > 0:   #x is greater than zero--True
   pass
```

OR

```
def testFunc():
    pass
```

Loop *else*

- Loop *else*: this gets executed if and only if the loop is exited without any *break*.

```
def dnaDt(dna):
   for base in dna:
            if base not in 'ATCG':
          print('Invalid')
                    break
   else:
            print('Valid DNA')
dnaDt('ATGCTGAC')  # printed: Valid DNA
dnaDt('ATGWGT')    # Printed: Invalid
dnaDt('12344')     # Printed: Invalid
```

Lesson-81

Regular Expressions

Regular Expressions

- Regular expressions are also referred as "REs" or "regex" or "regexp".

- Use RE to specify the rules for the set of possible strings that you want to match.
- It's a powerful string manipulation tool to search, replace, and parse complex patterns of characters or words.
- Strings methods for searching (***index***, ***find***, and ***count***), replacing (***replace***), and parsing (***split***) use hard-coded substrings and are case-sensitive.
- REs offers a flexible way to:
  - Search a string (methods: ***search*** and ***match***)
  - Replace parts of a string (method: ***sub***)
  - Break strings into smaller pieces (method: ***split***)
- REs are very useful in bioinformatics. We can use them to find DNA and Protein patterns
- DNA patterns
  - restriction sites
  - promoters/transcription factor binding sites
- Protein patterns
  - conserved domains (motifs)
  - active sites

Pattern and String Description

- **Pattern:** this is the regular expression to be matched
- **String:** This is the string, which would be searched to match the pattern anywhere in the string.

REs Syntax

- A pattern can be just a text string: 'GATC'
- It can have alternative characters: 'G[AT]TC'

The 'G[AT]TC' pattern will match a DNA string that starts with G; has A or T in the 2nd position, followed by TC.

- We can specify characters individually or as a range. For example 'G[abc]TC' or 'G[a-c]TC'
- A caret (^) symbol at the start of the character class matches any one character except those given in the square brackets **'G[^AT]TC'**

  This **'G[^AT]TC'** pattern will match a DNA string that starts with G; has C or G in the 2nd position, followed by TC. Here: **'GCTC'** or **'GGTC'**.
- A pattern can contain a wildcard (a dot) that matches any single character except newline: **'G.TC'**

Repeated Things

- x* matches zero or more number of x

'ca*' matches 'c', 'ca', 'caa', etc.

- x+ matches one or more number of x

  'ca+' matches 'ca', 'caa', etc.

- x? matches zero or one x

  'home-?work' matches 'homework' or 'home-work'

- x{m, n} matches i number of x, where m≤i≤ n 'a{2,3}' matches 'aa' or 'aaa'

Repeated Things: Equivalence

- *x{m, n}* matches i number of *x*, where *m≤i≤ n* **'a{2,3}' matches 'aa' or 'aaa'**
- *x*{0,} is same as *x**
- *x*{1,} is same as *x*+
- *x*{0,1} is same as *x?*

REs Syntax

- Parentheses can be used for grouping '(abc)+' matches 'abc', 'abcabc', 'abcabcabc'.
- A vertical line "|" means "or"; it allows you to look for either of two completely different patterns:

  'GAAT|ATTC'

Lesson-82

Regular Expressions-II

How to Use REs in Python?

- In order to use regular expressions in the code, we need to import *re* module.
- The two basic methods *re.search()* and *re.match()* are used to search a pattern.
  - Search looks for a pattern anywhere in a string
  - Match looks for a match at the beginning
- Both methods return None if the pattern isn't found and a "match object" instance if it is.

Search and Match

```
import re
pat = 'a*b'
re.search(pat, 'fooaaabcde')
<_sre.SRE_Match object; span=(3, 7), match='aaab'>
re.match(pat, 'fooaaabcde')
None
```

```
re.match('fooa*b', 'fooaaabcde')
<_sre.SRE_Match object; span=(0, 7), match='fooaaab'>
```

Match Object

- It is an instance of the match class with the details of the match result.

```
import re
res = re.search('a*b', 'fooaaabcde')
res.group()  # group returns string matched
'aaab'
res.start()  # index of the match start
3
res.end()    # index of the match end
7
res.span()   # tuple of (start, end)
(3, 7)
```

- It is an instance of the match class with the details of the match result.

```
import re
res = re.match('fooa*b', 'fooaaabcde')
res.group()  # group returns string matched
‘fooaaab'
res.start()  # index of the match start
0
res.end()    # index of the match end
7
res.span()   # tuple of (start, end)
(0, 7)
```

***split()* method**

- The re.split() is like split but can use patterns.

```
re.split('\W+', 'This... is a test,  short and sweet, of split()')
   ['This', 'is', 'a', 'test', 'short', 'and', 'sweet', 'of', 'split', '']
```

The '\W+' means split on all non-alphanumeric characters.

sub() and _findall()_ methods

- ***re.sub()*** substitutes all occurrences of RE pattern in string with a new pattern.

re.sub('(blue|white|red)', 'black', 'blue socks and red shoes')

'black socks and black shoes'

- ***re.findall()*** finds all matches and returns them as a list

re.findall('\d+','12 dogs,11 cats, 1 egg')

['12', '11', '1']

Compile Patterns

- If you plan to use a re pattern more than once, compile it to a ***re*** object

```
import re
p = re.compile('a*b')
res = p.search('fooaaabcde')
res.group()  # group returns string matched
'aaab'
res.start()  # index of the match start
3
res.end()    # index of the match end
7
res.span()   # tuple of (start, end)
(3, 7)
```

Backslash and Raw Strings

- RE use backslash character ('\') to indicate special meanings. However, if we want to use them without their special meaning then we have escape them either using backslashes or raw strings (preferable).

Regular String	Raw string
"ab*"	r"ab*"
"\\\\section"	r"\\section"
"\\w+\\s+\\1"	r"\w+\s+\1"

Lesson-83

Strings

Ways to Code Strings?

- So far we have declared strings that are either enclosed in single (‘’) or double (“”) quotation marks.
- Strings can declared with triple quotation marks (single or double). These are called block strings or triple-quoted strings.
- Python concatenates all lines together into a single line, and end-of-line (‘\n’) character is added where line breaks.

Block Strings: Example

```
dna = ”””abcd
efgh
Ijkl”””
print(dna)
print(dna.split('\n'))
```

Output:

```
abcd
efgh
ijkl
['abcd', 'efgh', 'ijkl']
```

Multiline Comments: Triple Quotes

- Remember: single line comment starts by adding # symbol at the start of a line.
- We can also use triple quotes (single or double) to add multi line comments in the code.

```
'''
this is a multi-line
comment'''
dna = ”””abcd
efgh
Ijkl”””
print(dna)
```

Implicit and Explicit Concatenation

- Explicit: using '+' operator
- Implicit: python automatically concatenates adjacent strings.

```
s = "Bio" "infor" "matics"
print(s)
Bioinformatics
ss = "Bio"+"infor"+"matics"
print(ss)
Bioinformatics
```

Escape Sequences

- Escape sequences are used to introduce an alternative interpretation of a series of characters.
- In Python, backslash is used to introduce escape sequences in the code. Backslash allows us to add difficult to type characters into the strings.
- Some examples you already know: '\n', '\t'

Escape	Meaning
\\	Backslash (stores one \)
\'	Single (stores ')
\"	Double (stores ")
'\n'	Newline
'\t'	Horizontal tab
'\v'	Vertical tab
'\b'	Backspace

Raw Strings

- Problems using escape sequences

```
>>> path = "C:\new\test.txt"
>>> print(path)
C:
ew      est.txt
```

- Use letter 'r' or 'R' to suppress the escape sequences

Raw Strings Suppress Escape Sequences

- Examples of raw strings:

```
>>> path = r"C:\new\test.txt"

>>> print(path)

C:\new\test.txt

>>> path = R"C:\new\test.txt"

>>> print(path)

C:\new\test.txt
```

Alternative Way to Suppress Escape Sequences

- An alternative way is to use double backslash (not a raw string).

```
>>> path = "C:\\new\\test.txt"

>>> print(path)

C:\new\test.txt
```

Functions That Return a Value

- We can write functions that can return a single or multiple values. The value can be numbers, strings, lists, dictionaries. e. g.,

```
def add_s(a, b):
    return a+b,a,b

ss,a,b = add(3,4)

print(ss,a,b)

7 3 4
```

Lesson-84

Lists

Lists

- List are ordered collection of values. Each value is called an element.
  - Left to right order among elements.
  - Can contain any object types such as number, strings, lists, dictionaries etc.
  - Like strings, these are sequences. Sequence methods that were used for strings can also be used for lists.
  - Lists are similar to Arrays. However, size of the list is not fixed and list can handle multiple data types.

Ways to Create a List

- Square brackets are used create a list

```
>>> [10,20] #list of two integers
[10,20]
>>> ["A", "g", "T"] #three strings
['A', 'a', 'T']
>>> ["A",10,["T",20],30.0] #diff.types
['A', 10, ['T', 20], 30.0]
>>> [] #empty
[]
```

Assigning List to Variables

- Like strings, we can assign lists to variables.

```
>>> a = ["A", "T"] #list
>>> b = [] #empty list
```

- We can pass lists to functions.

```
>>> print(a,b) # passing to functions
['A', 'T'] []
>>> type(a) #type is list
<class 'list'>
```

***while* Loop**

- Loops are used to perform repetitive tasks.
- A *while* loop needs a condition that will eventually becomes False.

```
>>> DNA = ["A", "T", "C", "G"]
>>> i = 0
>>> while i < len(DNA):
        print(DNA[i], end=",")
        i += 1
    A,T,C,G,
```

***for* Loop**

- for loop provides an easy way to access elements of the string.

```
>>> DNA = ["A", "T", "C", "G"]
>>> for base in DNA:
```

```
    print(base, end=",")
  print("\noutside loop")
```

Output:

A,T,C,G,

Outside loop

Ways to Create List of Integers

- range() function can be used in many ways to create list of consecutive integers.

```
>>> range(5) #one argument
range(0,5)
>>> range(1,5) #two arguments
range(1,5)
>>> range(1,10,2) #Three argument
range(1,10,2)
```

- Third argument is called step size that specifies the numbers of digits to skip.
- We can use loop to iterate the newly created lists.

```
>>> for i in range(5):
      print(i,end=",")
0,1,2,3,4,
>>> for i in range(1,10,2):
      print(i,end=",")
1,3,5,7,9,
```

Finding Length of a List

- Use ***len()*** function to find the number of elements in a list.

```
>>> DNA = ["A", "T", "G", "C"]
>>> len(DNA)
4
>>> DNA = [] #empty list
>>> len(DNA)
0
```

- Like strings, ***len()*** function returns zero for the empty lists

Length of a List of Lists

- Nested lists are counted as a single element by the *len()* function.

```
>>> DNA = ["A", "T",["T", "G"]]
>>> len(DNA)
3
>>> len([[90,3],["T",20]])
2
```

List Indexes and Slices

- A single element and a part of a list is accessed by indexing and slice operator, respectively.
- List index starts from zero.

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA[0] # first element
'A'
>>> DNA[3] # fourth element
'G'
```

List Negative Indices

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA[-1]    #last element
'G'
>>> DNA[-4] #first element
'A'
>>> DNA[-5]   #out of bound
IndexError: list index out of range
```

List Slices (Segments)

- Use a range to specify a slice (sub-list)
- **Format:** *sample[start:stop:step]*
- **start:** the beginning index of the slice, it will include the element at this index unless it is the same as *stop*, defaults to 0, i.e. the first index. If it's negative, it means to start n items from the end.
- **stop:** the ending index of the slice, it does *not* include the element at this index, defaults to length of the sequence being sliced, that is, up to and including the end.

- **step:** the amount by which the index increases, defaults to 1. If it's negative, you're slicing over the iterable in reverse.

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA[0:2]    # first two
['A', 'T']
>>> DNA[2:]    # last two
['C', 'G']
>>> DNA[50:100] # out of range
[]
```

Lesson-85

Lists-II

List Operations

- Concatenate

```
>>> dna_at = ["A", "T"]
>>> dna_cg = ["C", "G"]
>>> print(dna_at + dna_cg)
['A', 'T', 'C', 'G']
```

- Repeat

```
>>> ["A"]*4
['A', 'A', 'A', 'A']
```

List Membership Test

- “in” operator is used to check membership.

```
>>> "W" in ["A","T","C","G"]
False
>>> "W" not in ["A","T","C","G"]
True
>>> "A" in ["A","T","C","G"]
True
```

Strings and Lists

- We discussed split() function that returns a list.

```
>>> DNA = "A T C G"
>>> DNA.split()
['A', 'T', 'C', 'G']
>>> a = DNA.split()
>>> type(a[0])
<class 'str'>
```

- We can use join() function to convert a list to string. It's opposite of split().

```
>>> DNA = ["A", "T", "C", "G"]
>>> print("".join(DNA))
'ATCG'
```

Lists are Mutable: Updating

- Unlike strings, one or more than one elements of the lists can be changed using assignment operation.
- Index assignment: we can update single element using both positive and negative indexes.

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA[0] = "T"
>>> DNA[-2] = "G"
>>> print(DNA)
['T', 'T', 'G', 'G']
```

Strings to list conversion: *list()*

- Built-in list() function can be used to convert strings to lists

```
>>> DNA = "ATGC" #change A to G
>>> DNA = list(DNA)
>>> print(DNA)
['A', 'T', 'G', 'C']
>>> DNA[0] = "G"
>>> print("".join(DNA))
'GTGC'
>>> list(range(5))
```

[0,1,2,3,4]

Methods to Grow List

- New elements can also be inserted into Lists to increase it's size.
- Objects can be added to the end of the list using *append()* function.

```
>>> DNA = ["A", "T"]
>>> DNA.append("C")
>>> print(DNA)
['A', 'T', 'C']
>>> DNA.append(["G", "A"])
>>> print(DNA)
['A', 'T', 'C',['G', 'A']]
```

Methods to Delete Elements: *del*

- Built-in *del* function can be used.

```
>>> DNA = ["A", "T", "C", "G"]
>>> del DNA[0]   # single
>>> print(DNA)
['T', 'C', 'G']
>>> del DNA[0:2]   # slice
>>> print(DNA)
['G']
>>> DNA = ["A", "T", "C", "G"]
>>> del DNA[-1]   # single
>>> print(DNA)
['A', 'T', 'C']
```

Methods to Delete Elements: *pop()*

- Built-in pop() function deletes an element by index.

```
>>> DNA = ["A", "T", "C", "G"]
>>> DNA.pop()   #deletes from end
'G'
>>> print(DNA)
```

['A', 'T', 'C']

>>> DNA.pop(1) #deletes from middle

'T'

>>> print(DNA)

['A', 'C']

Lesson-86

Lists-III

STACK – LIFO

- Stack concept: Last In First Out (LIFO)
- The *append()* and *pop()* functions can be used together.

>>> DNA = ["A", "T", "C"]

>>> DNA.append("G")

['A', 'T', 'C', 'G']

>>> DNA.pop()

'G'

>>> DNA

['A', 'T', 'C']

Methods to Delete Elements: *remove()*

- Built-in remove() function takes a single argument. The argument will be the element to remove.

>>> DNA = ["A", "T", "C", "G"]

>>> DNA.remove("A")

>>> DNA

['T', 'C', 'G']

>>> DNA.remove("t")

ValueError: list.remove(x): x is not in list

- Only the first occurrence of the element is removed from the list.

>>> DNA = ["A", "T", "C', "G", "A"]

>>> DNA.remove("A")

>>> DNA

['T', 'C', 'G', 'A']

Other Useful Methods

- The minimum, maximum, number of occurrences, and sum of the elements of list.

```
>>> a = [1,4,20,2,5]
>>> sum(a) # add all elements
32
>>> min(a) # find minimum value
1
>>> max(a) # find maximum value
20
>>> a.count(2) # count occurrences
1
```

- We can also use slices with these methods.

```
>>> a = [1,4,20,2,5]
>>> sum(a[3:]) # add all elements
7
>>> min(a[3:]) # find minimum value
2
>>> max(a[3:]) # find maximum value
5
>>> a[1:].count(1) #count occurrences
0
```

***reverse()* and *index()* Functions**

- The reverse() function will reverse the list.

```
>>> a = [1,4,20,2,5]
>>> a.reverse() # reverse order
>>> print(a)
[5,2,20,4,1]
```

- The index() function will find the index of the element. Returns error when not found.

```
>>> a = ["A", "T", "C"]
>>> a.index("T")
```

```
1
 >>> a.index("G")
ValueError: 'G' not in list
```

Matrix

- The matrix can be declared in the form of nested lists.

```
>>> b = [[4,5],[2,3],[8,0]]
>>> b[0]
[4,5]
>>> b[0][0]
4
```

Sorting Lists: sort() function

- The *sort()* function is used for lists.

```
>>> a = [1,4,20,2,5]
>>> a.sort() # ascending order
[1,2,4,5,20]
>>> a.sort(reverse=True) #descending
[20,5,4,2,1]
```

  - Can be used with nested lists.

```
>>> b = [[4,5],[2,3],[8,0]] #nested
>>> b.sort() #by zero index
[[2,3],[4,5],[8,0]]
```

  - We can specify index as well for sorting.

```
>>> from operator import itemgetter
>>> b = [[4,5],[2,3],[8,0]]
>>> b.sort(key=itemgetter(1))
[[8,0],[2,3],[4,5]]
>>> b.sort(key=itemgetter(1),\
        reverse=True)
[[4,5],[2,3],[8,0]]
```

- Alternatively, sorted() function can also be used. Unlike sort(), it won't change the original list.

```
>>> from operator import itemgetter
>>> b = [[4,5],[2,3],[8,0]]
>>> sorted(b)
[[2,3],[4,5],[8,0]]
>>> print(b)
[[4,5],[2,3],[8,0]]
>>> sorted(b, reverse=True)
[[8,0],[4,5],[2,3]]
```

- Alternatively, *sorted()* function can also be used.

```
>>> from operator import itemgetter
>>> b = [[4,5],[2,3],[8,0]]
>>> sorted(b,key=itemgetter(1))
[[8,0],[2,3],[4,5]]
>>> sorted(b, key=itemgetter(1),\
        reverse=True)
[[4,5],[2,3],[8,0]]
```

Lesson-87

Files

File, Directory, and Path??

- A file is a named entity that is used to read and write data from/to a hard drive.
- Directory/folder: is an organizing unit in computer's file system for storing and locating files. It can contain both subdirectories and files.
- Path: a sequence of directory names that specifies the exact location of the file.

Text Files

- Files are used to read and write data from/to a hard drive.
- Text File: a text file is a sequence of lines that are separated by newline characters.

Files: *open()* function

- In Python, we specify the name of the file and tell interpreter whether we want to read it or write it.
- Python has an open() function that is used to read or write files.

- Function open() returns a handle (file object) used to perform operations on the file.

How to Open Files?

- Function open takes **two arguments**.
  - Name and location of the file
  - Mode (optional): read, write, append etc.
  - Syntax: **fout = open(filename, mode)**
  - **fout:** is a new file object. You can use any name instead of fout.
  - **filename:** a string specifying the name and the location of the file.
  - **Mode:** special characters to specify what to do with the file. Default is to read a file if you skip this argument.

Reading Files - Assign Handle

- *Default mode is read.* We can either skip it or specify 'r' to read a file.

```
# read a file without a mode (default is 'r')
fr = open('test.txt')
```

or

```
fr = open('test.txt','r')
print(fr)
```

- If only file name is listed and not full location on the system, Python tries to find it in *current working directory (where your source code is).*

Read Entire File as String

File: Test.txt

```
>fasta sequence
ATGCATGCGCTGC
CGTCGCGTACGTA
```

```
fr = open('test.txt','r')
a = fr.read()
print(a)
```

- *reads entire file as a string*. We can split on newline character to get individual lines.

Read *N* Characters from File

File: Test.txt

```
>fasta sequence
```

ATGCATGCGCTGC

CGTCGCGTACGTA

```
fr = open('test.txt','r')
a = fr.read(7) # read first seven
print(a)
a = fr.read(15) # read next 15
print(a)
```

Reading Files with a *for* loop

- The lines from a file object can be read using a for loop.
  - Open a file in read mode
  - Read each line using a for loop
  - Perform actions on lines

```
fr = open('test.txt','r')
for line in fr:
  print(line)
```

Output:

```
>fasta sequence
  ATGCATGCGCTGC
  CGTCGCGTACGTA
```

Reading Files and *print()* Function

- The *print()* function adds a newline character to each line. Also, each line from the file has a new line.
- We can use *rstrip()* function to remove whitespaces from the right side of the string.

```
fr = open('test.txt','r')
for line in fr:
  print(line.rstrip())
```

Output:

```
>fasta sequence
  ATGCATGCGCTGC
  CGTCGCGTACGTA
```

Reading Files with a *for* loop

- We can apply conditions inside *for* loop to process only the desired lines.

```
fr = open('test.txt','r')
for line in fr:
    if not line.startswith('>'):
        print(line.rstrip())
```

Output:

```
ATGCATGCGCTGC
CGTCGCGTACGTA
```

Lesson-88

Files-II

Reading Files with *readline()*

- The *readline()* function reads next line from the file as a string.

```
fr = open('test.txt','r')
print(fr.readline().rstrip())
print(fr.readline()[:-1])
print(fr.readline().replace('\n',''))
fr.close()
```

Output:

```
>fasta sequence
ATGCATGCGCTGC
CGTCGCGTACGTA
```

Using *print()* Argument to Suppress newline

- We can also use *end* argument to suppress newline of *print()*.

```
fr = open('test.txt','r')
print(fr.readline())
print(fr.readline(),end='')
print(fr.readline(),end='')
```

Output:

```
>fasta sequence
```

ATGCATGCGCTGC

CGTCGCGTACGTA

***while* Loop to Read File**

- The while will terminate when the line reaches end of file

```
fr = open('test.txt','r')
line = fr.readline()
while line:
print(line.rstrip())
line = fr.readline()
```

Output:

>fasta sequence

ATGCATGCGCTGC

CGTCGCGTACGTA

Reading Files with *readlines()*

- The *readlines()* function reads the whole file's content as a list of lines.

```
fr = open('test.txt','r')
all_lines = fr.readlines()
print(all_lines)
```

Output:

['>fasta sequence\n', 'ATGCATGCGCTGC\n', 'CGTCGCGTACGTA\n']

```
fr = open('test.txt','r')
all_lines = fr.readlines()
for line in all_lines:
print(line,end='')
```

Output:

>fasta sequence

ATGCATGCGCTGC

CGTCGCGTACGTA

Writing Files

- The *fout = open('filename','w')* opens a file to write. If a file already exists, it deletes all previous content from the file.

```
fout = open('test_write.txt','w')
fout.write('This is an example\n')
fout.close()
```

- The *write()* function writes a string to the file. Use newline if you want to write line by line.
- The *close()* function saves the file once writing is complete and makes it available for other uses.

Writing Files: append mode

- The *fout = open('filename','a')* opens a file to write. If a file already exists, the new data is appended to previous content in the file.

```
fout = open('test_write.txt','a')
fout.write('This is a rewrite\n')
fout.close()
```

- The *write()* function writes a string to the file. Use newline if you want to write line by line.
- The *close()* function saves the file once writing is complete and makes it available for other uses.

Lesson-89

Tuples

Tuple

- Tuple are written as series of items in parenthesis (not square brackets).
- Tuple is a compound data type (collection of other data types or objects).
- These *immutable* – cannot be changed in-place. Similar to strings.
- Fixed size – if you want to modify it create a new tuple
- Like lists and strings, tuples are ordered collection of objects that **maintain left to right order**.
- Items are accessed by position based indexing just like in Lists and Strings. Tuple **support indexing and slicing operations**.
- Works exactly like lists and strings in terms of methods and operations.

Ways to Create a Tuple

- A comma after the single item, before the closing parenthesis tells Python that it's a single item tuple and not a regular expression (As we can write expression in parenthesis; we need to differentiate).

```
a = () # an empty tuple
```

```
x = (40) # An integer
y = (40,) # A one item tuple
z = (40,50) # A two item tuple
```

Examples

```
x = (40)
type(x)
```

OUTPUT: <class 'int'>

```
y = (40,)
type(y)
```

OUTPUT: <class 'tuple'>

```
x = (0,[4],1.2,'3')
print(x)
```

OUTPUT: (0, [4], 1.2, '3')

Nested Tuple

- Tuple allows nesting of different data types.

```
x = ('ATG',('ATA', 'TAA')) # tuple
print(x)
```

OUTPUT: ('ATG', ('ATA', 'TAA'))

```
y = (['ATA', 'TAA']) # a tuple???
print(type(y))
```

OUTPUT: <class 'list'>

```
z = (['ATA', 'TAA'],) #tuple?
print(type(z))
```

OUTPUT: <class ‘tuple'>

Index and Index of Index

- *Index & Index of index:* similar to strings and lists

```
x = ('ATG', 'ATA', 'TAA') # tuple
print(x[0],x[-1])
```

OUTPUT: ATG TAA

```
y = (('ATG','start'),('TAA','stop'))
print(y[0])
OUTPUT:('ATG', 'start')
print(y[0][1])
OUTPUT: start
  print(y[-1][1])
OUTPUT: stop
```

Slicing

- Similar to strings and lists. *Format: T[start:end]*

```
x = ('ATG', 'ATA', 'TAA', 'TAG')
print(x[:])
OUTPUT: ('ATG', 'ATA', 'TAA', 'TAG')
print(x[:3])
OUTPUT: ('ATG', 'ATA', 'TAA')
print(x[1:])
OUTPUT: ('ATA', 'TAA', 'TAG')
print(x[:-2])
OUTPUT: ('ATG', 'ATA')
print(x[5:6])
OUTPUT: ()
```

Length and Concatenate

- The *len()* method returns the number of elements in the tuple.

```
x = ('ATG', 'ATA')
print(len(x))
OUTPUT: 2
```

- The "+" operator will combine two tuples.

```
 y = ('TAA', 'TAG')
 print(x + y)
 OUTPUT: ('ATG', 'ATA', 'TAA', 'TAG')
```

Iteration

- We can use *for* and *while* loops.

```
x = ('ATG', 'ATA', 'TAA', 'TAG')
for entry in x:
    print(entry, end=',')
OUTPUT: ATG,ATA,TAA,TAG,
i = 0
while i < len(x):
    print(x[i], end=',')
    i += 1
OUTPUT: ATG,ATA,TAA,TAG,
```

Conversions: List to/from Tuple

```
T = ('cc', 'aa', 'dd', 'bb')
tmp = list(T) # Make a list from tuple
OUTPUT: ['cc', 'aa', 'dd', 'bb']
tmp.sort() # Sort the list
print(tmp)
OUTPUT: ['aa', 'bb', 'cc', 'dd']
T = tuple(tmp) # Make a tuple from list
print(T)
OUTPUT: ('aa', 'bb', 'cc', 'dd')
```

Conversions

- Converting string to tuple and sorted method.

```
T = tuple('spam')
print(T)
OUTPUT: ('s', 'p', 'a', 'm')
print(sorted(T)) # sorted built-in
OUTPUT: ['a', 'm', 'p', 's']
```

Lesson-90

Dictionaries

Dictionary

- The compound data types such as lists and strings use integer indices to access elements.
- Dictionaries are also compound data types (collection of other data types or objects).
- Dictionary are known as *mappings* because these simply map keys to their associated values (*"key-value"* pairs).
- These are useful whenever we need to associate a set of values with keys. e.g., name and phone number; protein name and sequence.
- Dictionary don't maintain left to right order. In fact no order at all because we will use keys to access them.
- These are ***mutable*** – can update elements, remove, and add new elements.

Ways to Create a dictionary

- Start with an empty dictionary and add elements.

```
a = {} #this step creates an empty dictionary
a['one'] = 1      #add 1st element
a['two'] = 'two'  #add 2nd element
a['three'] = 3.0   #add 3rd element
a['four'] = []    #add 4th element
print(a)
{'four': [], 'one': 1, 'three': 3.0, 'two': 'two'}
```

- Provide a list of "key:value" pairs.

```
a = {'four': [], 'one': 1, 'three': 3.0, 'two': 'two'}
print(a)
{'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
```

- Note: key-value pairs are not in order.

Key and Values

- Elements (i.e., key-value pairs) in the dictionary are separated by commas.
- Each key can occur only once. Dictionary holds unique elements only.
- A key can be any immutable type (string, numbers, tuples). List is mutable and cannot be a key.

Keys are unique

- Each key can occur only once. Dictionary holds unique elements only.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
a['four'] = 4
print(a)
{'four': 4, 'two': 'two', 'three': 3.0, 'one': 1}
```

Mutable containers can't be Key

- A key can be any immutable type (string, numbers, tuples). List is mutable and cannot be a key.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
a[['A']] = 'a'
TypeError: unhashable type: 'list'
```

Remove Elements: del method

- The *del* statement removes a key-value pair from the dictionary. It's similar to *del* in list except that it accepts key not index.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
del a['four']
print(a)
{'three': 3.0, 'two': 'two', 'one': 1}
del a['Three']
KeyError: 'Three'
```

Remove Elements: pop method

The pop() function removes a key from the dictionary and returns it's value. It takes a key as an argument.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
a.pop('four')
[]
a.pop()
TypeError: pop expected at least 1 arguments, got 0
```

Finding the Number of Elements

- The len() function returns the number of key-value pairs.

```
a = {'four': [], 'two': 'two', 'three': 3.0, 'one': 1}
```

```
print(len(a))
```

4

```
dna = {}
print(len(dna))
```

0

Lesson-91

Sets

Sets

- An *unordered* collection of *unique and immutable* objects.
- Set class in Python is implemented using dictionaries so *all requirements are similar to dictionary keys*.
- Unique items: an item appears only once in a set regardless of how many times it is added.
- Sets ***cannot contain mutable types*** such as dictionaries and lists as elements. However, it can contain immutable types such as tuples.
- It is a compound data type and can contain variety of other objects or data types.
- Sets are traversable, can grow and shrink in size.
- These are neither sequences nor mappings.
- Sets are like valueless dictionaries (i. e., no key-pair mappings) in which elements do not maintain an order.
- Do not support indexing and slicing operations because these are not sequences.

How to Create a Set?

- To make a set object, pass in a sequence or other iterable object to the built-in *set()* function. Syntax: *set([iterable])*

```
a = set() # an empty set
print(a)
```

set()

- The iterable argument is optional. If it's not provided Python creates an empty set.

Unordered Collection and Optional Argument

```
a = set('spam') # string to set
print(a)
```

{'p', 'm', 'a', 's'}

- Note: Curly brackets, sets are more similar to dictionary than other objects.

```
y = set(list('happy')) # list to set
print(y)
{'p', 'h', 'a', 'y'}
```

- Note: duplicates are removed and no order among items.

Size of the Set Can Grow: Add an Element

```
a = set('spam') # string to set
print(a)
{'p', 'm', 'a', 's'}
a.add('k')
print(a)
{'p', 'm', 'a', 's', 'k'}
```

- The *add()* function can be used to add new element to the set.

Size of the Set Can Grow: Merging

```
a = set('spam') # string to set
print(a)
{'p', 'm', 'a', 's'}
a.update(set('xy'))
print(a)
{'p', 'a', 'y', 'm', 'x', 's'}
```

- The *update()* method or "|=" is used to merge two sets.

Size of the Set Can Shrink

- Various functions are available to delete elements from the sets.

s.remove(x): removes element x from set s; raises KeyError if not present.

s.discard(x): removes x from set s if present. All or only one x?

s.pop(): remove and return an arbitrary element from s; raises KeyError if empty

s.clear(): remove all elements from sets

- As an exercise, try these functions on sets are check their functionality.

Set Operations

Operation	Equivalent	Result
len(s)		cardinality of set *s*
x in s		test *x* for membership in *s*
x not in s		test *x* for non-membership in *s*
s.issubset(t)	s <= t	test whether every element in *s* is in *t*
s.issuperset(t)	s >= t	test whether every element in *t* is in *s*
s.union(t)	s | t	new set with elements from both *s* and *t*
s.intersection(t)	s & t	new set with elements common to *s* and *t*
s.difference(t)	s - t	new set with elements in *s* but not in *t*
s.symmetric_difference(t)	s ^ t	new set with elements in either *s* or *t* but not both
s.copy()		new set with a shallow copy of *s*

S1 = {1, 2, 3, 4}

S1 & {1, 3} # Intersection

{1, 3}

{1, 5, 3, 6} | S1 # Union

{1, 2, 3, 4, 5, 6}

S1 - {1, 3, 4} # Difference

{2}

S1 > {1, 3} # Superset

True

Lesson-92

Arguments

Defining New Functions

‘’

Syntax for function definition

>>> def any_name(list of parameters):

statement one

---

statement last

return answer

Parameters: argument required to make function work.

Calling a Function

- The syntax for a function call is:

```
>>> def multiply(x, y):
    return x * y
>>> multiply(5, 4)
 20
```

- Argument is a value passed to a function (or method) when calling the function.

Function Arguments

- There are two types of arguments:
- *keyword argument*: an argument preceded by an identifier (e.g. name=) in a function call or passed as a value in a dictionary preceded by **.
- *positional argument*: an argument that is not a keyword argument. Positional arguments can appear at the beginning of an argument list and/or be passed as elements of an iterable preceded by *.

Keyword Arguments

- We can call a function with some or all of its arguments out of order as long as we specify their names

```
def example(x, y, z):
 return x-y
example(2, 1, 43)
1
example(z=43, y=1, x=2)
1
example(2, z=43, y=1)
1
example(z=43,2,y=1)
SyntaxError: non-keyword arg after keyword arg
example(**{'x'=2, 'y'=1, 'z'=43})
1
```

Positional Arguments

Positional arguments can appear at the beginning of an argument list and/or be passed as elements of an iterable preceded by *.

```
def example(x, y, z):
 return x-y

example(2, 1, 43)

1

example(*(2,1,43))

1

example(*[2,1,43])

1
```

Python's Higher-Order Functions

- Python supports higher-order functions that operate on lists or iterable.
- The *map()* method applies a given function to each element of a sequence. It returns a new sequence that holds the return values of each of the function calls. *map(function, sequence)*.
- The *filter()* method takes a function that returns true or false. Applies the function to each item in a list. A new list is returned that contains all the items for which the function was true. filter(function, sequence)

```
def square(x):
    return x*x
def even(x):
    return 0 == x % 2
list(map(square, range(10,20)))
  [100, 121, 144, 169, 196, 225, 256, 289, 324,          361]
  list(filter(even, range(10,20)))
  [10, 12, 14, 16, 18]
list(map(square, filter(even, range(10,20))))
  [100, 144, 196, 256, 324]
```

- More than one sequence may be passed; the function must then have as many arguments as there are sequences.
- The function is called with the corresponding item from each sequence (or None if some sequence is shorter than another).

```
seq = range(8)
def add(x, y):
    return x+y
map(add, seq, seq)
```

[0, 2, 4, 6, 8, 10, 12, 14]

Lesson-93

Comprehensions

List Comprehensions

- A list *comprehension* is a programming language construct for creating a list based on existing lists.
- A powerful and popular feature in Python
  - Generate a new list by applying a function to every member of an original list
- Python's notation:

  [expression for name in list]

```
>>> li = [3, 6, 2, 7]
>>> [elem*2 for elem in li]
[6, 12, 4, 14]
```

For each member of the list, the list comprehension

1. Sets variable elem equal to that member,
2. Calculates a new value using expression elem*2,
3. It then collects these new values into a list which is the return value of the list comprehension.

- If list contains elements of different types, then expression must operate correctly on the types of all of list members.
- If the elements of list are other containers, then the name can consist of a container of names that match the type and "shape" of the list members.

```
li = [('a', 1), ('b', 2), ('c', 7)]
print([ n * 3 for (x, n) in li])
[3, 6, 21]
```

- Expression can also contain user-defined functions.

```
def add(a, b):
  return a + b
Tlist = [(6, 3), (1, 7), (5, 5)]
print([add(x, y) for (x, y) in Tlist])
[9, 8, 10]
```

Filtered List Comprehensions

- An *if* clause after the *for* loop allow us to select values based on a certain condition.

```
>>> [x for x in range(5) if x % 2 == 0]
[0, 2, 4]
>>> list(filter((lambda x: x % 2 == 0), range(5))) # using filter method
[0, 2, 4]
res = []   # regular coding
for x in range(5):
    if x % 2 == 0:
        res.append(x)
>>> res
[0, 2, 4]
```

Nested List Comprehensions

- We can code any number of nested *for* loops in a list comprehension, and each may have an optional associated *if* test.
- The general structure of list comprehensions looks like this:

```
[expression for target1 in iterable1 [if condition1]
        for target2 in iterable2 [if condition2]
        for targetN in iterableN [if conditionN]     ]
```

List Comprehensions and Matrixes

- A way to code matrix (i. e., multidimensional array) is to use nested list structures.

```
>>> M = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]]
```

- We can use normal index operations to get rows and particular columns within rows.

```
>>> M[1]
   [4, 5, 6]
>>> M[1][2]
    6
```

List Comprehensions and Files

- The *readlines()* method returns lines with "\n" end-of-line characters at the end.

```
>>> open('myfile').readlines()
 ['aaa\n', 'bbb\n', 'ccc\n']
>>> [line.rstrip() for line in open('myfile').readlines()]
['aaa', 'bbb', 'ccc']
>>> [line.rstrip() for line in open('myfile')]
['aaa', 'bbb', 'ccc']
>>> list(map((lambda line:line.rstrip()), open('myfile')))
['aaa', 'bbb', 'ccc']
```

Dictionary and Sets Comprehensions

```
>>> {base:0 for base in "ATGC"}
   {"T":0, "C":0, "G":0, "A":0}
>>> {base for base in "ATGCA"}
   {"T", "C", "G", "A"}
```

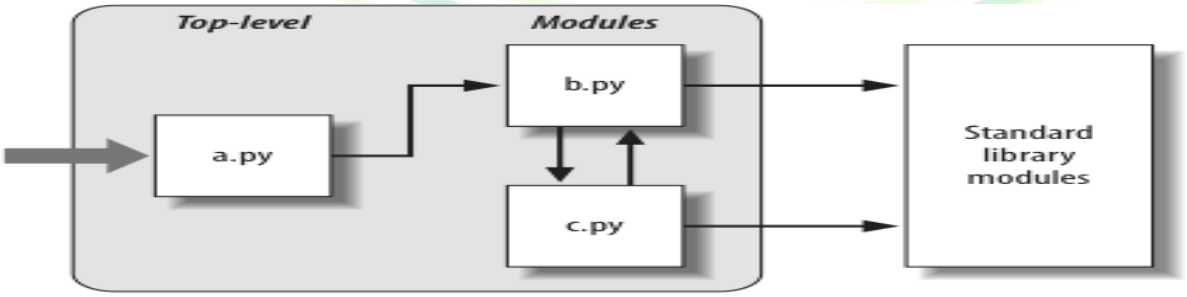
Lesson-94

Modules

Modules

- Script files contain Python variables, statements, and functions (for example name.py)
- In simple terms, every file of Python source code whose name ends in a *.py* extension is a *module*.
- Module is the highest-level program organizational unit. It is the collection of related functions.
- Large programs are divided into multiple modules (or files), that import content from each other.
- One of the module act as the main or top-level file. It is executed to start the entire program.

Program Architecture in Python



File: b.py

```
def spam(text):
  print(text,“spam”)
```

File: a.py

```
import b
b.spam(“test”)
```

Importing Modules

- To import entire module:

import module_name

e.g., import math

math.sqrt(4)

- To import specific functions

from module_name import function_name

- To import all the functions

*from module_name import **

- If functions (rather than module) are imported than there is no need to use the dot notation.

Importing Whole Module

File: module1.py

```
def spam(text):
  print(text, "spam")
def printer(x):
  print(x)
```

```
>>> import module1
>>> module1.printer("welcome!")
welcome!
>>> printer("welcome!")
NameError: name 'printer' is not defined
```

Importing Specific Functions from Modules

File: module1.py

```
def spam(text):
  print(text, "spam")
def printer(x):
  print(x)
```

```
>>> from module1 import printer
>>> printer("welcome!")
welcome!
>>> spam("hello")
NameError: name 'spam' is not defined
>>> module1.spam("hello")
NameError: name 'module1' is not defined
```

How import Works?

- Python performs three steps when a Module is imported.
  - Finding the module file
  - Compiling it to byte code
  - Running to modules code
- The first one is most important because you may need to tell Python where to find files to import.
- You need to know how module search path works in order to customize it.

Lesson-95

Packages

Packages

- Each package in Python is a directory, which must contain a special file called "__init__.py".
- The "__init__.py" file can be empty, and it indicates that the directory it contains is a Python package.
- The package can be imported the same way a module can be imported.
- If we create a directory called foo, which marks the package name, we can then create a module inside that package called bar.
- We need to add the "__init__.py" file inside the foo directory.
- To use the module bar, we can import it in two ways:
  - import foo.bar
  - from foo import bar

Useful Modules/Packages

Plotting: Matplotlib

Scientific: Scipy, Python bindings for R

Web Development: Django, web2py

scikit-learn: classification, regression, clustering, dimensionality reduction, model selection, preprocessing data.

Others: numpy, pandas, os, sys, csv

Visit the following URL for details:
https://pypi.python.org/pypi?%3Aaction=search&term=bioinformatics

Lesson-96

Classes

Everything in Python is Object

- Everything in Python is object.
  - Lists are objects

```
type([])
<class 'list'>
```

  - 42 is an object

```
type(42)
<class 'int'>
```

  - Functions are objects

```
def foo():
  pass
type(foo)
  <class 'function'>
```

Object: collection of data + methods

- Objects are data structures containing data (or attributes or variables) and methods (or functions).

```
s = "ACAGATC" # object of type string
```

  - objects can contain data

```
s = "ACAGCTC"
```

  - It can be modified/manipulated

```
s.count('A')
s.lower()
```

Standard data types *vs.* User-defined types

- Python standard data types

For most simple programs, we can use standard Python data types. E.g., numbers, strings, tuples, lists, sets, dictionaries.

- Defining your own data types

It might be useful though to create your own data types: objects built to your own specifications and organized in the way convenient to you

Object Oriented Programming (OOP)

- Python class is used to implement user-defined data types.
- Classes apply and extend the ideas we have covered so far.
- Classes are packages of functions that use and process built-in object types.

OOP is Optional in Python

- OOP is optional in Python and we don't need to declare a class to get started.
- Classes require some upfront planning, they are more useful for the long-term project development.
- The proper use of classes can cut the development time significantly.

How to Declare Class?

- To declare a class, use class statement followed by a name and a colon.

class <name>:

<body>

- Assignment statements inside class generates class attributes.

class FirstClass:

x = 0

y = 0

Generating Instances and Accessing Attributes

class FirstClass:

x = 0

y = 0

- Calling a class generates it's instances. We can generate as many instances as we want.

p1 = FirstClass()

```
print(p1.x)     # use dot notation to access
0
p1.x = 8
print(p1.x)
8
class FirstClass:
        x = 0
    y = 0
```

- We can generate as many instances as we want.

```
p1 = FirstClass()
p2 = FirstClass()
p1.x = 8
p2.x = 5
print(p1.x, p2.x)
8 5
```

Defining Class Methods

- The function's def statements inside a class generates methods to process data.

```
def name(self, parameter,..,parameter):
        statements
```

- *self* must be the first parameter to any object method. Attributes must be accessed through self.

sign of bright future

Lesson-97

Classes-II

Constructors

- A constructor is a special method with the name *__init__*.

```
def __init__(self, parameter,..,parameter):
        statements
class FirstClass:      # example
    def __init__(self, m, n):
```

```
        self.x = m
        self.y = n
p = FirstClass(3,4)
print(p.x, p.y)
```

3 4

Superclass

- A class can extend the definition of another class
  - Allows use (or extension) of methods and attributes already defined in the previous one.
  - New class is called *subclass*
  - Original class is called *parent*, *ancestor* or *superclass*

```
class subclass(superclass):
```

- Multiple inheritance is also supported.

```
class subclass(superclass,superclass):
class C1 (C2, C3):
        statements
```

- In terms of search tree, an instance inherits attributes from it's class, and a class inherits attributes from all classes above it in the tree.
- Search for a attribute starts from bottom to top, and from left to right.

Redefining Methods in Subclass

- To redefine a method of the superclass, include a new definition using the same name in the subclass.
  - The old code in the superclass won't get executed.

```
class FirstClass:
    x, y = 0, 0
        def add(self, m, n):
            self.x += m
            self.y += n
class SecondClass (FirstClass):
    z = 0
    def add(self, t):
            self.z += t
p = SecondClass()
```

```
p.add(3)
print(p.x, p.y, p.z)
```

0 0 3

Extending Constructors

- Like other methods, we can call constructor of the superclass in subclass.

```
class FirstClass:
        def __init__(self, m, n):
            self.x = m
            self.y = n
class SecondClass (FirstClass):
    def __init__(self, m, n, t):
            FirstClass.__init__(self, m, n)
            self.z = t
p = SecondClass(6, 9, 3)
print(p.x, p.y, p.z)
```

6 9 3

Private Data and Methods

- Any attribute or method with two leading under-scores in its name (but none at the end) is private and can't be accessed outside of class

```
class FirstClass:
        def __init__(self, m, n):
            self.__x = m
            self.__y = n
p = Firstlass(6, 9)
print(p.__x, p.__y)
```

AttributeError: 'FirstClass' object has no attribute '__x'

Private Methods

- **Private methods.**

```
class FirstClass:
        def __init__(self, m, n):
            self.__x = m
```

```
        self.__y = n
    def __add(self):
        return self.__x + self.__y
    def getHidden(self):
        print(self.__add())
p = Firstlass(6, 9)
p.__add()
AttributeError: 'FirstClass' object has no attribute '__add'
p.getHidden()
15
```

Positional and Keyword Arguments

- The positional and keyword arguments hold in case of classes as well.

```
class FirstClass:
        def __init__(self, m, n):
            self.x = m
            self.y = n
p = Firstlass()
TypeError: __init__() missing 2 required positional arguments: 'm' and 'n'
```

- The positional and keyword arguments hold in case of classes as well.

```
class FirstClass:
        def __init__(self, m = 5, n = 4):
            self.x = m
            self.y = n
p = Firstlass()
print(p.x, p.y)
5 4
p1 = Firstlass(n=10)
print(p1.x, p1.y)
5 10
p2 = Firstlass(n=10,m = 50)
```

```
print(p2.x, p2.y)
```

```
50 10
```

Lesson-98

Exceptions

Exceptions

- Thus far when our Python scripts have crashed we've assumed there is something wrong with the script and tried to fix it.

```
dictionary = {}
data = open(filename, 'r')
for line in data:
    key, value = line.split()
    dictionary[key] = value
data.close()
```

- Sometimes external factors can cause our script to crash.

```
>>> line = "Too many values"
>>> key, value = line.split()
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: too many values to unpack (expected 2)
>>> line = "notenough!"
>>> key, value = line.split()
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: need more than 1 value to unpack
```

- The goal is to write scripts that can deal with failures caused by "unexpected" changes in external circumstances.

Error Handling

- When an error occurs in our script, Python stops it.
- So we need to take control from Python (when errors occurs).
- We need to define some remedial actions by ourselves. This is called error handling.

How does Python deal with errors?

- Errors in Python are called ***"exceptions"***.
- When something goes wrong, Python will ***"raise an exception"***, i.e. generate an error.
- The code that deals with such errors is called an ***"exception handler"***.
- If our script does not have an exception handler for the exception that occurred, then ***Python's default exception handler*** is used.

Python's default exception handler

- It halts our script and prints out some information about what has gone wrong.

wrong1.py

Traceback (most recent call last):

File "wrong1.py", line 1, in <module>

print(Hello) 

NameError: name 'Hello' is not defined

Types

- Python has a variety of different types of exception. These are called *built-in exceptions*.
- We define our own types of exceptions. These are called *user-defined exceptions*.

Exception handling: try…except

```
try:
        Python commands
except:
        Exception handler
try:
        Python commands
except ExceptionType:
        Exception handler
dictionary = {}
data = None
try:
    data = open('filename', 'r')
    for line in data:
```

```
                key, value = line.split()
                dictionary[key] = value
        data.close()
except IOError:
        print('Problem with file')
        print('Aborting')
        if type(data) != type(None):
                data.close()
```

Fix the Exception handler

```
dictionary = {"A":4,"C":5}
try:
    print (dictionary["5"])
except IOError:
    print('Problem with the dictionary')
```

Lesson-99

BioPython

BioPython

- The Biopython Project is an international association of developers of freely available Python (http://www.
- python.org) tools for computational molecular biology.
- The Biopython web site (http://www.biopython.org) provides an online resource for modules, scripts,
- and web links for developers of Python-based software for bioinformatics use and research.
- Python is easy to learn, has a very clear syntax and can easily be extended with modules written in C, C++ or FORTRAN

What can I find in the Biopython package

The ability to parse bioinformatics les into Python utilizable data structures, including support for

the following formats:

- Blast output - both from standalone and WWW Blast
- Clustalw
- FASTA

- GenBank
- PubMed and Medline
- ExPASy les, like Enzyme and Prosite
- SCOP, including `dom' and `lin' les
- UniGene
- SwissProt

- Files in the supported formats can be iterated over record by record or indexed and accessed via a Dictionary interface.
- Code to deal with popular on-line bioinformatics destinations such as:

  NCBI { Blast, Entrez and PubMed services

  ExPASy-Swiss-Prot and Prosite entries, as well as Prosite searches
- Interfaces to common bioinformatics programs such as:

  Standalone Blast from NCBI

  Clustalw alignment program

  EMBOSS command line tools
- A standard sequence class that deals with sequences, ids on sequences, and sequence features.
- Tools for performing common operations on sequences, such as translation, transcription and weight calculations.
- Code to perform classication of data using k Nearest Neighbors, Naive Bayes or Support Vector Machines.
- Code for dealing with alignments, including a standard way to create and deal with substitution matrices.
- Code making it easy to split up parallelizable tasks into separate processes.
- GUI-based programs to do basic sequence manipulations, translations, BLASTing, etc.
- Extensive documentation and help with using the modules, including this le, on-line wiki documentation,the web site, and the mailing list.
- Integration with BioSQL, a sequence database schema also supported by the BioPerl and BioJava projects.

Lesson-100

Installing Biopython

Installing Biopython

(http://biopython.org/wiki/Download)

This is usually as simple as the standard commands:

python setup.py build

python setup.py test

sudo python setup.py install

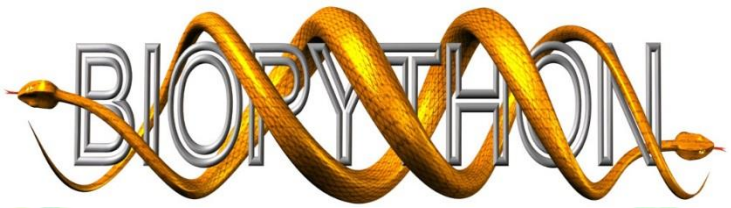
The longer version of our installation instructions covers installation of Python, Biopython dependencies and Biopython itself. It is available in

PDF http://biopython.org/DIST/docs/install/Installation.pdf

HTML formats http://biopython.org/DIST/docs/install/Installation.html

Lesson-101

File Parsing Biopython



Simple FASTA parsing example

If you open the lady slipper orchids FASTA le ls orchid.fasta in your favourite text editor, you'll see that the le starts like this:

```
>gi|2765658|emb|Z78533.1|CIZ78533 C.irapeanum 5.8S rRNA gene and ITS1 and ITS2 DNA
CGTAACAAGGTTTCCGTAGGTGAACCTGCGGAAGGATCATTGATGAGACCGTGGAATAAACGATCGAGTG
AATCCGGAGGACCGGTGTACTCAGCTCACCGGGGGCATTGCTCCCGTGGTGACCCTGATTTGTTGTTGGG
```

```
from Bio import SeqIO
for seq_record in SeqIO.parse("ls_orchid.fasta", "fasta"):
    print(seq_record.id)
    print(repr(seq_record.seq))
    print(len(seq_record))
```

You should get something like this on your screen:

```
gi|2765658|emb|Z78533.1|CIZ78533
Seq('CGTAACAAGGTTTCCGTAGGTGAACCTGCGGAAGGATCATTGATGAGACCGTGG...CGC', SingleLetterAlphabet())
740
...
gi|2765564|emb|Z78439.1|PBZ78439
Seq('CATTGTTGAGATCACATAATAATTGATCGAGTTAATCTGGAGGATCTGTTTACT...GCC', SingleLetterAlphabet())
592
```

GenBank file ls_orchid.gbk

```
from Bio import SeqIO
for seq_record in SeqIO.parse("ls_orchid.gbk", "genbank"):
    print(seq_record.id)
    print(repr(seq_record.seq))
    print(len(seq_record))
```

This should give:

```
Z78533.1
Seq('CGTAACAAGGTTTCCGTAGGTGAACCTGCGGAAGGATCATTGATGAGACCGTGG...CGC', IUPACAmbiguousDNA())
740
...
Z78439.1
Seq('CATTGTTGAGATCACATAATAATTGATCGAGTTAATCTGGAGGATCTGTTTACT...GCC', IUPACAmbiguousDNA())
592
```

Lesson-102

Quick Start Biopython

Working with sequences

>>> from Bio.Seq import Seq

>>> my_seq = Seq("AGTACACTGGT")

>>> my_seq

Seq('AGTACACTGGT')

>>> print(my_seq)

AGTACACTGGT

>>> my_seq.alphabet

Alphabet()

>>> my_seq

Seq('AGTACACTGGT')

```
>>> my_seq.complement()

Seq('TCATGTGACCA')

>>> my_seq.reverse_complement()

Seq('ACCAGTGTACT')
```

Lesson-103

Sequence and Alphabets Biopython

Simple FASTA parsing example

The alphabet object is perhaps the important thing that makes the Seq object more than just a string.

The currently available alphabets for Biopython are dened in the Bio.Alphabet module.

IUPAC alphabets (http://www.chem.qmw.ac.uk/iupac/)

Bio.Alphabet.IUPAC provides basic denitions for proteins, DNA and RNA, but additionally provides the ability to extend and customize the basic denitions

```
>>> from Bio.Seq import Seq
>>> my_seq = Seq("AGTACACTGGT")
>>> my_seq
Seq('AGTACACTGGT')
>>> my_seq.alphabet
Alphabet()
```

However, where possible you should specify the alphabet explicitly when creating your sequence objects, in this case an unambiguous DNA alphabet object:

```
>>> from Bio.Seq import Seq
>>> from Bio.Alphabet import IUPAC
>>> my_seq = Seq("AGTACACTGGT", IUPAC.unambiguous_dna)
>>> my_seq
Seq('AGTACACTGGT', IUPACUnambiguousDNA())
>>> my_seq.alphabet
IUPACUnambiguousDNA()
```

Unless of course, this really is an amino acid sequence:

```
>>> from Bio.Seq import Seq
>>> from Bio.Alphabet import IUPAC
>>> my_prot = Seq("AGTACACTGGT", IUPAC.protein)
>>> my_prot
Seq('AGTACACTGGT', IUPACProtein())
>>> my_prot.alphabet
IUPACProtein()
```

Lesson-104-110 Is not included

Lesson-111

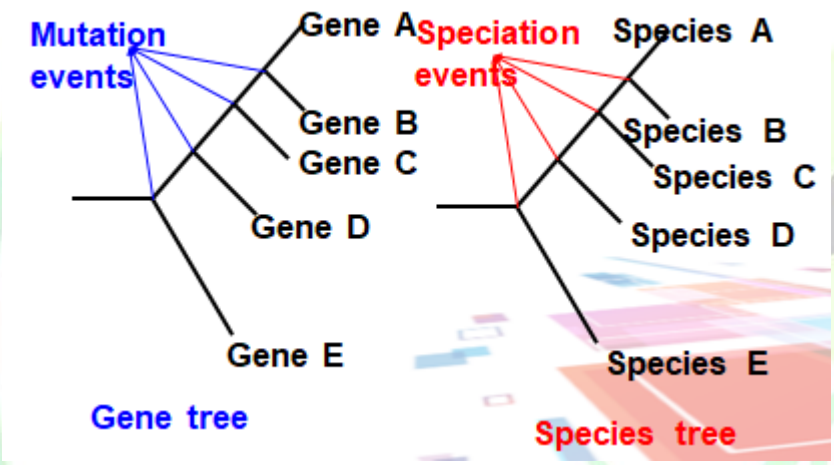
Phylogenetics

Outline

- Introduction
- Vocabulary
- Conclusion

Introduction

- It is the study of inferring evolutionary relationships
- Phylogenetic Trees



Vocabulary

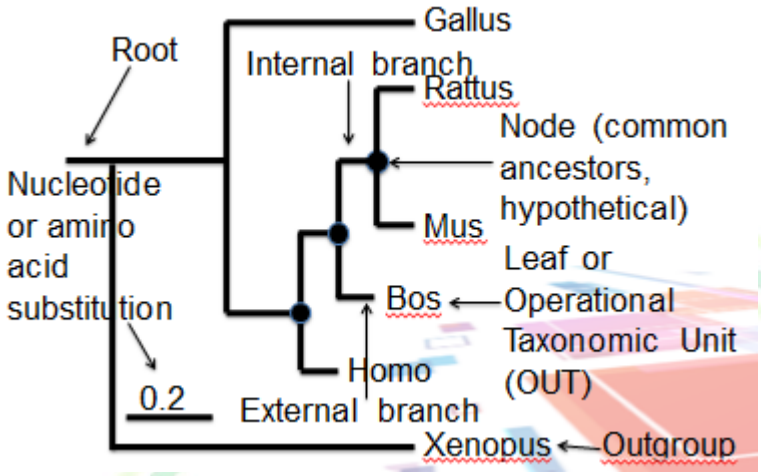
- **Nodes:** Represent taxonomic units

  Internal nodes correspond to ancestral species that are not part of the data
- **Internal branch:** Between 2 nodes

  Internal nodes are connected by internal brances
- **Leaf / Operational Taxonomic Units:**

  The terminal units appear at the tip of the branches (Species, genes or populations)
- External branch: Between a node and a leaf or OTU (Operational Taxonomic Units)

  Leaves are connected to the rest of the tree by the external branches emanating from an internal node



Conclusion

It is the study of evolutionary relationships

Lesson-112

Phylogenetic Analysis

- Introduction
- Choosing a study group
- Sampling a group of interest
- Sequence retrieval
- Data alignment and tree construction
- Conclusion

Introduction

The main aim of phylogeny reconstruction is to describe evolutionary relationships in tems of common ancestry

Choosing a study Group:

- A good first step is to survey the litaerature pertaining to the group of interest
- This will inform about the chocie of species and genes to be included in phylogeny reconstruction

Sampling the Group of Interest:

Four main sources of data for reconstructing the phylogeny of a gene family:

- Published sequences of characterized genes
- Gene databases such as NCBI
- EST project databases
- Unpublished data from colleagues

Sequence Retrieval:

When sequences are retrieved from BLAST searches they are allocated an e-score

- The closer the e-value is to 0, the higher the degree of similarity between two sequences

Sequence Alignment

Sequence alignment can be achieved automatically or manually

- There are many softwares for the data alignment and tree construction (e.g., ClustalW, MEGA etc.)

Conclusion

The main aim of phylogeny reconstruction is to describe evolutionary relationships in tems of common ancestry

Lesson-113

Phylogenetic Analysis and MSA

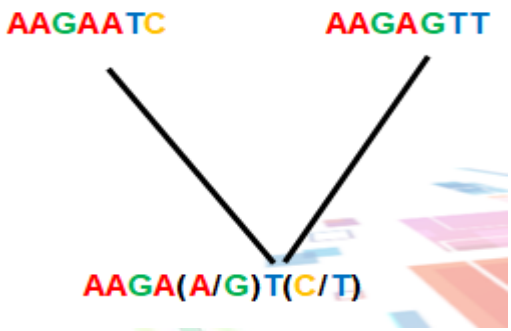
Introduction

Correct and accurate phylogenetic analysis depends upon quality of multiple sequence alignment

Significance

If two sequences in two organisms are very similar they are assumed to be derived from common ancestor

- The sequence alignments reveal which positions are conserved from the ancestor sequence



Progressive Approach

The progressive multiple alignment of a group of sequences first align ost similar pair

- Then it adds the more distant pairs
- The alignment is influenced by the “most similar“ pairs
- Does not always correclty represent the evolutionary history of the occured changes

Alternative Methods

- Most phylogenetic methods assume that each position in a sequence can change independently from the other positions

Gaps Resolution

- Gaps in alignements represent mutations in sequences
  - insertion, deletion, genetic rearrangements
- Gaps are treated in various ways by the phylogenetics methods.
- Most of them ignore gaps

Alternative to treat gaps is by **using sequence similarity scores** as the base for phylogenetic analysis

- instead of using the alignment itself, and trying to decide what happened at each position

The similarity scores based on scoring matrices (with gap scores) are used by the **Distance methods**

Conclusion

Correct and accurate phylogenetic analysis depends upon quality of multiple sequence alignment

Lesson-114

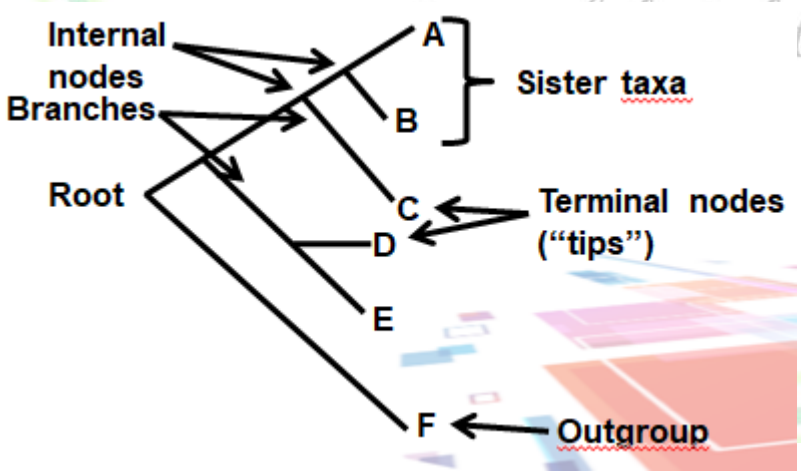
Evolutionary Tree

- Introduction
- Topologies of evolutionary tree
- Types of evolutionary tree
- Conclusion

Introduction

Also known as phylogenetic tree or dendograms

- It is a representation of evolutionary relationships between a set of organisms



Topologies of Evolutionary Tree

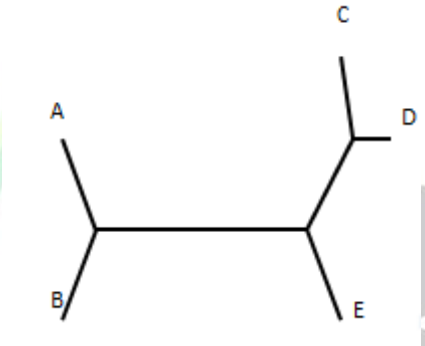
- Rooted tree
- Unrooted Tree

Rooted trees

- In a rooted tree, here exists a particular node called root, from which a unique path leads to any other node
- The direction of each path correspond to the evolutionary time
- The root is the common ancestor of all the OTU (Operational Taxonomic Units) under study

Unrooted trees

- A tree that only specifies the relationships among the OTUs but does not define the evolutionary path.
- Un-rooted trees do not make assumptions or require knowledge about common ancestors



Number of Trees

$N_R = (2n-3)!/2^{n-2}(n-2)!$

$N_U = (2n-5)!/2^{n-3}(n-3)!$

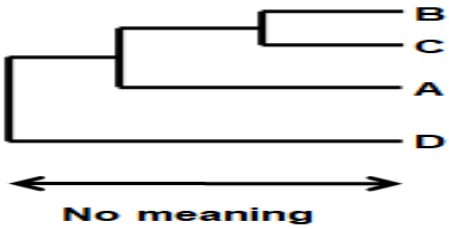
n	N_R	N_U
2	1	1
3	3	1
4	15	3
5	105	15
10	34459425	2027025

Types of Evolutionary Tree

- Cladogram
- Phylogram
- Ultrametric Tree

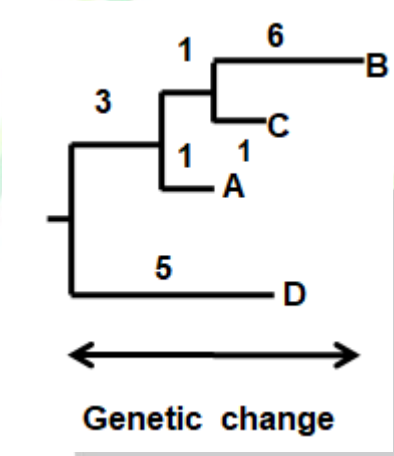
Cladogram

- They depict only the branching order which shows relationship among organisms
- The branches can be traced back to the point where they branch off representing a hypothetical ancestor



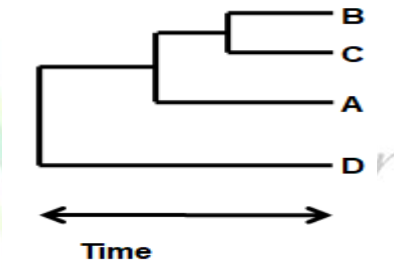
Phylogram

- These trees present branch lengths as proportional to some measure of divergence between species
- These trees typically include a scale bar to indicate the degree of divergence represented by a given length of branch



Ultrametric Tree

An ultrametric tree can be rooted so that all paths from the root to leaf have the same length



Conclusion

- Evolutionary tree is a representation of evolutionary relationships between a set of organisms

Lesson-115

Molecular Phylogenetics

Introduction

Study of evolutionary relationships between genes and species

- The actual pattern of evolutionary history is the phylogeny or evolutionary tree which we try to estimate

Evolutionary tree

- A tree is a mathematical structure which is used to model the actual evolutionary history of a group of sequences or organisms

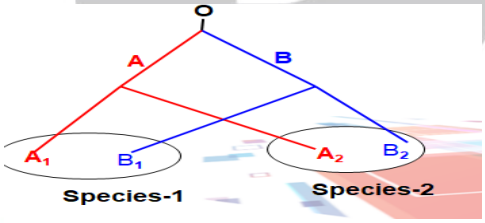
Evolutionary Process

Three major forces are at work in modifying the genetic information in any genome:

- Expansion (gene duplication)
- Deletion (gene loss)
- Exchange (HGT)

Phylogenetic Tree Reconstruction

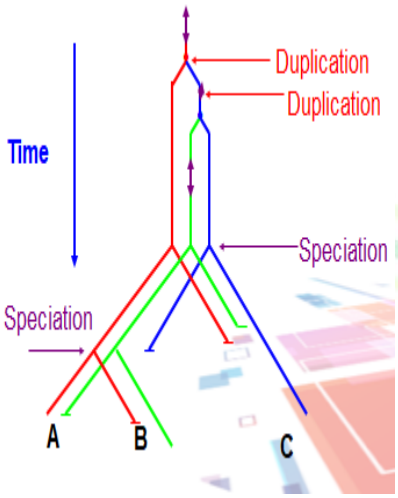
- Molecular phylogeny methods allow;
- Reconstruction of phylogenetic trees (inferred trees) from a given set of aligned sequences
- Aiming at reconstructing the history of successive divergence which took place during the evolution, between the considered sequences and their common ancestor
- These trees may not be the same as the true tree



Homologs: A_1, B_1, A_2, B_2

Paralogs: A_1 vs B_1 and A_2 vs B_2

Orthologs: A_1 vs A_2 and B_1 vs B_2



Evolutionary Process

Reconstruction of phylogenetic trees is a statistical problem

- reconstructed tree is an estimate of a true tree and the accuracy of this estimation should be statistically established

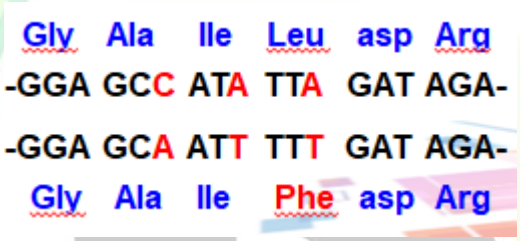
Nucleotides vs Amino Acids

- The nucleotide sequences of a pair of homologous genes have a **higher information content** than the amino acid sequences of the corresponding proteins
- synonymous changes alter the DNA sequence but do not affect the amino acid sequence

DNA yields more phylogenetic information than proteins

Synonymous vs non- Synonymous mutations

synonymous changes alter the DNA sequence but do not affect the amino acid sequence



Conclusions

Study of evolutionary relationships between genes and species

- The actual pattern of evolutionary history is the phylogeny or evolutionary tree which we try to estimate

Lesson-116[not available]

Tree Reconstruction Methods

Lesson-117

Maximum Parsimony Method

Maximum Parsimony Approach

Also known as minimum evolution method

- Evolution takes the shortest route
- To find a tree that explains the data with as few evolutionary changes as possible

Maximum Parsimony Fundamentals

1:- Mutations are exceedingly rare events

2:- A model that invokes more unlikely events (like mutations) is less likely correct

- The relationship that requires the fewest number of mutations to explain the data is more likely a correct relationship

Information content

- Sequence positions or columns in **MSA** (Multiple Seuence Alignment) may be informative or non-informative

Informative sites

- The sites or positions having **at least 2 different nucleotides**, and **each of these nucleotides has to be present at least twice** are informative

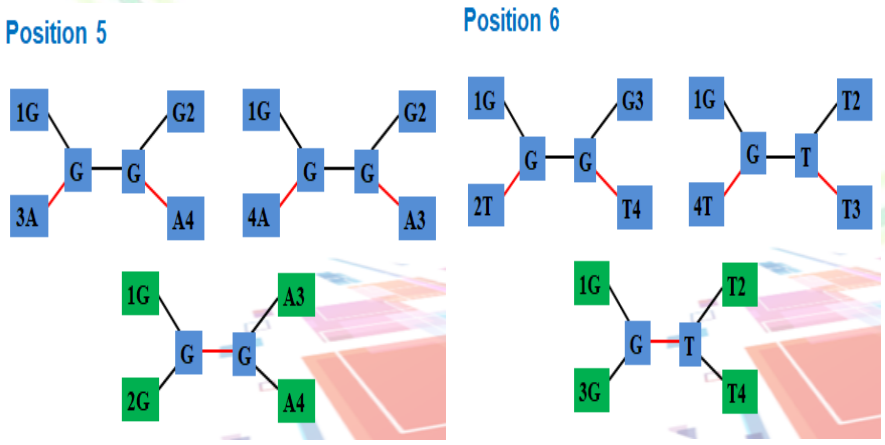
	1	2	3	4	5	6
1	G	G	G	G	G	G
2	G	G	G	A	G	T
3	G	G	A	T	A	G
4	G	A	T	C	A	T

Position 1 is said invariant and therefore uninformative, because all trees invoke the same number of mutations (0);

Position 2 is uninformative because 1 mutation occurs in all three possible trees;

Position 3 idem, because 2 mutations occur; Position 4 requires 3 mutations in all possible trees.

Positions 5 and 6 are informative, because one of the trees invokes only one mutation and the other 2 alternative trees both require 2 mutations.



Conclusions

- Also known as minimum evolution method
- To find a tree that explains the data with as few evolutionary changes as possible

Lesson-118

Maximum Likelihood Method

Introduction

Maximum Likelihood (ML) method finds the tree that maximizes the probability of genetic data given the tree

- Using a tree model for nucleotide substitutions, it will try to find the most likely tree out of all the trees given in the data set

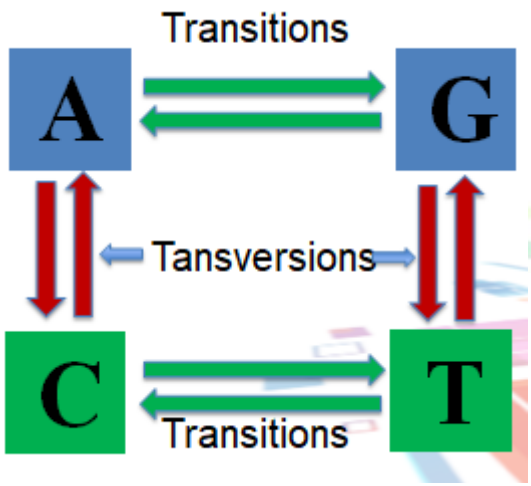
Approach

The bases are considered independent

- log-likelihood of having these bases are computed for a given topology by using a particular probability model
- This log-likelihood is added for all sites, and the sum of the log-likelihood is maximized to estimate the branch length of the tree
- This procedure is repeated for all possible topologies, and the topology that shows the highest likelihood is chosen as the final tree

Statistical Approach

- ML approach is purely a ststistical approach
- Probabilities are considered for each nucleotide substitution in a set of aligned sequences
- Thorne et al. (1991,1992) have introduced the method of sequence alignment based on a model (Bishop and Thompson 1986)
- Predicts the manner in which DNA sequences change during evolution



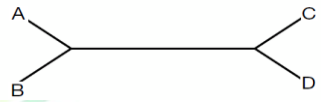
Statistical Approach

- Transitions are more common than tansversions
- The sequence with C and T are more closely related to each other than they are to the sequence with G

A. Sequences

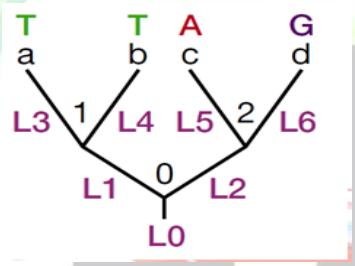
sequence a	A C G C G T T G G G
sequence b	A C G C G T T G G G
sequence c	A C G C A A T G A A
sequence d	A C A C A G G G A A

B. An unrooted phylogenetic tree for the sequences A-D

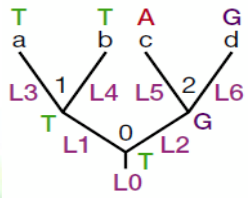


C: A rooted phylogenetic tree for one set of aligned sequences

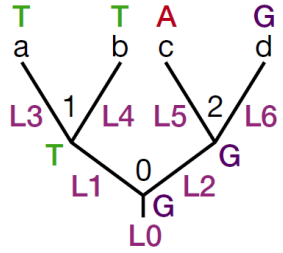
L1-L6 : Six Likelihoods



D: A rooted phylogenetic tree for one set of aligned sequences with one version of bases assigned to nodes 0,1, 2



E: A rooted phylogenetic tree for one set of aligned sequences with another version of bases assigned to nodes 0,1, 2



Likelihood of the tree

Likelihood L of the tree is

L(Tree) = L(Tree1)+L(Tree2)+---+L(Tree64)

Since there are three interior nodes (0,1,2) & there are 4X4X4=64 possible base asignments

Likelihood L of the tree1 is

$L(Tree1) = L_0 X L_1 X L_2 X L_3 X L_4 X L_6$

Likelihood of;

L_0 = starting with any nucleotide = ¼=o.25

Transitions = $2X10^{-6}$

Travrsions = 10^{-6}

Likelihood L of the tree1 is

= $0.25X2X10^{-6}X10^{-6}$

= $5X10^{-13}$

Conclusions

ML approach is a purely statistically based method. Probabilities are considered for every individual nucleotide substitution in a set of sequence alignment

Lesson-119

Distance Matrix

Distance Based Methods

- Feng and Doolittle (1996)
- Phylogenetic methods at first compute the pairwise distances from the sequences and then use these distances to obtain the trees
- Different phylogenetic programs have different distance calculation methods
  - Distance matrices are used as inputs to infer phylogenies using different Algorithms

Convert sequence data into a set of discrete pairwise distance values, arranged into a matrix

- Distance methods fit a tree to this matrix

$D_{i,j}$ = the distance between *i* and *j sequences*

$d_{i,j}$ = sum of branches on the tree path from i to j

- The phylogeny makes an estimation of the distance for each pair as the sum of branch lengths in the path from one sequence to another through the tree

Distance

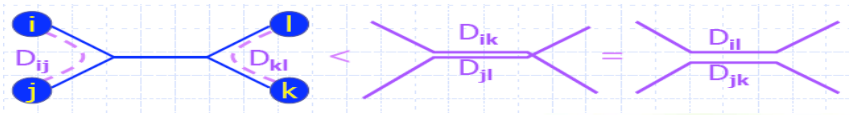
- A measure of how close is the tree to D is given by the least square criterion;
- $\sum(D_{i,j} - d_{i,j})^2 / D^2_{ij}$

Additivity

D is additive if and only if:

- For every four indices i,j,k,l, the maximum and median of the three pairwise sums are identical:

  $D_{ij}+D_{kl} < D_{ik}+D_{jl} = D_{il}+D_{jk}$



A. Sequences

sequence A	ACGCGTTGGGCGATGGCAAC
sequence B	ACGCGTTGGGCGACGGTAAT
sequence C	ACGCATTGAATGATGATAAT
sequence D	ACACATTGAGTGATAATAAT

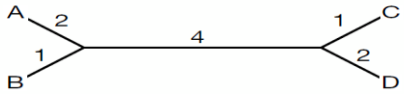
B. Distances between sequences, the number of steps required to change one sequence into the other.

n_{AB}	3
n_{AC}	7
n_{AD}	8
n_{BC}	6
n_{BD}	7
n_{CD}	3

C. Distance table

	A	B	C	D
A	–	3	7	8
B	–	–	6	7
C	–	–	–	3
D	–	–	–	–

D. The assumed phylogenetic tree for the sequences A-D showing branch lengths. The sum of the branch lengths between any two sequences on the trees has the same value as the distance between the sequences.



Conclusions

Distance based methods at first compute the pairwise distances from the sequences and then use these distances to obtain the trees

Lesson-120

UPGMA

UPGMA

- (Unweighted Pair – Group Method using arithmetic Averages)
- Assume constant mutation rate
- i.e Molecular clock hypothesis
- The two sequences with the shortest evolutionary distance between them are assumed to be the last two diverged and represented by the most racent internal node

Distance Matrix

	A	B	C	D	E
A		22	39	39	41
B			41	41	43
C				18	20
D					10
E					

Merge closely related pair

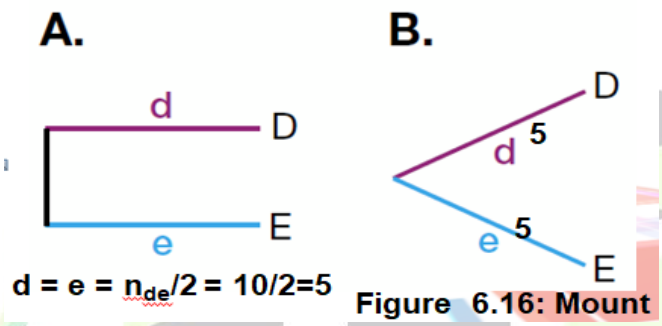
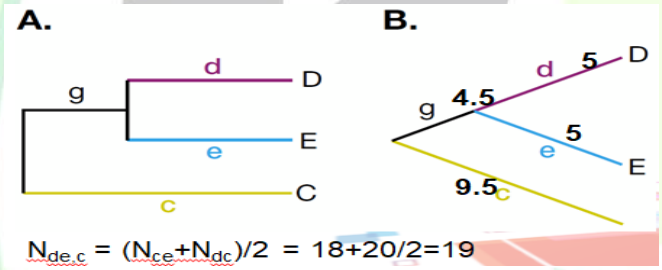


Figure 6.16: Mount



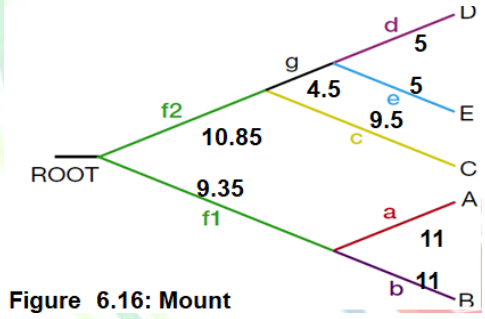


Figure 6.16: Mount

Conclusions

UPGMA assumes constant mutation rate i.e Molecular clock hypothesis

Lesson-121

Fitch-Margoliash Method

Introduction

Fitch-Margoliash (1987) method does not make the assumption of constant mutation rate

- Assume that the distances are additive

Methodology

- The sequences are combined in the groups of threes to define the branches of predicted tree and to calculate the branch lengths

Methodology

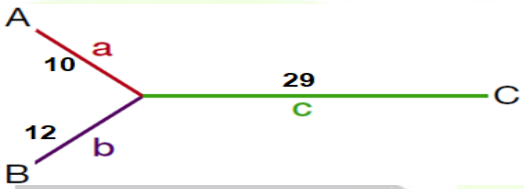
- Branch lengths a-c can be calculated using Algebra

	A	B	C
A		22	39
B			41
C			

distance from A to B $= a + b = 22$ (1)
distance from A to C $= a + c = 39$ (2)
distance from B to C $= b + c = 41$ (3)
subtract (3) from (2), $a - b = -2$ (4)
add (1) and (4), $2a = 20$, $a = 10$
from (1) and (2), $b = 12$, $c = 29$

Tree

- Showing relationship between a, b and c



Weaknesses

- This method of combining groups performs well for trees with short branches
- Presence of long branches decreases the reliability of results

Conclusions

- Assume that the **distances are additive**
- The sequences are combined in the groups of threes to define the branches of predicted tree and to calculate the branch lengths

Lesson-122

Microarray

Microarray

Tool for analyzing gene expression

- consists of a small membrane or glass slide containing samples of many genes arranged in a regular pattern
- have become incredibly popular since their inception in 1995 (Schena et al. (1995) *Science* **270**:467-70)

Gene Expression

Analyzing gene expression involves

- **Detection of mRNA** (transcriptome) present in a cell or tissue at a particular point in time
- mRNA isolated from a cell gives a snap-shot of the events occurring at that time

Gene Expression Studies

Analyzing gene expression leads to;

- Methods capable of giving a "snapshot" of RNA expression of all genes
- Can be used as diagnostic profile (e.g cancer diagnosis)
- Can show how RNA levels changes during development, exposure to stimulus, cell cycle, etc
- Provides large amounts of data
- Can help understand functions of whole systems

Principal

- The underlying principle of microarray technology is the **ability of DNA to bind to itself and to RNA**

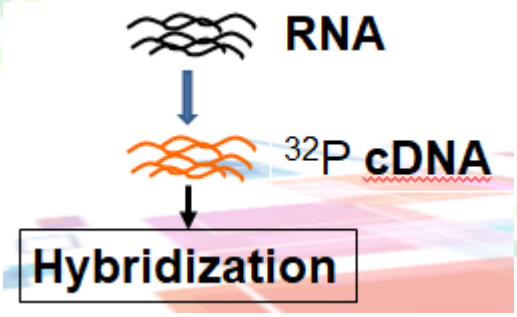
Evolution of microarray

One gene / one experiment

- Northern blots
- RNase assays
- RT-PCR

Macroarray

- Nylon Membranes
- ^{32}P labelled probes



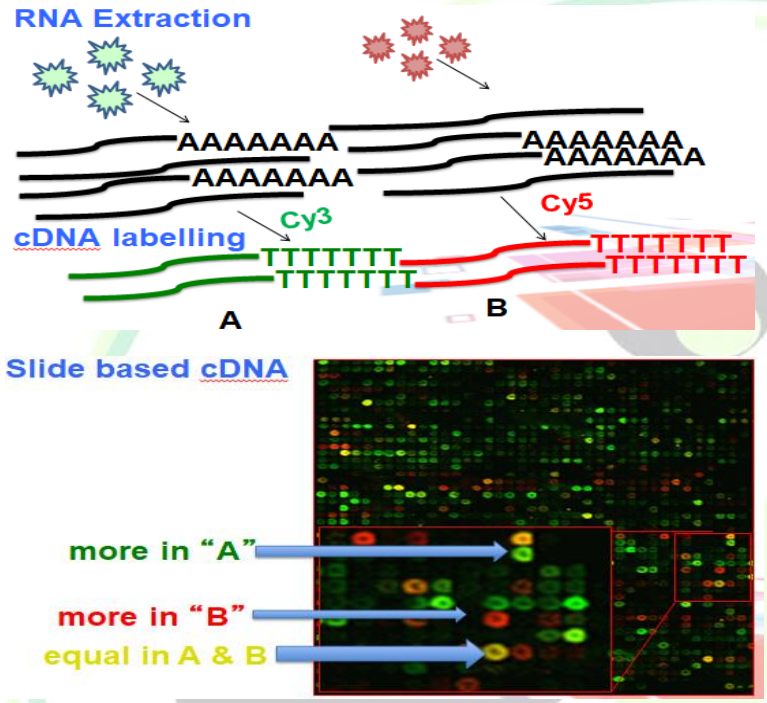
Slide based Arrays

The probe DNA is affixed directly to the surface of a glass microscope slide

- The probe DNA can range from a medium-length oligonucleotide (e.g., 60 nt) to an entire cDNA clone or larger

cDNA arrays

- >1000 nucleotides in length
- cDNA libraries



Uses of microarrays

- Gene discovery
- tissue profiles
- time course data
- altered genetic backgrounds
- Comparing tissues/genotypes

Conclusions

- Microarrays are used to investigate gene expression
- The underlying principle of microarray technology is the **ability of DNA to bind to itself and to RNA**

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-123

Microarray Types

Introduction

Microarrays are used to investigate gene expression

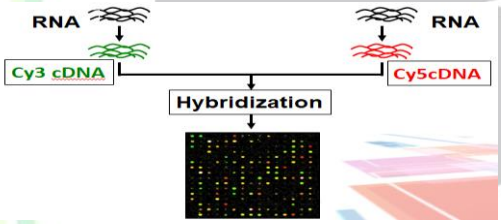
Main types of microarrays

- Slide based Arrays
  - cDNA arrays
  - oligonucleotide arrays
- Affymetrix

Slide based Arrays

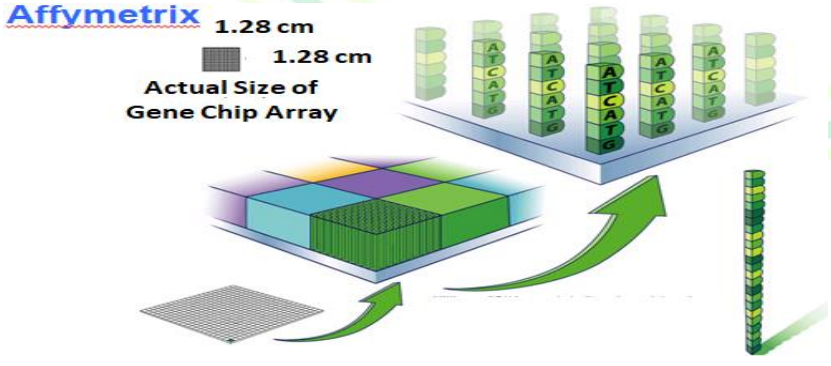
The probe DNA is affixed directly to the surface of a glass microscope slide

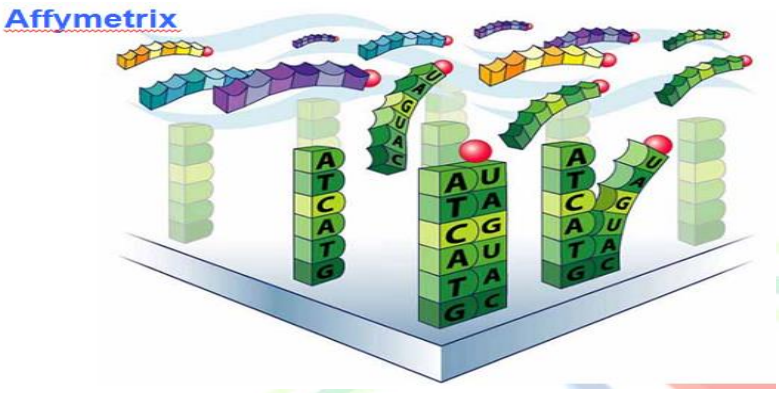
- The probe DNA can range from a medium-length oligonucleotide (e.g., 60 nt) to an entire cDNA clone or larger

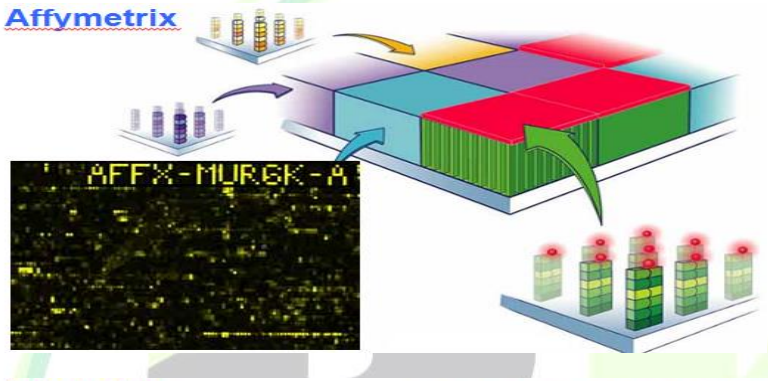


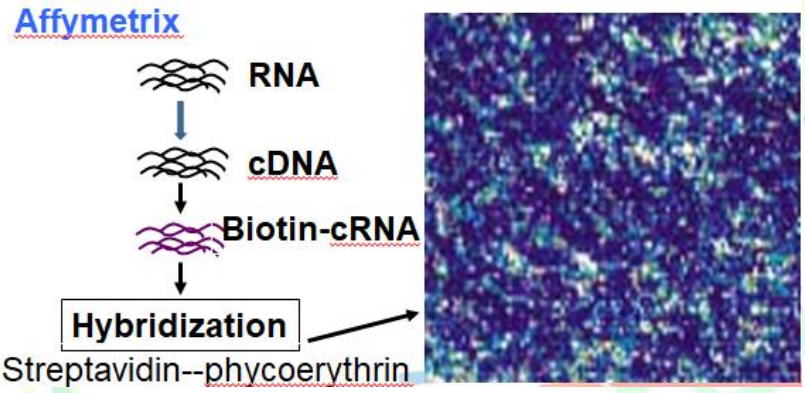
Affymetrix

- 25 nucleotides in length
- Mismatch pair
  - Single mismatch at centre of probe
  - Used to detect & eliminate false results
- 11 probe pairs per gene
- Photolithography









Slide vs Affymetrix

Affymetrix (high density oligonucleotide) and **glass slide** (cDNA, long oligo, etc) are conceptually similar, with differences in manufacture and details of design and analysis

Conclusions

Microarrays are used to investigate gene expression

Main types of microarrays

- Slide based Arrays
  - cDNA arrays
  - oligonucleotide arrays
- Affymetrix

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-124

Array design

Introduction

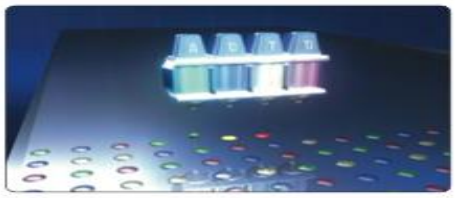
- Microarrays are based on the ability of complementary strands of DNA (or DNA and RNA) to hybridize to one another in solution with high specificity

Array Manufacturing

The DNA is deposited on the slide by any of a number of these methods;

- **"Printing"** with what is essentially an ink-jet printer
- Spotting by a robotically controlled set of fine-tipped metal pins
    - Spotted arrays

InkJet (HP/Canon) technology



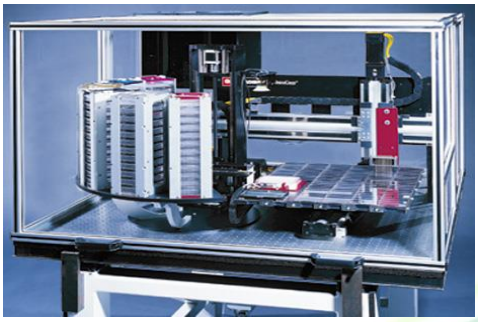


DNA spotting

- DNA attached to solid support
    - Glass, plastic, or nylon
- RNA is labeled
    - Usually indirectly
- Bound DNA is the probe
    - Labeled RNA is the "target"



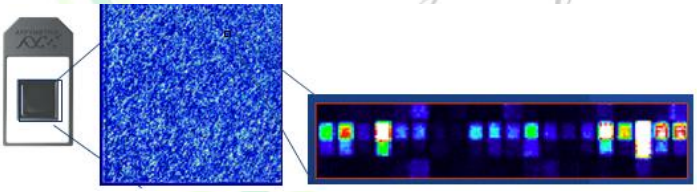
Commercial DNA Spotter

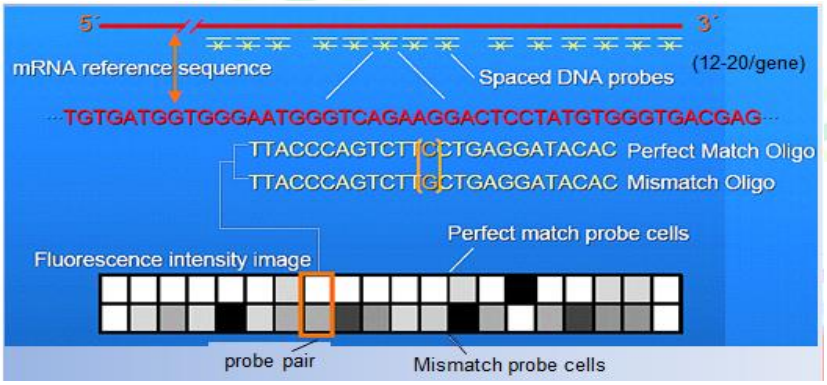


Affymetrix GeneChips

- Oligonucleotides synthesized on silicon chip
  - One base at a time
- Uses the process of photolithography
  - Developed for printing computer circuits
- Oligonucleotides for each gene selected by computer program to be the following:
  - Unique in genome
  - Nonoverlapping
  - Each gene is represented by a "probe set" consisting of 12-20 probes of 25 nt each
  - Each probe has a corresponding "mismatch" probe with a single base difference at the 13th nucleotide
  - Labeled RNA is hybridized to the array
  - Abundance is calculated using hybridization for the entire probe set, correcting the mismatch probes (indicates non-specific effects)

Benfey and Protopapas, "Genomics" © 2005 Prentice Hall Inc. / A Pearson Education Company / Upper Saddle River, New Jersey 07458





- Can hybridize only one sample/chip (i.e., no *direct* comparisons of 2 samples)
  - Standardized production tends to give good reproducibility
- Limited amount of probe sequence can be problematic
  - miss alternative splices, bias toward one end of transcript, (dependent on good genome annotation)

Attachment of sequences to slide

Covalent:

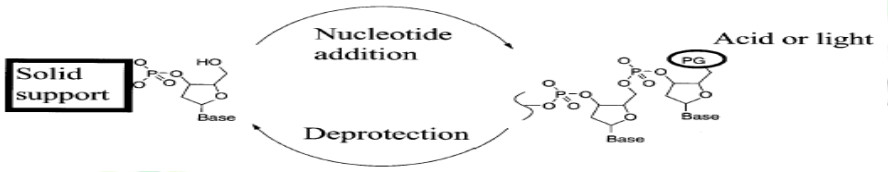
- Aliphatic amine group is added to 5' end of probe
- Probes are attached to glass from 5' end

Non-covalent:

- electrostatic attraction between amine and phosphate on probe backbone
- cDNA is attached to glass by the backbone
- only cDNA microarrays
- (oligonucleotides are too short for attraction)

In-Situ Synthesized Oligonucleotide Arrays

- built base-by-base on array surface
- Covalent reaction between the 5' hydroxyl and phosphate groups
- protective group on added 5' position to prevent multiple additions
- protective group is then converted to a hydroxyl group



Labelling

biotin: oligonucleotide Affymetrix

fluorescent dyes:

- **Cy3** (excited by laser at approximate wavelength 532 nm, fluoresces green)
- **Cy5** (excited by laser at approximate wavelength 635 nm, fluoresces red)

Conclusions

- In microarrays, complementary strands of DNA (or DNA and RNA) to hybridize to one another
- **Affymetrix** (high density oligonucleotide) and **glass slide** (cDNA, long oligo, etc) are two main methods

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-125

Array Applications

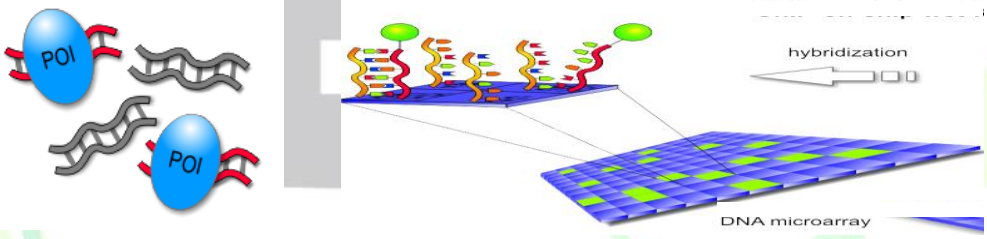
Introduction

Microarray can be combined with other methods in addition to looking at transcription ("transcriptional profiling")

- For instance, it can be used along with ChIP Chromatin Immuno-Precipitation4

This works best with whole genome tiling arrays to look at;

- Transcription factor binding
- Post Translation Modifications (PTMs) to histone proteins



RIP-chip

- Similar to ChIP-chip but for discovering RNA binding proteins rather than DNA binding proteins

aCGH

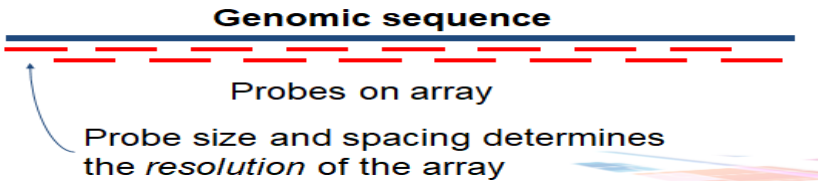
CGH (comparative genomic hybridization) looks at cytogenetic abnormalities

- Genomic DNA hybridized to array
- Often uses large clones (e.g., BACs) as array features

Tiled Arrays

- Cover a genomic region (or the whole genome!) at high coverage
- Probes are designed to cover whole sequence usually excluding only simple sequence repeats
- There is no bias toward known transcribed regions

sign of bright future



Protein binding microarrays (PBMs)

Protein-binding microarrays can be used to identify transcription factor binding sequences (motifs)

- double-stranded DNA probes used on array
- purified protein hybridized to array
- detected by antibody to protein
- can use real genomic sequence or carefully designed oligonucleotides
- possible to look at all possible 10-mer nucleotide sequences on a single array!

Conclusions

- In microarrays, complementary strands of DNA (or DNA and RNA) to hybridize to one another
- **Affymetrix** (high density oligonucleotide) and **glass slide** (cDNA, long oligo, etc) are two main methods

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

sign of bright future

Lesson-126
Experimental Design

Introduction

There are a number of important experimental design considerations for a microarray experiment:

- **Replicates**
- **amplification of RNA**
- **dye swaps**
- **reference samples**

Replicates

Replicates reduce variability in summary statistics and allow to use formal statistical methods

- At least three replicates per treatment are recommended

Lee et al. (1999)

Technical vs biological replicates

Technical replicates are repeat hybridizations using the same RNA isolate

Biological replicates use RNA isolated from separate experiments/experimental organisms

- Although technical replicates can be useful for reducing variation due to hybridization, imaging, etc., biological replicates are necessary for a properly controlled experiment

RNA Amplification

- Amplification methods can be used to increase the amount of RNA so that microarray experiments can be performed using very small numbers of cells

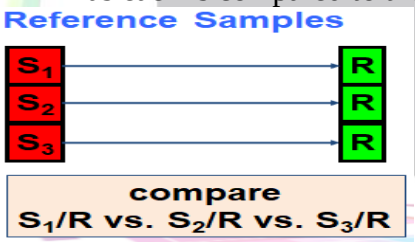
Dye Swaps

- When using 2-color arrays, it's important to hybridize replicates using a dye-swap strategy in which the colors (labels) are reversed between the two replicates.
- There can be biases in hybridization intensity due to **which dye is used**



Reference Samples

- One common strategy is to use a ***reference sample*** in one channel on each array.
- This is usually something that will hybridize to most of the features (e.g., a complex RNA mixture).
- Using a reference sample allows comparisons to be made between different experimental conditions, as each is compared to the common reference



Data Validation

It's strongly recommended that any key findings be verified by independent means

- Northern blots and quantitative RT-PCR are the typical ways of doing this; real-time, quantitative RT-PCR is generally the method of choice

MIAME (Minimal Information About a Microarray Experiment)

To publish a microarray experiment, it is expected to make available the following minimal information
This allows others to evaluate the data and compare it to other experimental results

EXPERIMENT DESIGN

- Type, factors, number of arrays, reference sample, qc, database accession (ArrayExpress, GEO)

Samples used, preparation and labeling
Hybridization procedures and parameters
Measurement data and specifications

- Quantitations, hardware & software used for scanning and analysis, raw measurements, data selection and transformation procedures, final expression data

Array design

- Platform type, features and locations, manufacturing protocols etc.

Conclusions

There are important experimental design considerations for a microarray experiment:

- **Replicates**
- **Amplification of RNA**
- **dye swaps**
- **reference samples**

It is expected to make available the minimal information about array

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-127
Microarray Image processing

Introduction

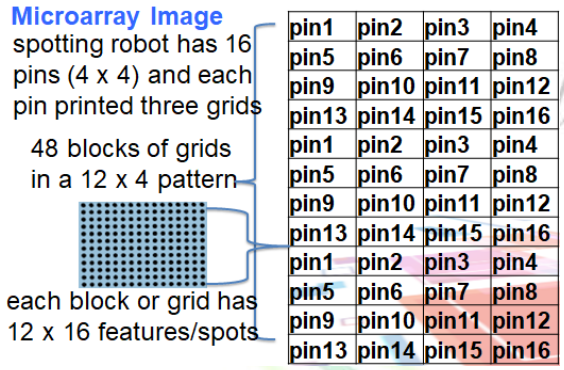
- Need to transform the fluorescence intensity image obtained after scanning into a measurement of gene expression for each spot
- Image processing for feature extraction affects the quality of the data
- In-situ synthesized oligonucleotide have integrated image-processing
- Pin-spotted arrays have multiple image processing approaches

Feature Extraction

Quantification of the digital TIFF image

Four steps:

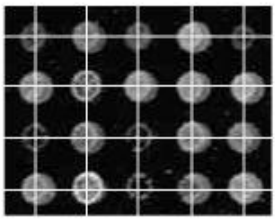
1. Identify the positions of the features on the microarray
2. For each feature, identify the pixels of the feature or foreground.
3. For each feature, identify background pixels
4. Calculate intensity of the feature, background and quality control



Feature Identification

Problems when identifying the position of the features:

- Shifted grid positions
- Curve within a grid
- Uneven feature spacing
- Uneven feature size



Software determines which pixels are in the area of a feature (segmentation):
• Fixed size circle (features may have different size)
• Variable circle (features may have irregular shape)
• Histogram (uses the frequency of intensities of the pixels)
• Adaptive shape (from center, extends to pixels with similar intensity)

Background Identification

Background adjustment accounts for:
- non-specific hybridization
- fluorescence from the glass

Image Quantification

Image-processing software provides:
• **Signal or foreground mean:**
mean of the pixels in the feature
• **Background mean:**
mean of the pixels in the background
• **Signal median:** median of pixels
(more robust to outliers than mean)
• **Background median:**
median of the pixels in the background

Image-processing software provides:
• **Signal standard deviation:**
standard deviation of pixels in the feature
• **Background standard deviation**
Median vs. standard deviation plot

File Format

The structure of a typical data file (for example .gpr file), after the images are scanned and the fluorescence intensities are translated into numbers, consists of:
- rows= microarray probe, and
- columns= information on the microarray probe (i.e. location, intensity value)

First lines describe the settings

ATF 1
15 9
..
DateTime=2006/01/05 00:32:05
Settings=
"Wavelengths=635 532"
"ImageFiles=C:\array1.tif 0 C:\array1.tif 1"
NormalizationMethod=None
..
Creator=GenePix Pro 5.0.1.24

Creator=GenePix Pro 5.0.1.24
Scanner=GenePix 4000B [50000]
Temperature=26.66
"ScanPower=100 100"

block	column	row	Flags	cy5_fore	cy5_back	cy3_fore	cy3_back
3	2	3	0	500	40	1000	50
3	2	3	0	1460	180	460	80
3	3	3	0	1030	55	1490	45
3	2	3	-100	25	20	30	21
.	.	.	.	.	.	.	.
32	24	20	0	20000	80	1000	60

Block	Column	Row	ID	F635 Median	B635 Median	F532 Median	B532 Median	Flags
1	1	1	EST1	65535	89	19969	59	0
1	2	1	EST2	45379	89	12517	59	0

Conclusions

Need to transform the fluorescence intensity image obtained after scanning into a measurement of gene expression for each spot

- Image processing for feature extraction affects the quality of the data

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

sign of bright future

Lesson-128
Microarray Normalization

Introduction

- Normalization comprises of a set of methods to remove systematic experimental errors and bias
- The normalization process includes:
  - **Data Cleaning and Transformation**
  - **Within-Array Normalization (for 2-dye data)**
  - **Between-Array Normalization (for multiple arrays)**

Data Cleaning and Transformation

The process involves:

- **Identification of flagged features**
- **Background subtraction**
- **Log transformations**

Identification of flagged features

Flag values represent quality of features like;

- Bad features (high coefficient of variation)
- Negative features (background > foreground)
- Dark features (weak foreground signal)

Flagged features

Good quality features have flag of 0

Flagged features may be dealt as;

- remove flagged features from the data set
- refer back to the original image
- adopt criterion

Background Subtraction

Background signal represents:

- non-specific hybridization to the glass
- natural fluorescence of the glass slide itself

We need to subtract background signal from the feature intensity

When background > feature intensity (foreground – background < 0):

- Remove these features from the analysis
- Use the lowest signal-intensity as background-subtracted intensity
- Estimate the true feature intensity

Log2 Transformation

Log2(Cy5/Cy3 raw intensities) = log2(Cy5)-log2(Cy3)

- Log ratio = 1 = 2-fold up-regulated gene in Cy5 sample
- Log ratio = -2 = 4-fold down-regulated in Cy5 sample
- Log ratio = 0 = not differentially expressed

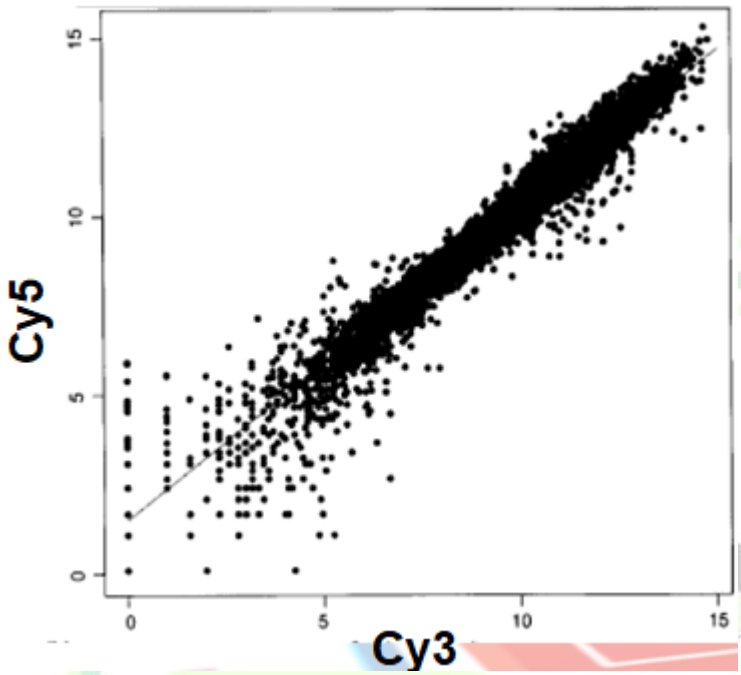
N/D = 1/16, 1/8, 1/4, 1/2, 0 1, 2, 4, 8,16

log2 = -4, -3, -2, -1, -INF, 0, 1, 2, 3, 4

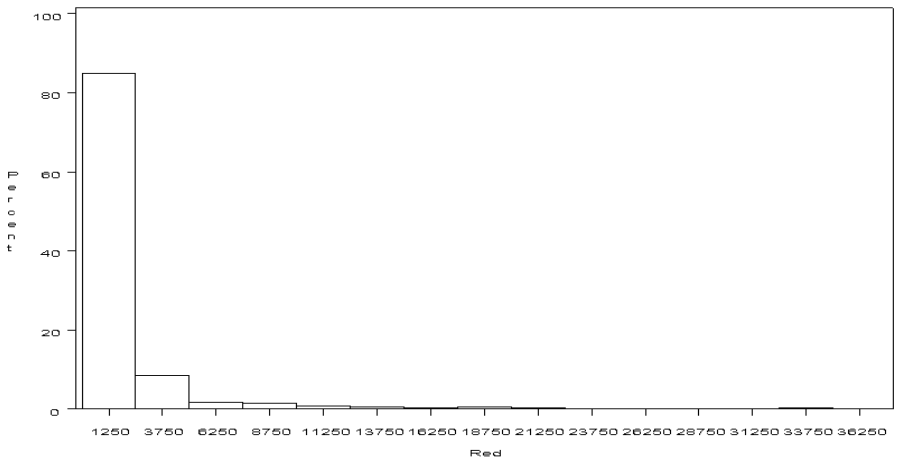
Transformation

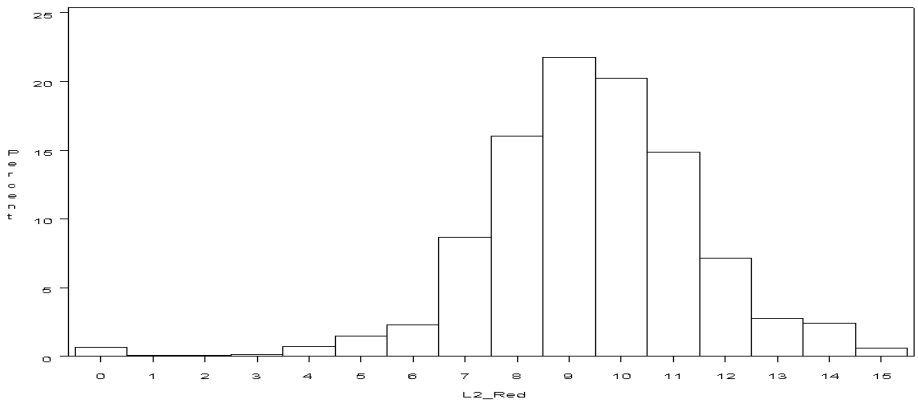
Plot of log2(Cy3) against log2 Cy5

sign of bright future



If intercept = 0 slope = 1the channels behave identically





Conclusions

Normalization removes experimental errors and bias, process includes:

- Data Cleaning and Transformation
- Within-Array Normalization
- Between-Array Normalization

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic

tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-129
Within-Array Normalization

Introduction

- Bias, experimental error must be removed prior to analysis of samples
- Most within-array normalizations are for spotted two-dye platforms
- Two-dye system measurements may introduce bias

Biases because of 2 Dye system

- Cy3 and Cy5 labels differentially incorporated (at different abundances)
- Cy3 and Cy5 dyes have different emission or response to the lasers (at different abundances)
- Cy3 and Cy5 emissions differentially measured at different intensities
- Cy3 and Cy5 intensities can vary between regions (array was not horizontal when scanned)

Genes selection

The set of genes to use for normalization must be identified:

a) All genes on the array. Assumes:

- small proportion of genes are expected to be differentially expressed
- there is a symmetry in the expression of the up/down-regulated genes
- remaining genes have constant expression

b) Constantly expressed or housekeeping genes

- expected to have a constant expression across a variety of conditions

Limitations:

- housekeeping genes tend to be highly expressed
- may not be representative of other genes of interest

c) Controls (spiked controls or titration series of control sequences)

i) Spiked controls

synthetic or foreign organism DNA sequences included in the two mRNA samples at equal amount

- expected to have equal red and green intensities

ii) Titration series approach:

- spots of the same sequence at different concentrations (intensities)
- red and green intensities are expected to be equal across concentrations

Methods to correct differential response of Cy3 and Cy5 channels

- All methods assume that most genes are not differentially expressed Otherwise use reference sample and control or spiked sequences

MA or RI plot

scatter plot of log ratio against average of log intensities of each feature

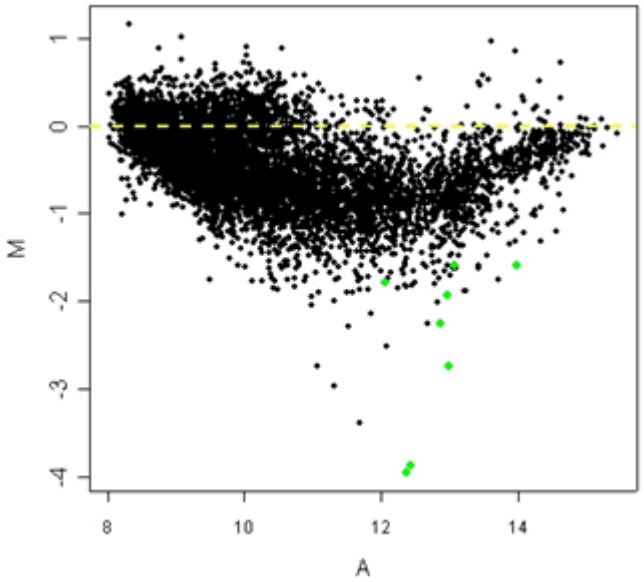
- Each point on the MA graph is a feature
- **x** coordinate: average intensity of the Cy3 and Cy5 channels
- **y** coordinate: log of the Cy5/Cy3 ratio
- Geometrically related to the log of Cy5 against the log of Cy3 scatter plots

MA or RI plot

$\mathbf{M=\log_2 R - \log_2 G}$

$\mathbf{A=(\log_2 R+\log_2 G)/2}$

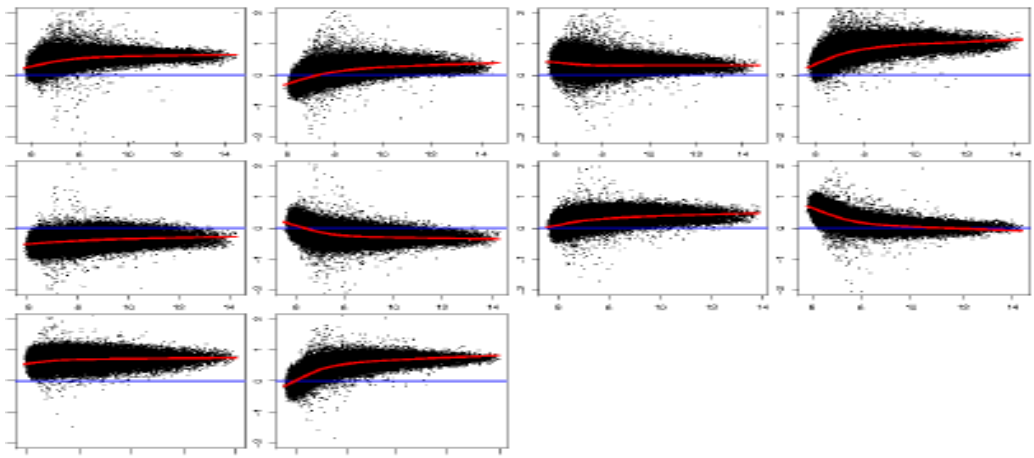
Values should scatter around 0



Linear normalization using MA plot:

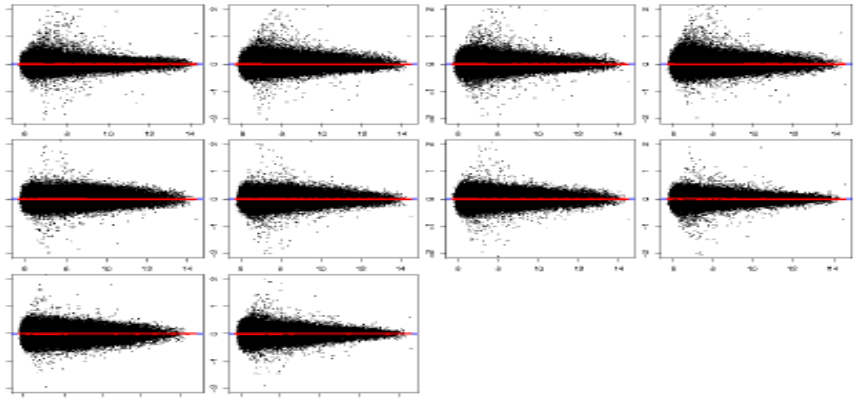
1. Perform a regression of the log ratio on the average log intensity **(MvsA)**
2. For each feature, the normalized log ratio is the observed M or log ratio value minus the predicted M value

Pairwise MA plot for 5 arrays



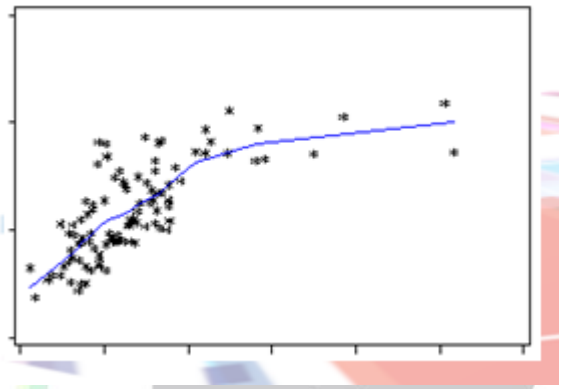
$M=\log_2 R - \log_2 G$
$A=(\log_2 R+\log_2 G)/2$

Pairwise MA plot after normalization



$M=\log_2 R - \log_2 G$
$A=(\log_2 R+\log_2 G)/2$

LOESS normalization
LOcally WEighted Scatterplot Smoothing
Fit a smooth curve
Use robust local linear fits
Effectively applies different scaling factors at different intensity levels
$Y = f(X)$
Transform X to $X' = f(X)$
Y and X' are comparable



Conclusions
Bias, experimental error must be removed prior to analysis of samples
Most within-array normalizations are for spotted two-dye platforms
Two-dye system measurements may introduce bias

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

sign of bright future

Lesson-130

Between-Array Normalization

Introduction

To compare samples in different arrays

- hybridization reactions may vary
- correct for the variability among arrays.
- variations in the distributions are due to experiment

Centring Normalization

The means of all the distributions are equal

Method: subtract the array mean log ratio or intensity from each feature

- All arrays will have a mean of zero after scale normalization.
- Alternative: median is more robust to outliers

Scaling normalization

The means and the standard deviations (SD) of all distributions are equal

- **Also known as standardization**
- subtract the mean of the array from the feature and divide by the standard deviation
- The mean of the measurements on each array will be zero
- The standard deviation will be 1 after scaling
- Alternative: median and median absolute deviation from median (MAD)
- Mean is preferred for computing distances and correlation coefficients

Quantile or distribution normalization

- The distributions of the data on each of the arrays are identical

1. Standardize the data.
2. For each array, order centered measurements from lowest to highest.
3. Compute a new distribution (use the average across all arrays for each ordered feature)
4. Replace each measurement with the corresponding new average

Original data

	A1	A2	A3
G1	1	6	7
G2	2	5	8
G3	3	4	9

Ranked data

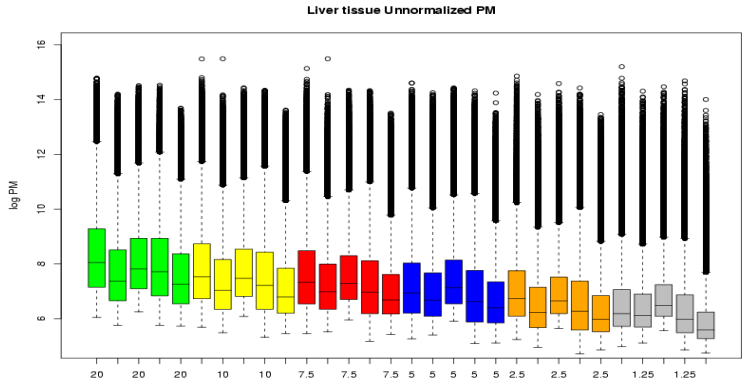
A1	A2	A3
1	4	7
2	5	8
3	6	9

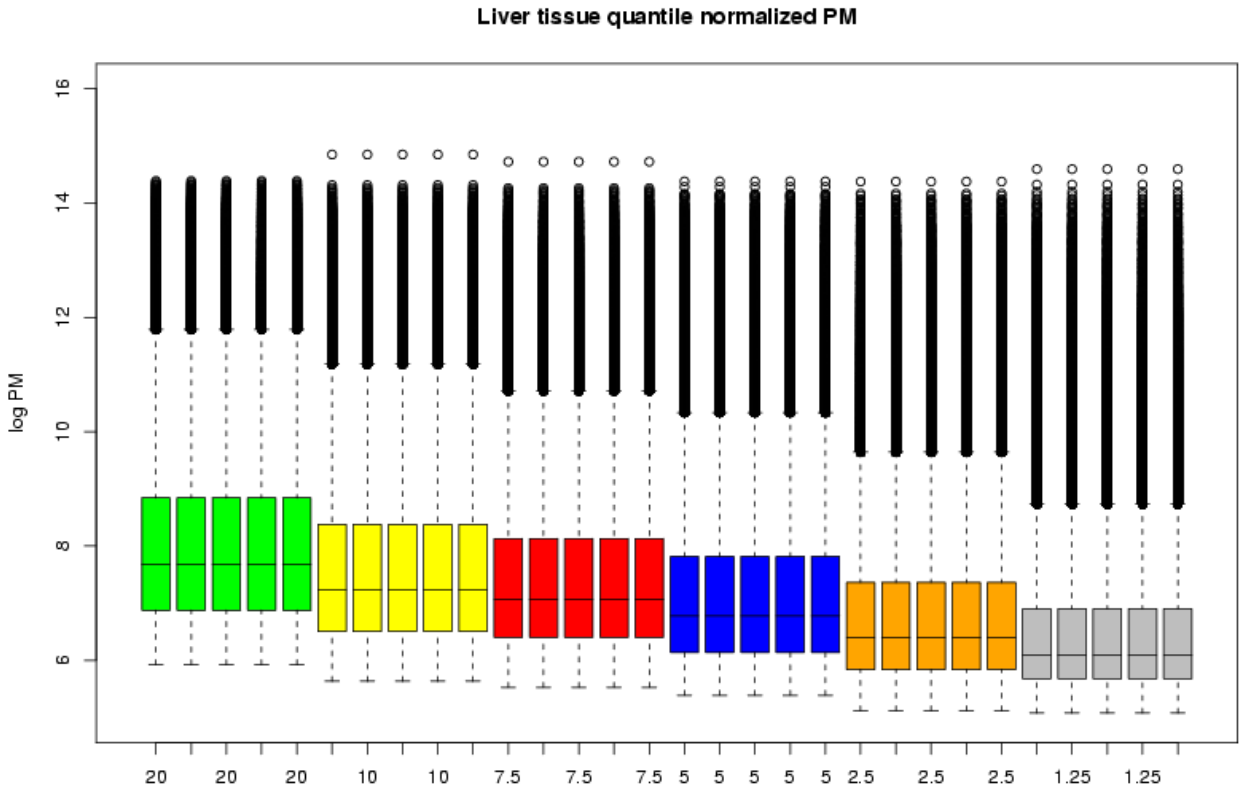
Average

4
5
6

Normalized values

	A1	A2	A3
G1	1	6	7
G2	2	5	8
G3	3	4	9





Conclusions

To compare samples in different arrays

- hybridization reactions may vary
- correct for the variability among arrays.
- variations in the distributions are due to experiment

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-131

Data Analysis

Introduction

Microarray and RNASeq can measure the expression of thousands of genes simultaneously

- Vast amounts of data require computers

Types of analysis

Gene-by-gene

- Method: Statistical techniques

Categorizing groups of genes

- Clustering algorithms

Deducing patterns of gene regulation

- Under development

Defining Gene Expression

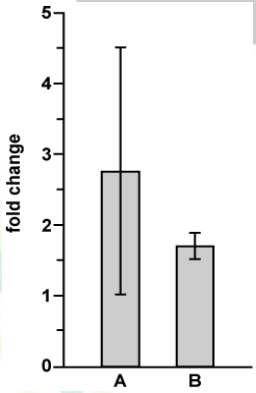
Ramsköld, *et al*. 2009, defines a threshold of

- 0.3 RPKM is optimal at different sequencing depth
- (for human and mouse data).

Differential Gene Expression

log2(A/B) = log2(average expression A) - log2 (average expression B)

- **2** $^{[\log2(A/B)]}$ = fold change in expression between conditions A and B
- Ignores variation of expression within and across conditions
- Does not discriminate absolute magnitude of expression e.g 10000/100 = 100/1



Benfey and Protopapas, "Genomics" © 2005 Prentice Hall Inc. / A Pearson Education Company / Upper Saddle River, New Jersey 07458

"A" has a fold change >2.5, but varies greatly between replicate experiments.

"B" has a fold change of only 1.75, but changes reliably each time the experiment is performed.

Hypothesis Testing

Null hypothesis: no differential expression.

Alternative hypothesis: differential expression

Hypothesis testing steps:

1) Compute test statistic of differential expression from experiment

2) Compare test statistic to distribution of test statistics

3) **Obtain P-value:** probability of a test statistic more extreme than that obtained in the experiment, by chance (if we run another experiment)

4) **Identify differentially expressed genes** as those with P-value < threshold

The smaller the P-value, the more statistically significant the differential expression of the gene and the less likely the result is to be false positive

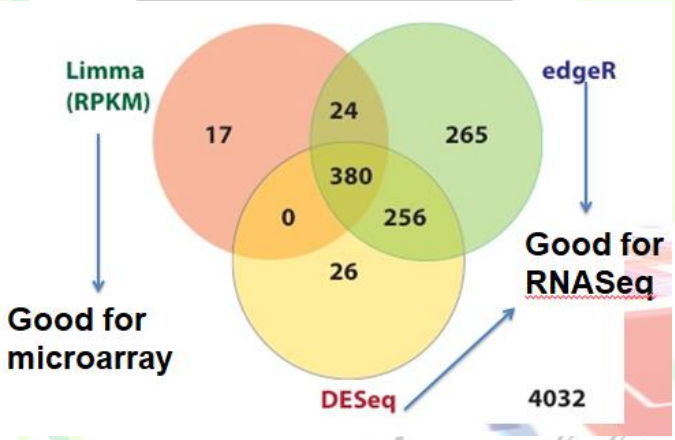
(that the observation have occurred by chance)

Bio-conductor packages:

Limma RPKM + t tests : linear models (Gordon Smyth, *2005*)

edgeR RPM + empirical Bayes estimation+exact tests : negative binomial distribution (Robinson and Oshlack, 2010)

DESeq RPM+ Estimate variance-mean dependence: negative binomial Anders and Huber, 2010)



Conclusion

Microarray and RNASeq can measure the expression of thousands of genes simultaneously

- Vast amounts of data require computers
- Bioconductor packages are efficiently used

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-132

t test

Introduction

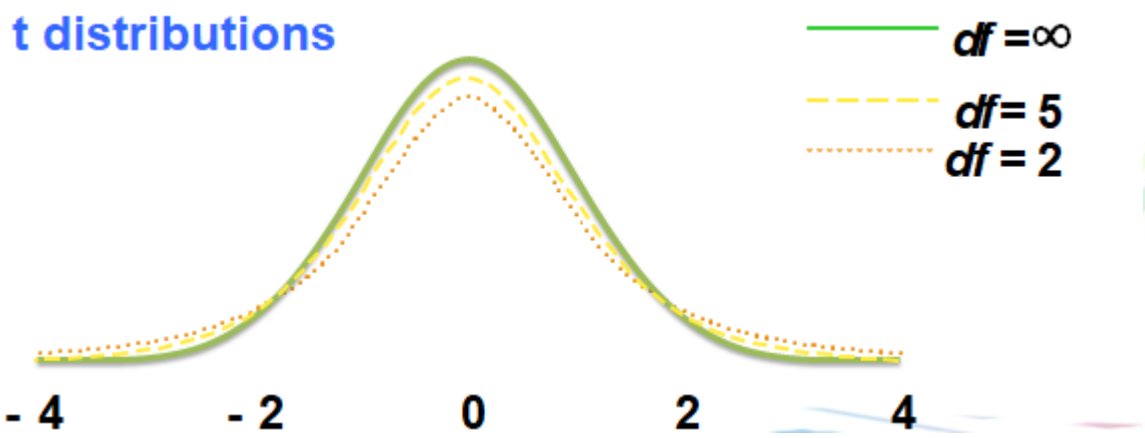
The Student's *t*-distribution is bell shaped and centered at 0 — like the Standard Normal distribution but more variable (larger spread)

- GOSSET, William Sealy 1876-1937



Assumptions

1. Population is normal although this assumption can be relaxed if sample size is "large"
2. Random sample was drawn from the population of interest

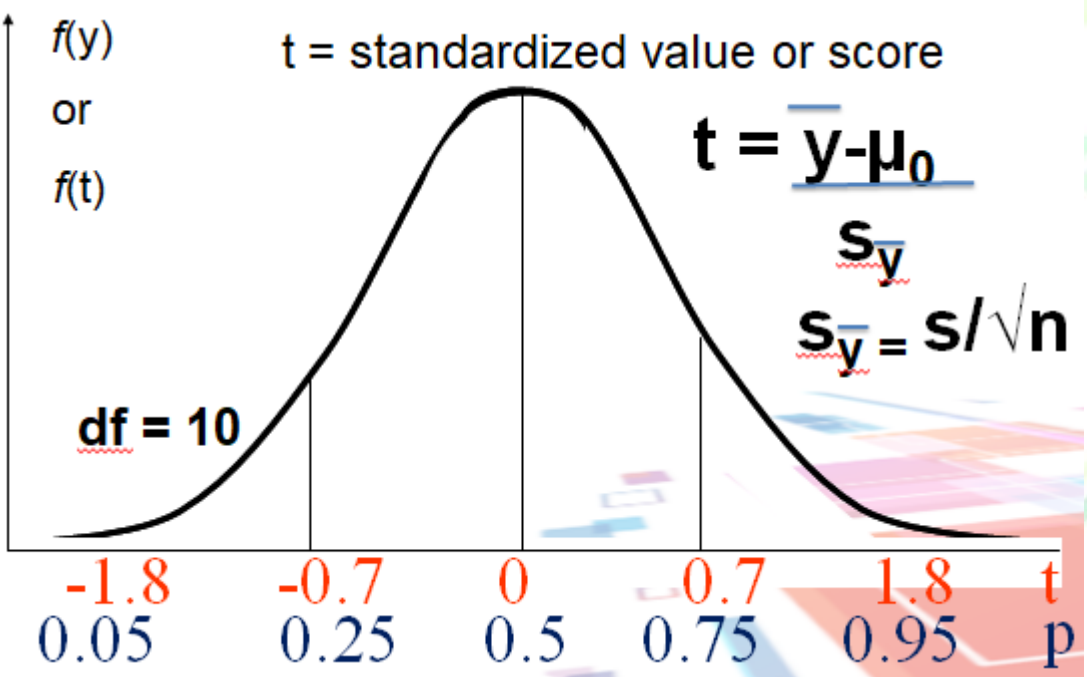


The t-distribution has one parameter that controls it's shape called the degrees of freedom.

As df increases, the t-distribution becomes more and more like the standard normal.

Degree of Freedom





(There are two red fours in a deck of 52, the 4 of hearts and the 4 of diamonds).

t or Z

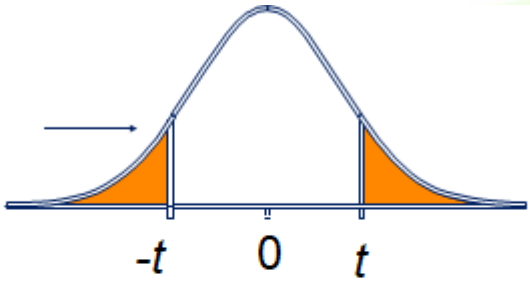
If n < 30 apply t else Z

t approaches z if $n \geq 30$

Form of Hypothesis

Ho: $\mu = \mu_o$

H_A: $\mu \neq \mu_o$ (two-tailed test)

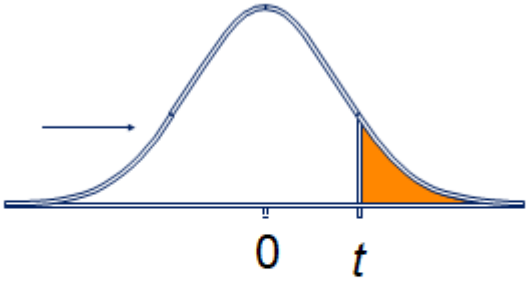


P-value = Shaded Area

t is either positive or negative

Ho: $\mu \leq \mu_o$

H_A: $\mu > \mu_o$ **(right-tailed test)**

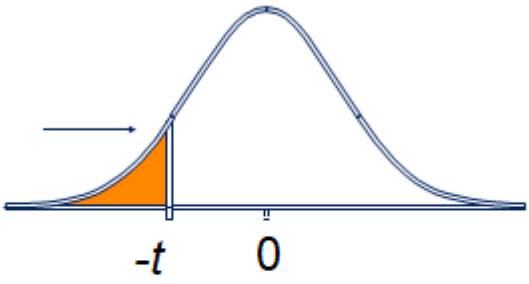


P-value = Shaded Area

t is positive

Ho: $\mu \geq \mu_o$

H_A: $\mu < \mu_o$ (left-tailed test)



P-value = Shaded Area

t is negative

One Sample t test

Testing against a standard or a claim

e.g Fuel efficiency of a car is less than companies claim

Gene expression is greater than 10 RPKM

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-133

t Test Example

sign of bright future

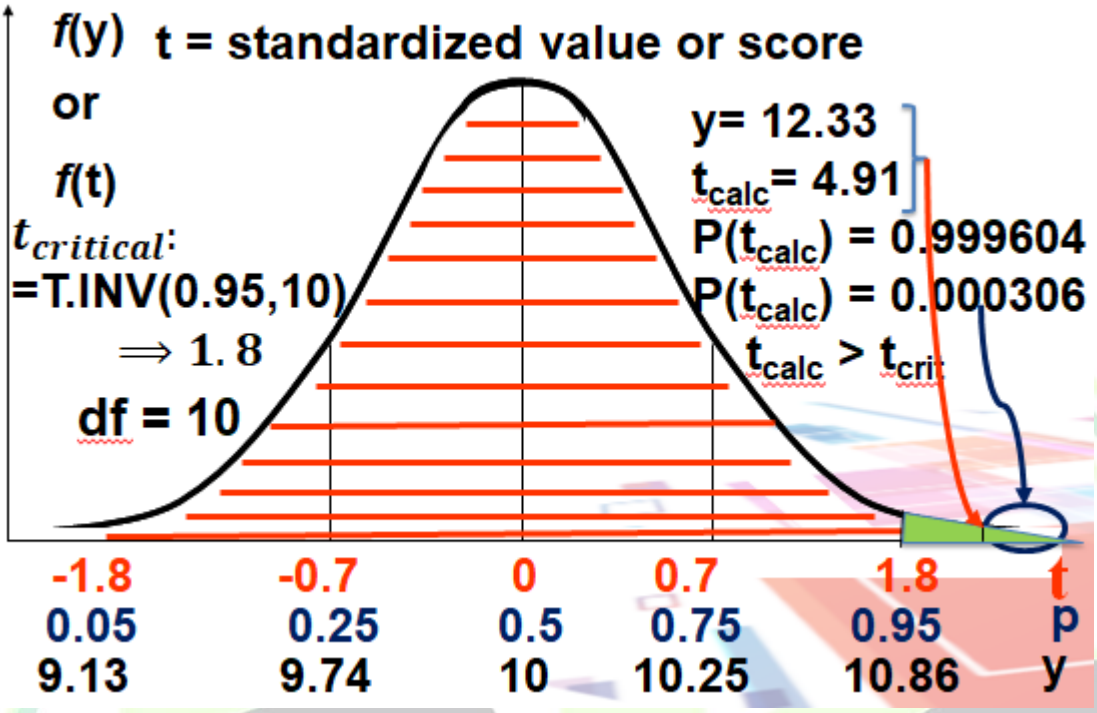
Example:

Expression of TP53 gene family is greater than 10 RPKM

We sample 11 genes belonging to TP53 gene family and found that the mean expression was 12.35 RPKM with a standard deviation of 1.5

n = 11, y = 12.35 s = 1.5

sy = s/√n = 1.5/ √10 = 0.475



(There are two red fours in a deck of 52, the 4 of hearts and the 4 of diamonds).

Binomial Tests

Since RNASeq data is a binomial data, t tests that assume normal distributions may not be appropriate for that

Negative binomial tests are used by Bioconductor packages like **edgeR** and **DESeq**

Conclusions

P < 0.05 (alpha)

0.000306 << 0.05

We conclude that the expression of TP53 gene family is greater than 10 RPKM

Probability of the expression > 10 is 0.0003 i.e 3/10000

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-134
Two sample t Test

Introduction

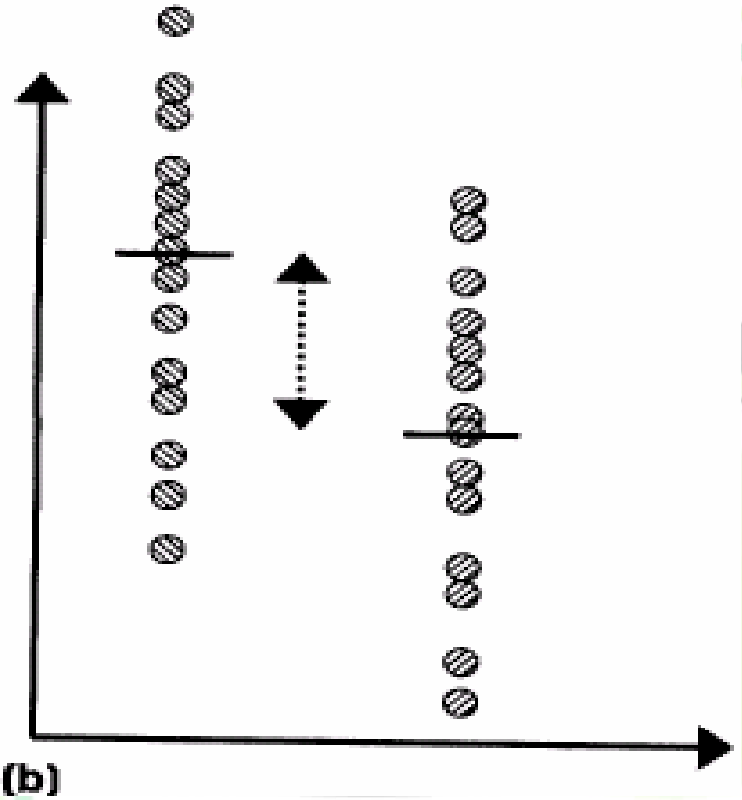
- Differential gene expression studies involve comparison of gene expression across two or more treatments

- Most models assume independence of observations i.e the value of one observation does not give information about other

Unpaired t-test

Gene expression from two groups of samples (one per condition)

- Test if the difference between the means of two conditions is zero



(b)

$y1$ = average of Trt1
$y2$ = average of Trt2
S_1 = Std dev of Trt1
S_2 = Std dev of Trt2

$$S_{\bar{Y}_B - \bar{Y}_A} = \sqrt{\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}}$$

$$t_{calculated}: \frac{(\bar{Y}_B - \bar{Y}_A) - \mu_0}{S_{\bar{Y}_B - \bar{Y}_A}}$$

Consider the expression of **TP53** gene in patients treated with $ZnCl_2$ vs normal persons as control

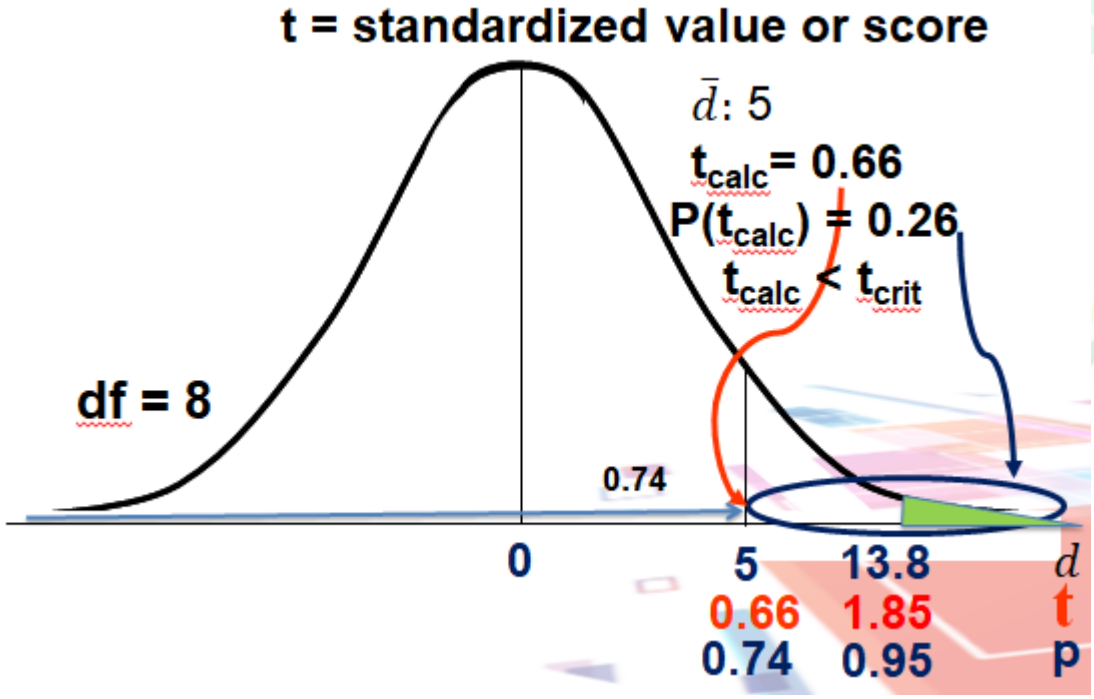
Control	Treatment
12	15
61	64
44	45
10	13
47	49

Notice that we have two **independent** or **uncorrelated** samples in this question
So we have to run a t-test for two **independent** samples

$\boldsymbol{H_0}$: $\boldsymbol{\mu_d \leq 0}$
$\boldsymbol{H_A}$: $\boldsymbol{\mu_d > 0}$
$\bar{Y}_B - \bar{Y}_A$: $46 - 41 = 5$

$S_{\bar{Y}_B - \bar{Y}_A} = \sqrt{\frac{143.2}{6} + \frac{194.8}{6}} = 7.5$
$\boldsymbol{t_{calculated}}$: $\frac{5-0}{7.5} = 0.66$

Null Distribution
If H_0 is true, $t_{calculated}$ follows t distribution with $(n_A - 1) + (n_B - 1)$
$= 5 - 1 + 5 - 1 = 8$
degrees of freedom



(There are two red fours in a deck of 52, the 4 of hearts and the 4 of diamonds).

Conclusions

Differential gene expression studies involve comparison of gene expression across two or more treatments
Most models assume independence of observations

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-135

Paired t Test

Introduction

Most models assume independence of observations i.e the value of one observation does not give information about other

Some times observations are dependent

Dependencies

Dependencies may arise:

- within array duplicate spots located next to each other
- across arrays same sample used in multiple slides

Dependencies may arise:

- across samples

samples from same patient or subject

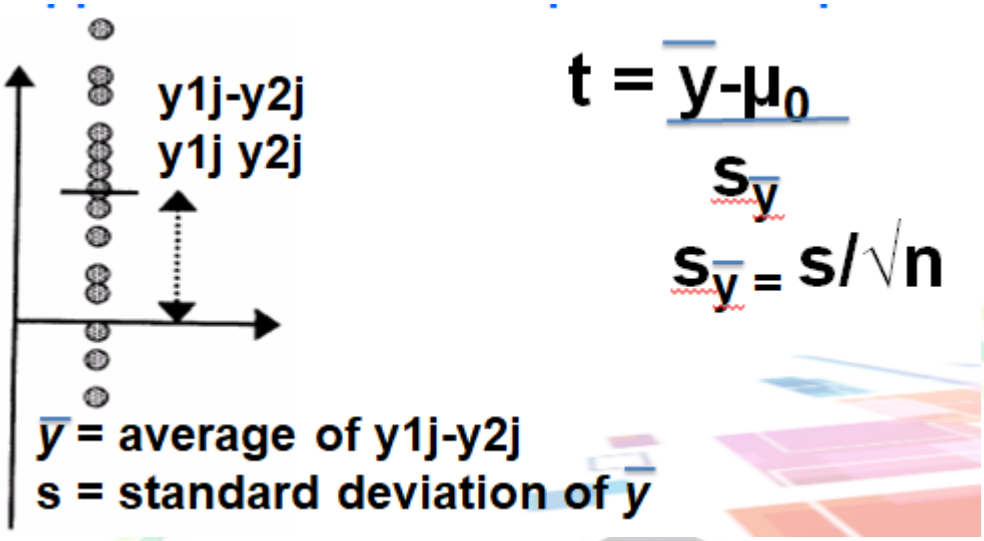
Approaches to resolve potential dependencies:

1) Use models that account for correlations

2) Combine dependent observations into a single observation

i) Average observations from replicated spots within microarray

ii) Pair data: when different conditions are measures on the same subject otherwise, unpaired data analysis



Paired t-test

- Data points are somehow correlated
- We have to pair up the data points and calculate their differences
- Test our hypothesis on the distribution of these differences
- Consider the expression of **TP53** gene in patients before and after a treatment with $ZnCl_2$

Patient	Cont.	Trt.
1	12	15
2	61	64
3	44	45
4	10	13
5	47	49

Test whether the drug is effected or not?

TP53 encodes a tumor suppressing protein **P53**

We want to see a higher expression of TP53 to control tumor

correlated samples

we have to run a paired t-test. Compute the difference in expression for each pair of genes

d = Treatment - Control

Patient	Cont.	Trt.	d
1	12	15	3
2	61	64	3
3	44	45	1
4	10	13	3
5	47	49	2

we will only use d in our statistical test

One sample T-test

H_0: $\mu_d \leq 0$

H_A: $\mu_d > 0$

$\bar{d}$: 2.4 (this is the average of all d values and our sample mean for d)

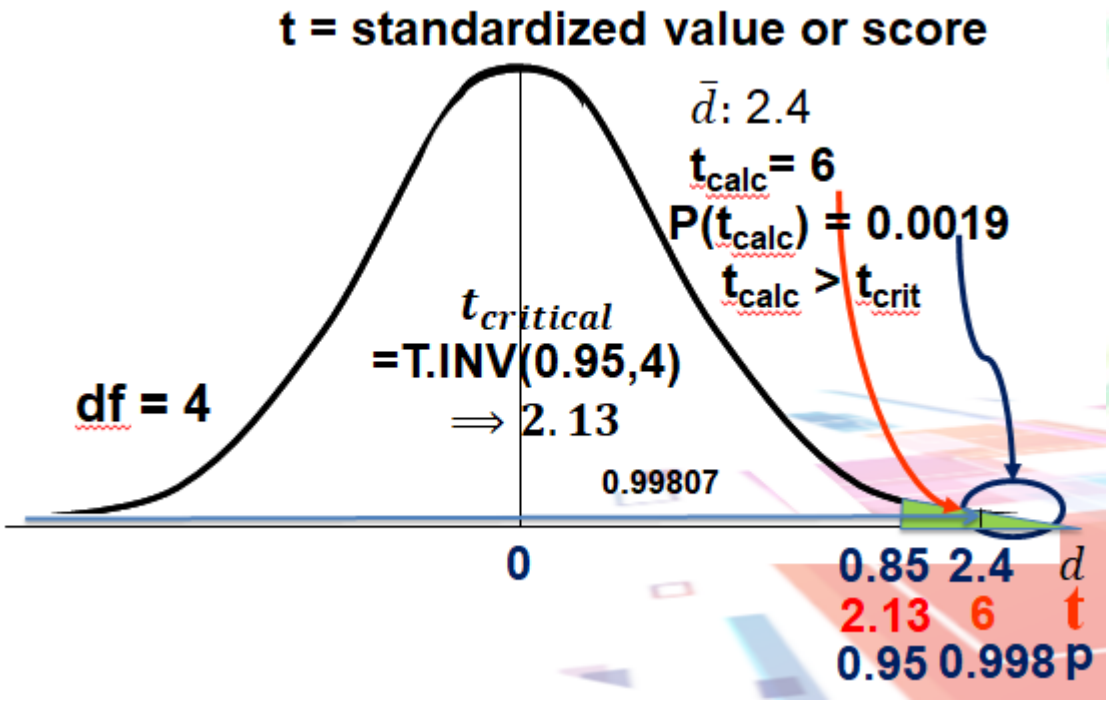
S_d: 0.89 (this is the sample standard deviation of the d values)

$S_{\bar{d}} = \frac{S_d}{\sqrt{n}} = \frac{0.89}{\sqrt{5}} = 0.40$

$t_{calculated}$: $\frac{\bar{d}-\mu_d}{S_{\bar{d}}} = \frac{2.4-0}{0.40} = \mathbf{6.0}$

Null Distribution:

If H_0 is true, $t_{calculated}$ follows t distribution with $(n-1) = 5-1 = 4$ degrees of freedom (because we have n=5 d values)



Potential disadvantages of combining data:

- Loose information, does not account for original variability
- Assumes all combinations are equally effective

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-136

Multiplicity Issue

P values

Using statistical tests is usually a better way of determining which genes are truly changed in an experiment

The p-value tells you how much confidence to place in your result

Sensitivity

How many of the truly changed genes you detect without sacrificing specificity (i.e., incorrectly thinking that a gene's expression is altered)

Sensitivity vs Specificity

Type I error (α, "p-value") false positives

Type II error (β)

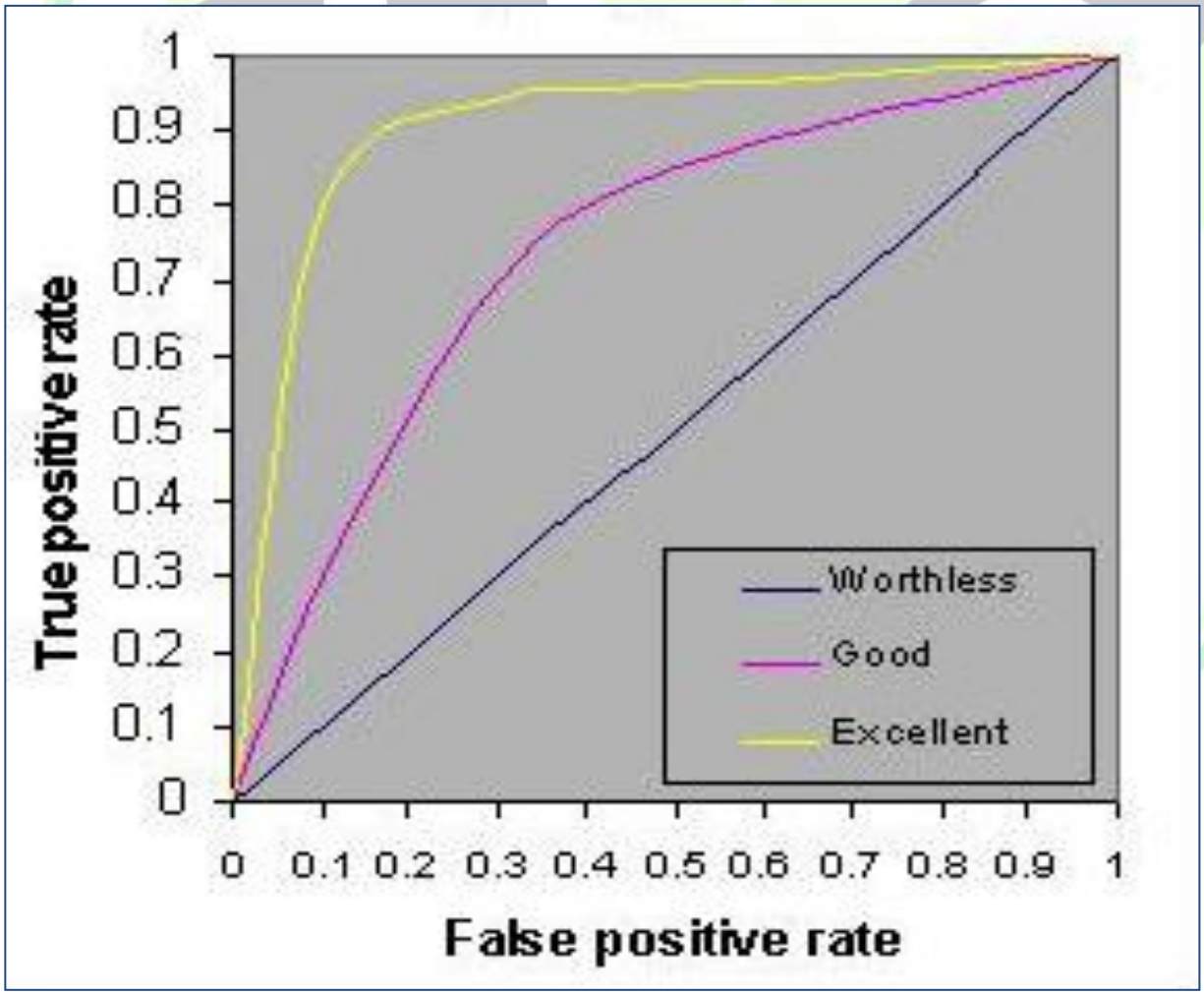
false negatives

the higher the one,

the lower the other

sign of bright future

Decision	Null Hypothesis	
	Correct	Wrong
Reject H_0	Type I error α	Correct $1 - \beta$
Accept H_0	Correct $1 - \alpha$	Type II error β



"ROC" (receiver-operator characteristic) curve

Multiplicity Issue

Gene	WT1	WT2	WT3	MU1	MU2	MU3	Unpaired	Paired
G1	2.2	2.3	1.8	20.9	21.2	20.7	0.00	0.00
G6	3.8	4.6	4.7	4.3	5.2	5.2	0.12	0.00
G4	34.3	34.3	33.6	2.0	2.8	1.3	0.00	0.00
G3	2.6	2.8	3.4	12.7	12.5	13.4	0.00	0.00
G23	19.5	21.5	16.7	33.0	34.6	29.6	0.00	0.00
G72	1.8	2.3	1.9	21.1	22.2	22.2	0.00	0.00
G9	6.7	9.4	12.1	58.3	59.5	65.6	0.00	0.00
G10	2.4	2.5	1.6	31.7	33.2	30.1	0.00	0.00

Multiplicity Issue

Multiple testing problem occurs when the chance of anyone of the results being called significant while in really it is not is small, but since many tests are performed altogether, guaranteed that many of the results will be incorrect (false positives)

Expression studies pose unique statistical challenges due to interrogation of so many gene simultaneously. This leads to extreme cases of what statisticians refer to as the *multiple testing problem*.

Example

At a = 0.05, 5 of every 100 results might be called significant by mistake (p<0.05)

How many tests will be wrong in an experiment with 20,000 tests

Bonferoni corrrection

Set α to desired α/number of tests

i.e 0.05/20,000 =

Flase Discovery Rate (FDR)

It is often OK to have *some* false positives, just not too many to control the *False Discovery Rate (FDR)*

Often, this is an acceptable trade-off as it allows you to find many more genes

(FDR q-value)

The expected proportion of false-positives among the positive results

i.e., at q=0.05, if you call 1000 genes significantly changed, up to 50 of those might be false positives, but the

rest will be true positives

Conclusions

Multiple testing problem occurs many tests are performed altogether

It is often OK to have *some* false positives, just not too many to control the *False Discovery Rate (FDR)*

References

- Trevino, V., Falciani, F., and Barrera-Saldana, H.A. 2007. DNA microarrays: a powerful genomic tool for biomedical and clinical research. Mol Med 13: 527-541.
- Kato, H., Saito, K., and Kimura, T. 2005. A perspective on DNA microarray technology in food and nutritional science. Curr Opin Clin Nutr Metab Care 8: 516-522.

Lesson-137

cBioPortal Database for Cancer Genomics

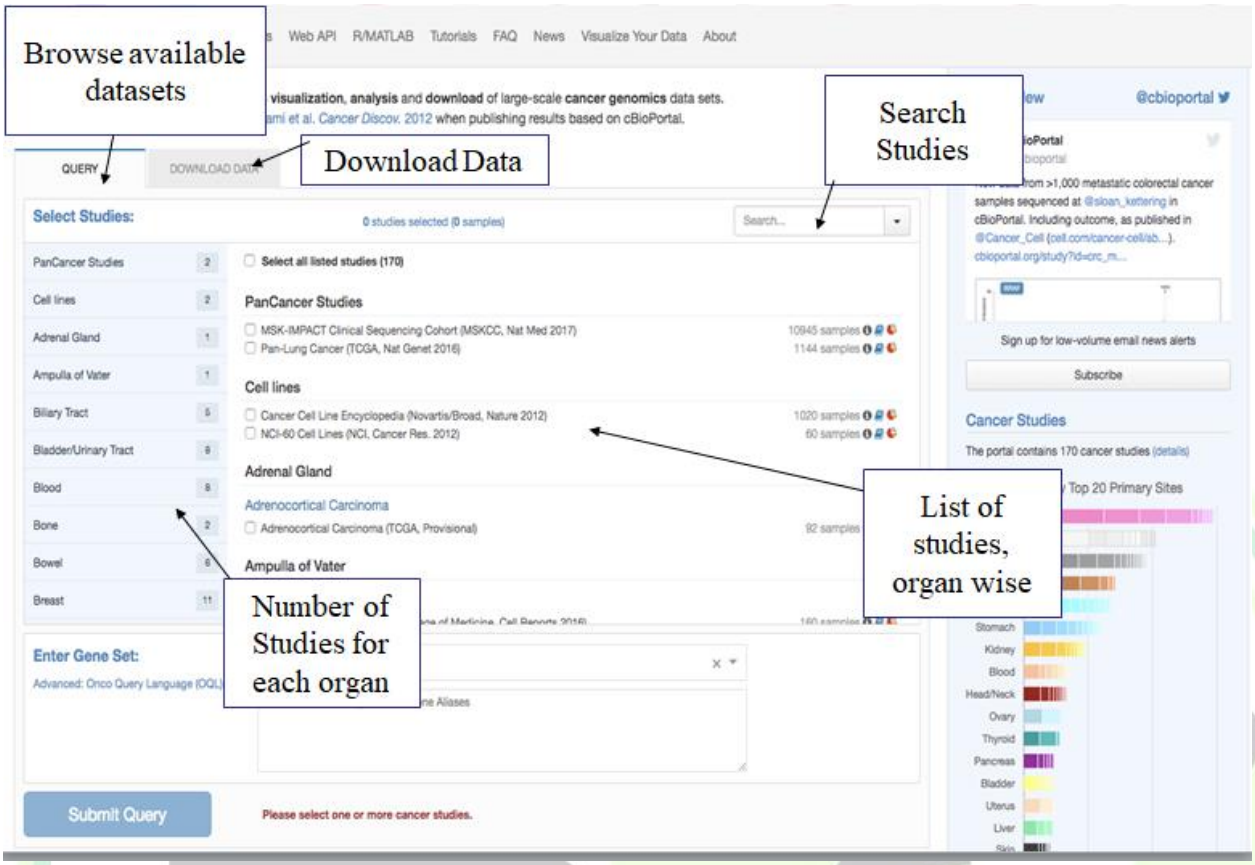
Introduction

- The cBioPortal for Cancer Genomics is an open-access, open-source resource for interactive exploration of multidimensional cancer genomics data sets.
- The cBioPortal significantly lowers the barriers between complex genomic data and cancer researchers
- Rapid, intuitive, and high-quality access to molecular profiles and clinical attributes from large-scale cancer genomics projects are always required for researches.

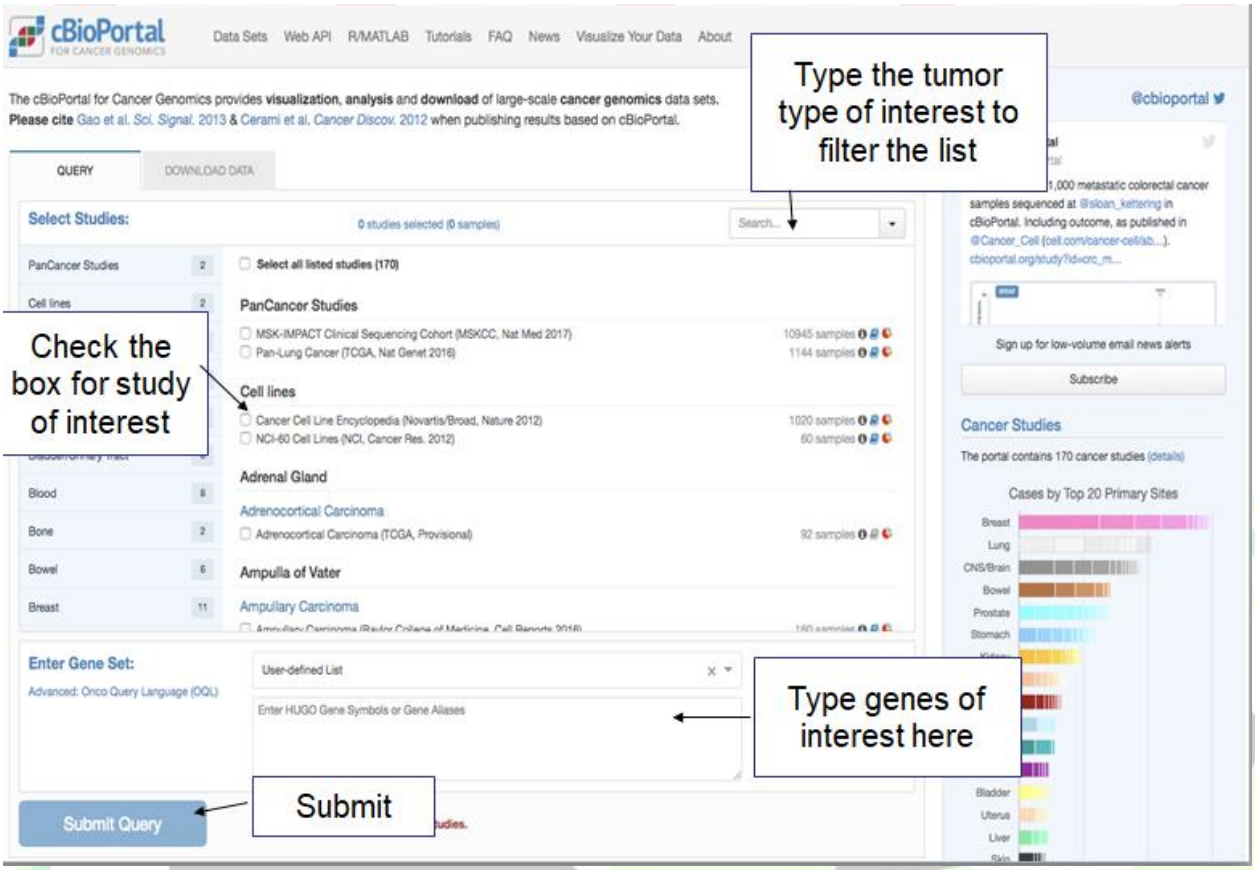
Data Types

- The portal stores
  - DNA copy-number data
  - mRNA and microRNA expression data,
  - non-synonymous mutations,
  - protein-level and phosphoprotein level (RPPA) data,
  - DNA methylation data, and
  - limited de-identified clinical data

Main Page:



Submitting a query

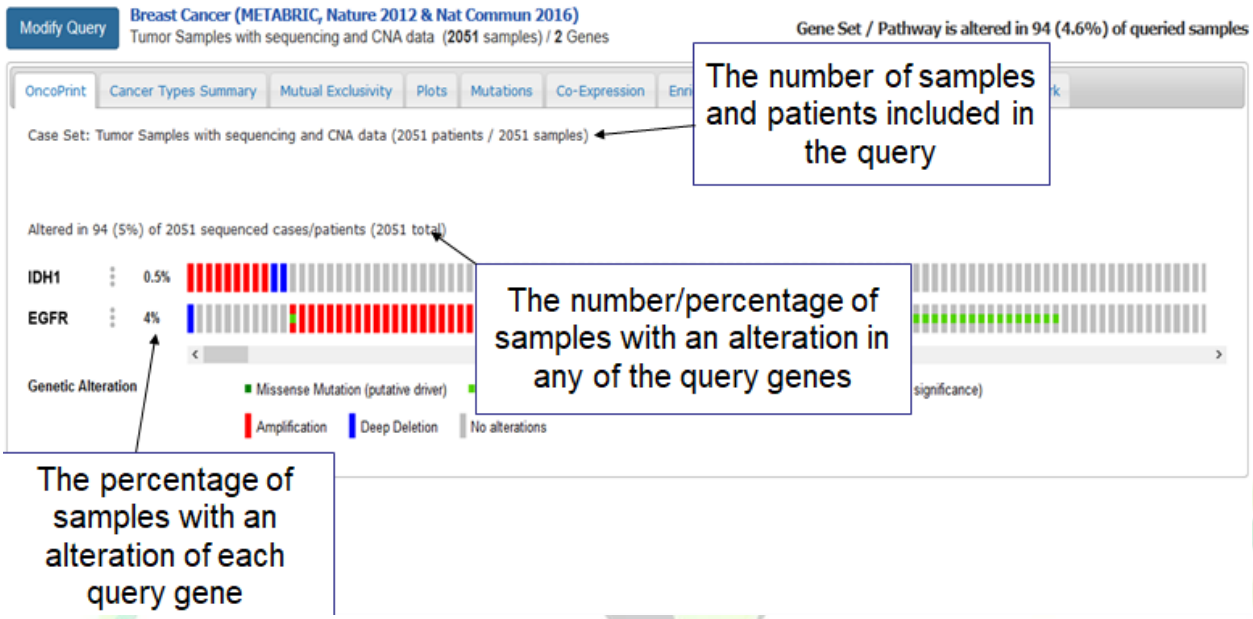


Querying multiple genes in a single dataset

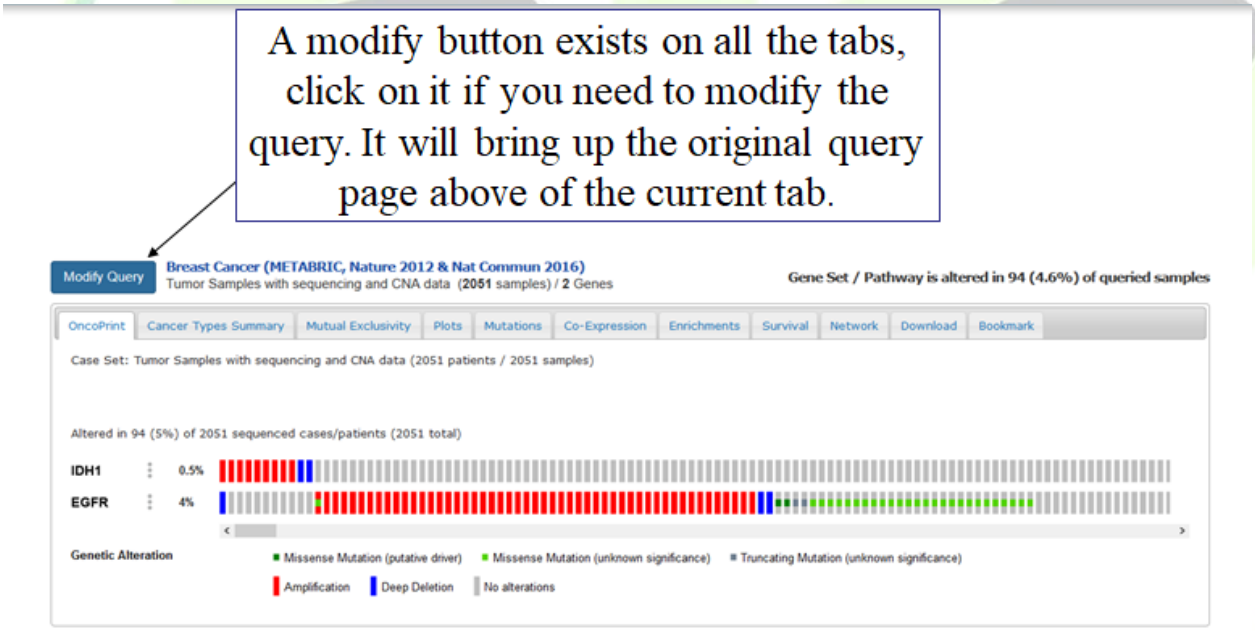
- Data/analysis tabs:
    - OncoPrint
    - Mutual Exclusivity
    - Plots
    - Mutations
    - Co-expression
    - Enrichments
    - Survival
    - Network
    - Download
    - Bookmarks

Performing the previous query will bring you to the OncoPrint Tab

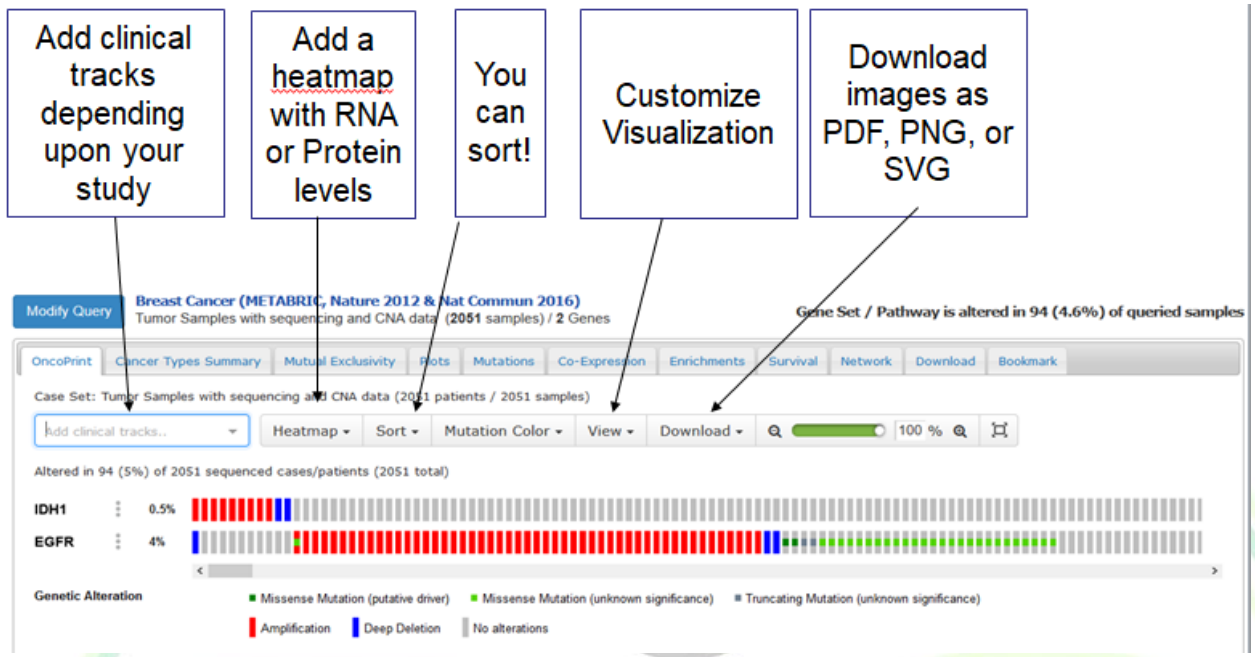
OncoPrint

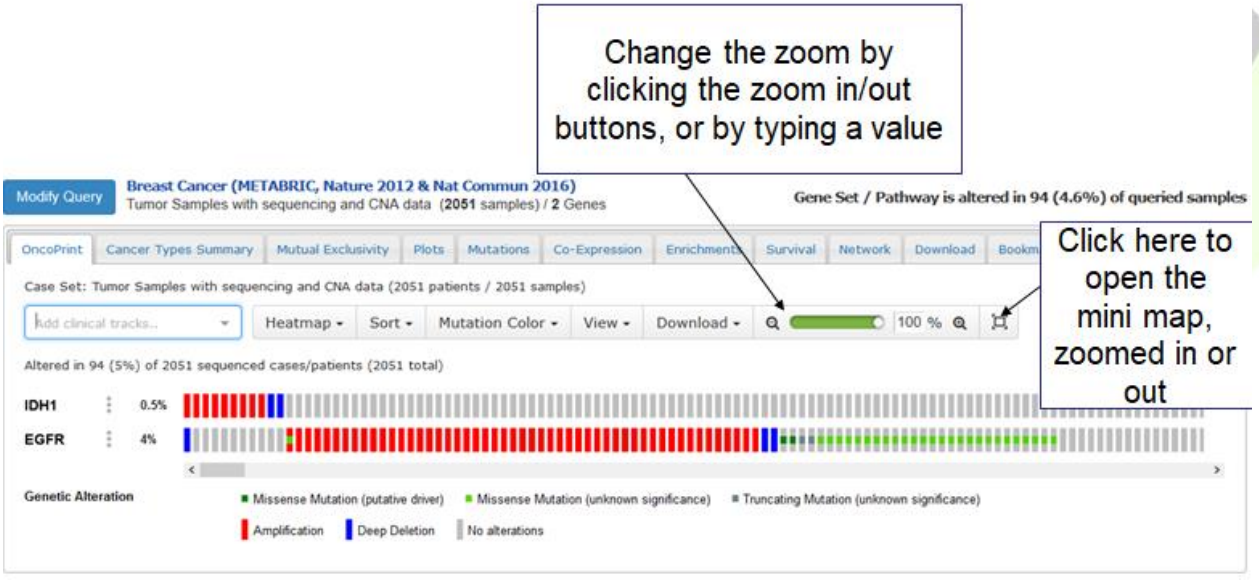


Modifying a query

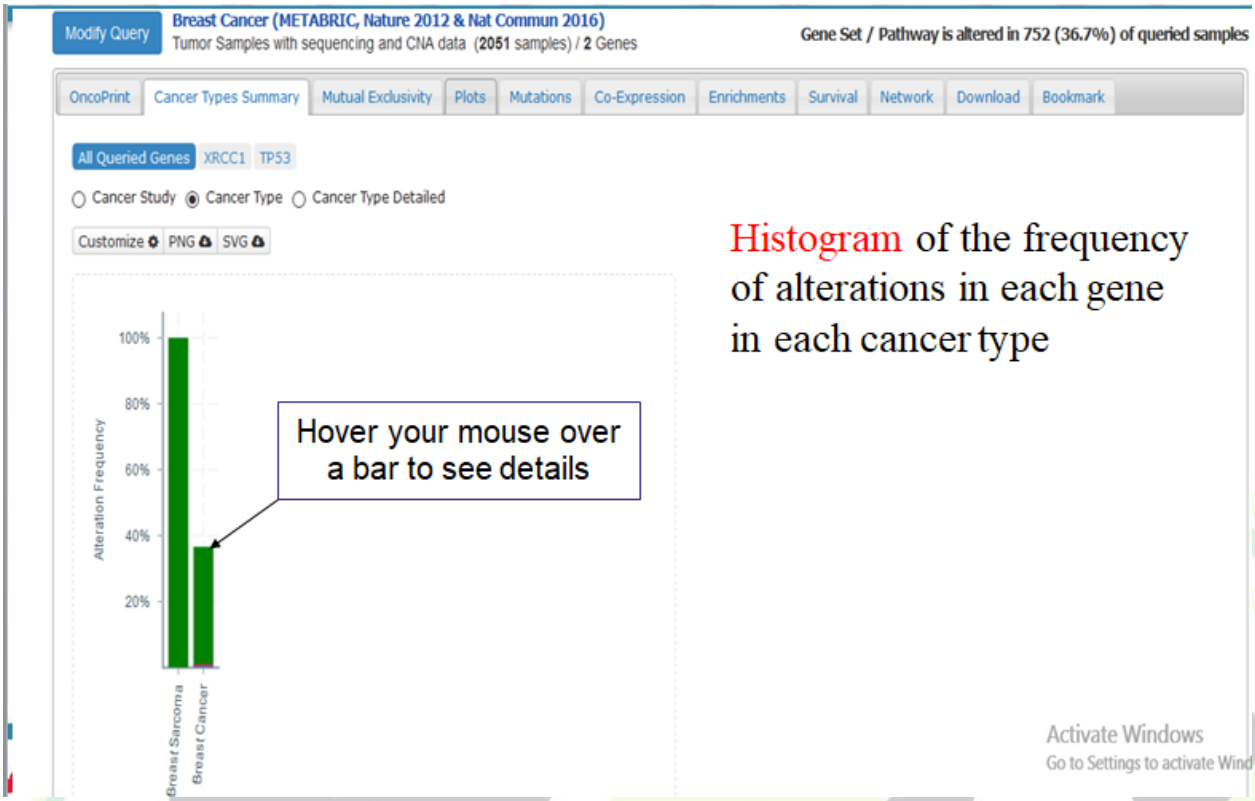


OncoPrint: Advanced Features



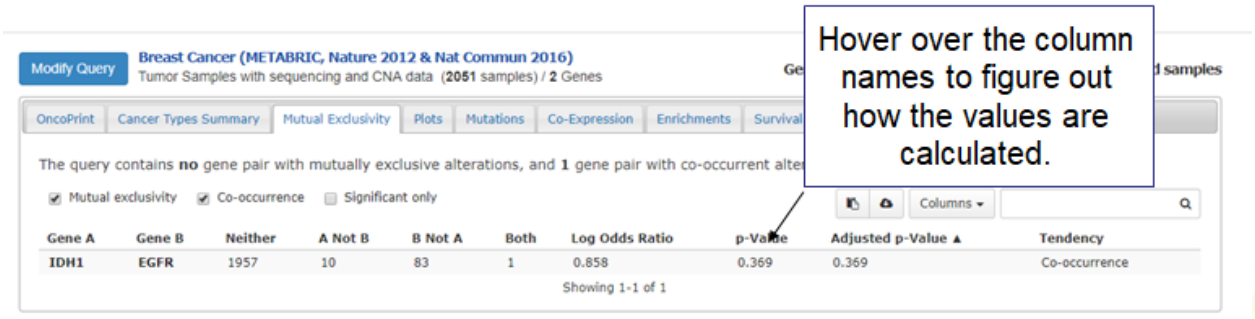


Cancer types Summary

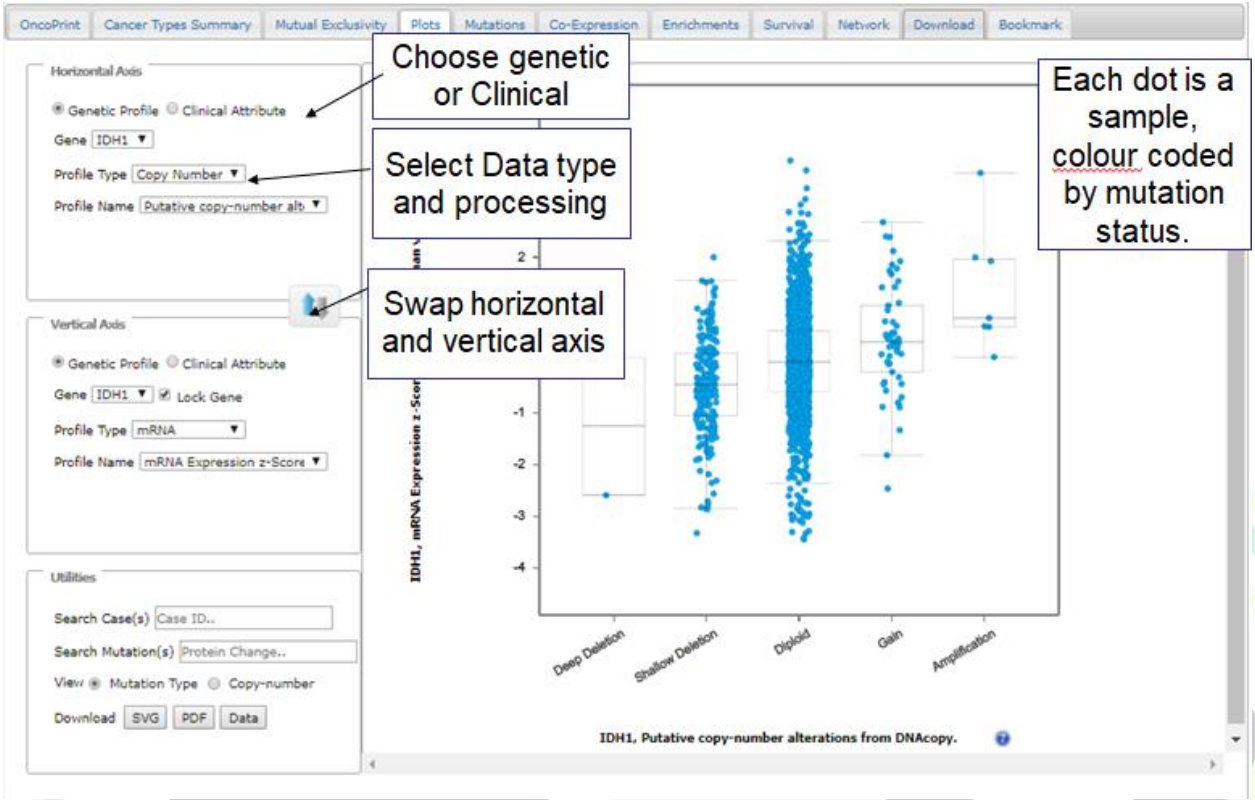


Mutual Exclusivity

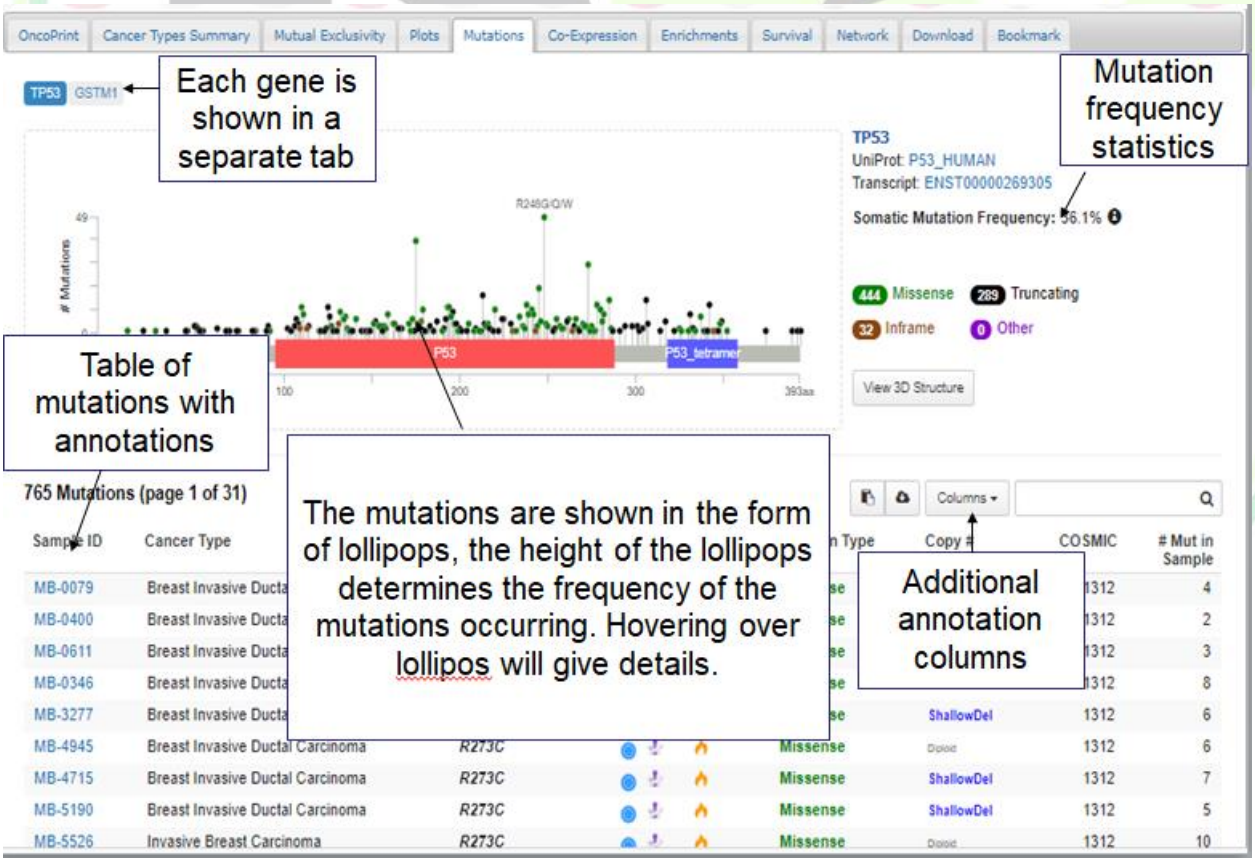
- The Mutual exclusivity tab provides a statistical analysis of the alterations depicted by the OncoPrint tab.
- All the pairwise combinations of query genes are analyzed for mutual exclusivity or co-occurrence.



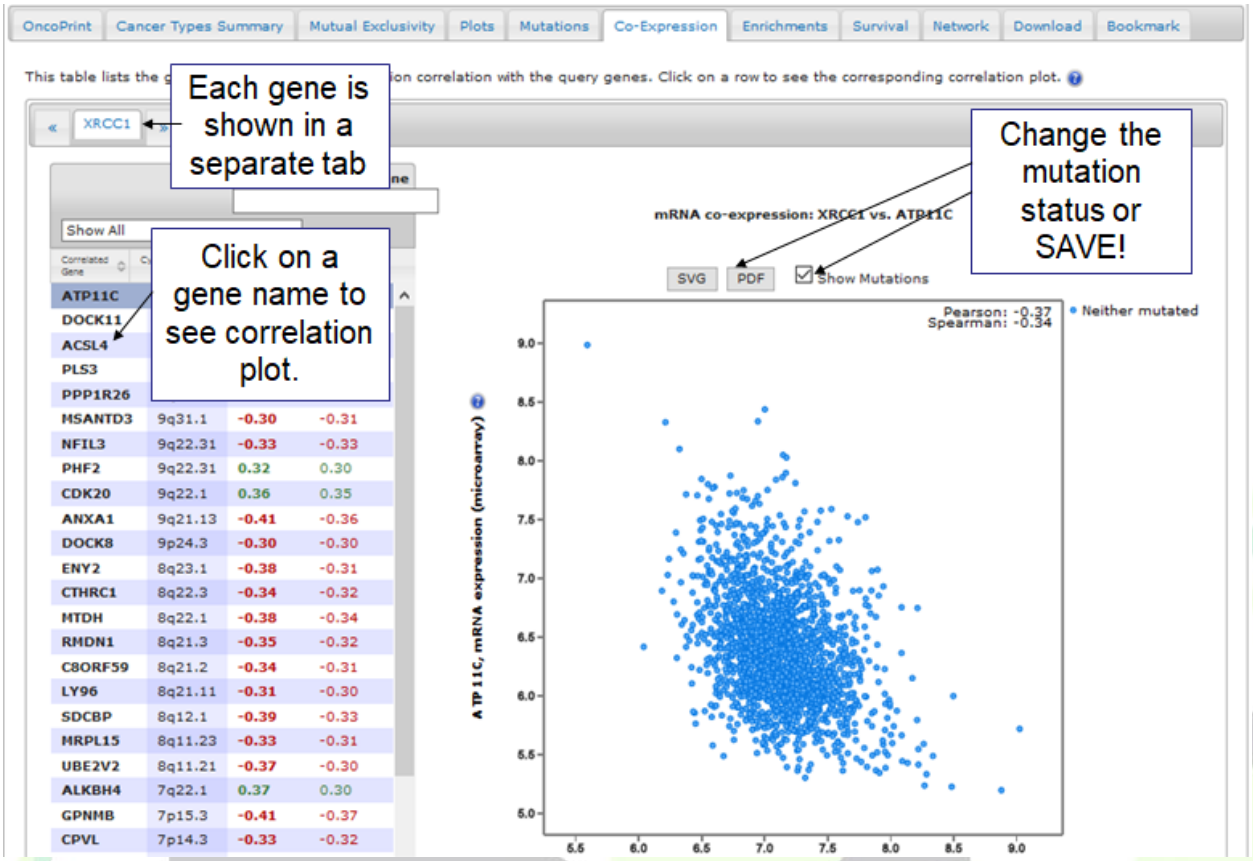
Plots



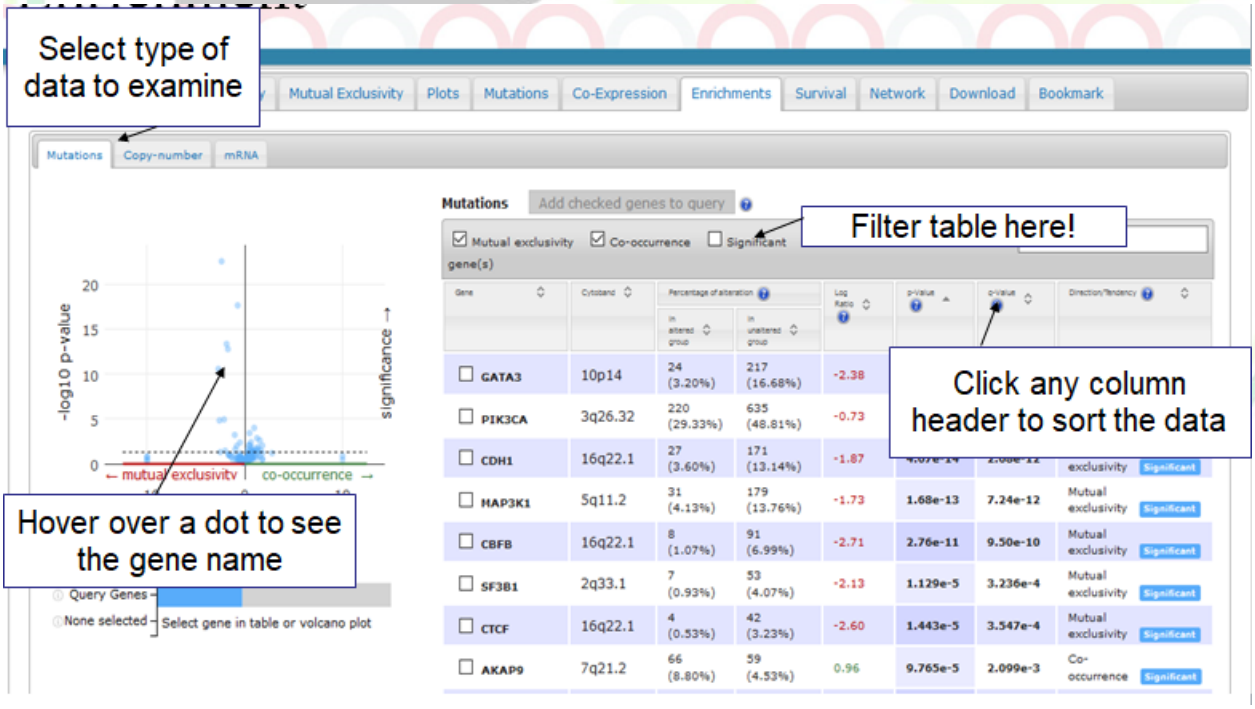
Mutations



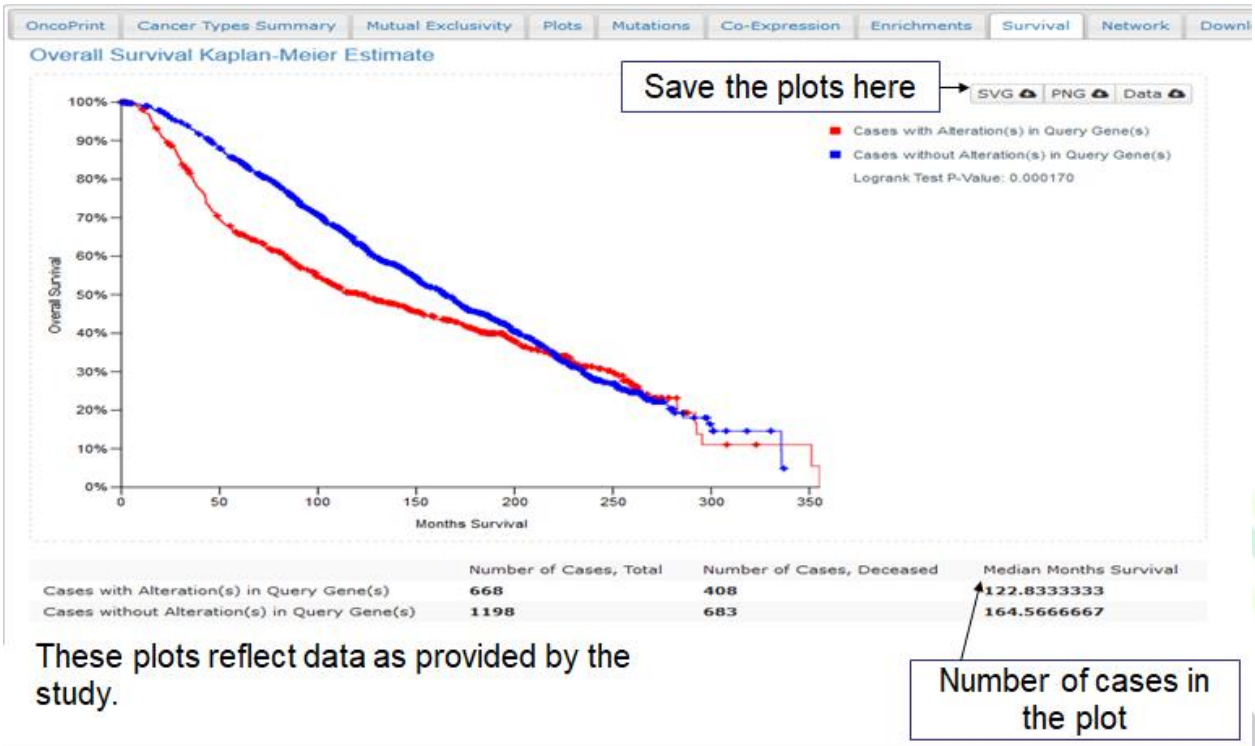
Co-Expression



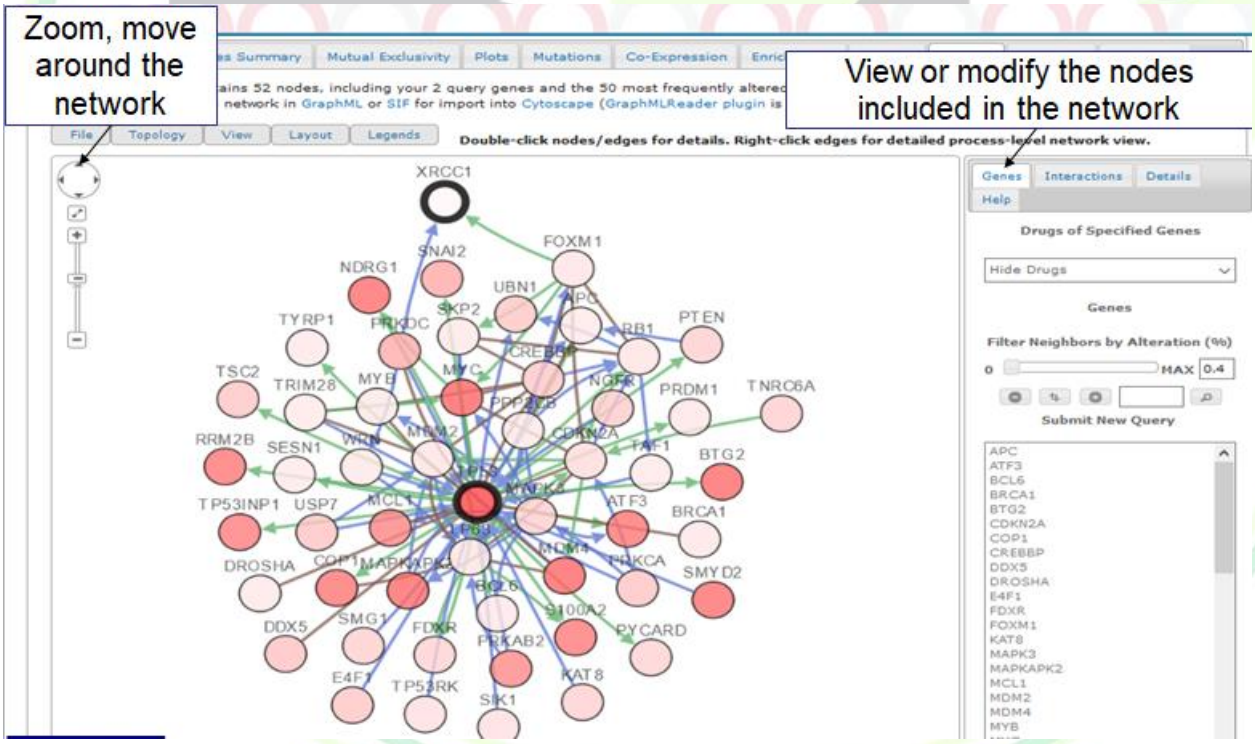
Enrichment

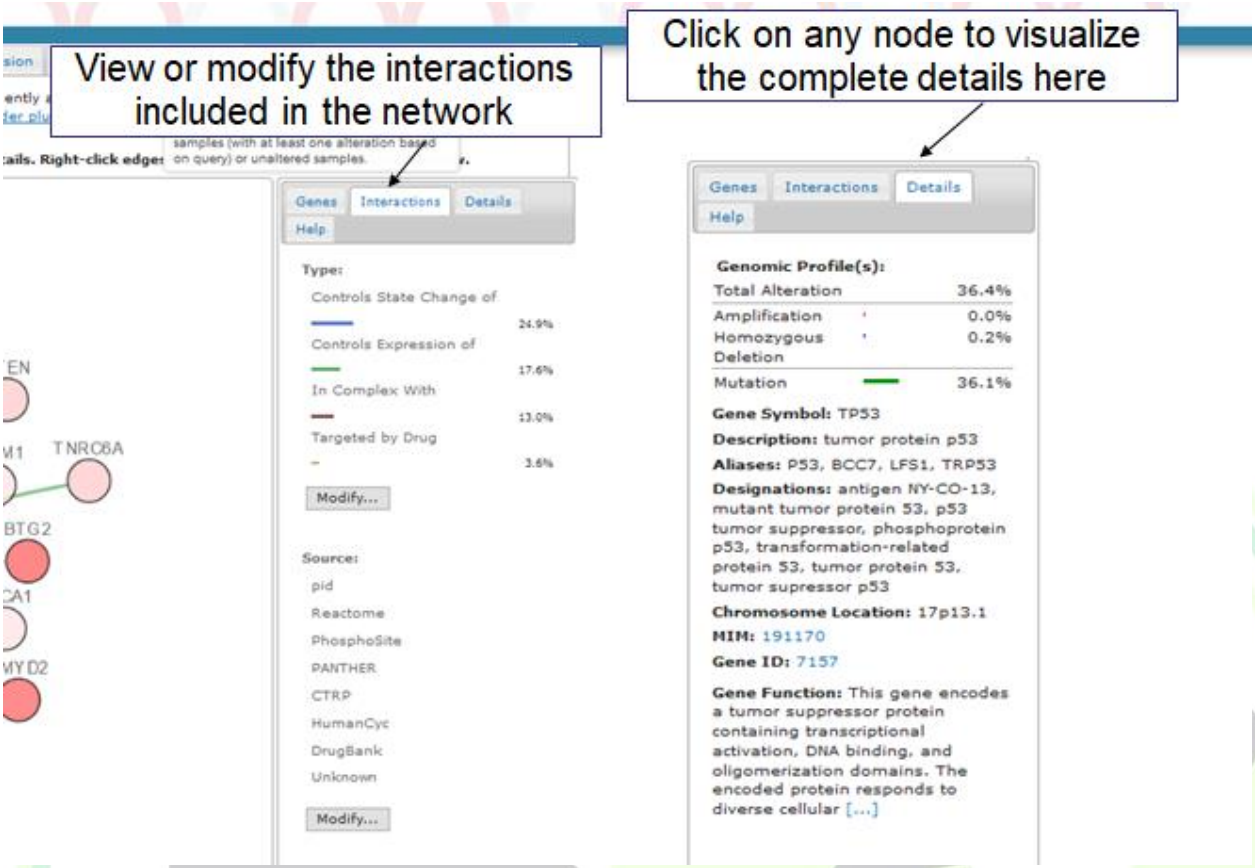


Survival

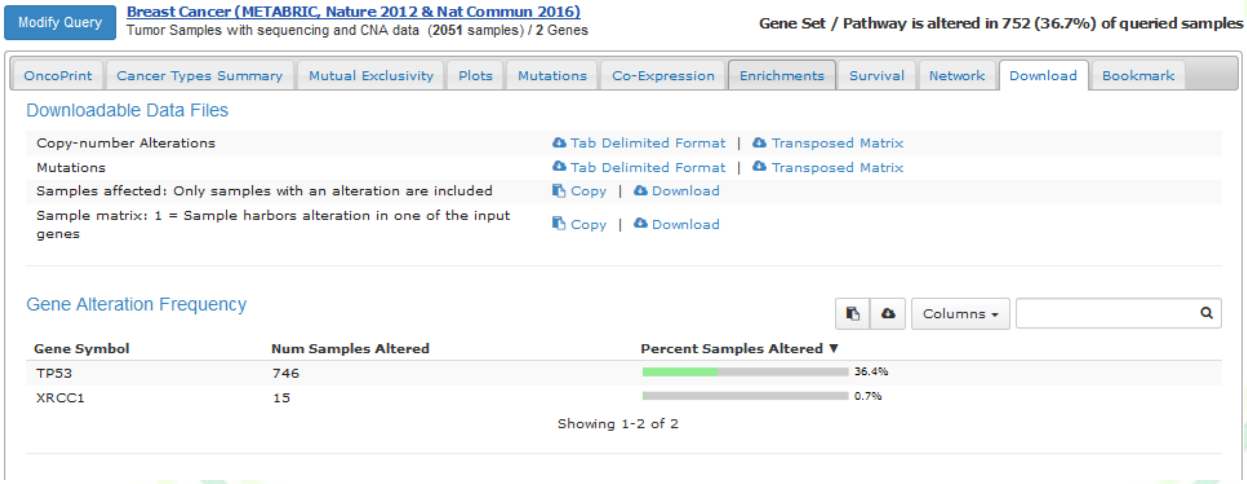

OncoPrint
Cancer Types Summary
Mutual Exclusivity
Plots
Mutations
Co-Expression
Enrichments
Survival
Network
Overall Survival Kaplan-Meier Estimate
Save the plots here
SVG
PNG
Data
Cases with Alteration(s) in Query Gene(s)
Cases without Alteration(s) in Query Gene(s)
Logrank Test P-Value: 0.000170
Overall Survival
Months Survival
Number of Cases, Total
Number of Cases, Deceased
Median Months Survival
Cases with Alteration(s) in Query Gene(s) 668 408 122.8333333
Cases without Alteration(s) in Query Gene(s) 1198 683 164.5666667
These plots reflect data as provided by the study.
Number of cases in the plot

Network

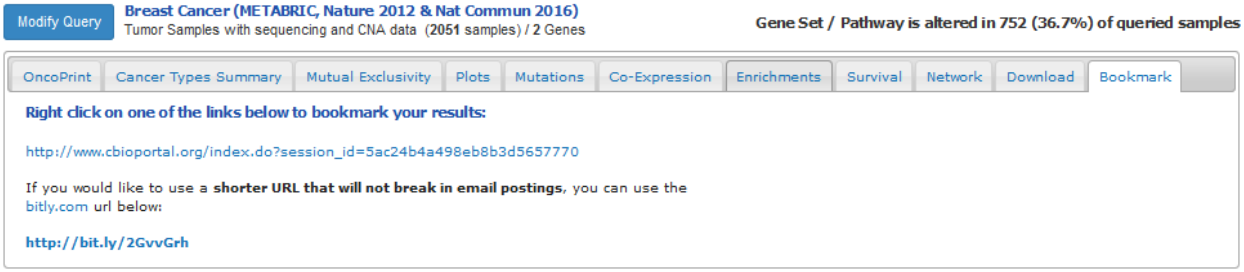

Zoom, move around the network
View or modify the nodes included in the network
File
Topology
View
Layout
Legends
Double-click nodes/edges for details. Right-click edges for detailed process-level network view.
Genes
Interactions
Details
Help
Drugs of Specified Genes
Hide Drugs
Genes
Filter Neighbors by Alteration (%)
MAX
Submit New Query
XRCC1
SNAI2
FOXM1
NDRG1
UBN1
SKP2
PRKDC
TYRP1
RB1
PTEN
TSC2
TRIM28
MYB
MYC
PRDM1
TNRC6A
RRM2B
SESN1
BTG2
TP53INP1
USP7
MCL1
ATF3
BRCA1
DROSHA
MDM4
SMYD2
DDX5
SMG1
FDXR
PYCARD
E4F1
TP53RK
KAT8
SIK1



Download



Bookmark



Bio Mates
sign of bright future